

Hardware Verification Test System

SW13-511

LCN Service - 3

Hardware Verification Test System

**SW13-511
March 2013**

Copyright, Notices, and Trademarks

Copyright 2013 by Honeywell International S rl.

Revision 07 – March, 2013

While this information is presented in good faith and believed to be accurate, Honeywell disclaims the implied warranties of merchantability and fitness for a particular purpose and makes no express warranties except as may be stated in its written agreement with and for its customers.

In no event is Honeywell liable to anyone for any indirect, special or consequential damages. The information and specifications in this document are subject to change without notice.

Honeywell, **TotalPlant**, and TDC 3000 are U.S. registered trademarks of Honeywell International Inc.

Other brand or product names are trademarks of their respective owners.

Other brand or product names are trademarks of their respective owners.

Honeywell Process Solutions
1860 W. Rose Garden Lane
Phoenix, AZ 85027 USA

1-800 822-7673

Support and Other Contacts

Location	Organization	Address	Phone/Facsimile
United States and Canada	Honeywell Solution Support Center	Honeywell TAC, MS P13, 2500 West Union Hills Drive, Phoenix, AZ, 85027	Phone: 1-800 822-7673 Facsimile: 1-973-455-5000
Note: Calls are answered by dispatcher between 6:00 A.M. and 4:00 P.M. Mountain Standard Time. Emergency calls outside normal working hours are received by an answering service and returned within one hour.			
Europe, Middle East, and Africa (EMEA)	Honeywell TAC-EMEA	TAC-BE02 Hermes Plaza Hermeslaan, 1HB-1831 Diegem, Belgium Email: tac- be02@honeywell.com	Phone: +32-2-728-2345 Facsimile: +32-2-728-2696
Pacific	Honeywell Global TAC – Pacific	Honeywell Limited Australia 5 Kitchener Way Burswood 6100, Western Australia Email: GTAC@honeywell.com	Phone: 1300-300-4822 (toll free within Australia) +61-8-9362-9559 (outside Australia) Facsimile: +61-8-9362-9564
India	Honeywell Global TAC – India	Honeywell Automation India Ltd. 56 and 57, Hadapsar Industrial Estate Hadapsar, Pune –411 013, India Email: Global-TAC- India@honeywell.com	Phone: +91-20- 66039400 Facsimile: +91-20- 66039800
Korea	Honeywell Global TAC – Korea	Honeywell Co., Ltd 17F, Kikje Center B/D, 191, Hangangro-2Ga Yongsan-gu, Seoul, 140- 702, Korea Email: Global-TAC- Korea@honeywell.com	Phone: +82-2-799-6317 +82-11-9227-6324 Facsimile: +82-2-792-9015

Location	Organization	Address	Phone/Facsimile
People's Republic of China	Honeywell Global TAC – China	Honeywell (China) Co., Ltd 33/F, Tower A, City Center, 100 Zunyi Rd., Shanghai 200051, People's Republic of China Email: Global-TAC-China@honeywell.com	Phone: +86- 21-52574568
Singapore	Global TAC – South East Asia	Honeywell Private Limited Honeywell Building 17, Changi Business Park Central 1, Singapore 486073 Email: GTAC-SEA@honeywell.com	Phone: +65-6823-2215 Facsimile: +65-6445-3033
Japan	Global TAC – Japan	Honeywell Japan Inc. New Pier Takeshiba, South Tower Building, 20th Floor, 1-16-1 Kaigan, Minato-ku, Tokyo 105-0022, Japan Email: Global-TAC-JapanJA25@honeywell.com	Phone: +81-3-6730-7160 Facsimile: +81-3-6730-7228
Elsewhere	Call your nearest Honeywell office.		
World Wide Web	Honeywell Solution Support Online: http://www.honeywellprocess.com/		
Training Classes	Honeywell Automation College: http://www.automationcollege.com		

Contents

1	INTRODUCTION	13
1.1	What HVTS Is and How It Works	13
1.2	Uses of HVTS.....	13
1.3	HVTS Restrictions with TPN Bridge Nodes	13
1.4	References	16
2	CLOCK SUBSYSTEM TESTS (CLKS)	17
2.1	Clock Subsystem Test Elements	17
2.2	Use Information	17
2.3	Setup Parameters	19
2.4	Preset Parameter Values for Clks	21
2.5	Mode Characteristics for Clks	22
2.6	Individual Clks Tests	22
2.7	Clks Alarm Categories	24
3	LOCAL CONTROL NETWORK EXERCISER (LCNX).....	25
3.1	Local Control Network Elements.....	25
3.2	Use Information	25
3.3	Setup Parameters	26
3.4	Preset Parameter Values for LCNX	27
3.5	Mode Characteristics for LCNX.....	27
3.6	Individual LCNX Tests.....	27
3.7	LCNX Alarm Messages.....	27
4	KEYBOARD SUBSYSTEM TESTS (KEYB)	29
4.1	Keyboard Subsystem Elements	29
4.2	Use Information	29
4.3	Setup Parameters	30
4.4	Preset Parameter Values for KEYB	31
4.5	Mode Characteristics for KEYB.....	32
4.6	Individual Keyboard Tests	33
4.7	KEYB Alarm Categories.....	33
5	OPERATOR INTERFACE SUBSYSTEM TESTS (OPIF)	35
5.1	Operator Interface Subsystem Elements	35
5.2	USE INFORMATION.....	35

5.3	Setup Parameters	36
5.4	Preset Parameter Values for OPIF	37
5.5	Mode Characteristics for OPIF	38
5.6	INDIVIDUAL OPERATOR INTERFACE TESTS	39
5.7	OPIF Alarm Categories	41
6	WINCHESTER DISK SUBSYSTEM TESTS (WINC)	43
6.1	Winchester Disk Subsystem Elements	43
6.2	Use Information	43
6.3	Setup Parameters	44
6.4	Preset Parameter Values for WINC	46
6.5	Mode Characteristics for WINC	47
6.6	Individual Winchester Disk Tests	47
6.7	WINC Alarm Categories	49
7	FLOPPY DISK SUBSYSTEM TESTS (FLPY)	51
7.1	Floppy Disk Subsystem Elements	51
7.2	Use Information	51
7.3	Setup Parameters	51
7.4	Preset Parameter Values for FLPY	52
7.5	Mode Characteristics for FLPY	52
7.6	Individual Floppy Disk Tests	53
7.7	FLPY Alarm Categories	56
8	DATA HIWAY INTERFACE SUBSYSTEM TESTS (DHIF)	57
8.1	Data Hiway Interface Subsystem Elements	57
8.2	Use Information	57
8.3	Setup Parameters	57
8.4	Preset Parameter Values for DHIF	59
8.5	Mode Characteristics for DHIF	60
8.6	Individual Hiway Interface Subsystem Tests	60
8.7	How to Interpret DHIF Alarms	63
9	STATUS INPUT/OUTPUT SUBSYSTEM TESTS (SIOS)	71
9.1	Status I/O Subsystem Elements	71
9.2	Use Information	71
9.3	Setup Parameters	71
9.4	Preset Parameter Values for SIOS	73

9.5	Mode Characteristics For SIOS	74
9.6	Individual System I/O Tests	74
9.7	SIOS Alarm Categories.....	75
10	MEMORY EXERCISER MODULE TESTS (MEMX).....	77
10.1	Memory Module Elements	77
10.2	Use Information	77
10.3	Setup Parameters	77
10.4	Preset Parameter Values for MEMX.....	78
10.5	Mode Characteristics for MEMX	78
10.6	Individual Memory Tests	78
10.7	MEMX Alarm Categories.....	78
11	COMMUNICATION LINE INTERFACE TEST (CLIF).....	79
11.1	COMMUNICATION LINE INTERFACE ELEMENTS	79
11.2	USE INFORMATION.....	79
11.3	SETUP PARAMETERS	79
11.4	Preset Parameter Values for CLIF	80
11.5	Mode Characteristics for CLIF	80
11.6	Individual CLIF Tests	81
11.7	CLIF Alarm Categories.....	82
12	DATA HIWAY EXERCISER (DHEX)	83
12.1	Data Hiway Exerciser Elements.....	83
12.2	Use Information	83
12.3	Setup Parameters	85
12.4	Preset Parameter Values For DHEX	92
12.5	Mode Characteristics for DHEX	93
12.6	Individual DHEX Tests	94
12.7	DHEX Alarm Categories	94
13	PDG SUBSYSTEM TESTS (PDGI).....	95
13.1	PDG Subsystem Elements.....	95
13.2	Use Information	95
13.3	Setup Parameters	96
13.4	Preset Parameter Values for PDGI.....	98
13.5	Mode Characteristics for PDGI	99
13.6	Individual PDGI Tests	99
13.7	PDGI Alarm Categories.....	100

14	SMART CONTROLLER MAGNETIC DISK TESTS (SCMD)	101
14.1	Smart Controller Subsystem Elements	101
14.2	Use Information	101
14.3	Setup Parameters	102
14.4	Preset Parameter Values for SCMD	107
14.5	Mode Characteristics for SCMD	108
14.6	Individual Smart Controller Tests	108
14.7	SCMD Alarm Categories	115
14.8	SCSI Status Definitions	115
15	APPLICATION MODULE REDUNDANCY TESTS (AMRT)	125
15.1	AM Redundancy Elements	125
15.2	Use Information	125
15.3	Setup Parameters	126
15.4	Preset Parameter Values for AMRT	127
15.5	Mode Characteristics for AMRT	128
15.6	Individual AM Redundancy Tests	128
15.7	AMRT Alarm Categories	130
16	PROCESS NETWORK INTERFACE TESTS (PNIF)	133
16.1	Process Network Interface Elements	133
16.2	Use Information	133
16.3	Setup Parameters	133
16.4	Preset Parameter Values for PNIF	134
16.5	Mode Characteristics for PNIF	134
16.6	Individual Process Network Interface Tests	135
16.7	Common Test Sequences	153
16.8	PNIF Alarm Categories	158
17	PROGRAMMABLE LOGIC CONTROLLER INTERFACE TESTS (PLCT)	159
17.1	Programmable Logic Controller Interface Elements	159
17.2	Use Information	159
17.3	Setup Parameters	161
17.4	Preset Parameter Values for PLCT	163
17.5	Mode Characteristics fr PLCT	164
17.6	Individual PLC Interface Subsystem Tests	164
17.7	How to Interpret PLCT Alarms	167

18	COMMUNICATION NETWORK INTERFACE TESTS (CNIF)	175
18.1	Communication Network Interface Elements	175
18.2	Use Information	175
18.3	Setup Parameters	175
18.4	Preset Parameter Values for CNIF	177
18.5	Mode Characteristics for CNIF	178
18.6	Individual CNIF Tests	178
18.7	CNIF Alarms	180
19	NETWORK GATEWAY INTERFACE TESTS (NGIF)	181
19.1	Network Gateway Interface Elements	181
19.2	Use Information	181
19.3	Setup Parameters	182
19.4	Preset Parameter Values For NGIF	183
19.5	Mode Characteristics for NGIF	183
19.6	Individual Network Gateway Interface Tests	183
19.7	Common Test Sequences	203
19.8	NGIF Alarm Categories	210
20	PROCESS NETWORK INTERFACE ENHANCED TESTS (PNIE)	211
20.1	Process Network Interface Elements	211
20.2	Use Information	211
20.3	Setup Parameters	211
20.4	Preset Parameter Values for PNIE	214
20.5	Mode Characteristics for PNIE	214
20.6	Individual Process Network Interface Tests	215
20.7	Common Test Sequences	245
20.8	PNIE Alarm Categories	253
21	LCN SEQUENCE OF EVENTS TEST (LSOE)	255
21.1	LCN Sequence Of Events Elements	255
21.2	Use Information	255
21.3	Setup Parameters	255
21.4	Preset Parameter Values for LSOE	256
21.5	Mode Characteristics for LSOE	256
21.6	LSOE Test	257
21.7	LSOE Messages	258
22	GENERIC GATEWAY SERIAL INTERFACE TEST (GGSF)	259

22.1	Generic Gateway Serial Interface Elements	259
22.2	Use Information	259
22.3	Setup Parameters.....	259
22.4	Preset Parameter Values for GGSF	260
22.5	Mode Characteristics for GGSF	260
22.6	Individual Generic Gateway Serial Interface Tests	260
22.7	GGSF Alarm Messages.....	262
23	WSI SUBSYSTEM TESTS (WSIF).....	263
23.1	WSI Subsystem Elements	263
23.2	WSIF HVTP Use Information.....	263
23.3	Setup Parameters.....	264
23.4	Mode Characteristics for WSIF.....	265
23.5	Individual WSIF Tests.....	266
23.6	Alarm Messages	274
23.7	Alarm Numbers.....	276
24	U^XS SUBSYSTEM TESTS (XUSF).....	285
24.1	U ^X S Subsystem Elements	285
24.2	XUSF HVTP Use Information	285
24.3	Setup Parameters.....	286
24.4	Mode Characteristics for XUSF	292
24.5	Individual XUSF Tests	292
24.6	XUSF Alarm Messages	299
25	LCN SUBSYSTEM TEST (LCNS).....	313
25.1	Common LCN Subsystem Test Elements.....	313
25.2	Use Information	313
25.3	Setup Parameters.....	313
25.4	Preset Parameter Values For LCNS	314
25.5	Mode Characteristics For LCNS.....	314
25.6	individual LCNS Tests	315
25.7	LCNS Alarm Messages	316
26	A^XM SUBSYSTEM TESTS (XAMF)	317
26.1	XAMF Subsystem Elements.....	317
26.2	XAMF HVTP Use Information.....	317
26.3	Setup Parameters.....	317

26.4	Mode Characteristics For XAMF	319
26.5	Individual XAMF Tests	319
26.6	XAMF Alarm Numbers	319
27	CONTROL AND COMMUNICATION PROCESSOR TESTS (CCPT)	321
27.1	Control And Communication Processor Tests Elements	321
27.2	CCPT Use Information	321
27.3	Setup Parameters	321
27.4	Preset Parameter Values for CCPT	322
27.5	Mode Characteristics for CCPT	322
27.6	Individual CCPT Tests	322
27.7	CCPT ALARM NUMBERS	324
APPENDIX A		327
	Terms And Abbreviations	327
APPENDIX B		329
	UCLM: CCP and UCIM Redesigned	329

Tables and Figures

Table 1-1 Test Program to Hardware Module Relationship..... 15

Table 14-1 — Sense Key Code Descriptions 120

Table 14-2— Sense Error Code Descriptions..... 121

Figure 1-1 Test System Supported Hardware 14

1 Introduction

This section introduces you to the characteristics and uses of HVTS. It also provides a summary of Release changes and a list of related publications.

1.1 What HVTS Is and How It Works

The Hardware Verification Test System (HVTS) is a collection of test programs that work under the Test System Executive and are used to verify the correct operation of **TotalPlant** Solution (TPS) system. Each test program is concerned with a specific subsystem that may be common to several nodes (sometimes called hardware modules) on the Local Control Network. Table 1-1 shows which test programs are used for each of the various node types.

The test programs are subdivided into individual tests that are designed to exercise specific functions within the subsystem. The service technician controls the operation of HVTS by keyboard entry of commands that specify the hardware to be tested and the number and sequence of tests to be run.

HVTS test programs are packaged with the Test System Executive (EXEC) on cartridge disk number 51152049 or on floppy diskette number 51150670. Follow the loading and test system setup instructions found in the Test System Executive manual.

1.2 Uses of HVTS

HVTS is useful during both system installation and initial hardware validation, and, in a limited sense, for troubleshooting during on-line system operation. System validation is accomplished before loading operational software and, in that instance, HVTS has unconstrained use of the system hardware. If HVTS is to be used for troubleshooting during normal operation, the selected TOCS and the nodes to be tested must be removed from service before HVTS can be loaded.

1.3 HVTS Restrictions with TPN Bridge Nodes

TPS system may consist of two or more separate LCN segments that are joined via TPN Bridge nodes. These bridge nodes must be powered down, and not loaded with HVTS, and do not partake of the HVTS testing. Testing of the nodes on each individual LCN segment may occur, but not across the bridges.

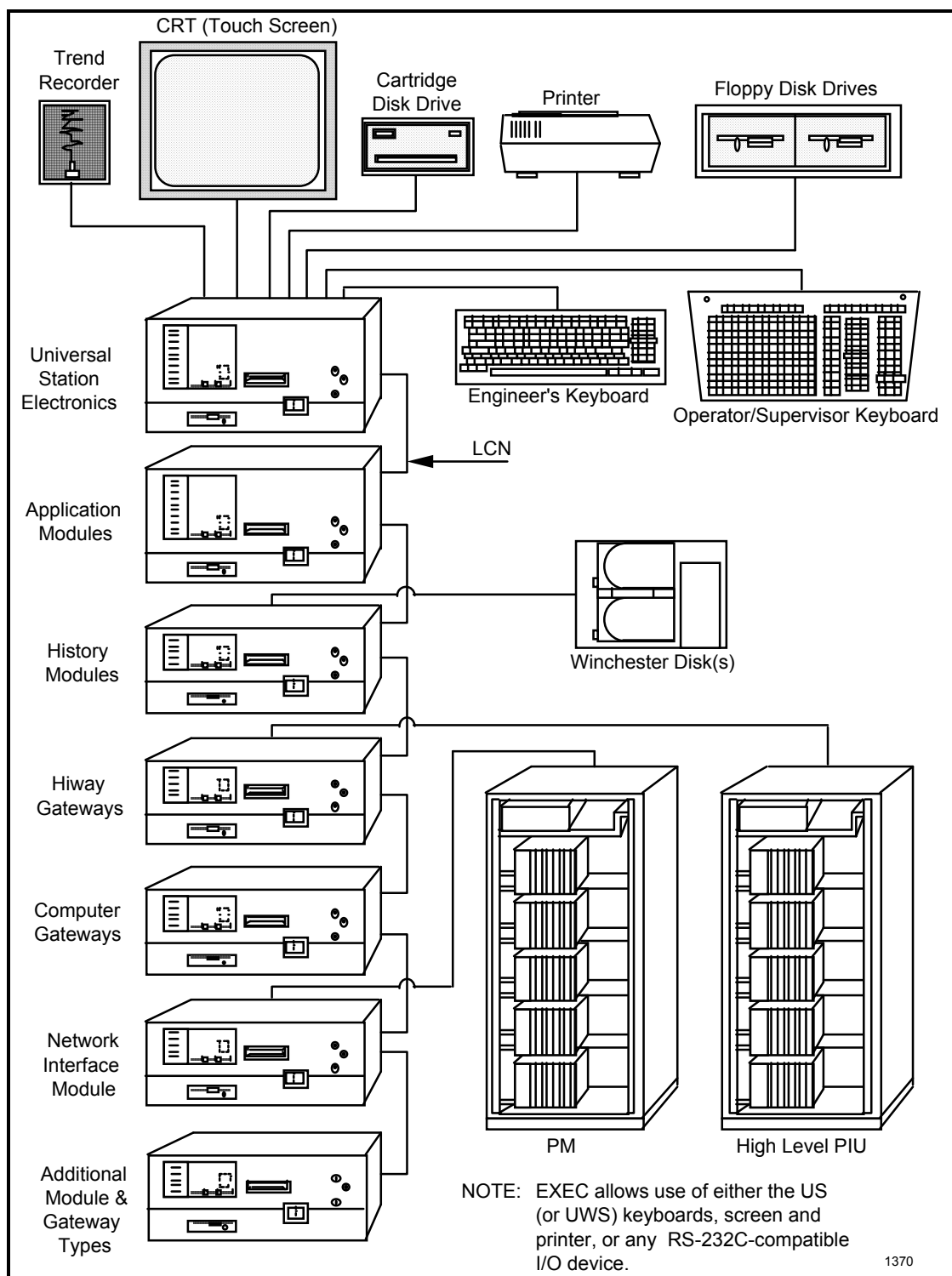


Figure 1-1 Test System Supported Hardware

Table 1-1 Test Program to Hardware Module Relationship

LCN Module Type											
Program Name	US	UXS	UWS	AM/AXM	HM	HG	CG	PIM/NIM	PLCG	PLNM	NG
AMRT				X							
CLIF						X					
CLKS	X	X	X	X	X	X	X	X	X	X	X
CNIF										X	
DHEX						X					
DHIF						X					
FLPY	X	X									
KEYB	X	X	X								
LCNS	X	X	X	X	X	X	X	X	X	X	X
LCNX	X	X	X	X	X	X	X	X	X	X	X
LSOE								X			
MEMX	X	X	X	X	X	X	X	X	X	X	X
NGIF											X
OPIF	X										
PDGI	X		X								
PLCT									X		
PNIF								X			
PNIE								X			
SCMD	X	X	X		X						
SIOS	X	X									
WINC					X						
WSIF	X	X		X							
XAMF				X							
XUSF		X									

1.4 References

The following are other TPS publications that will be of use during installation and maintenance of TPS systems.

Publication Title	Publication Number	Binder Title	Binder Number
<i>Maintenance Test Operations</i>	SW11-502	LCN Service - 1	3060-1
<i>Test System Executive</i>	SW13-510	LCN Service - 3	3060-3
<i>Core Module Test System</i>	SW13-512	LCN Service - 3	3060-3
<i>LCNI Network Communications Test</i>	SW13-513	LCN Service - 3	3060-3
<i>Universal Station Service</i>	US13-500	LCN Service - 1	3060-1
<i>Universal Station^X Service</i>	UX13-510	Universal Station ^X	3095
<i>History Module Service</i>	HM13-501	LCN Service - 2	3060-2
<i>Five/Ten-Slot Module Service</i>	LC13-500	LCN Service - 2	3060-2
<i>Dual Node Module Service</i>	LC13-510	LCN Service - 2	3060-2
<i>LCN System Checkout</i>	SW20-510	LCN Installation	3025
<i>Universal Workstation Installation, Operation, and Service</i>	UW02-500	LCN Service - 2	3060-2

2 Clock Subsystem Tests (CLKS)

2.1 Clock Subsystem Test Elements

The Clock Subsystem Test elements consist of one or more LCN modules. One of the following "normal" network topologies is expected:

- Both A and B clock sources and one or more 12.5 kHz listeners (a non-enhanced network).
- Both A and B clock sources, one or more 12.5 kHz listeners, a K2LCN/K4LCN node acting as a 12.5 kHz to LCN translator, and one or more K2LCN/K4LCN listeners (a hybrid network).
- A K2LCN/K4LCN LCN clock master and one or more K2LCN/K4LCN listeners (an enhanced network).
- An LCNP4-GPS or TPNB Time Link clock master and one or more LCNP4-GPS or TPNB listeners (an enhanced network). Note: The GPS provides the source of time and is incompatible with CLKS control of expected time values. The GPS antenna connection to the Time Link cable is not supported during CLKS testing. The GPS antenna must NOT be connected to the Time Link cable.

2.2 Use Information

CLKS uses stimulus/response test methodology wherever possible. Proper behavior of the clock subsystem is modeled by CLKS and compared to the actual clock subsystem response. CLKS operates in a master/slave relationship with copies of itself in the other LCN nodes. (All copies of CLKS in the network must be set up to reflect the capabilities of the clock subsystem in that individual LCN node.)

Whenever CLKS is started or restarted, the copy running in the TOCS is designated as master. Alarms can come from either the master or slave(s). If a breakdown in LCN communications is detected, the master copy CLKS prints an alarm. To restart a dropped node, all nodes must be stopped and restarted.

A check is made by each test for resources required. If the resources are not present, the test is removed from the test list and the operator is notified.

CLKS executes two types of test: Local and Network. Local testing checks clock operation within the individual nodes. Network testing checks clock subsystem communication through the LCN and is coordinated by the TOCS.

CAUTION

Do not run CLKS in Network operating scope on an LCN with operating RNOS nodes. CLKS in Network operating scope interferes with RNOS time keeping. This causes the time displayed at Universal Stations to change randomly and will upset journal event time tags. Operations performed in Local operating scope do not involve network clock activity and can be safely performed in the presence of operating RNOS nodes.

For proper operation, do not add nodes, delete nodes, or change the TOCS node while CLKS is running. Additionally, CLKS should be started in all nodes simultaneously by using the “LF” key to initiate a Network-directed command. Stopping the master CLKS node also stops all slave node(s).

CLKS in the TOCS uses the EXEC's network communication facilities to coordinate the operation of CLKS in the other LCN nodes. This causes one major complication: a task that is halted on error cannot receive network messages. This cuts off communications to the CLKS in a remote node, causing the CLKS in the TOCS to print operator information messages. If this occurs during an overnight run, it wastes a lot of paper and, if the halted CLKS was the A or B cable clock source, causes the loss of the run. To prevent this, CLKS simulates halt on error by inhibiting error messages while otherwise continuing to operate as usual. This allows the A or B cable source to reach the “Halt on Error Limit” without invalidating the testing performed by the rest of the nodes.

If the system does not contain 12.5 kHz clock transmitters (Clock Source Repeaters—CSR boards), crossed cable connection testing must be performed using the Network Communications Test (LCNI). Please refer to the crossed cable testing procedure documented in *LCNI Network Communications Test* in the *LCN Service* binder.

2.3 Setup Parameters

2.3.1 CLKS-Specific Parameters

OPERATING_SCOPE (Can be changed only with CLKS off) Allowed values are

LOCAL = No actions are taken by CLKS that could disturb the proper operation of any RNOS-based nodes that may co-exist on the network.

NETWORK = CLKS can perform actions that will upset proper operation of RNOS-based nodes co-existing on the network. As a safeguard against accidental disturbance of RNOS nodes, the operator is required to set the PERMISSION parameter to "Y" before NETWORK scope is allowed.

LISTENER (Can be changed only with CLKS off) Allowed values are

ENABLE = Indicates that this node has the ability to receive 12.5 kHz clock data.

DISABLE = Indicates that this node cannot receive 12.5 kHz clock data.

TRANSLATE (Can be changed only with CLKS off) Allowed values are

ENABLE = Indicates that this is a K2LCN/K4LCN node and can perform 12.5 kHz to 5 MHz translation if required by the test (the LISTENER value also must be ENABLE).

DISABLE = Indicates that this node should not be used as the 12.5 kHz to 5 MHz translator.

This parameter defaults to ENABLE in nodes that contain a K2LCN/K4LCN node and to DISABLE in nodes that do not.

MASTER (Can be changed only with CLKS off) Allowed values are

ENABLE = Indicates that this is a K2LCN/K4LCN node and is available to be used as the 5 MHz master clock source.

DISABLE = Indicates that this node will not be used as the 5 MHz master clock source.

This parameter defaults to ENABLE in nodes that contain a K2LCN/K4LCN node and to DISABLE in nodes that do not.

AC_LINE_SYNCH (Can be changed only with CLKS off) This parameter controls the use of the AC line reference signal. It affects all nodes in local mode, K2LCN/K4LCN nodes with MASTER enabled, and the A and B cable clock sources. Note that Test 3 ignores this parameter value and attempts to synchronize all non-K2LCN/K4LCN nodes to the power line. Allowed values are

ENABLE = The ac reference signal will be used if it is an option in the current test.

DISABLE = The ac reference signal will not be used.

PROCESSOR_TYPE (Display Only) Identifies the processor type present in this node.

A_SOURCE (Must be changed with CLKS off) Allowed values are

- = No cable A 12.5 kHz clock source is present

0-127 = LCN address (decimal) of the cable A 12.5 kHz clock source

B_SOURCE (Must be changed with CLKS off) Allowed values are

- = No cable B 12.5 kHz clock source is present

0-127 = LCN address (decimal) of the cable B 12.5 kHz clock source

IDLE_TIME (Can be changed at any time.) Allowed values are

1-30 = Specifies the length (in minutes) of the exercise period in Test 8.

PERMISSION (Can be changed at any time) Allowed values are

Y = Allow Network operating scope

N = Do not allow Network operating scope

Set by the program to N each time it starts with OPERATING_SCOPE set at NETWORK. The operator is then prompted to authenticate the network operating scope by changing this value to Y. This parameter will now accept any valid command processor abbreviation for YES or NO.

CLOCK_STATUS Display only, contains the current clock status-word value (in hex).

DEBUG_COMMUNICATIONS This parameter is used only for developer testing and must be left DISABLED.

2.3.2 General Parameters

TESTS	Allowed values are 1 through 9
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE	not used
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

2.4 Preset Parameter Values for Clks

OPERATING SCOPE = LOCAL

LISTENER = ENABLE

TRANSLATE = (hardware-dependent—see parameter explanation)

MASTER = (hardware-dependent—see parameter explanation)

AC_LINE_SYNCH = ENABLE

A_SOURCE = -

B_SOURCE = -

IDLE_TIME = 5

PERMISSION = N

TESTS = 1-9

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER, TESTNUMBER

SCALE = 0

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

2.5 Mode Characteristics for Clks

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

2.6 Individual Clks Tests

(System Modes)

Test 01	NOP and Local Timesets	SUB, MOD
	Verifies the Clock Subsystem's ability to perform basic clock operations such as NOP commands and local time sets. All enabled nodes are asked to print the current test number, send an NOP command to the Clock Subsystem and set up the expected clock status and settling delay. For each of the 32 bits in the seconds register, all nodes are asked to set the current time to the test value and verify that the time set was properly performed.	
Test 02	Programmable Timer	SUB, MOD
	Verifies proper operation of the programmable timer present on HMPU, HPK2, and K2LCN/K4LCN processor boards. If sufficient resources are present, all enabled HMPU/HPK2/K2LCN/K4LCN nodes are asked to print the test number, and a timer check is performed for each of the four programmable intervals. An alarm is issued whenever the difference between actual elapsed time and expected elapsed time varies beyond a predetermined margin.	
Test 03	AC Power Line Synchronization	SUB, MOD
	Verifies the Clock Subsystem's ability to lock to the ac power line in local mode. (This test is performed only in nodes that do not contain a K2LCN/K4LCN.) If the required resources are available, all enabled non-K2LCN/K4LCN nodes are asked to print the test number, send an enable AC line synch command to the clock subsystem, set the final clock status, and perform a delayed clock status check. All non-K2LCN/K4LCN nodes are allowed to remain in synch for 10 seconds, then all that have AC_LINE_SYNCH = DISABLE are asked to unlock from the ac line and perform a delayed clock status check.	
Test 04	12.5 kHz Network Operations	SUB, MOD
	Verifies the Clock Subsystem's ability to transmit and receive 12.5 kHz data and its ability to detect and process network time sets. If AC_LINE_SYNCH is enabled at the A and B clock source nodes, it will be used. This test is performed only if OPERATING_SCOPE = NETWORK and both A and B clock sources are present. Nodes with LISTENER = ENABLE are instructed to listen to the A and B clock sources. On even-numbered passes, the A cable clock source is used as the master and the B cable clock source functions as the slave. On odd-numbered passes, the roles are reversed.	
Test 05	12.5 kHz Clock Source Failovers	SUB, MOD
	Verifies the Clock Subsystem's ability to perform 12.5 kHz clock source failovers (the slave clock source takes over clock master operations). If AC_LINE_SYNCH is enabled at the A and B clock source nodes, it will be used. This test is performed only if OPERATING_SCOPE = NETWORK and both A and B clock sources are present. Nodes with LISTENER = ENABLE are instructed to listen to the A and B clock sources. On even-numbered passes, the A cable	

clock source is used as the master and the B cable clock source functions as the slave. On odd-numbered passes, the roles are reversed.

Test 06 LCN-Based Network Operations SUB, MOD

Verifies the Clock Subsystem's ability to transmit and receive 5 MHz data and its ability to detect and process network time sets. Also verifies the LCN ASIC's ability to detect and indicate the source of the clock data. If AC_LINE_SYNC is enabled at the master clock source node, it will be used. This test is performed only if OPERATING_SCOPE = NETWORK, at least one node is present with MASTER enabled, and at least one additional K2LCN/K4LCN node is present.

Test 07 12.5 kHz to LCN Based Translation SUB, MOD

Verifies the Clock Subsystem's ability to perform 12.5 kHz to 5 MHz translation and the detection/translation of time set events. Also verifies the LCN ASIC's ability to detect and indicate the source of the clock data. If AC_LINE_SYNC is enabled at the A and B clock source nodes, it will be used. This test is performed only if OPERATING_SCOPE = NETWORK, both A and B clock sources are present, at least one node is present with both LISTENER and TRANSLATE enabled, and at least one additional K2LCN/K4LCN node is present. On even-numbered passes, the A cable clock source is used as the master and the B cable clock source functions as the slave. On odd-numbered passes, the roles are reversed.

Test 08 Exerciser Test SUB, MOD, EXER

Operations performed in this test have already been performed in previous tests and are considered reliable at this point. The resources available are examined and a test setup is selected to use each node in the most useful manner. If OPERATING_SCOPE = LOCAL (local network) or a node does not fit into one of the normal topologies, it is placed in local operating mode and time sets will be performed and verified locally. If AC_LINE_SYNC is enabled at the A clock source, the B clock source, the K2LCN/K4LCN clock source or non-K2LCN/K4LCN nodes operating in local mode (as applicable), it will be used. If the A and B cable clock sources are used on even-numbered passes, the A cable clock source is used as the master and the B cable clock source functions as the slave. On odd-numbered passes, the roles are reversed.

Test 09 Time Link Based Network Operations SUB, MOD

Verifies the Clock Subsystem's ability to transmit and receive 5 MHz data and its ability to detect and process network time sets sent over the Time Link cable. Also verifies the LCN ASIC's ability to detect and indicate the source of the clock data. This test is performed only if OPERATING_SCOPE = NETWORK, at least one node is present with MASTER enabled, and at least two Time Link nodes (LCNP4-GPS or TPNB) are present. Note: The GPS provides the source of time and is incompatible with CLKS control of expected time values. The GPS antenna connection to the Time Link cable is not supported during CLKS testing. The GPS antenna must NOT be connected to the Time Link cable.

2.7 Clks Alarm Categories

CLKS prints alarms in either of two formats:

- As relatively simple English language descriptions of the expected vs actual values.
- The standard HVTs alarm header line followed by additional information including:
 - Description of the operation in progress
 - Starting random number (if a random number was in use)
 - Description of data area in error that includes the was and should be values, the mask used, and the differences (Exclusive OR of was and should be values ANDed with the mask).

3 Local Control Network Exerciser (LCNX)

3.1 Local Control Network Elements

The Local Control Network Exerciser tests correct communication between LCNI boards in three or more individual nodes on the same network.

3.2 Use Information

LCNX consists of one test that instructs each listed node in the Test System network to talk to every other listed node in the network during each pass.

The TOCS node is the master; all other nodes in the network are slaves and are started and controlled by the TOCS. All error detection occurs at the TOCS node. It administers all transmissions and prints errors if the expected results do not occur.

The TOCS randomly picks two slave nodes and transmits messages between the three, noting results. This process continues until all nodes in the network are used two at a time. This procedure is random but orderly, so the LCNX for an n-node system requires the same execution time each time.

3.3 Setup Parameters

3.3.1 LCNX-Specific Parameters

NODE_LIST_MODE Allowed values and their meanings are

AUTO_MODE—LCNX uses the network node list

USER_MODE—LCNX uses a node list specified through use of the command "USER_NODE_LIST".

NOTE

USER_NODE_LIST is an EXEC command that controls a node list used by only LCN test and exerciser programs. Whenever USER_NODE_LIST is used by a test or exerciser program, only the nodes that appear in **both** the list of LOADED nodes (as modified by use of the DEL and ADD commands), and the list controlled by the USER_NODE_LIST command are affected.

The initial value for USER_NODE_LIST is "-" (therefore, all loaded nodes). Changes to the list are invoked by the command—or command abbreviation—followed by a list of node numbers separated by commas or "-" (to signify a group of consecutive node numbers).

ERROR_TABLE_PRINT Allowed values and their meanings are

END_OF_TEST—Print the error table at test completion

NO_PRINT—Do not print the error table at completion

NOW—Print the error table at the end of the current pass (entered while LCNX is running). After the error table is printed, the value reverts to END_OF_TEST or NO_PRINT.

3.3.2 General Parameters

TESTS Allowed value is 1

ERROR_LIMIT Allowed values are 1 through 32767 or "-"

PASS_LIMIT Allowed values are 1 through 32767 or "-"

REPORT Allowed values are PASSNUMBER, TESTNUMBER, LOG

INHIBIT Allowed values are PASSNUMBER, TESTNUMBER, LOG

SCALE Allowed values are 0 through 999

MINUTES_LIMIT Allowed values are 0 through 32766 or "-"

ABBREVIATION Allowed values are 0 through 4

AFTER_ERROR_LIMIT Allowed values are ABORT or HALT

3.4 Preset Parameter Values for LCNX

NODE_LIST_MODE = AUTO MODE

ERROR_TABLE_PRINT = END OF TEST

TESTS = 1

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER

SCALE = 100

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

3.5 Mode Characteristics for LCNX

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One	Zero
Module	One or more	One	Random
Exerciser	One or more	One	Random

3.6 Individual LCNX Tests

(System Modes)

Test 01SUB, MOD, EXER

3.7 LCNX Alarm Messages

LCNX alarm numbers do not have meaning to the user. The alarm messages describe specific problems encountered in English text. Examples are

"*Startup Error* TOCS cannot communicate with Node 5"

"Node 5 has been dropped from the node list"

4 Keyboard Subsystem Tests (KEYB)

4.1 Keyboard Subsystem Elements

Keyboard subsystem elements include keyboard electronics, operator keyboard, engineer keyboard, Y.H. parallel keyboard, touchscreen, PDG, EPDG, EPDG2, or TPDG (or PIC and VDG), and monitor.

4.2 Use Information

The KEYB tests assume that the appropriate Operator Interface test has successfully run (that is, OPIF, PDGI, or XUSF). KEYB tests can be run on any US on the network except the TOCS itself.

Restriction—When testing a dual function board (for example, EPDG, EPDG2, TPDG, etc.), only one of the following test systems can run on that dual function board at a time without errors occurring in the systems. The test systems are KEYB, PDGI, SCMD, and XUSF.

Tests 1 through 3 properly execute without any changes to the initialized PBlock parameters.

Tests 4 and 5 are utility tests. Once started, only an Abort or Stop command halts the program.

Test 4 is a keyboard interrupt logger.

Test 5 allows keyboard tasks (jobs) to be sent to the keyboard electronics. Typical jobs are LED control, relay control, and status requests. PBlock parameters USER PORT IS and JOB are used in this utility. The JOB is sent to the keyboard connected to the Peripheral Interface Controller port USER PORT IS. Keyboard interrupts resulting from the job request are logged as in test 4 above.

NOTE

UNIX must be manually shut down before the test system KEYB can run on a U^XS.

4.3 Setup Parameters

4.3.1 KEYB-Specific Parameters

ENABLE Allowed values and their meanings are

INTERRUPTS—Allow board interrupts to be asserted

INITIALIZE_BOARD—Issue a board reset when KEYB is RUN

TRACE—Turn on special software-debugging mode

SPARE—Not used

DISABLE Allowed values and their meanings are

INTERRUPTS—Inhibit board-interrupt assertion

INITIALIZE_BOARD—Do not issue a board reset when KEYB is RUN

TRACE—Turn off special software-debugging mode

SPARE—Not used

TURN_ON Allowed values and their meanings are

DVTS_OUTPUT — Send all logs and alarms to the printer

VIDEO_OUTPUT — Send all logs and alarms to the Universal
Station video screen

TURN_OFF Allowed values and their meanings are

DVTS_OUTPUT — Do not send logs and alarms to the printer

VIDEO_OUTPUT — Do not send logs and alarms to the video screen

JOB Allowed values are \$0001 through \$FFFF

This is the hexadecimal code for a keyboard-task request, see

Use Information for Test 5 above

USER_PORT_IS Allowed values are 0 and 1

Select which of two ports on a Peripheral Interface Controller the user wishes to send a "Job" to. See test 5

0 = Main port (PIC channel 3)

1 = Expansion port (PIC channel 1)

C_n_HOOK_UP_TO Allowed values when n=0 are OPEN, SINGLE, DUAL_VDG
Allowed when n=1 or n=3 are OPEN, KEYBOARD, DEVICE
Meanings are
OPEN—No connection is made to this channel
SINGLE—There is one Video Display Generator
DUAL_VDG—There are two Video Display Generators
KEYBOARD—A keyboard is connected to this channel
DEVICE—A serial printer is connected to this channel

C_n_DATA_SELECTION Not used

4.3.2 General Parameters

TESTS Allowed values are 1 through 6
ERROR_LIMIT Allowed values are 1 through 32766 or "-"
PASS_LIMIT Allowed values are 1 through 32766 or "-"
REPORT Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE Allowed values are 0 through 999
MINUTES_LIMIT Allowed values are 0 through 32766 or "-"
ABBREVIATION Allowed values are 0 through 4
AFTER_ERROR_LIMIT Allowed values are ABORT or HALT

4.4 Preset Parameter Values for KEYB

ENABLE = INTERRUPTS, INITIALIZE_BOARD

DISABLE = TRACE, SPARE

TURN_ON = VIDEO_OUTPUT

TURN_OFF = DVTS_OUTPUT

JOB = \$0

USER_PORT_IS = \$0

C0_HOOK_UP_TO = SINGLE

C0_DATA_SELECTION = AUTO

C1_HOOK_UP_TO = OPEN

C1_DATA_SELECTION = AUTO
C3_HOOK_UP_TO = KEYBOARD
C3_DATA_SELECTION = AUTO
TESTS = 1-6
ERROR_LIMIT = 5
PASS_LIMIT = -
REPORT = LOG
INHIBIT = PASSNUMBER,TESTNUMBER
SCALE = 100
MINUTES LIMIT = -
ABBREVIATION = 4
AFTER_ERROR_LIMIT = ABORT

4.5 Mode Characteristics for KEYB

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

4.6 Individual Keyboard Tests

(System Modes)

Test 01	Keyboard Channel and Status Checks	SUB, MOD
	Determines operability of the PIC interface with respect to the keyboard. Self-test indications and device presences are read and checked.	
Test 02	Keyboard Kernel Electronics Check	SUB, MOD
	Keyboard-kernel electronics-status indications (Firmware-Revision response, Hardware-Revision response, Subsystem-Status response, and Request Engineering-Attributes response) are requested and checked for validity.	
Test 03	Keyboard Task Request Checks	SUB, MOD
	LED outputs and annunciator checks are made.	
Test 04	Keyboard Interrupt Interpreter	SUB, MOD
	Waits for keyboard interrupts, acknowledges them, reads input data, and processes a log of the interrupts.	
Test 05	Keyboard Task Utility	SUB, MOD
	Provides for issuing a user-defined Task Request to the keyboard kernel electronics.	
Test 06	Lamp Exerciser	SUB, MOD, EXER
	Randomly lights and checks status of keyboard LEDs.	

4.7 KEYB Alarm Categories

<u>Alarm Range</u>	<u>Most likely causes</u>
0100 – 0299	Keyboard controller
0300 – 0399	May be Keyboard controller—The IOCB-termination status and Operational-status registers must be analyzed to determine the error
0400 – 0499	Keyboard controller.
0600 – 0699	May be Keyboard controller—The IOCB-termination status and Operational-status registers must be analyzed to determine the error.

NOTE

Any alarms less than 100 or greater than 900 are PIC-board failures. These tests assume that the OPIF/PDGI/XUSF tests have been passed.

5 Operator Interface Subsystem Tests (OPIF)

NOTE

The Operator Interface (OPIF) subsystem test will not run in X-Layer node. Use the Universal Station^X Interface (XUSF) subsystem test instead.

5.1 Operator Interface Subsystem Elements

Operator Interface Subsystem elements include Peripheral Interface Controller board, Video Display Generator board, Operator and Engineer Keyboards, Video Monitor, Matrix Printer, and cables.

5.2 USE INFORMATION

OPIF tests can be run on any US on the network except the TOCS itself.

Special Requirements—The nature of the device connected to a particular port must be defined to this program. Where the program requires actual devices, it bypasses those channels that are configured "HOOK_UP_TO = OPEN." Devices on each channel are required to fully test all defined functions.

Special Feature—Test 10 of OPIF can be used to verify the peripheral cabling and setup of the Operator Interface subsystem. Upon starting test 10, a peripheral-status log is printed. If any status change occurs (i.e., cables from the peripherals get disconnected and subsequently reconnected) the status is redisplayed for each state change. The status log indicates the state of all channels and the PBlock setup for each channel. To invoke test 10 the following is required:

```
00:01 OPIF?  TEST 10
00:01 OPIF?  MODE S
00:01 OPIF?  RUN
```

To stop test 10 the following is required:

```
00:01 OPIF?  ABORT
```

5.3 Setup Parameters

5.3.1 OPIF-Specific Parameters

Note that in some of the information that follows, reference is made to "C_n". This convention is used to indicate the assignment of parameter values to each of the five I/O channels. The device assignment for each channel is:

C0—Video

C1—Spare Printer

C2—Printer

C3—Keyboards

C4—Spare

C_n_HOOK_UP_TO Allowed values for Printer and Keyboard Channels are OPEN and DEVICE; allowed values for Video Channels are OPEN, SINGLE, DUAL

Meanings are

OPEN—No device is connected

DEVICE—A physical device (Printer, Keyboard, etc.) is connected and ready

SINGLE—One video-display generator is available

DUAL_VDG—Two video-display generators are available.

INITIALIZE_BOARD Allowed values are YES, NO

This parameter controls whether or not the board under test is initialized following the RUN command. Normally set to YES.

C_n_SCALE Allowed values are 0 - 100

Establish a ratio of test executions between the channels (used by test 9)

COLOR_TO_VDG Allowed values are DARK, RED, GREEN, YELLOW, BLUE, MAGENTA, LT BLUE, WHITE

Selects which color is used by the Video Flood test

DATA_SELECTION Allowed values and their meanings are
 AUTO—Incremental change of a set data pattern
 RANDOM_STRING—Generate random data
 USER_STRING—Use the data supplied in "USER_STRING"
 Ij_STRING—Alternates I's and j's for printer (only) testing

USER_STRING Allowed value is any string of up to 40 ASCII characters

5.3.2 General Parameters

TESTS Allowed values are 1 through 18

ERROR_LIMIT Allowed values are 1 through 32767 or "-"

PASS_LIMIT Allowed values are 1 through 32767 or "-"

REPORT Allowed values are PASSNUMBER, TESTNUMBER, LOG

INHIBIT Allowed values are PASSNUMBER, TESTNUMBER, LOG

SCALE Allowed values are 0 through 999

MINUTES_LIMIT Allowed values are 0 through 32767 or "-"

ABBREVIATION Allowed values are 0 through 4

AFTER_ERROR_LIMIT Allowed values are ABORT or HALT

5.4 Preset Parameter Values for OPIF

C0_HOOK_UP_TO = SINGLE

C1_HOOK_UP_TO = OPEN

C2_HOOK_UP_TO = OPEN

C3_HOOK_UP_TO = DEVICE

C4_HOOK_UP_TO = OPEN

INITIALIZE_BOARD YES

C0_SCALE = 0

C1_SCALE = 0

C2_SCALE = 0

C3_SCALE = 0

C4_SCALE = 25

COLOR_TO_VDG = DARK

DATA_SELECTION = AUTO

USER_STRING =

TESTS = 1-18

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER

SCALE = 0

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

5.5 Mode Characteristics for OPIF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

5.6 INDIVIDUAL OPERATOR INTERFACE TESTS

		(System Modes)
Test 01	Basic Operator Interface Subsystem Operations Tests basic functions of the PIC.	SUB, MOD
Test 02	Execute OPIF Valid Commands Executes and checks valid commands for the subsystem.	SUB
Test 03	Generate IOCB Interrupts Exercises the seven implemented interrupt vectors.	SUB, MOD
Test 04	Verify Invalid CCB Detection Tests the Channel Control Block error-detection mechanisms for each channel.	SUB, MOD
Test 05	Illegal Command/Parameter Detection Checks error conditions (interrupts and status bits) for proper trapping of invalid commands.	SUB, MOD
Test 06	IOCB Chaining and Pause Verifies transfer pause and restart execution and the trapping of invalid commands.	SUB
Test 07	Random Transfers Exercises all available channels on a random-selection basis (one channel for each pass).	SUB, MOD
Test 08	Verify IOCB Error Detection Tests the Input Output Control Block error-detection functions for each channel.	SUB, MOD
Test 09	Channel Transfers Exercises all available channels according to the channel-parameter settings.	SUB, MOD
Test 10	Monitor Subsystem Status Continuously monitors and logs the device-status bits until the program is ABORTed. This test requires manual intervention, and must be the only test selected.	SUB, MOD
Test 11	Module Bus Error Detection Verifies the ability of the OPIF to correctly detect and report bus errors asserted while it is the temporary bus master.	SUB
Test 12	System Exerciser Verifies transmission of data on channels 0 through 4.	SUB, MOD, EXER
Test 13	Video Flood Fills (floods) the CRT with the color specified by COLOR_TO_VDG. Repeats with changes when this parameter is changed until the test is aborted. This test requires manual intervention and must be the only test selected.	SUB, MOD
Test 14	Diagnostic Commands	SUB

	Tests the PIC diagnostic commands for board-crash handling.	
Test 15	Touchscreen Video	SUB, MOD
	Writes an image on screen that allows testing of the touchscreen. Moves cursor to current hit point. Beeps and flags error if hit is not in white square. This test requires manual intervention and must be the only test selected.	
Test 16	Loopback and Configuration	SUB, MOD
	Selects a random valid configuration and performs a series of loopback commands on each port at that configuration.	
Test 17	Incoming CRT Test Pattern	SUB, MOD
	Writes an image on the screen used for inspection of the CRT. This test disables the cursor; therefore, after running this test you must set INITIALIZE_BOARD to YES before Test 15 can run correctly. This test must be the only test selected.	
Test 18	Crosshatch Test Pattern	SUB, MOD
	Writes a crosshatch test pattern on the screen that is useful in verifying operation of the printer. This test disables the cursor; therefore, after running this test you must set INITIALIZE_BOARD to YES before Test 15 can run correctly. This test must be the only test selected.	

5.7 OPIF Alarm Categories

<u>Alarm Range</u>	<u>Most likely causes</u>
0100 – 0699	PIC-controller failure
0700 – 0999	Peripheral failure—Use the IOCB status and Operational status to isolate the failed peripheral and the error (paper out, off-line, etc.)
1000 – 1099	PIC-controller failure—The change-detect log can reveal intermittent peripheral connections
1100 – 1199	PIC-controller failure
1200 – 1399	Peripheral failure
1400 – 1499	PIC-controller failure
1500 – 1599	Touchscreen/Video failure
1600 – 1699	PIC controller failure

NOTE

Any alarms smaller than 100 or greater than 9000 are PIC-board failures.
Some peripheral alarms can be caused by improper channel setup.
Any Channels with keyboards must be configured as OPEN.
The VDG board is considered here to be a "peripheral."

6 Winchester Disk Subsystem Tests (WINC)

6.1 Winchester Disk Subsystem Elements

Winchester Disk Subsystem elements include Winchester Disk Controller, Winchester Disk Interface, Data Cable(s), and one or two Winchester Disk Drives.

6.2 Use Information

CAUTION

Please be aware that running any WINC tests other than tests 24 and 27 destroys whatever data is stored on the disk(s) being tested.

On a new, non-initialized drive, test 25 needs to be run first to format the disk and initialize (write) the Bad Sector Table for the disk.

Before RUNning test 25, FORMAT must be enabled and MI_TESTS =1 (enabled). If CLR_BST is disabled (to allow operator entry of known bad sectors), test 25 issues the following message before it executes:

```
>>>> BUILD BAD SECTOR TABLE (BST) FOR DRIVE 0
```

Enter

```
BUI n,n,n;n,n,n;n,n,n;
```

n,n,n = Cylinder, Head, and Byte of a bad sector. This information is listed on top of the disk for each bad sector. Additional bad-sector sets included on the same entry line are separated by semicolons (to a maximum of 40 characters for each line), or can be entered by repeating the BUI command. Note that a semicolon separates each sector set and ends the data portion of the entry.

To print contents of the Bad Sector Table, enter

```
BUI P; (or: BUI PRINT;)
```

To edit contents of the Bad Sector Table if an error is made, enter

```
BUI E; (or: BUI EDIT;)
```

No deletion function is provided; however, a bad-sector set can be deleted by changing its contents to be identical with that of another set.

To signal completion of a satisfactory BST and to begin the disk formatting (and recording of the BST on the disk), enter

```
BUI D; (or: BUI DONE;)
```

To delete all information entered in the preliminary BST, you must ABOrt the test before executing the DONE command.

If there are two drives, the test program repeats the Build Bad Sector Table message for drive 1. Additional guidance on the building and editing of a BST is provided by explanatory messages displayed in response to each command. The content of these messages is controlled by the selected ABBreviation level.

Once Test 25 has completed a BST (following your entry of DONE command), additions to the BST require the running of Test 24.

6.3 Setup Parameters

WINC-Specific Parameters

DRIVE_n (0,1) 0 = No Drive or Drive Not Under Test
 3 = Hitachi Drive (85 Mwd)
 5 = Wren Drive (I) (35 Mwd)
 6 = Wren Drive (II) (86 Mwd)

MI_TESTS 0 = Manual Intervention Testing is Disabled
 1 = Manual Intervention Testing is Enabled

NOTE

The parameter `RETRY_LIMIT` has been deleted from WINC starting with HVTs Rel 6.1. If you have an earlier version of HVTs, use the default value of 0 (zero) for this parameter.

ENABLE Allowed values and their meanings are
 FORMAT—Allow test 25 (format the disk)
FAST—Skip two of the four data patterns used in test 23
 BST—Allow read/write tests to use the Bad Sector Table
 CLR_BST—Clear and Initialize the Bad Sector Table during test 25.
 EXERCISER_READ_ONLY—Test 21 can only read from disk.
 SYSTEM_SOFTWARE_CHECK—Abort test if there is a valid system boot on Drive 0.

DISABLE Allowed values and their meanings are
 FORMAT—Prohibit test 25 from formatting the disk(s)
 FAST—Do all data patterns during the test 23 surface check
 BST—Prevent write/read tests from using the BST
 CLR_BST—Allow entry of user-defined bad-sector data during test 25
 EXERCISER_READ_ONLY—Test 21 both reads and writes to disk.
 SYSTEM_SOFTWARE_CHECK—Do not check for system boot.

BUILD BST See paragraph 6.2 for use of the BUI command.

6.3.1 General Parameters

TESTS	Allowed values are 1 through 27
ERROR_LIMIT	Allowed values are 1 through 32767 or "-"
PASS_LIMIT	Allowed values are 1 through 32767 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
SCALE	Allowed values are 0 through 999
MINUTES	Allowed values are 0 through 32767 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

6.4 Preset Parameter Values for WINC

DRIVE_0 = 5

DRIVE_1 = 0

MI_TESTS = 0

DISABLE = FORMAT,CLR_BST,EXERCISER_READ_ONLY

ENABLE = BST,FAST,SYSTEM_SOFTWARE_CHECK

BUILD_BST = READY

TESTS = 1-22,26

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER,SUSPENDED

SCALE = 100

MINUTES = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

6.5 Mode Characteristics for WINC

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

6.6 Individual Winchester Disk Tests

(System Modes)

Test 01	Basic Winchester Disk Controller Responses Ensures a basic ability to communicate with the Winchester Disk Controller and to generate and control the two program interrupts.	SUB
Test 02	Bus Error Detection Verifies that the Winchester Disk Controller can detect and report module bus-transaction errors.	SUB
Test 03	Test DMA Commands Verifies the Module Data Bus read/write capability and ensures the integrity of the data paths between the memory and the Winchester Disk Controller.	SUB
Test 04	Load and Verify Registers Ensures the integrity of the data paths between the memory and the Winchester Disk Controller, and checks the ability of the Winchester Disk Controller to store and retrieve data from its on-board RAM memory.	SUB, MOD
Test 05	Paged Status Reads and verifies (if possible) the contents of the Paged Status Register.	SUB, MOD
Test 06	CCB and IOCB Chaining Verifies that the Winchester Disk Controller accesses the Channel Control Block (CCB) and sequences through a series of Input/Output Control Blocks (IOCB). The "Start IOCB Chain" and "Stop IOCB Chain" commands are tested.	SUB
Test 07	Verify Illegal Command Detection Verifies that the Winchester Disk Controller detects illegal commands stored into the Command Word Register.	SUB, MOD
Test 08	Verify Invalid CCB and IOCB Detection Verifies that the Winchester Disk Controller detects invalid Channel Control Blocks (CCB) and invalid Input/Output Control Blocks (IOCB). These conditions are detected through the "CCB Format Error" bit and "IOCB Format Error" bit in the Operational Status Register, and the "Invalid Operation Code" termination status in the IOCB.	SUB, MOD

Test 09	Verify Retry Counter	SUB, MOD
	Checks that the WDC retries a transfer having known error(s) the specified number of times before returning a "Fatal Error" termination status. Also, that transfers without error are not retried.	
Test 10	Sector Addressing	SUB, MOD
	Ensures that the WDS can write and read to all sectors of a track, and that all sectors are uniquely addressable. Bad Sectors are not handled.	
Test 11	not used	
Test 12	Drive Uniqueness Check	SUB, MOD
	Verifies that each Winchester-disk drive is uniquely addressable. An unique data pattern is written to a test sector on each drive, followed by the read back from all drive(s) for verification.	
Test 13	Memory Addressing	SUB
	Checks that the WDC can address all of memory (DRAM) by writing or reading from representative memory areas. Every available 64 k bank of memory is tested. Each memory address line is tested in both "zero" and "one" states.	
Test 14	Write/Read Fixed Data Patterns	SUB, MOD
	Ensures that the WDS can write and read over the entire disk surface(s) available for testing.	
Test 15	Transfer Length Test	SUB
	Verifies that the WDS can transfer any number of sectors (128 or 256 bytes per sector), from 0 to 255 sectors. The transfer length is incremented by powers of 2 (i.e., 0,1,2,4,8 ... 128 and the maximum 255). The write operation is started on sector 25 of the test track.	
Test 16	Check Head Movement	SUB, MOD
	Ensures that the drive(s) under test can perform each increment of head movement in both forward and reverse directions. All increments of head movement from 0 through Max Cylinder are used.	
Test 17	Disk Drive Format Check	SUB, MOD
	Verifies that the Winchester Disk Subsystem can correctly indicate (through configuration-status and page-status registers) the type of drive that it is controlling.	
Test 18	Verify Track 00 Protection	SUB, MOD
	Verifies that the WDS allows data to be written on track 00 only if the "Write Normal Track 00" IOCB command is used. If the "Write Data—Normal" IOCB command is used, the transfer should not take place and an error-termination status should be indicated.	
Test 19	Error Detection and Correction (EDAC) Check	SUB, MOD
	Checks that the WDC returns a fatal error when the EDAC test sector is read, and that the data pattern returned by the controller is hex DB6D.	
Test 20	Verify Recovery From an Aborted Transfer	SUB, MOD
	Ensures that the "Conditional Abort" command correctly terminates a transfer (on a sector boundary if a transfer is in progress), and allows any subsequent transfers to take place without error.	

Test 21	Exerciser Test	SUB, MOD, EXER
	Uses random parameters to verify that the WDS can write and read.	
Test 22	Program Load	SUB
	Uses the "Boot Load" command to verify that the WDS program-load function operates correctly.	
Test 23	Check Surface & Append to BST	SUB
	Tests the surface for bad spots and appends a list of any bad sectors found to the Bad Sector Table on the disk(s). The first data pattern used is hex DB6D and the second is its complement, hex 2492. This test is classified as a utility and is forced to run alone.	
Test 24	Print/Append to Bad Sector Table	SUB
	Prints the Bad Sector List contained in BST for the disk(s) and allows the operator to add user-discovered bad sectors to that list. Prompts the operator during BST data entry (for cylinder, head, and sector). You must set MI_TESTS = 1 and DISABLE both FORMAT and CLR_BST before starting Test 24.	
Test 25	Format and Build Bad Sector Table	SUB
	Used to prepare a new, non-initialized drive before loading any data on it. Formats the disk(s), tests the surface for bad spots, and provides for entry of a vendor-supplied list of known bad sectors to the BST. The data pattern after formatting is hex DB6D. Prompts the operator for BST information (cylinder, head, and byte). You must set MI_TESTS = 1 and ENABLE FORMAT to run Test 25. DISABLE CLR_BST if you need to enter a vendor-supplied list of bad sectors.	
Test 26	Write/Read Using Random Parameters	SUB, MOD
	Uses random parameters to verify that the WDS can write and read.	
Test 27	Read-Only Test	SUB, MOD
	This is a read-only adaptation of Test 14. It reads the entire disk except head 0 cylinder 0 looking for IOCB errors.	

6.7 WINC Alarm Categories

<u>Alarm Range</u>	<u>Most likely causes</u>
0001 - 0099	System Software Check
0101 – 0599	Controller or Power Supply
0601 – 0699	Device or Media or Controller
0701 – 0899	Controller or Power Supply
0901 – 2799	Device or Media or Controller

Alarm code 2524 is the Bad Sector alarm.

7 Floppy Disk Subsystem Tests (FLPY)

7.1 Floppy Disk Subsystem Elements

Floppy Disk Subsystem elements include Floppy Disk Controller, Floppy Disk Interface, Data Cable(s), and 1 to 4 Floppy Disk Drives.

7.2 Use Information

Special requirements—Scratch diskettes for each drive to be tested. If unformatted diskettes are used, Test 11 should be run first.

Tests 19 and 23 require manual intervention during test execution (to open drive doors, insert special diskettes, etc.). Parameter MI TESTS controls skipping of these tests.

7.3 Setup Parameters

7.3.1 FLPY-Specific Parameters

DRIVE_n (0 to 3)	0 = No Drive or Drive Not Under Test 1 = Single Density, Single Sided 2 = Single Density, Double Sided 3 = High Density, Single Sided 4 = High Density, Double Sided (normal TPS 3000 format) (1.2 MB capacity at 96 TPI.)
MI_TESTS	0 = Manual Intervention Testing is Disabled 1 = Manual Intervention Testing is Enabled (allows tests 19 & 23 to run)
RETRY_LIMIT	Allowed values are 0 through 14

7.3.2 General Parameters

TESTS	Allowed values are 1 through 23
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

7.4 Preset Parameter Values for FLPY

DRIVE_0 = 4

DRIVE_1 = 0

DRIVE_2 = 0

DRIVE_3 = 0

MI_TESTS = 0

RETRY_LIMIT = 3

TESTS = 1-24

ERROR_LIMIT = 20

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER

SCALE = 0

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

7.5 Mode Characteristics for FLPY

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

7.6 Individual Floppy Disk Tests

(System Modes)

Test 01	Basic Floppy Disk Controller Responses	SUB
	Tests basic ability to communicate with the FDC and to generate and control the two program interrupts.	
Test 02	Bus Error Detection	SUB
	Verifies that the Floppy Disk Controller detects and reports module bus-transaction errors.	
Test 03	Test DMA Commands	SUB
	Verifies Module Data Bus read/write capability and ensures the integrity of the data paths between memory and the FDC.	
Test 04	Load and Verify Registers	SUB, MOD
	Ensures the integrity of the data paths between memory and the FDC, and checks the ability of the FDC to store and retrieve data from its on-board RAM memory.	
Test 05	Paged Status	SUB, MOD
	Reads and verifies contents of the Paged Status Register.	

Test 06	CCB and IOCB Chaining	SUB
	Verifies that the FDC accesses the Channel Control Block (CCB) and sequences through a series of Input/Output Control Blocks (IOCB). The "Start IOCB Chain" and "Stop IOCB Chain" commands are tested.	
Test 07	Verify Illegal Command Detection	SUB, MOD
	Verifies that the FDC detects illegal commands stored into the Command Register.	
Test 08	Verify Invalid CCB and IOCB Detection	SUB, MOD
	Verifies that the FDC detects invalid Channel Control Blocks (CCB) and invalid Input/Output Control Blocks (IOCB).	
Test 09	Verify Retry Counter	SUB, MOD
	Checks that the FDC retries a transfer having a known error(s) the specified number of times before returning a "Fatal Error" termination status. Also, that transfers without error are not retried.	
Test 10	Sector Addressing	SUB, MOD
	Ensures that the FDS can write and read to all sectors of a track, and that all sectors are uniquely addressable.	
Test 11	Diskette Formatting	SUB
	Formats the diskette in each of the drives as specified by the setup parameters (single/double density and single/double sided).	
Test 12	Drive Uniqueness Check	SUB, MOD
	Verifies that each floppy-disk drive is uniquely addressable.	
Test 13	Memory Addressing	SUB
	Checks that the FDC can address all of memory (DRAM) by writing and reading from representative memory areas.	
Test 14	Write/Read Fixed Data Patterns	SUB, MOD
	Ensures that the FDS can write and read over the entire diskette surface(s) available for testing.	
Test 15	Transfer Length Test	SUB
	Verifies that the FDS can transfer any number of sectors (128 or 256 bytes per sector) from 0 to 255 sectors.	
Test 16	Check Head Movement	SUB, MOD
	Ensures that the drive(s) under test can perform every increment of head movement in both forward and reverse directions.	

Test 17	Cyclic Redundancy Code (CRC) Checker	SUB, MOD
	Verifies that FDC can detect an invalid CRC code.	
Test 18	Verify Track 00 Protection	SUB, MOD
	Verifies that the FDC allows data to be written on track 00 only if the "Write Normal Track 00" IOCB command is used.	
Test 19	Drive Ready Check	SUB
	Verifies that the FDC can detect and indicate a "Drive Not Ready" status for the drive(s) under test.	
Test 20	Verify Recovery From an Aborted Transfer	SUB, MOD
	Ensures that a "Conditional Abort" command or multiple "Start" commands correctly terminate a transfer that is in progress.	
Test 21	Write/Read Using Random Parameters	SUB, MOD
	Uses random parameter to verify that the FDC can write and read.	
Test 22	Program Load	SUB
	Uses the "Boot Load" command to verify that the FDC program-load function operates correctly.	
Test 23	Whole Diskette Write Protection	SUB
	Ensures that the FDC does not allow data to be written on any track if diskette write-protect is enabled.	
Test 24	Exerciser Test	SUB, MOD, EXER
	This is essentially the same as Test 21 except that it writes/reads just one block for each pass.	

7.7 FLPY Alarm Categories

<u>Alarm Range</u>	<u>Most likely causes</u>
0101 – 0599	Controller or Power Supply
0601 – 0699	Device or Media or Controller
0701 – 0899	Controller or Power Supply
0901 – 2499	Device or Media or Controller

8 Data Hiway Interface Subsystem Tests (DHIF)

8.1 Data Hiway Interface Subsystem Elements

Data Hiway Interface Subsystem elements include Data Hiway interface board (in context with other Hiway Gateway elements), redundant Data Hiway, and a High Level or Low Level PIU.

8.2 Use Information

To provide a thorough test of the Data Hiway Interface Subsystem, up to 256 passes of some tests and at least one pass of all tests must be run in the Subsystem Mode.

Tests 8 through 23 require either an HLPIU or LLPIU on the Data Hiway with the Data Hiway Interface card “Grant” and “Request” lines tied together (jumper together terminals 7 to 9 and terminals 8 to 10 on the DHI paddle board) or with a Hiway Traffic Director to grant requests. For tests 18, 19, 22, and 23, it must be an R320B HLPIU with Rev B or later firmware.

If Tests 8 through 23 are to be run, make sure that the specified PIU does not have customer wiring connected to it.

If the PIU used is “small,” make sure that it has both “base” and “base + 32” addresses enabled and that the 'base + 32' address is not occupied by another box.

If the HG is connected to a "hot" Data Hiway, Test 14 should be run only in loopback mode (see FLAG parameter).

When no Hiway Traffic Director is present, Test 22 exercises both Hiway cables and the ability to swap in both directions; however, when a Hiway Traffic Director is present Test 23 has very limited ability to test Hiway swapping. If any of Tests 1, 2, 3, or 5 are in the test list, only Hiway A is used by Test 23. If none of these are in the test list, Test 23 swaps from Hiway A to Hiway B once, and stays on Hiway B until the Hiway Traffic Director is manually switched back to Hiway A.

8.3 Setup Parameters

8.3.1 DHIF-Specific Parameters

FLAG Allowed values are 0 through FFFF

Each bit represents a flag controlling exception paths, etc.

Bit 0 set: Bypass reading of page-status words after each operation.

Bit 1 set: Test 14 to use loopback commands, not reads.

Bit 3 set: Test 19 to bypass PIU-status verification after a General Report request.

Other bit positions are not defined.

Example of use: 0003 requests no page-status reads and use of loopback commands

PERSIST_INTERVAL	Allowed values are 0 through 200 alue representing a percentage of the half-second pause before steady-state recomparing 0 – Do not compare (should be used for looping) 10 – Recompare after 50 ms (minimum) 50 – Recompare after 1/4 sec. (preset value) 200 – Recompare after one sec. (maximum)
BOX_ADDRESS	Allowed values are 5 through 31 Hiway address of HLPIU
CB_ADDRESS	Allowed values are 0 (none) through 63 Hiway address of CB
HIWAY_ADDRESS	Allowed values are even numbers 2 through 62 Hiway address of the DHI
SUPPORTED_MODES	Internal switch settings for tests 1-24; always use preset values

8.3.2 General Parameters

TESTS	Allowed values are 1 through 24
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

8.4 Preset Parameter Values for DHIF

FLAG = 8
PERSIST_INTERVAL = 50
BOX_ADDRESS = 30
CB_ADDRESS = 0
HIWAY_ADDRESS = 2
SUPPORTED_MODES = 551155555555575555555550
TESTS = 1-23
ERROR_LIMIT = 5
PASS_LIMIT = -
REPORT = LOG
INHIBIT = PASSNUMBER,TESTNUMBER,SUSPENDED
SCALE = 100
MINUTES_LIMIT = -
ABBREVIATION = 4
AFTER_ERROR_LIMIT = ABORT

8.5 Mode Characteristics for DHIF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

8.6 Individual Hiway Interface Subsystem Tests

(System Modes)

Test 01A	Valid Configuration Status Register Commands	SUB, MOD
	Assures that the DHI hardware correctly responds to valid commands to the Configuration Status Register.	
Test 01B	Valid Simple Commands, No Interrupts	SUB, MOD
	Assures that the DHI firmware correctly responds to selected Command-Register commands without the use of interrupts.	
Test 02	Valid Simple Commands Using Interrupts	SUB, MOD
	Assures that the DHI firmware correctly responds to selected Command-Register commands using vector 0 interrupts in two ways.	
Test 03	Proper Detection/Response to Module Bus Errors	SUB
	Verifies that module-bus data-parity errors during MCPU writes to the shared status bus are properly detected and reported.	
Test 04	Test DMA Read and Test DMA Write Commands	SUB
	Assures that the DHI firmware correctly responds to the two Test DMA commands.	
Test 05A	Invalid Commands to the Flag Control Register	SUB, MOD
	Assures that undefined commands to the Subsystem Flag Control Register are properly ignored.	
Test 05B	Invalid Commands and Parameters	SUB, MOD
	Assures that undefined Command-Register commands are properly rejected with the correct error indications.	
Test 06A	Simple NOP IOCB and Three Loopback IOCBs	SUB, MOD
	Assures that the simplest of the IOCB commands in their simplest form correctly function (without using a Data Hiway).	

Test 06B	Three Loopback IOCBs with Multiple Words	SUB, MOD
	Assures that multiple words can be looped back.	
Test 06C	Disabling Interrupts and Starting an IOCB	SUB, MOD
	Assures that the basic electronics and firmware correctly respond to interrupts being disabled when a valid Start IOCB chain 3 is commanded.	
Test 07	Poll Check and BCH Generation/Detection IOCBs	SUB, MOD
	Assures that the "Generate BCH" and "Check (Detect) BCH" operations properly perform.	
Test 08A	Corrupted CCB Address and Board Type ID	SUB, MOD
	Assures that corrupted fields in the CCBs are properly recognized and that the CCB Format Error Bit is set in the Operational Status Word.	
Test 08B	Corrupted Parameter Fields/Invalid Operations	SUB, MOD
	Assures that corrupted fields in the IOCBs are properly recognized and properly processed.	
Test 09	Basic Hiway Reads and Writes	SUB, MOD
	Assures that the simplest Read and Write commands correctly work and that Hiway "housekeeping" commands correctly function.	
Test 10	Chained IOCBs with Simple IOCB Commands	SUB, MOD
	Assures that chained IOCBs correctly work.	
Test 11	Multiple and Reversed Multiple CCB Slots	SUB, MOD
	Assures that multiple Start IOCB commands can be successively invoked to various Channel Control Block slots.	
Test 12	Stop on Error and Stop Skip Reset Functions	SUB, MOD
	Assures that the Stop/Skip on Error and Stop/Skip Reset correctly work with nonrecoverable Hiway errors.	
Test 13	Stop IOCB Command and IOCB Pointer Words	SUB, MOD
	Assures that the "Stop IOCB" command correctly works.	
Test 14	System Exerciser Test	SUB, MOD, EXER
	Assures that there is no unintentional interaction between the DHI and other boards in the Hiway Gateway.	

- | | | |
|---------|--|----------|
| Test 15 | Intentionally Forcing/Detecting Hiway Errors | SUB, MOD |
| | Verifies that the Data Hiway detects and properly handles (and reports) as many types of errors as can easily be caused without manual intervention. | |
| Test 16 | Four Read, Compare, Report IOCB Operations | SUB, MOD |
| | Assures that the four types of Read, Compare, and Report operations work correctly, and that FIFOs and FIFO Overflow is working. | |
| Test 17 | Conditional Write with Preread IOCB Operation | SUB, MOD |
| | Assures that the "Conditional Write With Preread" IOCB-operation correctly functions. | |
| Test 18 | Buffer Grant/Report IOCB and HG FIFO Management | SUB, MOD |
| | Assures that Poll and Call Up and the associated protocol correctly work. It also tests "Buffer Grant with Report." | |
| Test 19 | Verify Broadcast IOCB and SOE Synchronize Tally | SUB, MOD |
| | Verifies that the Broadcast Operation correctly functions for both non-Sequence of Events Synchronize-PIU commands and for the Sequence of Events Synchronize-PIU command. | |
| Test 20 | Start and Compare Protocol IOCB Operations | SUB, MOD |
| | Assures that the "Start of Protocol" and "Protocol Compare" IOCB-operations correctly function. | |
| Test 21 | Four Batch Controller IOCB Operations | SUB, MOD |
| | Verifies that the four Batch-Controller operations correctly function. | |
| Test 22 | Testing Without a Physical HTD on the Hiway | SUB, MOD |
| | Assures that the HG can act as a Hiway Traffic Director, including swapping between the primary and back-up Hiways. | |
| Test 23 | Testing With a Physical HTD on the Hiway | SUB, MOD |
| | Assures that the HG can function in a configuration with a Hiway Traffic Director, including commanding a swap from the primary Hiway to the back-up Hiway. | |

8.7 How to Interpret DHIF Alarms

NOTE

For these tests, all odd-numbered alarms indicate controller error.

<u>Alarm Range</u>	<u>Most likely causes</u>
0100 – 0598	Controller
0600 – 0798	Controller or Hiway
0800 – 0898	Alarms divisible by four indicate Controller; remaining even-numbered alarms indicate Controller or Hiway or PIU
0900 – xx98	Controller or Hiway or PIU (alarms in ranges 18xx, 19xx, 22xx or 23xx are most likely poll callup related and therefore indicative of an HLPIU or controller error)

Alarms that indicate some form of No Response, such as 81 or C1 error in the error FIFO (see below), 440000xx listen-error value, or 0000 poll responses when 0001 is expected, are frequently indicative of a bad I/O card (paddle board), particularly if they occur on only one of the Hiways (A or B). Alarms indicating component failures such as a State Generator Timeout (most-significant bit of the Hiway status set) are probably controller failures.

The complete set of possible DHI alarms is described below. The first line of every alarm is displayed in the standard alarm format with a generally meaningless alarm number. The first two digits of the alarm number identify the test number (or the test number plus 50 if the alarm results from the standard housekeeping commands that precede most tests). This alarm number is advanced by two for every subtest (i.e., command or operation), and often wraps in order to avoid exceeding 99. Thus it is common for alarm numbers to be duplicated within a test. Alarm numbers do not necessarily represent the same commands or operations from one pass to another when subtests are conditionally skipped (when driven by random numbers or by pass number).

An even alarm number always indicates that the primary subtest failed (e.g., no interrupt). An odd alarm number always indicates that the follow up comparison to assure steady state failed (i.e., unexpected late interrupt).

Following the alarm number on the first alarm line is a string of terse unique codes (see the "Difference" line explanation) that summarize the errors. This should be ignored unless running at abbreviation level 1. Running at abbreviation level 1 allows tracking of one persistent but perhaps intermittent error while using minimum displays. This will get as many errors on one screen as possible or will reduce the stack of paper and the time required to print it. If the same terse codes are on every alarm line, it is likely that only this one type of failure is occurring.

The second and subsequent alarm lines occur only at abbreviation level 2 and higher. They generally start with three lines of "Was," "Should Be," and "Difference." The "Was" lines are shown below as the first line of each paragraph. (Note that they seldom contain the word "Was.") The remainder of each paragraph defines or describes the various fields on that line. In all cases, the "Should Be" is displayed immediately underneath the "Was" line with all fields aligned.

The "Difference" line always is displayed immediately underneath the "Should Be" line, and identifies those hex digits that are wrong. An "N" indicates that there was no change to a field that should have changed. A "C" indicates that there was a change to a field that should not have changed. A "W" indicates that the hex digit changed to a wrong value. Note that this same "N/C/W" coding is used in the terse coding at the end of the first alarm line, but is based on comparison of the entire word, not hex digit by hex digit.

Often, several discrepancies occur within one failure. If they are in the same line, that line is displayed only once (e.g., two different hardware registers); however, several triplets of "Was/Should Be/Difference" lines can result if the discrepancies are in fields displayed in different lines.

When the abbreviation level is 3 or 4, a scenario describing the command, the operation, the IOCB chains, and the first eight data words is displayed after all "Was/Should Be." It is described in detail after the "Was" line descriptions.

8.7.1 Explanation of Level 1 DHIF Alarms

Registers were xx xxxx xxxx xxxx xxxx xxxx xxxx xxxxxxxx
 Paged Stat 0- E xxxxxxx xxxx xxxx xxxxxx xxxxxx xxxxxx xxxxxx
 Paged Stat 10-16 xxxx xxxxxxxxxx xxxxxxxxxx xxxxxxxxxx
 Pre Unsol xx xx xx xx xx xx xx xx xx xxx xxx xxx xxx xxx xxx xxx
 Post Unsol xx xx xx xx xx xx xx xx xx xxx xxx xxx xxx xxx xxx xxx
 Registers were aa bbbb cccc dddd 00ee ffff gggg hhhhhhh

aa = "Software Register" contents

00 = Operation completed

06 = Operation timed out

09 = No bus error exception from intentionally causing a bus parity error

(Any other values indicate software error, not DHI or Hiway error.)

bbbb = Hardware Status register

cccc = Configuration Status register before reading Operational Status register

dddd = Configuration Status register after reading Operational Status register.
 Interrupt Acknowledge Semaphore (bit 2) and Module Bus Data Parity
 Error (bit 15) should be cleared. Otherwise should be the same as "cccc".

00ee = Interrupt Vector register with the most significant byte cleared to "00".

ffff = Command register

gggg = Operational Status register

hhhhhh = Paged Status words. During self test, bits 0-6 show "Test in Error".

Paged Stat 0- E aaaaaaaa bbbb ccdd eeeeeee ffffffff gggggg hhhhhh iiiiii

aaaaaaaa = Firmware revision number (should remain constant)

bbbb = Hiway status

cc = Retry count

dd = Inter-block Time

00eeeeee = Address causing bus error (when no error, the value is \$FFFFFFFF)

00ffffff thru 00iiiiii = Current IOCB addresses for CCBs 0 through 3 respectively.

Paged Stat 10-16 aaaa 000bbbbbbb cccccccc ddddeeee

aaaa = Data Hiway address (most significant byte = Hiway Address status, least
 significant byte = address)

00bbbbbb = Starting address of the FIFO Address table

cccccccc = Hiway Listen error (most significant 6 bits is error code, least significant 26 bits is the Bad Received data—Hiway bits 1 through 26) Note if cccccccc = 440000xx, then the PIU polling adjustment is suspect. May require the minimum adjustment.

dddd = Poll counter (number of polls since last time this was read)

eeee = Poll Response counter (number of poll responses since last read)

Pre Unsol aa bb cc dd ee ff gg hh ii jjj kkk lll mmm nnn ooo ppp qqq

This line profiles any unsolicited interrupts that preceded the solicited interrupt. The field descriptions are: aa = the number of unsolicited interrupts preceding the solicited interrupt; bb through ii = the first eight or fewer Interrupt Vector register values; jjj through qqqq = the first eight or fewer Operational Status register values.

Post Unsol aa bb cc dd ee ff gg hh ii jjj kkk lll mmm nnn ooo ppp qqq

This line profiles any unsolicited interrupts that both preceded and followed the solicited interrupt (i.e., it is the continuation of "Pre Unsol" and uses the same field descriptions)

8.7.2 Explanation of Level 2 DHIF Alarms

```
IOCB Cx, Ix   xxxx xxxx xxxxxxxx xxxx xxxx xxxxxxxx
Data Cx, Ix, Bx  xxx xxx xxx xxx xxx xxx xxx xxx xxx xxx
SOE & FIFO Ovrfl  xxxx xxxx
$80410 & $80412  xxxx xxxx
FIFO x Hdr & E0  xxxx xxxx xxxx xxxx xxxx   xxxx xxxxxxx xxxxxxx xxxxxxx
FIFO x E1 & E2   xxxx xxxxxxx xxxxxxx xxxxxxx   xxxx xxxxxxx xxxxxxx xxxxxxx
FIFO x E3 & E4   xxxx xxxxxxx xxxxxxx xxxxxxx   xxxx xxxxxxx xxxxxxx xxxxxxx
```

```
IOCB Cx, Ix   aaaa bbbb ccdddddd eeee ffff gghhhhhh
```

Cx = IOCB Chain (CCB number) that this IOCB is on

Ix = IOCB's position on the chain (the first IOCB is I0)

ffff = the IOCB Status word (remainder of the IOCB should remain constant)

```
Data Cx, Ix, Bx  aaa bbb ccc ddd eee fff ggg hhh iii jjj kkk lll
```

Cx = IOCB Chain (CCB number) that the IOCB using this data is on

Ix = Position on the chain for the IOCB using this data (the first IOCB is I0)

Bx = the block number of the 12 data words

aaa through lll = the data words (Note that the locations to be read into are precoated with \$FFF5, thus a value of \$FFF5 in the "Was" line indicates that no read took place.)

```
SOE & FIFO Ovrfl  aaaa bbbb
```

aaaa = The Broadcast SOE Synchronize tally of 96 microsecond intervals (in Hex). This is only alarmed when more than three counts different from the time standard.

bbbb = the 16 FIFO Overflow bits

```
$80410 & $80412  aaaa bbbb
```

The contents of \$80410 and \$80412 after a "Test DMA Read" or a "Test DMA Write."

```
FIFO x Hdr & E0  000a bbb ccc ddd eee   fff gghhhh 0iiiiii 0jkkkkk
```

x = the Error FIFO number, from 0-7

000a thru eeee = is the Error FIFO header

bbb = the Append Pointer (indicates where the next FIFO entry goes). Note that this is usually the only header field to alarm; other header fields are constant.

gg = the Hiway error code

fff thru 0jkkkkk = First position Error FIFO entry

FIFO x E1 & E2 aaa bbbccc 0ddddd 0effff ggg hhhiii 0jjjjj 0kl111

 x = the Error FIFO number, from 0-7

aaa thru 0effff = Second position Error FIFO entry

ggg thru 0kl111 = Third position Error FIFO entry

 bb and hh = the Hiway Error code

FIFO x E3 & E4 aaa bbbccc 0ddddd 0effff ggg hhhiii 0jjjjj 0kl111

 x = the Error FIFO number, from 0-7

aaa thru 0effff = Fourth position Error FIFO entry

ggg thru 0kl111 = Last position Error FIFO entry

 bb and hh = the Hiway Error code

Interpreting the Hiway Error Code:

Ignore the first three bits of the Hex value Hiway Error code. That is, interpret a value of 81 as an alarm code of 01; a value of F5 as 15; etc.

<u>Code</u>	<u>Description</u>
\$00	No error
\$01	No response
\$02	Bad parameter address (used in buffer grant)
\$03	Function code error
\$04	Bad source address error
\$05	Echo data noncompare
\$06	Invalid BCH
\$07	BCH generation failure by the DHI
\$08	Hiway security location check failure
\$09	Grant not received from HTD within 30 msec
\$0A	Grant not dropped by the HTD within 300 µsec
\$0B	Unexpected loss of Hiway grant when DHI is not the HTD
\$0C	Excessive time on the Hiway (more than 30 msec)
\$0D	Hiway box busy
\$0E	Preferred device request aborted and disabled (would not give up the Hiway after 30 msec) when DHI is the HTD.
\$0F	Bad start bit in Hiway word
\$10	Bit underrun/correlation error or incomplete response
\$11	No response to Callup when the DHI is the HTD

\$12	BCH detection logic test failure
\$13	Hiway response to a broadcast operation (none expected)
\$14	Invalid HG security word was received
\$15	Bad destination address during buffer grant

8.7.3 Explanation of Level 3 DHIF Alarms

```

FIFO xx Header    xxxx xxxx xxxx xxxx xxxx
FIFO xx Values    xxxx xxxx xxx xxx  xxx xxx xxx xxx  xxx xxx xxx xxx
DAC FIFO          xxxx xxxx xxxx xxxx xxxx  xxxxxx xxxxxx xxxxxx xxxxxx xxxxxx
Hardwr  Config    Hardwr  Config    Hardwr  Config
Status Was        xxxx      xxxx      xxxx      xxxx      xxxx      xxxx
FIFO xx Header    000a bbbb cccc dddd eeee

```

xx = the Report FIFO number (08-14)

000a thru eeee = the report FIFO header

bbbb = the Append Pointer that indicates where the next FIFO entry goes. (Note that this is usually the only header field to alarm; the other fields are constant.)

```

FIFO xx Values    aaaa bbbb cccc ddd  eeee fff ggg hhh  iiii jjjj kkk lll

```

xx = the Report FIFO number (08-14)

aaaa thru dddd = contents of the Report FIFO entry in the first position

eeee thru hhhh = contents of the Report FIFO entry in the second position

iiii thru llll = contents of the Report FIFO entry in the last position

```

DAC FIFO  000a bbbb cccc dddd eeee  ff00ggg hh00iii jj00kkk ll00mmm nn00ppp

```

000a thru eeee = the DAC Receive FIFO header

bbbb = the Append Pointer that indicates where the next FIFO entry goes. (note that this is the only header field to alarm; other fields are constant) ff, hh, jj, ll, and nn are the five PIU addresses. gggg, iiii, kkkk, mmmm, and pppp are the five PIU Status Word values

Wrong "Reset Sequence" Register Values.

	After Writing a 9		After Writing a 1		One Second Later	
	Hardwr	Config	Hardwr	Config	Hardwr	Config
Status Was	aaaa	bbbb	cccc	dddd	eeee	ffff

The above alarm is displayed only during test startup. A "Reset Sequence" was initiated and the Hardware Status register and the Configuration Status register are tracked three times—after the first half of the reset sequence, after the last half, and after firmware self test has had time to complete.

8.7.4 Interpreting Commands and Operations Which Caused Alarms

A "scenario" is included at the end of each alarm when running at abbreviation level 3 or 4. If the command issued is to the Configuration Status register address (addressable latch), the following scenario is displayed:

Flag Control Value = xxx, Registers = xxx xxx xxx xxx xxx xxx xxx xxx

If the command issued is to the Command register, this scenario is displayed:

Commanded Value = xxx, Registers = xxx xxx xxx xxx xxx xxx xxx xxx

For every CCB that has active IOCBs chained to it, and for every IOCB in each chain, the final lines of the scenario contain the CCB (IOCB chain) number, the active chain of IOCBs, and the first eight data elements for each IOCB. These scenario lines typically will be a small subset of the following:

CCB	Active Chain of IOCBs	Data Elements for this IOCB
0	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx data xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx data xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx data xxx xxxx xxx xxxx xxxx
1	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa FFF5 xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa FFF5 xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa FFF5 xxxx xxx xxxx xxx xxxx xxxx
2	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
3	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx

Where "op" is the 6-bit operation code, "bapa" is the 16-bit Hiway address, consisting of a 6-bit box address (ba) and 10-bit parameter address (pa), and "data" is the 16-bit write data. For most alarms resulting from some form of "write" to a box, these three fields will be most useful in identifying what is being tested. For "read" type operations, the data is **not** read into the third data word, it goes into the words which contain \$FFF5.

Note that "Active Chain of IOCBs" normally applies to chains which are set up to be commanded with a "Start IOCB" command. However, on occasion, a chain will be left "Active" while other commands are being exercised. The Scenario Alarming algorithm considers a chain to be "active" only up to the place where it points to a canned terminating "NOP" IOCB. Normally, three CCBs point to this "NOP" and thus are inactive. For exercising non "Start IOCB" commands, all four CCBs usually are pointing to the terminating "NOP" IOCB, so no chains are active.

9 Status Input/Output Subsystem Tests (SIOS)

9.1 Status I/O Subsystem Elements

Status I/O Subsystem elements include Status Input/Output Subsystem interface electronics, Digital and Trend termination panels, and an external 24-volt power supply (for trend drives). Optional equipment: HLPIU Card-file assembly, up to eight Trend Pen assemblies.

9.2 Use Information

Special Feature—SIOS is supplied with a built-in HELP function that supplies setup and option data to a user who may not have immediate access to this document. To invoke the SIOS HELP function the following is required:

00:01	ALL ?	DEV SIOS	
00:01	SIOS?	ENABLE HELP	(or "ENA H")
00:01	SIOS?	RUN	

The HELP text appears on screen, the program delays, then stops, leaving HELP in the DISABLED state.

9.3 Setup Parameters

9.3.1 SIOS-Specific Parameters

COMPARE	Allowed values are \$00 through \$FF*
DOUT	Allowed values are \$0000 through \$0 FF* Specify the value assigned to digital outputs
RESET	Allowed values are 0 and 1* 0 = Reset the SIOS board, not board flags 1 = Reset the SIOS board and flags
INTERRUPT	Allowed values are 0 and 1* 0 = Disable SIOS Board interrupts 1 = Enable SIOS Board interrupts
MOD_TEST	Allowed values are 0 and 1* 0 = Turn OFF the Passed Module Test LED 1 = Turn On the Passed Module Test LED

CHART_DRIVE	Allowed values are 0 and 1* 0 = Turn OFF the Trend Pen chart drive motor 1 = Turn ON the Trend Pen chart drive motor
LOOPBACK	Allowed values are 0 and 1* 0 = Turn OFF the SIOS loopback mode 1 = Turn ON the SIOS loopback mode
CFREAD	Not used
CFWRITE	Not used
TREND	Allowed values (n,n,n) are: (1) — 1 through 8 (which trend pen) (2) — \$00 through \$FF (desired output counts) (3) — 0
ENABLE	Allowed values and their meanings are INTERRUPTS—Allow board interrupts to be asserted HELP—Turn on HELP routine when SIOS is run TRACE—Do not use SIMULATION—Indicate that a hardware simulator is connected LOOP—Utility, Loop on an operation TREND LOG—Utility, Log state of trend outputs
DISABLE	Allowed values and their meanings are INTERRUPTS—Inhibit board interrupts from being asserted HELP—Allow normal operation when SIOS is run TRACE—Always disabled SIMULATION—Indicate that no hardware simulator is connected LOOP—Utility, Stop looping TREND LOG—State set after the trend log is presented
READ	Allowed values* and their meanings are HARD—Test 8 logs the hardware status CONF—Test 8 logs the configuration status STATUS—Test 8 logs the status input register DOUTPUT—Test 8 logs the digital output status REGS—Test 8 logs all the above registers
DONT_READ	These values are set by the program (in response to the selected READ values), do not attempt to use this parameter.

*Value entries for these parameters are valid only when made during the running of Test 8. Entry of any of these parameters at any other time has no effect on test execution. They are each initialized to a not-used value by Test 8, then tested to detect user entries.

9.3.2 General Parameters

TESTS	Allowed values are 1 through 8
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

9.4 Preset Parameter Values for SIOS

COMPARE = \$0
DOUT = \$0
RESET = 0
INTERRUPT = 0
MOD_TEST = 0
CHART_DRIVE = 0
LOOPBACK = 0
CFREAD = \$0
CFWRITE = \$0,0,0
TREND = \$0,0,0
ENABLE = INTERRUPTS,LOOP,TREND LOG,SIMULATION
DISABLE = TRACE,HELP
READ =
DONT_READ = HARD,CONF,STATUS,DOUTPUT,REGS,NIL
TESTS = 1-8
ERROR_LIMIT = 5
PASS_LIMIT = -
REPORT = LOG
INHIBIT = PASSNUMBER,TESTNUMBER
SCALE = 0
MINUTES_LIMIT = -
ABBREVIATION = 4
AFTER_ERROR_LIMIT = ABORT

9.5 Mode Characteristics For SIOS

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

9.6 Individual System I/O Tests

(System Modes)

Test 01	SIO Hardware Register Checks	SUB
Verifies that the SIO board can be properly reset and that the hardware-register set can be read from and written to by the MCPU without invoking module-bus errors.		
Test 02	SIO Interrupts & Digital Change Detect	SUB, MOD
Verifies the functionality of the SIO-interrupt logic, digital-input change-detect logic and the digital-loopback circuits.		
Test 03	SIO Digital Output Checks	SUB, MOD
Verifies output uniqueness for the digital-output subsystem by use of all bit pattern combinations.		
Test 04	SIO Trend Pen (analog) Output Checks	SUB
Checks the accuracy of trend (analog) outputs and output uniqueness.		
scale % —25 0 10 20 30 40 50 60 70 80 90 100 102.5		
ma Out — 0 4.0 5.6 7. 8.8 10.4 12.0 13.6 15.2 16.8 18.4 20.0 20.4		
Counts —\$00 \$32 \$46 \$5A \$6E \$82 \$96 \$AA \$BE \$D2 \$E6 \$FA \$FF		
Test 05	Simulated HLPIU Card File Access & Interrupts	SUB, MOD
Factory use only; requires simulator equipment.		
Test 06	SIO Module Bus Error Handling	SUB
Tests functionality of the module-bus error-detection logic.		
Test 07	SIO Subsystem Exerciser	SUB, MOD, EXER
Makes random changes of trend pen position.		
Test 08	SIO Board Level Utility	SUB
Provides the ability to manually Read and Write SIO registers; Control Subsystem Flags; Log Status and Digital Change Detect Interrupts; Log Digital Output Readings; Log Trend (Analog) Outputs; and Read and Write HLPIU CFA DRAM space. Must be only test-selected and requires ABORT or STOP command to halt.		

9.7 SIOS Alarm Categories

<u>Alarm Range</u>	<u>Most likely causes</u>
0101 – 0289	SIO controller
0290 – 0299	SIO controller or faulty simulator hardware
0300 – 0399	SIO controller or a bad load on the digital outputs
0400 – 0499	SIO controller or a bad load on the Trend-Pen outputs or a faulty Trend-Pen power supply
0500 – 0599	SIO controller or a bad simulator
0600 – 0699	SIO controller
0700 – 0799	SIO controller or a bad load on the Trend-Pen outputs or a faulty trend pen power supply

NOTE

Any alarms smaller than 100 or greater than 900 are SIO controller-board failures.
Some alarms can be caused by improper parameter/hardware setup.
Simulation testing requires special hardware.

10 Memory Exerciser Module Tests (MEMX)

10.1 Memory Module Elements

Memory Module elements consist of Memory boards.

10.2 Use Information

The purpose of the Memory Exerciser Module (MEMX) is to exercise all memory boards in the node. It performs a write/read test on certain areas of memory that are reserved from the program-memory pool as well as all memory locations above the program and memory pool areas. The Module Configuration Table (MCT) is referenced to establish the memory size.

Because the test program resides on the first memory board, the number of memory locations available for write/read testing is fewer than on other memory boards in the module. The user can swap memory boards after 50% of testing has been completed, to more thoroughly test each memory board.

10.3 Setup Parameters

10.3.1 MEMX-Specific Parameters

None

10.3.2 General Parameters

TESTS	Allowed values are 1 and 2
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG SUSPENDED
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

10.4 Preset Parameter Values for MEMX

TESTS = 1-2

ERROR_LIMIT = 5

PASS LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER,SUSPENDED

SCALE = 0

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

10.5 Mode Characteristics for MEMX

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

10.6 Individual Memory Tests

(System Modes)

Test 01 Write/Read Test SUB

Verifies that the memory electronics can store, retain, and retrieve from each memory cell patterns of 1s and 0s. This test writes to and then reads from all available memory cells.

Test 02 Write/Read Test Exerciser SUB, MOD, EXER

Verifies that the memory electronics can store, retain, and retrieve from each memory cell patterns of 1s and 0s. This test first writes to heap and then reads the data back from heap. Then, starting with the first 256 kW in beyond pool, it reserves and tests each block in turn until all blocks in beyond pool have been tested. If an individual block cannot be reserved, it is skipped until the next pass of the test. The test then scans program memory and reports any EDAC and/or parity errors.

10.7 MEMX Alarm Categories

Any alarms from this test program indicate memory-board failure.

Error printout includes the address of the failing memory location, whether the failure was a single-bit error or multiple-bit error, whether the error was recoverable or unrecoverable, and the number of the bit that was incorrect.

11 Communication Line Interface Test (CLIF)

11.1 COMMUNICATION LINE INTERFACE ELEMENTS

The Communication Line Interface elements consist of a Communications Line Interface board contained within one of the following node types: CIU, Computer Gateway, or Processor Gateway.

11.2 USE INFORMATION

To provide a thorough test of the CLI Subsystem, at least one pass of all tests must be run in Subsystem Mode.

Some CLIF tests are affected by the presence of the I/O adapter and loopback connectors (51109041) or by the presence of the adapter with a loopback cable (51109040). The following table summarizes these effects.

	Normal Execution	Error Condition
No I/O adapter	Tests 01-12	Tests 13,14
Adapter without loopback hardware	Tests 01-12	Tests 13,14
Adapter with loopback connectors	Tests 01-12, 14	Test 13
Adapter with loopback cable	Tests 01-08, 10-13	Tests 9, 14

If the computer cable between the CG and the Host Processor is installed, only tests 01, 03, 05, and 14 can be used. Running any of tests 02, 04, or 06-13 results in errors. Test 14 runs properly only if the host processor is running a program that will receive and retransmit the test data.

11.3 SETUP PARAMETERS

11.3.1 CLIF-Specific Parameters

BAUD_RATE Allowed values are 0 through 9 (valid for tests 13 and 14)

- 0 – 9600 bps (default)
- 1 – 1200
- 2 – 1760
- 3 – 2152
- 4 – 2400
- 5 – 4800
- 6 – 19200
- 7 – 38400
- 8 – 57600
- 9 – 76800

11.3.2 General Parameters

TESTS	Allowed values are 1 through 14
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

11.4 Preset Parameter Values for CLIF

BAUD_RATE = 0
TESTS = 1-14
ERROR_LIMIT = 5
PASS_LIMIT = -
REPORT = LOG
INHIBIT = PASSNUMBER,TESTNUMBER
SCALE = 100
MINUTES_LIMIT = -
ABBREVIATION = 4
AFTER_ERROR_LIMIT = ABORT

11.5 Mode Characteristics for CLIF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

11.6 Individual CLIF Tests

(System Modes)

- | | | |
|---------|--|-----------|
| Test 01 | Basic CLI Responses | SUB |
| | Ensures a basic ability to communicate and to generate and control the two program interrupts. | |
| Test 02 | Load and Verify Registers | SUB, MOD |
| | Ensures the integrity of data paths between the DRAM and CLI and checks ability of the CLI to store and retrieve data from its on-board RAM. | |
| Test 03 | Configure Parameter Download Command | SUB, MOD |
| | Verifies that CLI detects illegal Download Initial parameters when the Configure command is executed. | |
| Test 04 | Paged Status | SUB, MOD |
| | Reads and verifies contents of the Paged Status Register. | |
| Test 05 | Test DMA Commands | SUB |
| | Verifies the Module Data Bus read/write capability and ensures the integrity of the data paths between the DRAM and the CLI. | |
| Test 06 | Last IOCB Flag & Interrupt Flag | SUB, MOD |
| | Verifies that the CLI accesses the Channel Control Block and sequences through a series of Input/Output Control Blocks. | |
| Test 07 | Stop IOCB Command | SUB |
| | Verifies that CLI accesses IOCBs through the "Stop" Command. | |
| Test 08 | Verify Illegal Command Detection | SUB |
| | Verifies that CLI detects illegal commands when stored into the Command-Word Register. | |
| Test 09 | Bus Error Detection | SUB |
| | Verifies that the CLI can detect and report module bus-transaction errors. | |
| Test 10 | Verify Invalid CCB & IOCB Detection | SUB, MOD |
| | Verifies that CLI detects invalid CCBs and invalid IOCBs through format error bits in the Operation Status Register. | |
| Test 11 | Loopback (Correct BCC) All Data ID at 76 k bps | SUB, MOD |
| | Verifies that all Data ID are transferred on the data link. | |
| Test 12 | Loopback (Correct BCC) All Data ID at 1200 baud | SUB, MOD, |
| | Verifies that all Data ID are transferred on the data link. | |

Test 13 Remote Loopback (0-3, 2-1) SUB, MOD, EXER

Uses loopback cables to verify data-pattern transfers. Channel 0 transmits to channel 3 and channel 2 transmits to channel 1.

Test 14 Remote Loopback (0-1, 2-3) MOD, SUB

Uses loopback connector to verify data-pattern transfers. Channel 0 transmits to channel 1 and channel 2 transmits to channel 3.

11.7 CLIF Alarm Categories

<u>Alarm Range</u>	<u>Most likely causes</u>
0101 – 1299	Controller Board
1301 – 1499	Controller Board or Paddle Board

12 DATA HIWAY EXERCISER (DHEX)

12.1 Data Hiway Exerciser Elements

The Data Hiway Exerciser elements include the Data Hiway Interface Board (in context of other Hiway Gateway elements), redundant Data Hiway, and at least one operational Hiway box (PIU, DHP, CB, etc.).

12.2 Use Information

12.2.1 Cautions for On-Process Use

While these tests may be used on an "On-Process" Hiway under most conditions, in certain circumstances it causes such intense periods of Hiway activity that other preferred access devices can get "No Grant" errors. This can occur if two or more HGs simultaneously run Test 2 on the same Hiway. If it is necessary to run DHEX under these conditions, the INTERBLOCK TIME parameter should be started at the highest possible value to reduce Hiway "hogging" by the HGs under test. It can then be incrementally reduced to increase the Hiway Loading. For more information, see the INTERBLOCK TIME parameter.

Because Test 1 is limited to activity levels significantly lower than those generated by Test 2, two or more HGs can simultaneously execute Test 1 without special precautions. Test 3 performs only polling/callup sequences and produces minimal Hiway activity.

If a redundant pair of HGs is split up with the primary on-line and the secondary running this test, AUTOMATIC FAILOVER TO THE SECONDARY DOES NOT OCCUR. If the primary HG fails under these circumstances, the operator must MANUALLY restart the primary or shut down the secondary and reload it with the On Process HG personality to recover.

If the HG selected to run the test was acting as the Hiway Traffic Director, the preferred access devices it serves are blind (no access grants) to the Hiway during the shutdown/ reload/restart period unless a redundant partner is available to take over the HTD functions. If there is no redundant HG available, the HTD functions are lost during periods when DHEX is not running. This condition is unacceptable for On Process operation.

12.2.2 Device-Specific Exclusions

DHEX does not need to be told the addresses of boxes connected to the hiway because it does a search during its start up; however, the presence of other preferred access devices on the Hiway requires guidance by the operator.

A list of "banned" devices prevents DHEX from accessing 45000 Hiway Interface Modules (or any other device the operator desires).

A list of "security-only" devices prevents DHEX from attempting to read from addresses other than the hiway-security location of a DHI (or any other device that responds only to the hiway-security location).

12.2.3 DHEX Logs

NOTE

Communications errors involving the Data Hiway and/or devices on the hiway are counted, not alarmed, and show up only when the various logs are displayed. It is the responsibility of the user to read the logs and determine if this HG and/or the Data Hiway and its boxes are communicating within specifications for use by On Process applications.

DHEX has its own unique set of logs. In order of highest-to-lowest resolution, these logs are Detail, Composite, and Aggregate.

When a higher resolution log is generated, **all logs with coarser resolution are also generated**. Thus, the Composite log is accompanied by the Aggregate log, and the Detail log is accompanied by the Composite and Aggregate logs.

Periodic scheduling of these logs is controlled by parameter LOG_INTERVAL; their demand output is controlled by parameter DEMAND_LOG_TYPE. Other parameters affecting the logs are LOG_TYPE, FINAL_LOG_TYPE, SLEW_CONTROL, and DSPLY_BOX_NUMBER.

Because of space limitations within this document, the formats are shown slightly compressed. All values shown as XXXXXXXXXX are actually 32-bit fields and can contain values to 4,294,967,295. Any error counts exceeding this maximum value are lost; however, the earliest this could occur would be after 18 days of continuous, full throughput operation.

Detail Log Format—This provides the highest resolution of error breakdown, with error counts displayed box by box. The following format is repeated for each box:

DATE TIME DHEX SLOT_NUMBER

BOX XX

TRANSFERS	XXXXXXXXXX	NO RESPONSE	XXXXXXXXXX	BAD ADDRESS	XXXXXXXXXX
FUNC ERROR	XXXXXXXXXX	BAD SOURCE	XXXXXXXXXX	ECHO ERROR	XXXXXXXXXX
BCH ERROR	XXXXXXXXXX	BCH GEN FAIL	XXXXXXXXXX	SECURITY FAIL	XXXXXXXXXX
NO GRANT	XXXXXXXXXX	EXCESS GRANT	XXXXXXXXXX	GRANT LOST	XXXXXXXXXX
EXCESS TIME	XXXXXXXXXX	BUSY RESPONSE	XXXXXXXXXX	REQST ABORTED	XXXXXXXXXX
BAD START BIT	XXXXXXXXXX	PARTIAL RESP	XXXXXXXXXX	NO CALLUP RESP	XXXXXXXXXX
BCH CHECK FAIL	XXXXXXXXXX	BRDCAST RESP	XXXXXXXXXX	INVLD SEC WRIT	XXXXXXXXXX
BAD DEST ADRS	XXXXXXXXXX				

Note that this format changes slightly for box number zero (HTD) and for the DHI (see DHI_ADDRESS parameter). In those instances, the field named NO RESP H2 is replaced by one named UNEXPECTED RSP.

Composite Log Format—Contains the sum of the individual errors by class as follows:

```
DATE TIME DHEX SLOT_NUMBER

                                TOTALS

TRANSFERS      XXXXXXXXXX NO RESPONSE  XXXXXXXXXX BAD ADDRESS   XXXXXXXXXX
FUNC ERROR     XXXXXXXXXX BAD SOURCE   XXXXXXXXXX ECHO ERROR    XXXXXXXXXX
BCH ERROR      XXXXXXXXXX BCH GEN FAIL  XXXXXXXXXX SECURITY FAIL XXXXXXXXXX
NO GRANT       XXXXXXXXXX EXCESS GRANT XXXXXXXXXX GRANT LOST    XXXXXXXXXX
EXCESS TIME    XXXXXXXXXX BUSY RESPONSE XXXXXXXXXX REQST ABORTED XXXXXXXXXX
BAD START BIT  XXXXXXXXXX PARTIAL RESP XXXXXXXXXX NO CALLUP RESP XXXXXXXXXX
BCH CHECK FAIL XXXXXXXXXX BRDCAST RESP XXXXXXXXXX INVLD SEC WRIT XXXXXXXXXX
BAD DEST ADRS  XXXXXXXXXX
```

Aggregate Log Format—Provides only the following summary:

```
DATE TIME DHEX SLOT NUMBER

PASS COUNT = XXXXXXXXXX NO OF TRIES = XXXXXXXXXX TOTAL ERRORS = XXXXXXXXXX
```

Addresses Log Format—This log contains the addresses of all hiway devices including the DHIF itself:

```
DATE TIME DHEX SLOT NUMBER    DEVICES PRESENT ON THIS HIWAY:

XX, YY, ZZ, etc.
```

12.3 Setup Parameters

12.3.1 DHEX-Specific Parameters

PERMISSION Allowed values are Y or N

To prevent accidental operation on an on-process Data Hiway, the program sets PERMISSION to N each time DHEX is started. The operator must set it to Y after DHEX starts.

DHI_ADDRESS Allowed values are 2 to 63

This is the Data Hiway address of the DHI. If another device type is detected at this hiway address the program aborts after informing the operator of the duplication. If Basic or Enhanced Operator Stations are present on the hiway, the DHI cannot be assigned hiway address 4.

HTD_FUNCTIONS Allowed values are

ENABLED — The DHI polls and supports both preferred-access device request/grant ports.

DISABLED — The DHI does not poll and uses the request/grant lines from an HTD or another DHI performing the traffic director function.

HWY_STATISTICS

Allowed values are CLEAR or ACCUMULATE

When CLEAR is set by the operator, the test program clears the error and transfer counters and sets this value back to ACCUMULATE. This provides a clean starting point without having to stop and restart the program.

PRIMARY_CABLE

Allowed values are

A_CABLE = Switch/swap to cable A

B_CABLE = Switch/swap to cable B

ALTERNATE = Periodically alternate between cables or follow the active cable.

When running with an HTD or with another DHI acting as Traffic Director, this parameter should be ALTERNATE and ALTERNATE INTERVAL should be set to “-” to allow the DHI to follow the active Hiway cable.

If running with an HTD, the primary hiway can be swapped to the backup hiway only once. Attempts to swap back to the primary will be unsuccessful. If the DHI is acting as the HTD, the primary/secondary switch can be performed as often as desired.

ALTERNATE_INTERVAL

Allowed values are 0 to 32767 or -

This specifies the number of passes between active cable swaps if PRIMARY_CABLE is set to ALTERNATE.

When running with an HTD or with another DHI acting as Traffic Director, this parameter should be set to “-” and PRIMARY_CABLE should be set to ALTERNATE to allow the DHI to follow the active hiway cable.

WRITE_INTERVAL

Allowed values are 0 to 32767 or -

This specifies the number of passes between hiway-security write operations. If this parameter is nonzero, hiway-security data is initially written to prevent an error on a freshly powered-up box. If this parameter is set to 0 or -, security writes are not performed unless SECURITY_COMPARE is set to ENABLE and a miscompare is detected. To completely disable writes, set this parameter to 0 or “-” and set SECURITY_COMPARE to DISABLE.

SCRAMBLE_INTERVAL	Allowed values are 0 to 32767 or - In order to force a variance in the traffic pattern on the hiway, the ordering of the hiway box-numbers list and of the “safe” box database addresses list are both 'scrambled' following the number of passes specified by this parameter. Setting it to 0 or - keeps the current lists as is.
BANNED_LIST	Sixteen values in range 0 to 64 This list specifies the hiway addresses of up to 16 devices that do not support data base reads or security locations (e.g., 45000 HIM) and are not to be accessed by DHEX. Unneeded locations should contain the value 64.
SECURITY_ONLY	Sixteen values in range 0 to 64 This list specifies the hiway addresses of up to 16 devices that support only security locations (other DHIs) and are to have only security operations performed on them. Unneeded locations should contain the value 64. The hiway addresses for box 0 and the DHI need not be entered. NOTE: As part of Test 1, DHEX directs the DHI to read its own security data and the security data of Box 0; both reads should be unsuccessful. The read attempts directed at Box 0 can be inhibited by adding it to this list. Read attempts directed at the DHI cannot be inhibited; entering the DHI address in this list has no effect.
DEMAND_LOG_TYPE	Requests an unscheduled log. Allowed values are INHIBIT—No unscheduled log output. AGGREGATE—Produces the Aggregate log once. COMPOSITE—Produces the Composite log once. DETAIL—Produces the Detailed log once. ADDRESSES—Produces the Addresses log once. After the specified log is output, this parameter is reset to INHIBIT.

SLEW_CONTROL When running with test output on a CRT, the detail report can scroll off the screen too fast to read. This parameter allows control of report generation to prevent such loss of information.

SLEW_CONTROL controls the generation of all Detailed reports, both demanded and periodic. If a report is due and SLEW_CONTROL is not SCROLL, testing stops until the operator completes or aborts the report by manipulation of this parameter and of DSPLY_BOX_NUMBER.

NOTE: As each device is reported, its values are accumulated. At the completion of the Detail report, the Composite report and Aggregate report use the accumulated values. Devices that were not displayed are not included.

SLEW_CONTROL allowed values are

SCROLL — When entered by the operator, allows the detail report to scroll at full speed (used for hard copy). The Composite and Aggregate reports contain the sum of all hiway device statistics.

NEXT — When entered by the operator, requests display of data for the hiway device specified by DSPLY_BOX_NUMBER.

After the data is displayed, the parameter DSPLY_BOX_NUMBER is set to the next device number in anticipation of another NEXT, and sets SLEW_CONTROL to WAITING.

When SLEW_CONTROL is NEXT, and the last available device has been reported, SLEW_CONTROL is set to DONE and the accumulated totals are output.

SKIP — When entered by the operator, causes display of data for the hiway device specified by DSPLY_BOX_NUMBER. DSPLY_BOX_NUMBER is not modified. If there are no statistics for the indicated device, a short warning is output and SLEW_CONTROL is set to WAITING.

DONE — When entered by the operator or set by the program, terminates the current report and prints the accumulated totals.

WAITING — The requested operation has been completed, and further instructions may be entered. There is no limit on the time spent waiting. To continue, change SLEW_CONTROL to one of the above values.

DSPLY_BOX_NUMBER Allowed values are 0 to 64 or -

DSPLY_BOX_NUMBER is used with SLEW_CONTROL to control the generation of detailed reports. If set to “-”, all available devices are displayed.

If DSPLY_BOX_NUMBER is set to “-” and SLEW_CONTROL is set to SKIP, the current report is aborted and no totals are output.

HIWAY_ERRORS This parameter controls the printing of hiway errors. It does **not** affect accumulation of hiway errors for the error logs. Because DEHEX can produce prodigious quantities of hiway errors in some circumstances (e.g., loss of a cable or HTD), these errors are not normally printed as they occur. However, it may at times be desirable to have hiway errors printed as they occur in addition to their accumulation for error logs. Allowed values are:

SUPPRESS— Hiway errors are accumulated for the logs but are not printed as they occur.

FIRST_ONLY— Only the first hiway error is printed. (This is useful when an error-free test run is required. The operator is notified without necessarily halting the test.)

REPORT— All hiway errors are printed as they occur. The use of a reasonable error limit is strongly suggested, as the loss of a single hiway device can cause an enormous eruption of error messages.

LOG_TYPE Specifies which log type is to be printed periodically. Its allowed values are DETAIL, COMPOSITE, AGGREGATE, and ADDRESSES.

LOG_INTERVAL Specifies the number of passes between periodic logs. Its allowed values are 0 to 32767 and “-”. If set to 0 or “-”, periodic logs are not generated.

FINAL_LOG_TYPE	Specifies the log to be printed when the program is stopped by the operator in an orderly manner. Its allowed values are INHIBIT, DETAIL, COMPOSITE, AGGREGATE, and ADDRESSES. No stop log is generated if set to INHIBIT or in the event that the program is aborted or terminates because of a fatal hardware error.
SECURITY_COMPARE	Specifies whether the data read from the hiway security location is to be compared to the expected hiway security data. Its allowed values are ENABLE — Any miscompare is noted by an alarm message, and the correct security data is written to the box. This eliminates repeated errors while waiting for WRITE_INTERVAL to force a write. DISABLE — No comparison is performed and writes are completely controlled by WRITE_INTERVAL. To completely disable writes, set this parameter to DISABLE, and WRITE INTERVAL to 0 or “-”.

INTERBLOCK_TIME

Controls the minimum time (in 100 microsecond intervals) between successive attempts to access the hiway. Its allowed values are 1 (.1 millisecond) to 127 (12.7 milliseconds).

As this value is increased, the DHI remains off the hiway for longer intervals between requests. In most cases, the program overhead exceeds the interblock time and effects of this parameter cannot be seen; however, in Test 2 an extremely large list of consecutive requests is started and the DHI runs at maximum throughput for up to 30 seconds. If two or more HGs simultaneously run this test, there may be periods of activity so intense that other preferred-access devices may be held off long enough to cause "No Grant" errors. Increasing the interblock time in one or more of the HGs may help alleviate the condition.

See the DHEX Use Information at paragraph 12.2.1 for cautions on running with an On Process Hiway.

NOTE

There are two additional "Display Only" parameters that you may find useful. CURRENT_CABLE

Shows current cable selection as CABLE A, CABLE B, or UNKNOWN.

BOX_COUNT Shows total hiway box count. Remember that both Box "0" and the HG are included in this total. Thus if you have one HG with no boxes the count will equal 2. Also, remember some box types have multiple addresses and therefore are counted as two boxes.

12.3.2 General Parameters

TESTS	Allowed values are 1 through 3
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

12.4 Preset Parameter Values For DHEX

PERMISSION = N
DHI_ADDRESS = 2
HTD_FUNCTIONS = DISABLED
HWY_STATISTICS = ACCUMULATE
PRIMARY_CABLE = ALTERNATE
ALTERNATE_INTERVAL = -
WRITE_INTERVAL = 20
SCRAMBLE_INTERVAL = 100
BANNED_LIST = 64,64,64,64,64,64,64,64,64,64,64,64,64,64,64,64
SECURITY_ONLY = 64,64,64,64,64,64,64,64,64,64,64,64,64,64,64,64
DEMAND_LOG_TYPE = INHIBIT
SLEW_CONTROL = SCROLL
DSPLY_BOX_NUMBER = -
HIWAY_ERRORS = SUPPRESS
LOG_TYPE = AGGREGATE
LOG_INTERVAL = 10000
FINAL_LOG_TYPE = COMPOSITE
SECURITY_COMPARE = ENABLE
INTERBLOCK_TIME = 127
TESTS = 1-2

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER

SCALE = 0

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

12.5 Mode Characteristics for DHEX

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	Only one	Random
Exerciser	One or more	Only one	Random

12.6 Individual DHEX Tests

(System Modes)

Test 01 Read/Compare/Write

SUB, MOD, EXER

Performs read/compare/write operations on the hiway-security word of each device available for use by the program.

Test 02 High-traffic Read

SUB

Reads from every “safe” database address in every available device capable of responding to data-base reads at the highest possible throughput. See CAUTION at paragraph 12.2.1.

Test 03 Hiway Poll Response Timing

SUB, MOD

Adjusts poll response timing of the hiway. This is a utility test and must be the only test selected for execution.

12.7 DHEX Alarm Categories

<u>Alarm Range</u>	<u>Most Likely Causes</u>
2-20, 34-36, 42-46, 50, 56-58	DHIF
30-32	Busy node
38-40, 44, 48, 54	Hiway box
52	Software detected

13 PDG Subsystem Tests (PDGI)

NOTE

The Peripheral Display Generator Interface (PDGI) subsystem test will not run in X-Layer node. Use the Universal Station^X Interface (XUSF) subsystem test instead.

13.1 PDG Subsystem Elements

PDG subsystem elements include the PDG board (Peripheral Display Generator, which combines the functions of the Peripheral Interface Controller and Video Display Generator) or the EPDG board (Enhanced Peripheral Display Generator, which adds a SCSI controller function to the PDG functions); Operator, Supervisor and Engineer Keyboards; Video Monitor; Matrix Printer; and cables.

13.2 Use Information

PDGI tests can be run on any US or UWS on the network except the TOCS itself.

Restriction—When testing a dual function board (for example, EPDG, EPDG2, TPDG, etc.), only one of the following test systems can run on that dual function board at a time without errors occurring in the systems. The test systems are KEYB, PDGI, SCMD, and XUSF.

Special Requirements—The nature of the device connected to a particular port must be defined to this program. Where the program requires actual devices, it bypasses those channels that are configured "HOOK_UP_TO = OPEN." Devices on each channel are required to fully test all defined functions.

Special Feature—Test 10 of PDGI can be used to verify the peripheral cabling and setup of the Operator Interface subsystem. Upon starting test 10, a peripheral-status log is printed. If any status change occurs (i.e., cables from the peripherals get disconnected and subsequently reconnected) the status is redisplayed for each state change. The status log indicates the state of all channels and the PBlock setup for each channel. To invoke test 10 the following is required:

```
00:01      PDGI?  TEST 10
00:01      PDGI?  MODE S
00:01      PDGI?  RUN
```

To stop test 10 the following is required:

```
00:01      PDGI?  ABORT
```

13.3 Setup Parameters

13.3.1 PDGI-Specific Parameters

Note that in some of the information that follows, reference is made to "C_n." This convention is used to indicate the assignment of parameter values to each of three I/O channels. The device assignment for each channel is

C0—Video

C2—Printer

C3—Keyboards

Use of the STATUS command displays dummy C1 and C4 values for HOOK_UP_TO and SCALE that cannot be changed.

C_n_HOOK_UP_TO Allowed values for Printer and Keyboard Channels are OPEN and DEVICE; allowed values for Video Channels are OPEN and SINGLE.

Meanings are

OPEN = No device is connected

DEVICE = A physical device (Printer, Keyboard, etc.) is connected and ready

SINGLE = One video-display generator is available

INITIALIZE_BOARD Allowed values are YES, NO.

This parameter controls whether or not the board under test is initialized following the RUN command. Normally set to YES.

C_n_SCALE Allowed values are 0 - 32767

Establish a ratio of test executions between the channels (used by test 9)

COLOR_TO_VDG Allowed values are DARK, RED, GREEN, YELLOW, BLUE, MAGENTA, LT BLUE, WHITE

Selects which color is used by the Video Flood test

DATA_SELECTION Allowed values and their meanings are

AUTO = Incremental change of a set data pattern

RANDOM_STRING = Generate random data

USER_STRING = Use the data supplied in "USER_STRING".

USER_STRING Allowed value is any string of up to 40 ASCII characters

ENABLE or **DISABLE** Allows the operator to enable or disable the specified function while running the program.

The allowed value is

KEYBOARD_ANNUNCIATORS—When enabled, allows the Exerciser Test (test 15) to toggle the external annunciators on and off by keyboard commands. When disabled, the annunciators will be turned off and remain off until enabled again.

13.3.2 General Parameters

TESTS	Allowed values are 1 through 18
ERROR_LIMIT	Allowed values are 1 through 32767 or "-"
PASS_LIMIT	Allowed values are 1 through 32767 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32767 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

13.4 Preset Parameter Values for PDGI

C0_HOOK_UP_TO = SINGLE
C2_HOOK_UP_TO = OPEN
C3_HOOK_UP_TO = DEVICE
INITIALIZE_BOARD YES
C0_SCALE = 0
C2_SCALE = 0
C3_SCALE = 0
COLOR_TO_VDG = DARK
DATA_SELECTION = AUTO
USER_STRING =
ENABLE = KEYBOARD_ANNUNCIATORS
DISABLE =
TESTS = 1-18
ERROR_LIMIT = 5
PASS_LIMIT = -
REPORT = LOG
INHIBIT = PASSNUMBER,TESTNUMBER
SCALE = 0
MINUTES_LIMIT = -
ABBREVIATION = 4
AFTER_ERROR_LIMIT = ABORT

13.5 Mode Characteristics for PDGI

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

13.6 Individual PDGI Tests

(System Modes)		
Test 01	Basic Operator Interface Subsystem Operations Tests basic functions of the PIC.	SUB, MOD
Test 02	Execute PDGI Valid Commands Executes and checks valid commands for the subsystem.	SUB
Test 03	Generate IOCB Interrupts Exercises the seven implemented interrupt vectors.	SUB, MOD
Test 04	Verify Invalid CCB Detection Tests the Channel Control Block error-detection mechanisms for each channel.	SUB, MOD
Test 05	Illegal Command/Parameter Detection Checks error conditions (interrupts and status bits) for proper trapping of invalid commands.	SUB, MOD
Test 06	IOCB Chaining and Pause Verifies transfer pause and restart execution and the trapping of invalid commands.	SUB
Test 07	Random Transfers Exercises all available channels on a random selection basis (one channel for each pass).	SUB, MOD
Test 08	Verify IOCB Error Detection Tests the Input Output Control Block error-detection functions for each channel.	SUB, MOD
Test 09	Channel Transfers Exercises all available channels according to the channel parameter settings.	SUB, MOD
Test 10	Monitor Subsystem Status Continuously monitors and logs the device-status bits until the program is ABORTed. This test requires manual intervention, and must be the only test selected.	SUB, MOD

13.7 PDGI Alarm Categories

100 **Hardware Verification Test System** **March 2013**

14 SMART CONTROLLER MAGNETIC DISK TESTS (SCMD)

14.1 Smart Controller Subsystem Elements

The Smart Controller Magnetic Disk Subsystem elements include the Smart Peripheral Controller (SPC), the Enhanced Peripheral Display Generator (EPDG/EPDG2) or the Turbo Peripheral Display Generator (TPDG), up to seven magnetically recorded peripherals (namely, removable and fixed-media disk subsystems), and the SCSI-to-Ethernet Adapter.

14.2 Use Information

Restriction—When testing a dual function board (for example, EPDG, EPDG2, TPDG, etc.), only one of the following test systems can run on that dual function board at a time without errors occurring in the systems. The test systems are: KEYB, PDGI, SCMD, and XUSF.

The Format Test (21) may be used to reformat a drive or change the usable disk capacity without reformatting (**9 GB hard drive only**).

The contents of the Defect Data List produced by the Format Test (21) vary by device. For some devices, this list identifies the defective blocks on the disk drive. Other devices identify the number of good sectors on a track-by-track basis. Because the form and content of the defect data vary from vendor to vendor, the defect data display is of raw hex data.

You can display the defect data without reformatting the disk by running test 21 with `FORMAT DISABLED` and `DISPLAY_DEFECT_LIST` set for the list(s) wanted.

To test the SCSI-to-Ethernet Adapter, run Test 24 for single-end testing using one node, or Test 25 for end-to-end testing using two nodes; however, most setup parameters are for magnetically recorded peripherals and do not apply to the SCSI-to-Ethernet Adapter. Refer to the Test 24 and 25 test descriptions in subsection 14.6 for additional information.

14.3 Setup Parameters

14.3.1 Smart Controller Parameters

DEVICE_n Identifies which devices are to be tested and the device type at each SCSI device address (0-7).

Allowed values are

NOT_IN_USE—Not available for testing

WREN3_FMD—Wren III fixed media disk

IOMEGA_RMD—Iomega removable media disk

FMD—Generic type for Fixed Media Disk

RMD—Generic type for Removable Media Disk

MASTER_CONTROLLER—Used by test 20 only. This is the single designated "master" controller in the Controller-to-Controller test. This parameter must be defined to SCMD in each controller to be tested.

ETHERNET_MASTER—SCSI-to-Ethernet Adapter in the designated "Master" test mode.

ETHERNET_SLAVE—SCSI-to-Ethernet Adapter in the designated "Slave" test mode.

LIMIT_TESTING_TO_READ_ONLY

Preserves the original contents of the disk(s) being tested by blocking any "write" operations to the disk(s). See the next parameter for a more limited write restriction.

Allowed values are

YES— No data is written to the disk(s). Tests that depend on data being written to the disk(s) are skipped. Read operations are performed, but no data comparisons are made.

NO— Allows the tests to write to the disks, thus destroying original disk contents.

CAUTION

The parameters LIMIT_TESTING_TO_READ_ONLY and AVOID_WRITING_TO_RESERVED_DISK_AREA do **not** inhibit writing to disk by the Utility Test (test 23).

AVOID_WRITING_TO_RESERVED_DISK_AREA

Preserves only the four logical blocks (8-11) reserved for the "boot" record on each of the disks being tested. See the previous parameter for full write restriction.

Allowed values are

YES— The program does not write to logical blocks 8-11, and the "Boot Load" test (Test 16) is skipped.

NO— Allows write operations to any logical block thus destroying the original disk contents.

CLEAR_DEFECT_LIST Used by test 21 to control treatment of the Growth and Certification defect lists. The Certification list identifies Logical Block Addresses (LBA) that fail during a disk certification operation. The Growth list identifies bad LBAs reassigned during normal use.

Allowed values are

YES— The Format test discards the existing Growth and Certification defect lists and builds a new Certification defect list.

NO— The Format test keeps both defect lists and adds to the Certification defect list any new defective blocks found during its certification phase.

DISPLAY_DEFECT_LIST

Used by tests 21 and 22 to control whether either or both of the Growth and Certification defect lists are displayed. Not all selections are valid on all drives. When disk formatting is enabled, the specified list is displayed both before and after formatting; when disk formatting is disabled, the specified list is displayed and test 21 terminates.

Allowed values are

NONE—No defect data is displayed by the Format test.

PRIMARY—Displays only the Certification defect list. This option is not valid for IOMEGA or RMD drives.

GROWTH—Displays only the Growth defect list. This option is not valid for WREN III or FMD drives.

BOTH—Displays all defect data returned from the device.

MASTER_CONTROLLER_NODE_NUMBER

Used by test 20 only. This is the LCN node number (0-127) of the "master" controller in the Controller-to-Controller test. This parameter must be defined to SCMD in each controller to be tested.

ENABLE or DISABLE Allows the operator to enable or disable specified functions associated with execution of certain tests.

Allowed values are

FORMAT—When enabled, the Format test (21) formats any disks specified in **DEVICE_0** through **DEVICE_7**, thereby destroying **all** original data on the disk(s).

CHANGE_CAPACITY_ONLY—This parameter only applies to 9 GB hard drives (ST39175LW, ST39236LW or later). When **CHANGE_CAPACITY_ONLY** is enabled and **FORMAT** is disabled, the Format test (21) will change the usable disk capacity without reformatting the disk. This operation takes one or two seconds to complete. The new Last LBA Value is specified by the parameter **VERIFY_DISK_LAST_LBA_VALUE**.

MANUAL_INTERVENTION_TESTS—When enabled, allows test 18 to run.

QUICK_SURFACE_TEST—When enabled, the Surface Check Test (19) uses only one "worst case" pattern in verifying the disk(s). When disabled, Test 19 uses two patterns.

COMPARE_SURFACE_TEST_DATA—When enabled, compares the data read in the Surface Check Test (19). When disabled, the program relies on the hardware to detect and indicate data errors. The default is **DISABLED**.

REASSIGN_DEFECTIVE_BLOCKS—When enabled, tests 19 and 22 can reassign defective blocks, but only if write operations are permitted (see the option **LIMIT_TESTING_TO_READ_ONLY**).

RESET_SCSI_BUS—When enabled, causes the SCSI bus to be reset each time the program is started and when test 8 runs. Note that this option can cause unexpected interrupts and aborted transfers if two or more controllers are on the same SCSI bus. The default is **ENABLED**.

CONTROLLER_TO_CONTROLLER_TESTING—When enabled, limits testing to "controller only" tests (i.e, tests 1-7, 11, 12, and 20). Must be enabled in SCMD in each controller to be tested by test 20.

UTILITY_HELP—When enabled, the Utility Test (23) displays operation information to support the user.

VERIFY_DISK_CAPACITY—When enabled, causes the program to verify the last Logical Block Address (LBA) on each of the disk drives. Assuming 256 byte blocks, the last LBA is:

WREN III—556,991

IOMEGA 20 MB—83,663

ST3283N—864,410

The default is DISABLED.

DISPLAY_DRIVE_ID_AND_CAPACITY—When enabled, causes the drive ID, from the INQUIRY command, and the block size and last logical block address to be displayed on starting the test. This option will be disabled by the program following the display. The default is ENABLED.

UDO or UDONT

Used to enable (UDO) or disable (UDONT) Utility Test (23) functions.

Allowed values are

COMPARE_DATA—When enabled (UDO), the Utility Test compares data following every "input" operation.

ERR_DISPLAY—When enabled (UDO), the Utility Test displays all available information when an error occurs. This does not apply to data comparison errors.

STATUS_DISPLAY—When enabled (UDO), the Utility Test unconditionally displays all available information about an operation upon its conclusion.

DISPLAY_DATA—When enabled (UDO), the Utility Test (23) will display the data read from any of the operations.

ENTRY

Specified by the operator when entering information requested by tests 22 and 23. May be specified as ENT or ENTRY. This parameter accepts any string of 1 to 40 alphanumeric characters.

BLOCK_SIZE_IN_BYTES

Establishes the expected block size, in bytes, for the disks being tested. This size is verified on each disk before any tests are run. It also establishes the block size used by test 21 for formatting the disks.

VERIFY_DISK_LAST_LBA_VALUE

Entered by the operator to specify the last Logical Block Address (in decimal) for the FMD or RMD drives. This parameter is used when VERIFY_DISK_CAPACITY is enabled, or when CHANGE_CAPACITY_ONLY is enabled.

OTHER_NODE_ETHERNET

Contains the SCSI-to-Ethernet Adapter address of the partner node in Test 25.

14.3.2 Display-Only Parameters

The following "display only" parameters cannot be changed by the operator, but they can be displayed either by typing the parameter name or in the STAtus display. These are decimal values that are filled in by the program once each run just before running any test that accesses the disk(s).

PEEK_DEVICE Contains the SCSI device address of the disk to which the next two values apply.

PEEK_LAST_LBA Contains the last Logical Block Address (in decimal) of the disk identified by PEEK_DEVICE.

PEEK_LAST_LBA_SIZE This parameter contains the size in bytes of the last Logical Block of the disk identified by PEEK_DEVICE.

14.3.3 General Parameters

TESTS Allowed values are 1 through 28

ERROR_LIMIT Allowed values are 1 through 32767 or "-"

PASS_LIMIT Allowed values are 1 through 32767 or "-"

REPORT Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED

INHIBIT Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED

SCALE Allowed values are 0 through 999

MINUTES_LIMIT Allowed values are 0 through 32767 or "-"

ABBREVIATION Allowed values are 0 through 4

AFTER_ERROR_LIMIT Allowed values are ABORT or HALT

14.4 Preset Parameter Values for SCMD

DEVICE_0 = NOT_IN_USE
DEVICE_1 = NOT_IN_USE
DEVICE_2 = NOT_IN_USE
DEVICE_3 = NOT_IN_USE
DEVICE_4 = NOT_IN_USE
DEVICE_5 = NOT_IN_USE
DEVICE_6 = NOT_IN_USE
DEVICE_7 = NOT_IN_USE
LIMIT_TESTING_TO_READ_ONLY = YES
AVOID_WRITING_TO_RESERVED_DISK_AREA = NO
CLEAR_DEFECT_LIST = NO
DISPLAY_DEFECT_LIST = NONE
MASTER_CONTROLLER_NODE_NUMBER = 0
ENABLE = RESET_SCSI_BUS, DISPLAY_DRIVE_ID_AND_CAPACITY
DISABLE = FORMAT, CHANGE_CAPACITY_ONLY,
 MANUAL_INTERVENTION_TESTS, QUICK_SURFACE_CHECK,
 COMPARE_SURFACE_TEST_DATA, REASSIGN_DEFECTIVE_BLOCKS,
 CONTROLLER_TO_CONTROLLER_TESTING, UTILITY_HELP,
 VERIFY_DISK_CAPACITY
UDO = COMPARE_DATA, ERR_DISPLAY, DISPLAY_DATA
UDONT = STATUS_DISPLAY
ENTRY =
BLOCK_SIZE_IN_BYTES = 256
VERIFY_DISK_LAST_LBA_VALUE =
OTHER_NODE_ETHERNET = FFFF, FFFF, FFFF
TESTS = 1-15,17-20
ERROR_LIMIT = 5
PASS_LIMIT = -
REPORT = LOG
INHIBIT = PASSNUMBER, TESTNUMBER, SUSPENDED
SCALE = 0
MINUTES_LIMIT = -
ABBREVIATION = 4
AFTER_ERROR_LIMIT = ABORT

14.5 Mode Characteristics for SCMD

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

14.6 Individual Smart Controller Tests

(System Modes)

- | | | |
|---|--|----------|
| Test 01 | Basic Controller Operations | SUB, MOD |
| Verifies that the program can communicate with the controller and that the controller can correctly perform basic functions. | | |
| Test 02 | Test DMA Write/Read Commands | SUB |
| Verifies that the controller DMA functions are operational by use of the DMA write and DMA read commands. It uses "walking ones," "walking zeros," all ones, all zeros, and random patterns each pass. | | |
| Test 03 | Verify "Display Paged Status" Commands | SUB, MOD |
| Verifies that all paged statuses can be read and, for those that are predictable, the contents are verified. | | |
| Test 04 | Interrupt Generation and Control | SUB, MOD |
| Verifies that the controller can generate and inhibit the three controller interrupts. | | |
| Test 05 | Invalid Commands | SUB, MOD |
| Verifies that the controller can detect and indicate the receipt of invalid commands. | | |
| Test 06 | Invalid CCB and IOCBs | SUB, MOD |
| Verifies that the controller can detect and indicate invalid CCBs in both the Initiator and Target IOCB chains. The Initiator IOCB chain is also tested for the detection and indication of invalid IOCBs. | | |
| Test 07 | Module Bus Error Detection | SUB |
| Verifies that the controller can detect, respond to, and recover from the following errors: parity error during a DMA read, parity error during a status bus write, and module bus "bus error" while the controller was the temporary bus master. | | |

Test 15 Exerciser Test

SUB, MOD, EXER

Verifies that there is no interaction between the controller disk subsystem and other concurrently running subsystems. One set of write/read operations (or read/read if writes are prohibited) is conducted on the available disks, using the following random parameters: disk selection, starting logical block address, transfer length (1-5 blocks), and data. This test is skipped if no disks are specified.

Test 16 Boot Load Operations

SUB

This test should not be run if there are any RNOS nodes on the network.

Verifies that the controller can correctly perform the "boot load" function on receipt of the BOOT command. Testing depends on the "type" pinning on the controller board, which can signify either "Winchester" or "floppy" disk compatibility. This test is skipped if no disks are specified, and no testing is conducted if there are any restrictions on writing to the disks.

Test 17 Abort and Stop Transfer Operations

SUB

Verifies that the controller can correctly terminate write and read operations in progress (without corrupting data) upon receipt of abort and stop commands. This test is skipped if no disks are specified.

Test 18 Removable Media "Media Status" Checks

SUB, MOD

Verifies that the removable media disks can indicate the removal of the removable media cartridge and that they will indicate a "write protect" violation and preserve data on the media when write protection is enabled. This test requires that the parameter MANUAL_INTERVENTION_TESTS is enabled and that removable media disks are specified.

Test 19 Surface Check

SUB, MOD

Verifies the entire surface on each of the specified disks. Any defective Logical Blocks are "reassigned" as they occur if REASSIGN DEFECTIVE BLOCKS is enabled. If the QUICK SURFACE CHECK option is enabled, only one "worst case" data pattern (\$DB6D) is used, otherwise two patterns (\$2492 and \$DB6D) are used. If COMPARE SURFACE TEST DATA option is enabled, the data read is compared by the test program. If COMPARE SURFACE TEST DATA option is disabled, the data read is not compared (the hardware checksum mechanism would be relied on to detect and report data error conditions). If the LIMIT TESTING TO READ ONLY option is set to YES, only "read" operations will take place with no data comparisons. This test is skipped if no disks are specified. Approximate run times, in minutes, for test 19 when running on an HMPU, HPK2, or K2LCN based systems are

Write/Read with

	Disable Quick Enable Compare	Disable Quick Disable Compare	Enable Quick Enable Compare	Enable Quick Disable Compare	Read Only
WREN III	36 min	31 min	18 min	15 min	9 min
IOMEGA	9 min	8 min	4 min	3 min	2 min
ST3283N	60 min	35 min	29 min	17 min	16 min
ST3600	1 hr 46 min	56 min	53 min	28 min	10 min
ST31230	4 hr 18 min	1 hr 48 min	2 hr 32 min	54 min	19 min
ST32171	7 hr 5 min	3 hr 35 min	3 hr 32 min	1 hr 47 min	39 min
MD150	1 hr 1 min	29 min	23 min	15 min	8 min
ZIP Z100si	38 min	16 min	14 min	8 min	4 min

Test 20 Controller to Controller Transfer Test

SUB, MOD

Verifies that two or more controllers on the same SCSI bus can communicate with each other. The controllers can reside in the same node or in different nodes on the same LCN. The primary purpose of this test is to allow controller board testing in the factory without the need for disks. Note: Only use SCSI addresses 0 and 1 or 6 and 7 for this test.

Parameters that must be set up in every copy of SCMD that is to participate in this test are DEVICE_n (MASTER_CONTROLLER),

CONTROLLER_TO_CONTROLLER_TESTING (ENABLE), and

RESET_SCSI_BUS (DISABLE in all cases).

Test 21 Format Disk

SUB

If the FORMAT option is enabled, formats all specified disks, unless prohibited by LIMIT_TESTING_TO_READ_ONLY = YES or AVOID_WRITING_TO_RESERVED_DISK_AREA = YES. **Note that formatting a disk destroys all original data on the disk.** The disk drives can be commanded to "clear" or "keep" the existing lists of defective logical blocks (see CLEAR_DEFECT_LIST). Lists of defective Logical Block Addresses maintained by each disk drive are displayed before and after the test runs if DISPLAY_DEFECT_LIST is selected (independent of whether FORMAT is enabled or disabled). The defect data display is in raw hex format because form and content of defect data vary from vendor to vendor. **This must be the only test selected to run.**

If the FORMAT option is disabled, the CHANGE_CAPACITY_ONLY option is enabled, and 9 GB hard drive (ST39175LW, ST39236LW or later), then the usable disk capacity will be changed **without reformatting the disk**. This operation takes one or two seconds to complete. The new Last LBA Value is specified by the parameter VERIFY_DISK_LAST_LBA_VALUE

Format Disk

ST3600	13 min
ST31230	21 min
ST32171	41 min
MD150	17 min
ZIP Z100si	9 min

Test 22 Manually Reassign Bad LBA

SUB

When the value of parameter LIMIT_TESTING_TO_READ_ONLY is NO, allows the test operator to manually reassign bad logical blocks. The operator is prompted by a series of questions. Unless the value of the parameter DISPLAY_DEFECT_LIST is NONE, the defect data is displayed both before and after a device is changed. Because the defect data list consists of logical (not physical) addresses, the same logical block number can appear more than once.

This must be the only test selected to run.

Other parameters that directly affect this test are DEVICE_n, which defines the device type; ENTRY, which allows switching of devices and input of the bad logical block address (in decimal); and ENABLE of REASSIGN_DEFECTIVE_BLOCKS, which allows reassignment. The ABORT command can be used to exit this test at any time.

Test 23 Utility Test

SUB

Allows a knowledgeable technician or engineer to specify IOCB operations to be conducted on specified device(s). **Note that original data will be destroyed on any disks written to.** A menu of requests that require specific responses guides you through the selection process. Options available are UTILITY_HELP (enable/disable), USE (used instead of “BUILD,” allows use of previously built IOCBs), and UDO/UDONT selections. **This must be the only test selected to run.**

Test 24 SCSI-to-Ethernet Adapter – Single-End Test

SUB

This test verifies that the SCSI-to-Ethernet Adapter is present, can recognize local commands, and the SCSI bus interface is functional. Test 24 does not loop back data. The ability to transfer data on the Ethernet cable is not tested. The parameter “DEVICE_n” must be set to either “ETHERNET_MASTER” or “ETHERNET_SLAVE.” This parameter is not used except to specify the SCSI device address. Test 24 reads and displays the SCSI-to-Ethernet Adapter’s physical address and the firmware revision for the node during the first pass only.

Test 25 SCSI-to-Ethernet Adapter – End-to-End Test

SUB

This test verifies communication between two nodes that use SCSI-to-Ethernet Adapters to interface to an Ethernet LAN. This is accomplished by sending a random data block from the “Master” node to the “Slave” node where it is compared. The same data block is then sent back to the “Master” node and compared again. The parameter “DEVICE_n” must be set to either “ETHERNET_MASTER” or “ETHERNET_SLAVE” in order to specify the SCSI device address and testing role. Test 25 reads and displays the node’s SCSI-to-Ethernet Adapter physical address and firmware revision during the first pass only. The unique Ethernet physical address value must be entered in the setup parameter “OTHER_NODE_ETHERNET” for each partner’s node before running Test 25 between the nodes. Start the Slave node first, and then the Master node within 60 seconds.

This test is intended for use by Engineering and performs write/read operations on the available disks during ESD testing. This test is skipped if no disks are specified.

Uses the disk subsystem in a manner similar to "on-line" operation by executing 100 sets of write/read operations (or read/read, if writes are prohibited) on the available disks, using the following random parameters: disk selection, starting logical block address, transfer length (1-255 blocks), and data. This test is skipped if no disks are specified.

Uses the disk subsystem in a manner similar to "on-line" operation by executing 100 sets of write/read extended operations (or read/read extended, if writes are prohibited) on the available disks, using the following random parameters: disk selection, starting logical block address, transfer length (1-255 blocks), and data. This test is skipped if no disks are specified.

14.7 SCMD Alarm Categories

<u>Alarm Range</u>	<u>Most likely causes</u>
0100 - 0799	Controller failure
0800 - 1099	Device or SCSI bus
1100 - 1299	Controller failure
1300 - 1999	Device or SCSI Bus
2000 - 2099	Controller failure
2100 - 2199	Device or SCSI Bus
2200 - 2299	Device or SCSI Bus
2300 - 2399	Device or SCSI Bus
2400 - 2499	Device or SCSI Bus
2500 - 2599	Device or SCSI Bus
2600 - 2899	Device or SCSI Bus

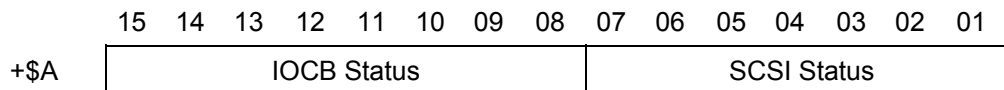
14.8 SCSI Status Definitions

The IOCB Status word displays the results of all I/O transactions between the SCSI controller board (SPC, EPDG, etc.) and a device on the SCSI bus. The IOCB Status word is divided into three parts; an IOCB Status byte and a SCSI Status byte.

The IOCB Status byte indicates the success or failure of (1) attempting to establish communication with the specific device on the SCSI bus, and (2) completing the SCSI bus I/O phases appropriate for the operation specified in the IOCB suffix (for example, the SCSI Command Descriptor Block) that is appended to the IOCB. The SCSI Command Descriptor Blocks are defined in the ANSI SCSI standard and the device vendor's product manual.

The SCSI Status byte reflects the success or failure of the targeted SCSI device to perform the operation that is specified in the IOCB suffix.

The format of the IOCB Status word and the code definitions follow.



14.8.1 IOCB Status

Valid IOCB Status codes are defined below. When appropriate, the SCSI Status codes are also defined.

IOCB Status Code

Definition

IOCB Complete (\$80) A value is entered into the IOCB Status byte upon the completion of the SCSI bus transaction. If the SCSI Status byte is zeros in bytes 1-4, the operation was completed without an error.

The Status byte is sent from the target to the initiator during the status phase of the termination of each command as listed below, unless the command is cleared by an ABORT message, by a BUS DEVICE RESET message, or by a “hard” RESET condition.

Status Byte								
Bit	7	6	5	4	3	2	1	0
Byte	R	V	V	Status Byte Code				V

R = Reserved bit

V = Vendor unique bit

<u>Status Byte Codes</u>								Status
7	6	5	4	3	2	1	0	
R	V	V	0	0	0	0	V	Good
R	V	V	0	0	0	1	V	Check Condition
R	V	V	0	0	1	0	V	Condition Met/Good
R	V	V	0	0	1	1	V	Reserved
R	V	V	0	1	0	0	V	Busy
R	V	V	0	1	0	1	V	Reserved
R	V	V	0	1	1	0	V	Reserved
R	V	V	0	1	1	1	V	Reserved
R	V	V	1	0	0	0	V	Intermediate/Good
R	V	V	1	0	0	1	V	Reserved
R	V	V	1	0	1	0	V	Intermediate/Condition Met/Good
R	V	V	1	0	1	1	V	Reserved
R	V	V	1	1	0	0	V	Reservation Conflict
R	V	V	1	1	0	1	V	Reserved
R	V	V	1	1	1	0	V	Reserved
R	V	V	1	1	1	1	V	Reserved

Good	This status indicates that the target has successfully completed the command.
Check Condition	Any error, exception, or abnormal condition that causes Sense data to be set, will cause a Check Condition status. The Request Sense command should be issued following a Check Condition status to determine the nature of the condition.
Condition Met	The search data commands return this status whenever a search condition is satisfied. This status does not break a chain of linked commands. The logical block address of the logical block that satisfies the search can be determined with a Request Sense command.
Busy	The target is busy. This status is returned whenever a target is unable to accept a command from an initiator. The normal initiator recovery action is to issue the command again at a later time.
Intermediate	This status is returned for every command in a series of linked commands (except the last one) unless an error, exception, or abnormal condition causes a Check Condition status or a Reservation Conflict status to be set. If this status is not returned, the chain of linked commands is broken. No further commands in the series are executed.
Reservation Conflict	This status is returned whenever a SCSI device attempts to access a logical unit or an extent within the logical unit that is reserved for that type of access to another SCSI device.
Select Timeout (\$81)	This value is entered in the IOCB Status byte if the SPC could not select the specified SCSI device within 2 seconds.
Parity Error (\$82)	Odd parity is carried on the SCSI bus. This value is entered in the IOCB Status byte if a parity error occurs on the SCSI bus.

Buffer Overflow (\$83)	If the number of bytes received from the SCSI bus exceeds the buffer size that is specified elsewhere in the IOCB, this value is entered in the IOCB status byte.
Other Than Send CDB Received when Selected as a Target (\$84)	The SPC to SPC communication is limited to only the use of the send command descriptor block (CDB) and associated data. This value is entered into the IOCB Status byte if the SPC is selected as a target and the CDB received is not a send CDB. This value is only valid in the target IOCB chain.
SCSI Abnormal Operation (\$85)	This value is entered into the IOCB Status byte if the specified operation was started but did not complete for a reason other than defined above. The SCSI controller IC generates a termination interrupt. The status from the SCSI controller IC that is associated with the interrupt is in the SCSI Status byte. The codes are defined below. When this status is received, the SCSI bus may be “hung,” and software will have to reset the SCSI bus with the reset bus command to continue.

SCSI Status Codes, Terminate Interrupt

7	6	5	4	3	2	1	0	Definition
0	1	0	0	0	0	0	0	An invalid command was issued.
0	1	0	0	0	0	0	1	An unexpected disconnect caused a command to terminate.
0	1	0	0	0	0	1	0	A Time-out occurred during a Select or Reselect command.
0	1	0	0	0	0	1	1	A parity error caused a command to terminate (ATN is not asserted).
0	1	0	0	0	1	0	0	A parity error caused a command to terminate (ATN is asserted).
0	1	0	0	0	1	0	1	The logical address exceeded the disk boundaries.
0	1	0	0	0	1	1	0	The wrong target device reselected the SCSI controller IC.
0	1	0	0	0	1	1	1	An incorrect message, status, or command type was received.
0	1	0	0	0	M	C	I	An unexpected information phase was requested. MCI = Message, command, and I/O defines the type of information being requested. See below.

A SCSI Status code associated with a Terminate interrupt can contain an MCI subcode to denote the requested SCSI bus phase. The subcodes are defined in the ANSI SCSI standard.

MCI Code Definitions

Message	Command	I/O	Bus Phase
0	0	0	Data out of target.
0	0	1	Data in from target.
0	1	0	Command to target
0	1	1	Status from target
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Message out from target
1	1	1	Message in from target

14.8.2 Sense Key Descriptions

When the IOCB status is \$80xx, and the SCSI status byte is \$02 (check condition as in \$8002), error codes are available in response to the Request Sense command. In this case, the error is defined by specifying both a Sense Key and a Sense Error code. The code indicates a particular error in a given class. The Sense Key groups errors together by type.

Sense Key codes are described in

Table 1-1 The Sense Error codes are described in Table 14-2.

Table 14-1 — Sense Key Code Descriptions

Sense Key (Hex)	Description
0	No Sense: In the case of a successful command, no specific Sense Key information needs to be reported for the drive.
1	Recovered Error: Indicates the last command was completed successfully with some recovery action performed by the drive. Note that for some settings, the last command may have terminated before completing.
2	Not Ready: Indicates the addressed logical unit cannot be accessed. Operator intervention may be required to correct the condition.
3	Medium Error: Indicates the command was terminated with a nonrecoverable error condition, probably caused by a flaw in the medium or an error in the recorded data.
4	Hardware Error: Indicates the drive detected a nonrecoverable hardware failure while performing the command or during a self-test. This includes a SCSI interface parity error, controller failure, device failure, etc.
5	Illegal Request: Indicates an illegal parameter in the Command Descriptor Block or in additional parameters supplied as data for some commands (Format Unit, Mode Select, etc.) If the drive detects an invalid parameter in the Command Descriptor Block, it terminates the command without altering the medium. If the drive detects an invalid parameter in the additional parameters supplied as data, the drive may have already altered the medium.
6	Unit Attention: Indicates the drive may have been reset.
B	Aborted Command: Indicates the drive aborted the command. The initiator may be able to recover by retrying.
E	Miscompare: Indicates that the source data did not match the data read from the medium.

Table 14-2— Sense Error Code Descriptions

Sense Error (Hex)	Description	Sense Key
00	No additional information	No Sense
01	No index/address mark found	Hardware Error
02	No seek complete	Hardware Error
03	Write fault	Hardware or Recovered Error
04	Drive not ready	Hardware or Recovered Error
06	No track 0 found	Hardware Error
10	ID CRC or ECC error	Hardware, Medium, or Recovered Error
11	Unrecovered read error	Medium or Recovered Error
12	No address mark (on byte sync) found in ID field	Medium or Recovered Error
13	No address mark (on byte sync) found in data field	Medium or Recovered Error
14	No record found	Medium or Recovered Error
15	Seek positioning error	Hardware, Medium, or Recovered Error
17	Recovered read data with target's read retries (not with ECC)	Recovered Error
18	Recovered read data with target's error correction (ECC) applied	Recovered Error
19	Defect list error	Medium Error
1A	Parameter overrun	Illegal Request
1B	Synchronous transfer error	Hardware Error
1C	Primary defect list not found	Medium Error
1D	Compare error	Miscompare
20	Invalid command operation code	Illegal Request
21	Illegal logical block address. Address greater than the LBA returned by the read capacity data with PMI bit not set in CDB.	Illegal Request
24	Illegal use of a bit in CDB	Illegal Request
25	Invalid LUN	Illegal Request
26	Invalid field in parameter list	Illegal Request
29	Power-on, reset, or bus device reset	Unit Attention
2A	Mode select parameters changed by another initiator.	Unit Attention

Sense Error (Hex)	Description	Sense Key
2B	Microcode download	Unit Attention
31	Medium format corrupted.	Medium Error
32	No spare defect location available	Medium Error
37	Rounded parameter	Recovered Error
3D	Invalid identify message	Illegal Request
43	Message reject error	Aborted Command
44	Internal controller error	Hardware Error or Not Ready
45	Selection/reselection failure	Aborted Error
47	SCSI interface bus parity error	Hardware Error
48	Initiator detected error	Aborted Command
49	Inappropriate or illegal message	Aborted Command
4E	Overlapped commands attempted	Aborted Command
80-8F	Correctable ECC (low nibble = length)	Recovered Error
90	Configuration error	Hardware Error
A0	Self-test error (program ROM)	Hardware Error
A1	Self-test error (processor RAM)	Hardware Error
A2	Self-test error (buffer RAM)	Hardware Error
A3	Self-test error (SCSI protocol)	Hardware Error
A4	Self-test error (DMA)	Hardware Error
A5	Self-test error (Disc sequencer)	Hardware Error
A6	Self-test error (Disc sequencer RAM)	Hardware Error
A7	Self-test error	Hardware Error
A8	Unable to read/write EEPROM	Hardware Error
A9	Read EEPROM directory checksum error	Hardware Error
AA	Incompatible EEPROM version #	Hardware Error
AB	Incompatible EEPROM reversion #	Hardware Error
AC	EEPROM data checksum error	Hardware Error
AD	Unknown or bad parameters in EEPROM	Hardware Error
AE	Wrong HDA/PCBA has been swapped	Hardware Error
AF	Wrong diskware	Software Error
B0	Servo command timeout	Hardware Error
B1	Servo command failure	Hardware Error
B2	Servo command rejected	Hardware Error

Sense Error (Hex)	Description	Sense Key
B3	Servo interface failure	Hardware Error
B4	PLO unlock	Hardware Error
B5	Internal servo error	Hardware Error
B6	Seek recovery	Hardware Error
C0	Defect list full	Medium Error
C1	Failure writing G list	Hardware Error
C2	Extended EEPROM writable life cycle limit. The data integrity cannot be assured. Extended Sense data byte 3 to 6 contains the accumulative write cycle counter. Note that if the specified EEPROM life span is exceeded, the drive may not function properly.	Hardware Error
C3	Illegal entry to G list	Medium Error
C4	Duplicate entry to G list	Medium Error

15 Application Module Redundancy Tests (AMRT)

15.1 AM Redundancy Elements

The normal Application Module Redundancy Subsystem includes two Application Modules that are cable-connected through AMR or EAMR boards. Limited testing can be done on a single AM with an AMR or EAMR board.

The Application Module(s) must contain either an HMPU processor board with an AMR or EAMR board, or a K4LCN processor board with an EAMR board.

15.2 Use Information

AMRT tests can be used with the following hardware configurations.

Processor AMR/EAMR Function	AMRT Tests
HMPU AMR Single	1 through 7
HMPU EAMR Single	1 through 7
K4LCN AMR	Not supported
K4LCN EAMR	1-3, 5-7, and 13
HMPU AMR Master/HMPU AMR Slave	1 through 12
HMPU AMR Master/HMPU EAMR Slave	1 through 12
HMPU EAMR Master/HMPU AMR Slave	1 through 12
HMPU EAMR Master/HMPU EAMR Slave	1 through 12
K4LCN AMR Master/K4LCN AMR Slave	Not supported
K4LCN AMR Master/K4LCN EAMR Slave	Not supported
K4LCN EAMR Master/K4LCN AMR Slave	Not supported
K4LCN EAMR Master/K4LCN EAMR Slave	1-3 and 5-15
HMPU/K4LCN (Redundancy with mixed processor types)	Not supported

Tests 8, 9, 10, 11, and 12 can be used for dual board configurations only.

The AMRT-specific parameters are required because AMRT is normally a 2-node test program and cannot be set up without knowledge of whether the "other node" exists.

When performing master/slave tests, tests to be run can be specified in only the master node. The slave node will follow whatever is specified in the master. The roles of master and slave can be reversed when all no tests are being performed.

When performing master/slave tests, the tests must be started with the NETWORK command (see the Test System Executive) or timeouts will occur.

15.3 Setup Parameters

15.3.1 AMRT-Specific Parameters

O_NODE	Identifies the "other node's" address. If there is no backpanel connector to another node, use the node number for this node. Allowed values are 0-127.
O_SLOT	Identifies the slot number of the other node's AMR or EAMR that the backpanel is connected to. If there is no backpanel connector to another node, then use the slot number of this node's AMR or EAMR. Allowed values are 2-10.
ROLE	Specifies the role type. When there is a backpanel connector to another node, one of the nodes must be the MASTER and the other the SLAVE. If there is no backpanel connector to another node, the role must be SINGLE. Allowed values are MASTER, SLAVE, SINGLE.

15.3.2 General Parameters

TESTS	Allowed values are 1 through 15
ERROR_LIMIT	Allowed values are 1 through 32767 or "-"
PASS_LIMIT	Allowed values are 1 through 32767 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 1 through 32767 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

15.4 Preset Parameter Values for AMRT

O_NODE = 1

O_SLOT = (the slot number of the AMRT's AMR/EAMR is used as an initial value)

ROLE = UNKNOWN

TESTS = 1-15

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER, SUSPENDED

SCALE = 0

MINUTES_LIMIT = -

ABBREVIATION = 3

AFTER_ERROR_LIMIT = ABORT

15.5 Mode Characteristics for AMRT

System Mode	Programs Running	Tests Running	Delay Between Tests
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

15.6 Individual AM Redundancy Tests

(System Modes)

Test 01 Reset AMR/EAMR Board SUB, MOD

This test first clears all flags in the CONFIGSTAT register, then issues a reset command, verifies the reset, issues a remove reset command, then verifies the reset removal.

Test 02 Verify Registers SUB, MOD

Verifies functionality of the register set on the AMR/EAMR boards so far as is possible because of the nature of the operation of the registers.

Test 03 Check Modes of Operation SUB, MOD

Writes to the command register to place the AMR/EAMR board in either primary or secondary mode, then reads the configuration status register to verify proper mode.

Test 04 Parity Error Test SUB

Sends bad parity data over the module bus to see that the AMR/EAMR boards detect the errors and take appropriate action. No parity error is generated over the front edge HMPU-AMR/EAMR interconnecting bus. This test can be run only on configurations with an HMPU processor board.

Test 05 Verify Single-byte Cleanpoint Package Transfers SUB, MOD

Writes a cleanpoint package, using loopback mode, to verify transfer integrity. Checks are made for expected/unexpected interrupts and the packets transferred are checked for correct command bits, size bits, address, and data. The check ends with a test of the Stop Stores command and its interrupts.

Test 06 Verify Single-word Guaranteed Store Transfers SUB, MOD

Writes a Guaranteed Store package, using loopback mode, to verify transfer integrity. Checks are made for expected/unexpected interrupts, and the packets transferred are checked for correct command bits, size bits, address, and data.

- Test 07 Guaranteed Store Transfer and Buffer Full SUB
- Writes two 8 k Guaranteed Store packages consisting of long word random data, using loopback mode, to verify package transfer, buffer full, and AMRT buffer integrity. Checks are made for expected interrupts, and the packets transferred are checked for correct command bits, size bits, address and data.
- Test 08 Verify Masking, External Address, and Cleanpoint SUB, MOD
- Writes a Cleanpoint package to verify transfer integrity between two boards. Checks are made for interrupt masking, expected/unexpected interrupts, and the packets transferred are checked for correct command bits, size bits, address, and data. This test can be run only for dual board configurations.
- Test 09 Verify Handshaking Signals SUB
- Verifies the operation of the various handshaking signals sent between the two AMR/EAMR boards. This test can be run only for dual board configurations.
- Test 10 Stores Across Clean Point Boundaries SUB, MOD
- Writes long word Guaranteed Store packets to verify the integrity over the backpanel of two AMR/EAMRs. The packets transferred are checked for correct command bits, size bits, address and random data. This test can be run only for dual board configurations.
- Test 11 Verify Random Parameters SUB, MOD
- Writes random data patterns of random data size and lengths in various combinations of clean point packages and guaranteed store packets and performs primary/secondary switching. A check is made that no half-full interrupt is received, and the packets transferred are checked for correct command bits, size bits, address, and data. The transfer is performed on both AMR/EAMR buffers. This test can be run only for dual board configurations.
- Test 12 Exerciser Test SUB, MOD, EXER
- Writes a Clean Point or a Guaranteed Stores package of random data patterns in random sizes and lengths. The maximum store length is hardware dependent and is 8 K packets for the HMPU kernel mode and 32 K packets for the K4LCN kernel mode. The K4LCN kernel mode is used for the dual K4LCN/EAMR hardware configuration. All other supported dual hardware configurations use the HMPU kernel mode. The packets transferred are checked for correct command bits, size bits, address, and random data. This test can be run only for dual board configurations.
- Test 13 EAMR Guaranteed Store Transfer and Buffer Full SUB
- Writes two sets of 32 K Guaranteed Stores packets that consist of long word random data that uses loopback mode to verify package transfer, buffer full, and EAMR buffer integrity. Checks are made for the expected interrupts, and the packets transferred are checked for the correct command bits, size bits, address, and data. This test can be run only on configurations with a K4LCN processor board.

Test 14 EAMR in K4LCN Kernel Board

SUB, MOD

Writes random data patterns of random data sizes and lengths in various combinations of Clean Point packages and Guaranteed Store packets and performs Master/Slave switching. A check is made that no half-full interrupt is received, and the packets transferred are checked for the correct command bits, size bits, address, and data. The transfer is performed on both EAMR buffers. This test can be run only on configurations with a K4LCN processor board.

Test 15 Mixed Kernel Test

SUB, MOD

Writes a Clean Point package to verify mixed kernel detection between two boards. This test can be run only on dual board configurations with a 68040 processor.

15.7 AMRT Alarm Categories

The AMRT tests are designed to verify correct operation of the AMR or EAMR board, and the majority of alarms produced indicate a failure of either the master or slave board. Other failure messages indicate discrepancies between operator-entered information and set up data gathered by the test program.

Two general types of test messages are produced. Operator Alert messages contain text explanations of the detected problems, while Alarm messages are formatted as described in subsection 2.8.2 of the *Test System Executive* manual. Please note that the message text that accompanies some numbered Alarm messages can vary. Careful analysis of Alarm messages is required to determine which AMR or EAMR board has failed, because both master and slave boards often issue messages associated with the same error.

AMRT Operator Alert Messages

SLAVE DID NOT RESPOND TO TEST NUMBER CALL

Master node lost communication with slave node.

MASTER HAS NOT COMMUNICATED FOR 1 MINUTE

Slave node lost communication with master node.

INVALID TEST NUMBER

Operator has specified an invalid test number.

OTHER NODE ABORTED THIS NODE VIA MAIL

Partner node has aborted operation and messaged this node to abort.

INVALID NODE AND/OR SLOT NUMBER FOR SINGLE NODE TESTING

See explanation of parameters O_NODE and O_SLOT.

INVALID NODE AND/OR SLOT NUMBER FOR TWO NODE TESTING

Each node must point to the other's node and slot number.

NODE AND/OR SLOT NUMBER OUT OF RANGE

Acceptable node range is 0-127 and slot range is 1-15.

UNEXPECTED MAIL RECEIVED FROM MASTER

Occurred while waiting to become the primary.

DID NOT BECOME PRIMARY WITHIN ONE MINUTE

Occurred while waiting to become the primary.

SLAVE TEST NOT COMPLETED IN ONE MINUTE

Occurred while attempting to swap node modes.

MASTER/SLAVE/SINGLE MUST BE SPECIFIED

See explanation for parameter role.

INITIAL SETUP FAILED

AMRT detected a parameter error or a hardware configuration that is not supported.

INITIAL ENVIRONMENT CONF. INCORRECTLY

AMRT detected a communications error between the nodes or a hardware configuration that is not supported.

INVALID TEST MODE

None of the tests selected are valid for the system mode.

ILLEGAL COMMAND DETECTED

AMRT software error.

SOFTWARE ERROR – MEMORY ALREADY RESERVED

AMRT software error.

SOFTWARE ERROR – BUFFER SIZE WRONG

AMRT software error.

HARDWARE CONFIGURATION NOT SUPPORTED

See subsection 15.2 (hardware configuration).

SOFTWARE ERROR – NO PACKETS SPECIFIED

AMRT software error.

SOFTWARE ERROR – MORE THAN 3 PACKETS

AMRT software error.

MIXED KERNEL PAIR NOT SUPPORTED

See subsection 15.2 (hardware configuration)

16 Process Network Interface Tests (PNIF)

16.1 Process Network Interface Elements

The Process Network Interface consists of a PNI/PNM board pair in the context of a Network Interface Module.

16.2 Use Information

In order to avoid conflicts in NIM use, do not attempt to switch to PMEX operating mode (NMA command) while PNIF is active.

The individual PNIF tests are described in detail beginning at heading 16.6. A number of sequences that are used by several tests are described beginning at 16.7. These common sequences are

- Clear PNI (16.7.1)
- Reset PNI (16.7.2)
- Board Response Check (16.7.3)
- Execute Test IOCB (16.7.4)
- Execute a TBC Full Duplex Loopback Test IOCB (16.7.5)

The vast majority of the alarms generated by PNIF are as a result of a Board Response Check (see heading 16.7.3 for details). Other common alarms are self-test failure, command timeout, IOCB timeout, and board not responding.

Specifics of test failures are spelled out in the text portion of the alarm messages; therefore, running at Abbreviation level 2 or higher is recommended.

Test 21 requires operator intervention. Note that the Test/Normal switch on the edge of the PNI card should be left in Normal state except during execution of this test; other PNIF tests do not run well when it is set to Test state.

Test 22 must be set up for multiple passes to get the full exerciser effect.

The value of the parameter RDR_NODE (used by tests 22 and 23) can be changed at any time during operation; the change will take effect the next time either test is run.

16.3 Setup Parameters

16.3.1 PNIF-Specific Parameters

RDR_NODE	Controls the operation of tests 22 and 23. Allowed values are 0 (no node polling) and 1 to 64 (UCN node addresses).
-----------------	--

16.3.2 General Parameters

TESTS	Allowed values are 1 through 23
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 1 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

16.4 Preset Parameter Values for PNIF

RDR_NODE = 0

TESTS = 1-23

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER,SUSPENDED

SCALE = 100

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

16.5 Mode Characteristics for PNIF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

16.6 Individual Process Network Interface Tests

(System Modes)

Test 01 Reset

SUB, MOD

Puts the PNI through its Reset/Self-test sequence to verify that all self-tests are passed. Also checks the operation of the Hardware Status and Configuration Status registers.

Test actions are

- Clears the PNI (see heading 16.7.1 for details).
- Resets the PNI (see heading 16.7.2 for details).
- If the board passes its self-test, the test delays 250 ms then does a Board Response Check (see heading 16.7.3 for details). Note that the initial Board Response Check is done in the Reset sequence.

Test 02 Load and Verify Registers

SUB, MOD

Ensures the integrity of data paths between the memory and the PNI and checks the ability of the PNI to store and receive data from the PNI-CPU shared RAM. Data patterns are transferred over the module bus by loading and verifying the following hardware registers:

Operational Status register

Paged Status words

Command register

As much as possible, the rest of the domain also is monitored and is alarmed on unexpected changes.

Seven basic word patterns are used: \$FFFF, \$5555, \$AAAA, \$0144, \$4401, \$DBDB, and \$0000. Two additional patterns (a random pattern and its complement) are used for the Operational Status register and the Paged Status words.

NOTE

A random pattern cannot be used to test the command register because the effect of these patterns is unpredictable and sometimes undesirable.

Test 02 actions are

- Clears the PNI (see heading 16.7.1 for details).
- Loads and verifies the Operational Status register. For each data pattern, including the random patterns
 - Writes the data pattern to the Operational Status register.

- Reads the Operational Status register and verifies the pattern as part of a board response check (see heading 16.7.3 for details).
 - Does another Board Response Check after a 250 ms delay.
- Loads and verifies the four Paged Status words. For each data pattern, including the random patterns
 - Writes the data pattern to the Paged Status words.
 - Reads the Paged Status words and verifies the pattern as part of a board response check (see heading 16.7.3 for details).
 - Does another Board Response Check after a 250 ms delay.
- Loads and verifies the Command register. For each data pattern, except the random patterns
 - Writes the data pattern to the Command register (as a command).
 - Waits for command completion.
 - Reads the Command register and verifies the pattern as part of a board response check (see heading 16.7.3 for details). Note that the Command register's busy bit cannot be verified because it is cleared by the PNI on command completion.
 - Does another Board Response Check after a 250 ms delay.
 - Clears the PNI (see heading 16.7.1 for details) in case the Command register test did something strange to the PNI.

Test 03 Configuration Status & Subsystem Flag Control Registers SUB, MOD

Checks the operation of the Configuration Status Register (read) and the Subsystem Flag Control Register (write) and verifies that the rest of the PNI data remains unchanged. This test includes assuring that the PNI hardware responds correctly to valid commands to the Subsystem Flag Control Register (CPU writes to the Configuration Status register).

Except for the Reset Sequence commands (checked in another test), all valid commands are written to the Subsystem Flag Control register (addressable latch). This includes the "clear all flags" command (similar to but not the same as a Reset sequence). This test verifies that the appropriate changes occur to the first two hardware registers and that all the remaining registers and the PNI data remain unchanged. Steady state should be achieved immediately and persist for at least 250 ms. Note that this level of testing cannot verify the functionality associated with these commands, only the indications that result in the registers.

Test 03 actions are

- Clears all flags, consisting of
 - Enable (set) Reset flag
 - Disable (clear) Reset flag with the clear all flags bit set (\$41).
- Waits for self-test sequence to end.
- Does a Board Response Check (see heading 16.7.3 for details) to verify that the PNI data matches what is expected after a Reset.
- Delays for 250 ms, then do another Board Response Check.
- Aborts the PNI by writing an Abort command, without interrupt, to the PNI's Command register.
- Does another Board Response Check to verify that the PNI data matches what is expected after an Abort.
- Delays for 250 ms, then do another Board Response Check.
- For the CPU hardware interrupt flag
 - Enables interrupts
 - Disables interrupts
 - Enables interrupts
 - Disables interrupts
- For the Passed module test flag
 - Enables (passed) module test
 - Disables (failed) module test
 - Enables (passed) module test
- For the Synchronization timer enable flag
 - Enables Synchronization timer
 - Disables Synchronization timer
 - Enables Synchronization timer
 - Disables Synchronization timer
- For the CPU/external watchdog timer enable flag
 - Enables MCPU/external watchdog timer
 - Disables MCPU/external watchdog timer
- Resets the PNI (see heading 16.7.2 for details)
- If the self-tests pass, Clears the PNI (see heading 16.7.1 for details), otherwise aborts the test.

When enabling and disabling the subsystem control flags for Test 03, two Board Response Checks are done after every operation to check for the expected changes. The first check is done immediately after the operation and the second is done after a 250 ms delay. The only exception is made when enabling the CPU/external watchdog timer that delays to allow the watchdog to expire before doing the Board Response Check.

Test 04 Paged Status

SUB, MOD

Reads and verifies contents of the Paged Status registers. All paged status words in shared RAM are precoated, then checked for proper status after commands to read Paged Status for each of the following:

- Firmware version
- Device status
- Bus error address
- Current IOCB addresses 0-3
- Synchronization timer value

As much as possible, the rest of the domain also is monitored and is alarmed on unexpected changes.

Seven basic word patterns are used: \$FFFF, \$5555, \$AAAA, \$0144, \$4401, \$DBDB, and \$0000. Two additional patterns (a random pattern and its complement) also are used.

Test 04 actions are

- Clears the PNI (see heading 16.7.1 for details).
- For each Paged status and each of the data patterns:
 - Does a Board Response Check (see Section 16.7.3 for details) to make sure that the writes to the Paged Status words worked.
 - Reads the selected Paged Status word by writing a Paged Status command—without interrupt—to the PNI's command register. The particular paged status is specified in the command word's parameter field. The PNI should write the data to the Paged Status words in shared RAM, overwriting the precoated pattern.
 - Does a Board Response Check to verify that the command worked and that the selected paged status was written to the Paged Status words in shared RAM.
 - Does another Board Response Check to check for steadiness after a 250 ms delay.

Test 05 PNI Interrupts of CPU

SUB, MOD

Tests the PNI's board and priority related interrupt generation logic, including the logic that generates the Operational Status Word. The following interrupts are generated and checked:

- Board-related interrupt (Vector 0)
- Priority 0 related interrupt (Vector 1)
- Priority 1 related interrupt (Vector 2)
- Priority 2 related interrupt (Vector 3)
- Priority 3 related interrupt (Vector 4)

Only the board and priority-related interrupts are tested in Test 05. The Synchronization interrupt (Vector 5) is tested during the Synchronization timer test (Test 13). Although interrupt Vectors 6 and 7 are not used by the PNI, they still are checked to make sure that neither interrupt occurs during this test.

Test 05 actions are

- Clears the PNI (see heading 16.7.1 for details).
- Enables the CPU hardware interrupts by writing a word to the Subsystem Flag Control register specifying
 - CPU hardware interrupt flag
 - Enable/True
 - Do Not clear all flags
- Makes sure that the interrupts are enabled and everything else is okay by doing two Board Response Checks (see heading 16.7.3 for details); the first immediately, and the second 250 ms later.
- Generates a board-related interrupt by writing a NO OP command—requesting a command completion interrupt—to the PNI's command register.
- Verifies that the correct interrupt occurred and all else is okay by doing two Board Response Checks; the first immediately, and the second 250 ms later.

- For each priority (0-3) generates and verifies a priority-related interrupt by executing a NO OP IOCB and specifying a completion interrupt:
 - Starts the priority by writing a PRIORITY START command—without a command completion interrupt—to the PNI's command register. The parameter field of the command word specifies which priority is to start.
 - Causes the PNI to execute the NO OP IOCB by incrementing by one the number of IOCBs issued by PNIF in the priority's control structure (shared RAM).
 - Waits for IOCB completion or timeout.
 - If IOCB times out, generates an alarm saying so.
 - Regardless of the completion/timeout outcome, does two board response checks; the first immediately and the second after a 250 ms delay.

- Clears the PNI (see heading 16.7.1 for details) to disable the hardware interrupts, etc.

Test 06 Module Bus Error Detection

SUB

Checks the ability of the PNI to detect Module Bus errors by forcing these types of errors:

- Bus error (by forcing the PNI to DMA from a non-existent memory location)
- Parity error (generated by the CPU with different data patterns)

Test 06 actions are

- Clears the PNI (see heading 16.7.1 for details)
- Forces the PNI to DMA from a nonexistent memory location by
 - Stores a nonexistent memory address in the Priority 0 CCB.
 - Starts Priority 0 by writing a PRIORITY START command—without a command completion interrupt—to the PNI's command register. Priority 0 is specified by setting the command's parameter field to a 0.
 - Causes the PNI to execute the nonexistent IOCB by incrementing by one the number of IOCBs issued by PNIF in the priority 0 control structure (shared RAM). This in turn should cause a bus error when the PNI attempts to access the IOCB using DMA.
- Does two Board Response Checks to verify that the bus error occurred; the first check after a 50 ms delay, and the second 250 ms after that.
- Clears the Data Transfer Timeout indicator on the CPU that was set during the bus error transaction.
- Clears the PNI.

- Performs the parity error detection test—skipped if the CPU is a Hyper Kernel 2 (HPK2) board. This test checks the PNI's ability to detect module bus parity errors by using 16 data patterns consisting of a 1 "walking" from left to right within a word of 0s. Its actions are
 - Writes the selected data pattern to the PNI's command register with a bad parity bit.
 - Does two Board Response Checks to verify that the bus error occurred; the first check after a 50 ms delay and the second 250 ms after that.
 - Clears the Bus error indicator on the CPU board that was set during the parity error transaction.
 - Resets the PNI (see heading 16.7.2 for details) to clear the error.
 - If the PNI passes its self-tests, Test 6 clears the PNI (see heading 16.7.1 for details) to get the board back to a known state; however, if the PNI fails its self-tests, Test 6 is aborted.

Test 07 Hardware & Firmware Watchdog Timers**SUB, MOD**

Verifies the proper operation and expiration of the two watchdog timers on the PNI board

- The Hardware (or External) Watchdog Timer—used by the PNI to detect CPU failure.
- The Firmware (or Internal) Watchdog Timer—used by the PNI to detect failure of the PNI's local CPU and/or firmware.

The CPU/external watchdog is tested by making sure that it does not expire when kicked, expires when not kicked, and recovers after expiration. The PNI/internal firmware watchdog is tested by running the corresponding test IOCB provided by the PNI firmware.

To test the CPU/external watchdog timer's ability to not fail when kicked, Test 07

- Clears the PNI (see heading 16.7.1 for details).
- Enables the CPU/external watchdog timer by writing a word to the subsystem flag control register that specifies
 - CPU/external watchdog timer flag
 - ENABLE/TRUE
 - DO NOT Clear all flags
- Kicks the watchdog continuously for one second.

- Disables the CPU/external watchdog timer by writing a word to the subsystem flag control register that specifies
 - CPU/external watchdog timer flag
 - DISABLE/FALSE
 - DO NOT Clear all flags
- Delays 400 ms, then writes a NO OP command—without a command completion interrupt—to the PNI's command register.
- Does a Board Response Check (see heading 16.7.3 for details) to make sure that nothing failed.

To test the CPU/external watchdog timer's ability expire when not kicked, Test 07

- Enables the CPU/external watchdog timer by writing a word to the subsystem flag control register that specifies
 - CPU/external watchdog timer flag
 - ENABLE/TRUE
 - DO NOT Clear all flags
- Kicks the watchdog continuously for one second.
- Waits for 250 ms (enough time for the watchdog to expire).
- Writes a NO OP command—without a command completion interrupt—to the PNI's command register.
- Does a Board Response Check to make sure that the watchdog expired.

To test the PNI's ability to recover after a CPU/external watchdog timer expiration, Test 7

- Disables the CPU/external watchdog timer by writing a word to the subsystem flag control register that specifies
 - CPU/external watchdog timer flag
 - DISABLE/FALSE
 - DO NOT Clear all flags
- Does a Board Response Check to make certain that the failure cleared.
- Resets the PNI (see heading 16.7.2 for details) to get the board working again.

SUB, MOD

Test 09 CPU/PNI DMA Test Command**SUB**

Tests the DMA read and write functions of the PNI board by using the DMA test commands (DMA read and DMA write) provided by the PNI. The DMA read command reads a word from a fixed location in CPU RAM and writes it to the paged status words in PNI/CPU shared RAM. The DMA write command reads a word from the paged status words in PNI/CPU shared RAM and writes it to a fixed location in CPU RAM. Both commands are run a multiple number of times with random data patterns. All of the PNI data is checked after each run.

Test 09 actions are

- Clears the PNI (see heading 16.7.1 for details)
- For each of 100 random data patterns, Test 09
 - Coats the Paged Status words in shared RAM with the complement of the data pattern.
 - Writes the pattern to the DMA read test location in CPU RAM.
 - Writes a DMA READ TEST command—without requesting a command completion interrupt—to the PNI's Command register.
 - Does two Board Response Checks; the first immediately after command completion and the second after a 250 ms delay.
- For each of 100 random data patterns, Test 09
 - Coats the Paged Status words in shared RAM with the complement of the data pattern.
 - Writes the complement of the pattern to the DMA write test location in CPU RAM.
 - Writes the pattern to the Paged status word (in shared RAM) used by the DMA write test.
 - Writes a DMA WRITE TEST command—without requesting a command completion interrupt—to the PNI's Command register.
 - Does two Board Response Checks; the first immediately after command completion and the second after a 250 ms delay.

Test 10 PNI Local RAM

SUB, MOD

Tests the RAM local to the PNI using "walking 1" and "walking 0." This test is more extensive than the PNI local RAM test performed by the PNI during its self-test.

Test 10 actions are

- Clears the PNI (see heading 16.7.1 for details)
- Writes a TEST PNI LOCAL RAM command—without requesting a command completion interrupt—to the PNI's Command register. The command's parameter is set to a one to specify that the PNI should run the test once.
- Waits for 250 ms after command completion to allow the PNI time to settle.
- Does a Board Response Check (see heading 16.7.3 for details) to make sure that the test worked and that nothing else changed.

Test 11 PNI EPROM Read & Checksum

SUB, MOD

Reads the entire contents of the PNI's EPROM, verifies its checksum, and checks parity. This test is more extensive than the EPROM test performed by the PNI during its self-test.

Test 11 actions are

- Clears the PNI (see heading 16.7.1 for details)
- Writes a TEST PNI EPROM command—without requesting a command completion interrupt—to the PNI's Command register. The command's parameter is set to a one to specify that the PNI should run the test once.
- Waits for 250 ms after command completion to allow the PNI time to settle.
- Does a Board Response Check (see heading 16.7.3 for details) to make sure that the test worked and that nothing else changed.

Test 12 Token Bus Controller Host Interface & DMA**SUB, MOD**

Checks the operation of the Token Bus Controller's DMA and its host interface by using the TBC Host Interface test IOCB. During the test, the TBC reads data from the input part of the test buffer through DMA and writes it back to the output part of the test buffer, also through DMA. The output data is then compared against the input data. Seven fixed-byte patterns are used: \$FF, \$55, \$AA, \$14, \$41, \$DB, and \$00. Two additional patterns (a random pattern and its complement) also are used.

Test 12 actions are

- Clears the PNI (see heading 16.7.1 for details)
- For each of the nine data patterns, Test 12 sets up and executes the TBC Host Interface test IOCB as follows
 - Sets up the test IOCB with the address of the test buffer and the proper checksum.

NOTE

PNIF uses only one test buffer at a time, so only one test buffer is allocated per running PNIF test. The test buffer is preallocated at initialization so that each test can use it without the need to allocate or deallocate it.

- Coats the input part of the test buffer with the data pattern.
- Coats the output part of the test buffer with the data pattern's complement.
- Executes the test IOCB (see heading 16.7.4 for details).
- Does two Board Response Checks; the first immediately after IOCB completion and the second after a 250 ms delay. The test buffer is checked for the expected result as part of the Board Response Check.

Test 13 Synchronization Timer Test IOCB**SUB, MOD**

Verifies proper operation of the PNI's Synchronization Timer and related interrupt logic by using the Synchronization timer test IOCB. The timer is preset to a count (number of ticks) and then allowed to expire, which should generate an interrupt. A PNI firmware timer stops the test and reports an error if the timer fails to generate an interrupt within six seconds.

Test 13 actions are

- Clears the PNI (see heading 16.7.1 for details).
- Sets up the Synchronization timer test IOCB with a random number of ticks.
- Executes the test IOCB (see heading 16.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 14 Silence Detect Timer Test IOCB

SUB, MOD

Verifies proper operation of the PNI's Silence Detect Timer and related interrupt logic by using the Silence Detect timer test IOCB. The timer is preset to a count (number of ticks) and then allowed to expire, which should generate an interrupt. A PNI firmware timer stops the test and reports an error if the timer fails to generate an interrupt within six seconds.

Test 14 actions are

- Clears the PNI (see heading 16.7.1 for details).
- Sets up the Silence Detect timer test IOCB with a random number of ticks.
- Executes the test IOCB (see heading 16.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 15 Token Bus Controller Receiver Test IOCB

SUB, MOD

Executes the TBC's receiver test IOCB to check proper operation of the receiver's buffer handling. This test finds problems in the TBC controller and support circuitry. Failures of this test can be caused by either the Transmitter or Receiver since both are used.

Seven fixed-byte patterns are used: \$FF, \$55, \$AA, \$14, \$41, \$DB, and \$00. Two additional patterns (a random pattern and its complement) also are used. Each pass of this test executes the test IOCB three times for each data pattern using in turn

- Internal loopback
- Modem A loopback
- Modem B loopback

NOTE

At present, the PNI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in PNIF to provide for the eventuality of two separate modems.

Test 15 actions—for each data pattern and for each loopback type—are

- Clears the PNI (see heading 16.7.1 for details)
- Sets up the TBC Receiver test IOCB specifying the loopback type and the test buffer address.
- Sets up the receiver test buffer that controls the test with
 - A random number of frames between 1 and 5
 - A random frame length between 16 and 33
 - A random preamble length between 1 and 8
 - A random preamble byte pattern
 - The data pattern byte
 - Zeros out unused pieces of the test buffer
- Executes the test IOCB (see heading 16.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 16 Token Bus Controller Transmitter Test IOCB SUB, MOD

Executes the TBC's transmitter test IOCB to check for proper operation of the TBC's transmitter. This test finds problems in the TBC controller and support circuitry. Failures of this test can be caused by either the Transmitter or Receiver since both are used.

Each pass of this test executes the test IOCB three times using in turn

- Internal loopback
- Modem A loopback
- Modem B loopback

NOTE

At present, the PNI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in PNIF to provide for the eventuality of two separate modems.

Test 16 actions—for each loopback type—are

- Clears the PNI (see heading 16.7.1 for details)
- Sets up the TBC Transmitter test IOCB specifying the loopback type and the test buffer address.
- Sets up the transmitter test buffer that controls the test with
 - A random data length between 2 and 1024
 - A random number of frames between 1 and 15
- Executes the test IOCB (see heading 16.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 17 Token Bus Controller Internal Loopback Test IOCB SUB, MOD

Checks the TBC's data paths by executing TBC Full Duplex Loopback test IOCB using an internal loopback. This IOCB checks the proper operation of the TBC' serial interface and the FCS generation/checking logic. The TBC has separate transmit and receive checksum hardware, so normal testing verifies valid checksum detection. The data is read from the input portion of the test buffer, transmitted around the loopback path and is written back to the output portion of the test buffer.

Seven fixed-byte patterns are used: \$FF, \$55, \$AA, \$14, \$41, \$DB, and \$00. Two additional patterns (a random pattern and its complement) also are used.

All of the PNI interaction in Test 17 is performed by the sequence that executes a TBC Full Duplex loopback test IOCB (see heading 16.7.5 for details).

Test 18 Token Bus Controller External Loopback Test IOCB SUB, MOD

Checks the TBC's data paths by executing TBC Full Duplex Loopback test IOCB using Modem A/B (external) loopback. This IOCB checks the proper operation of the TBC' serial interface and the FCS generation/checking logic. The TBC has separate transmit and receive checksum hardware, so normal testing verifies valid checksum detection. The data is read from the input portion of the test buffer, transmitted around the loopback path and is written back to the output portion of the test buffer.

Seven fixed-byte patterns are used: \$FF, \$55, \$AA, \$14, \$41, \$DB, and \$00. Two additional patterns (a random pattern and its complement) also are used. Each pass of Test 18 executes the test IOCB once for Modem A and once for Modem B.

NOTE

At present, the PNI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in PNIF to provide for the eventuality of two separate modems.

All of the PNI interaction in Test 18 is performed by the sequence that executes a TBC Full Duplex loopback test IOCB (see heading 16.7.5 for details).

Test 19 Modem Jabber Test IOCB SUB, MOD

Uses the Modem Jabber test IOCB to test the PNM's ability to detect jabbering on the UCN Cables, and also checks the PNI's ALS diagnostics and operation under traffic conditions.

For each modem (A&B), Test 19 actions are

- Clears the PNI (see heading 16.7.1 for details)
- Sets up the Modem Jabber test IOCB specifying the the Modem (A or B) and including the proper checksum.
- Executes the test IOCB (see heading 16.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 20 Cable Switch Logic (static) Test IOCB

SUB, MOD

Checks the redundant cable switching logic and functions in the static mode (the modem is offline and there is no traffic on the cables) by using the Cable Switch Static test IOCB. The ability of the cables to be swapped and the silence detector to report silence are tested.

Each pass of Test 20 executes the test IOCB once for Modem A and once for Modem B.

NOTE

At present, the PNI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in PNIF to provide for the eventuality of two separate modems.

For each modem (A&B), Test 20 actions are

- Clears the PNI (see heading 16.7.1 for details)
- Sets up the Cable Switch Static test IOCB specifying the the Modem (A or B) and including the proper checksum.
- Executes the test IOCB (see heading 16.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 21 Test/Normal and Node Address Switches

UTILITY

Checks the PNI's ability to correctly read the Test/Normal switch and the node address switches on the PNI PWB. It runs in Utility mode because it requires manual intervention. It reports the state of the Test/Normal switch when the test is started and every time the switch is flipped. Also the rest of the PNI data is checked for unexpected changes.

Test 21 loops continuously until it receives a STOP or ABORT command. A warning alarm is issued on exit if the Test/Normal switch state is not Normal.

Test 21 actions are

- Clears the PNI (see heading 16.7.1 for details)
- Reads the Configuration status register to determine the initial state of the switch.
- Reports the initial state of the switch as an information message (not an alarm).

Subsequently, Test 21 continuously reads the state of the switch and reports each change. Following each state change, Test 21 does two Board Response Checks; the first immediately after reporting the state change and the second after a 250 ms delay. It also continuously monitors the test mode looking for a STOP or ABORT command from the operator. A warning alarm is displayed if the switch is set to "Test" when Test 21 exits.

Test 22 Exerciser

EXER

Execution of this test is controlled by the value assigned to the parameter RDR_NODE. If the value of RDR_NODE is the address of a valid UCN node, this test executes the same logic as Test 23.

If the value of RDR_NODE is zero, this test exercises the PNI's data paths, randomly alternating between using the Internal and Modem A and Modem B loopback options of the Full duplex loopback test IOCBs. A different random data pattern is used for each pass of Test 22.

NOTE

At present, the PNI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in PNIF to provide for the eventuality of two separate modems.

Test 22 does not loop internally; the operator must set up for multiple passes to get the full exerciser effect.

All of the PNI interaction in Test 22 is performed by the sequence that executes a TBC Full Duplex loopback test IOCB (see heading 16.7.5 for details).

Test 23 UCN Node Polling

SUB, MOD

This test is controlled by the value assigned to the parameter RDR_NODE. If the RDR_NODE value is zero this is a "do nothing" test.

If the RDR_NODE value is the address of a valid UCN node, the UCN node is polled and the results are checked for validity. This test expects the selected node to be powered up and not be loaded with any software.

16.7 Common Test Sequences

The following common sequences are used in several PNIF tests. Their descriptions are included here (once) to avoid repeating the information in the description of each test.

16.7.1 Clear the PNI

This sequence clears the PNI board and related data. It sets all the PNI registers and externally visible data to a known state (mostly zeros). By setting the board to a known state, an extensive Board Response Check can be performed after each test (or test phase) to check for the expected changes and to make sure that nothing else has changed.

The following is a more detailed description of how PNIF interfaces to the PNI hardware in order to perform this function:

- **ABORT the PNI:** Writes an ABORT command, without interrupt, to the PNI's Command register.

NOTE

PNIF executes a PNI command by:

- Writing a command word to the PNI's command register
- Waiting for the command to complete (command register not busy) or for a timeout

Timeout length depends on the command. If the PNI times out, PNIF issues an alarm that specifies which command failed.

- Set the subsystem control flags to a default state: For each flag write a word to the Subsystem Flag Control register specifying the flag and the new state (TRUE = ENABLE; FALSE = DISABLE). The "clear all flags" function is not used. The default state of the subsystem control flags is
 - DISABLE PNI hardware interrupts
 - ENABLE PNI passed module test flag (green LED)
 - DISABLE PNI synchronization timer
 - DISABLE PNI watchdog timer
- Initialize CCBs, Priorities and FIFOs:
 - Initialize the CCBs—Write the test IOCB address and the CCB ID to the PNI's CCBs 0-3 (the address of the PNI's CCBs is calculated based on the slot number of the PNI board; note that the CCBs are in CPU RAM).

NOTE

PNIF uses only one IOCB at a time, therefore PNIF does not need to use the circular IOCB chains supported by the PNI. This document refers to the IOCB used by PNIF as the "test IOCB."

- Initialize the priority control structures in PNI/CPU shared RAM by—
 1. Writing the test IOCB address as the first IOCB in the priority's circular IOCB chain.
 2. Setting the number of IOCBs executed by the PNI to zero.
 3. Setting the number of IOCBs issued by PNIF to zero.
- Set up Receive buffer FIFOs for PNI priority 0—These structures in PNI/CPU shared RAM are not used during the execution of PNIF by the PNI or by PNIF, but are set up and initialized for the sake of completeness.
- Initialize the test IOCB to a NO OP: The IOCB is in CPU RAM and is initialized by the CPU writing directly to RAM and setting the IOCB fields as follows
 - NOT a group address
 - NOT a type 3 transmission
 - Re-synchronization NOT needed
 - NO OPERATION
 - 0 Frame size
 - 0 Checksum (calculated immediately before execution)
 - Null (\$080000) Transmit buffer pointer
 - NOT Complete
 - NOT a Resynchronize error
 - Successful status
 - 0 Retries
 - No completion interrupt
 - End of Chain
 - IOCB ID
 - DO NOT Execute the next IOCB
 - Null (\$000000) next IOCB pointer
- Do a Board Response Check (see heading 16.7.3 for details).

16.7.2 Reset the PNI

This sequence puts the PNI through its Reset/Self-test sequence and verifies that all self-tests have passed. The Configuration Status Register is checked for proper operation while the PNI is in the RESET state. A Board Response Check is done after a successful RESET/Self-test to verify that the PNI data matches what is expected after a RESET.

The following is a more detailed description of how PNIF interfaces to the PNI hardware in order to perform this function:

- Sets Passed module test flag to ENABLE (TRUE = Green LED on): This is done by writing a word to the Subsystem flag control register specifying
 - Passed module test flag
 - ENABLE/TRUE
 - DO NOT Clear all flags
- Puts PNI in RESET by setting the RESET flag to ENABLE (TRUE): This is done by writing a word to the Subsystem flag control register specifying
 - RESET flag
 - ENABLE/TRUE
 - DO NOT Clear all flags
- Checks Configuration status register for expected contents when PNI is in RESET.
- Takes the PNI out of RESET by setting the RESET flag to DISABLE (FALSE): This is done by writing a word to the Subsystem flag control register specifying
 - RESET flag
 - DISABLE/FALSE
 - DO NOT Clear all flags

At this point the PNI starts its self test sequence.

- Waits for the self-test sequence to finish or for time-out (up-to-six seconds). A successful self-test is indicated by
 - Board not failed
 - Passed module test flag = TRUE
 - Self-test that failed = 0
- If the self-test is successful, a Board Response Check is done to verify that the PNI data matches what is expected after a RESET.
- If the self-test is not successful or does not finish within the allowed time, an alarm is issued and any PNIF tests that are running are aborted.

16.7.3 Board Response Check

This sequence compares all PNI-related data to specified target (should be) data using a specified mask. The data compared includes

- In shared RAM
 - Hardware status
 - Configuration status
 - Interrupt vector
 - Command register
 - Operational status
 - Paged status words
 - Address switches
 - Network address in use
 - Group address mask
 - PNI internal error code
 - PNI internal error PC
 - Self test that failed (valid only immediately after a self-test sequence)
- Paged status
 - Firmware revision
 - Firmware type
 - Device errors
 - Network status
 - Bus error address
 - Current IOCBs for priorities 0-3
 - Synchronization timer value
- Miscellaneous
 - Operational status for interrupt vectors 0-7
 - Test IOCB
 - Test buffer
 - DMA read test command location
 - DMA write test command location

If a mismatch occurs, an alarm is generated that—at abbreviation level 4—specifies what the test was doing, plus a description of the data item that did not match, what it was, what it should be, and the mask used for the comparison. For the test buffer only, the first 10 mismatches are listed and a summary of mismatches is displayed at the end of the alarm.

To execute a Board Response Check, PNIF first collects all the data and moves it to the CPU RAM. The comparison is then done from CPU RAM. Data is collected from the PNI as follows:

- Shared RAM: PNIF collects data in shared RAM by word-at-a-time copies (byte copies are not allowed).
- Paged Status: PNIF collects the paged status data by writing a PAGED STATUS command—without interrupt—to the PNI's command register. The particular paged status is specified in the parameter field of the command word. Once the PAGED STATUS command is complete, PNIF reads the data from the Paged Status words in shared RAM a word at a time.
- Miscellaneous: This data already is in CPU RAM.

16.7.4 Execute Test IOCB

This sequence is used to execute a test IOCB through priority 0 by

- Starting priority 0 by writing a PRIORITY START command—without a command completion interrupt—to the PNI's command register with the command parameter field set to 0.
- Incrementing by one the counter that specifies the number of IOCBs issued by PNIF. The PNI executes an IOCB when it detects differences between that counter and the counter that specifies the number of IOCBs that have been executed by the PNI. Both of these counters are in the Priority's control structure in shared RAM.
- Waiting for IOCB completion or timeout. Note that timeout is set according to IOCB type because some IOCBs take longer than others.
- If the IOCB times out, this sequence generates an alarm.

A delay of 100 ms after IOCB completion is included in this sequence in order to allow time for the PNI to set up the Operational status that corresponds to an End-of-chain interrupt (generated even when interrupts are disabled).

16.7.5 Execute a TBC Full Duplex Loopback Test IOCB

- This sequence is used to set up and execute a TBC Full Duplex Loopback IOCB using a specified loopback type and byte data pattern. This IOCB checks the proper operation of the TBC's serial interface and the FCS generation/checking logic. The TBC has separate transmit and receive checksum hardware, so normal testing verifies valid checksum detection. The data is read from the input portion of the test buffer, transmitted around the loopback path and written back to the output portion of the test buffer.

This test can be executed with any of three loopback types:

- Internal loopback
- Modem A loopback
- Modem B loopback

NOTE

At present, the PNI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in PNIF to provide for the eventuality of two separate modems.

For the specified loopback type and data pattern, this sequence

- Clears the PNI (see heading 16.7.1 for details)
- Sets up the TBC Full Loopback Test IOCB by specifying the loopback type, the test buffer address, and that the TBC is to generate the FCS and NOT copy it to the test buffer. Also, the proper checksum is calculated for the IOCB.
- Sets up the transmitter test buffer which controls the test with
 - The input area coated with the specified data pattern
 - The output area coated with the complement of the data pattern
 - The length output field set to the complement of the expected length
 - The error output field set to the complement of a successful status
- Executes the test IOCB (see heading 16.7.4 for details).
- After the IOCB is complete, delays 250 ms for the PNI to settle, then does a Board Response Check to verify the results.

16.8 PNIF Alarm Categories

PNIF Alarm messages are formatted as described at heading 2.8.2 of the *Test System Executive* manual. The first digit of a 3-digit alarm code or the first two digits of a 4-digit alarm code is the number of the test reporting the failure. Specifics of the failure are contained in the test message text.

17 Programmable Logic Controller Interface Tests (PLCT)

17.1 Programmable Logic Controller Interface Elements

These tests are used to verify operation of the Programmable Logic Controller Interface board operating within a PLC Gateway module.

17.2 Use Information

Tests 1-17, 20, and 22 are primarily hardware verification tests, thus are useful in field testing. Tests 18, 19, and 23-26 are primarily firmware design verification tests. Tests 18, 19, and 23-26 are not intended for field use. These tests have special configuration requirements (a connected PLC) and normally are blocked from use by a zero (0) value assignment in the SUPPORTED_MODES parameter.

All tests must be run on PLCGs that are off line including the redundant “partner” PLCG if applicable. Before running tests 7 through 26 make certain that the PLC has no customer wiring attached to it.

Tests 7 and 13 require special loopback assemblies to be installed on the I/O card. Different assemblies are used depending on whether a PLCI I/O and a relay card are being tested or an EPLCI I/O.

- To test a PLCI I/O card without the relay card, test 7 requires that loopback plug (51195336) be installed on the I/O card's 50 pin connector and loopback plug (51195337) be installed on the Port 3 DB25 connector. PLCT should be in module mode.

IMPORTANT: When these assemblies are in place, tests 8-26 will not run properly.

- To test the EPLCI I/O card, test 7 requires that loopback cable (51195129) be connected between Ports 1 and 2 (DB25 connectors), loopback plug (51196073) be installed on Port 4 (DB9), and loopback plug (51196072-200) be connected to Port 3 through the Port 3 extension assembly (51196072-100). PLCI must be in module mode.

IMPORTANT: When these assemblies are in place, tests 8-26 will not run properly.

- Test 13 allows testing of the I/O channels when two PLCGs or EPLCGs are connected as a redundant pair.

- If the redundant pair is implemented with PLCI I/Os and the relay card, Port 3 on the PLCI I/Os are left open, the PLCI I/Os are both connected to the relay card through the 50 pin product cable, and Ports 1 and 2 on the relay card are connected to each other through the loopback cable (51195129).

The pinning on the relay card has to be set to “OTHER” instead of “ALLEN BRADLEY REDUNDANT” to enable both ports. Do not forget to reverse the setting when the testing is complete.

IMPORTANT: With this loopback cable in place, tests 7, 18, 19, and 23-26 will not run properly.

- If the redundant pair is implemented with EPLCI I/Os interconnected with the product interlink cable (51196074), Port 1 is looped back to Port 2 on both EPLCI I/Os using the loopback cable (51195129), and Port 3 on both I/Os can be left open.

IMPORTANT: With this loopback in place tests 7, 18, 19, and 23-26 will not run properly.

In general, the tests are to be run in three environments, Burn In, System Level, and Firmware Verification. These environments restrict which tests should be run.

Burn In testing is done in two parts. Tests 1-6, 8-10, 12, 14-17, 20, and 22 can be run on a single node (pinning for redundancy is required by factory test procedures, but is optional for field use). Test 7 requires pinning for redundancy and loopback connectors must be installed. Tests 8-26 will not run properly with loopback connectors installed.

System Level testing is performed in two modes, single node and redundant node.

Single (non-redundant) node: Tests 1-6, 8-17, 20, and 22. Restrictions apply if the Test 7 loop back connectors are in place.

Redundant node: Tests 1-6, 8-12, 14-17, 20, and 22 run on one node at a time. Test 13—which requires a special loop back cable—runs on both nodes at the same time with no other tests running in either node. Restrictions apply if the Test 7 loop back connectors are in place.

Firmware Verification (not intended for field use) requires the addition of a PLC device or devices to the proper ports, before tests 18, 19, and 23-26 can be run. Allen-Bradley PLCs are attached to port 1 and Modicon PLCs are connected to port 2. In a redundant node configuration, only one device type can be attached.

17.3 Setup Parameters

17.3.1 PLCT-Specific Parameters

FLAG	Not used
PERSIST_INTERVAL	Allowed values are 0 through 200 Value representing a percentage of the half-second pause before steady-state recomparing. 0 – Do not compare (should be used for looping) 10 – Recompare after 50 ms (minimum) 50 – Recompare after 1/4 sec. (preset value) 200 – Recompare after one sec. (maximum)
PLC_MODEL_TYPE	PLC model or device type (enumeration). Used in tests 18, 19, and 23 through 26. Allowed values are: - MB_384 - Modicon 384 - MB_484 - Modicon 484 - MB_584 - Modicon 584 - AB_PL2 - Allen-Bradley PLC 2, 3, or 5
O_NODE	(Other Node number) The node number of a redundant partner. Must be “-” if no redundant partner is present. Used in tests 13 and 26.
O_SLOT	(Other Slot number) The slot number of a redundant partner. Must be “-” if no redundant partner is present. Used in tests 13 and 26.

SUPPORTED_MODES Consists of one digit per test in sequence (the first test relates to test 1, the second to test 2, etc.). Each digit is the sum of the modes that the associated test can run in. Values assigned are:

- 0 = Test never runs
- 1 = Subsystem Mode
- 2 = Exerciser Mode
- 4 = Module Mode

For field use, this parameter should be left at its default value, thus blocking unsupported or inappropriate tests.

EPLCG_IO_INSTALLED Used in test 13 to determine which IO board is installed. Allowed values are "Yes" and "NO." This parameter determines which firmware the test will run.

17.3.2 General Parameters

TESTS	Allowed values are 1 through 26
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

17.4 Preset Parameter Values for PLCT

FLAG = 0

PERSIST_INTERVAL = 50

PLC_MODEL_TYPE = MB_584

EPLCG_IO_INSTALLED = YES

O_NODE = -

O_SLOT = -

SUPPORTED_MODES = 55115555550557555005050000

TESTS = 1-26

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER,SUSPENDED

SCALE = 100

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

17.5 Mode Characteristics for PLCT

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

17.6 Individual PLC Interface Subsystem Tests

(System Modes)

Test 01A	Valid Configuration Status Register Commands	SUB, MOD
	Assures that the PLCI hardware correctly responds to valid commands to the Configuration Status Register.	
Test 01B	Valid Simple Commands, No Interrupts	SUB, MOD
	Assures that the PLCI firmware correctly responds to selected Command-Register commands without the use of interrupts.	
Test 02	Valid Simple Commands Using Interrupts	SUB, MOD
	Assures that the PLCI firmware correctly responds to selected Command-Register commands using vector 0 interrupts in two ways.	
Test 03	Proper Detection/Response to Module Bus Errors	SUB
	Verifies that module-bus data-parity errors during MCPU writes to the shared status bus are properly detected and reported.	
Test 04	Test DMA Read and Test DMA Write Commands	SUB
	Assures that the PLCI firmware correctly responds to the two Test DMA commands.	
Test 05A	Invalid Commands to the Flag Control Register	SUB, MOD
	Assures that undefined commands to the Subsystem Flag Control Register are properly ignored.	
Test 05B	Invalid Commands and Parameters	SUB, MOD
	Assures that undefined Command-Register commands are properly rejected with the correct error indications.	
Test 06A	Simple NOP IOCB and Three Loopback IOCBs	SUB, MOD
	Assures that the simplest of the IOCB commands in their simplest form correctly function.	
Test 06B	Three Loopback IOCBs with Multiple Words	SUB, MOD
	Assures that multiple words can be looped back.	

- Test 15 Intentionally Forcing/Detecting FIFO-Reported Errors SUB, MOD
Verifies that the PLCI detects and properly handles (and reports) as many types of errors as can easily be caused without manual intervention.
- Test 16 Four Read, Compare, Report IOCB Operations SUB, MOD
Assures that the four types of Read, Compare, and Report operations work correctly, and that FIFOs and FIFO Overflow is working.
- Test 17 Conditional Write with Pre-read IOCB Operation SUB, MOD
Assures that the "Conditional Write With Preread" IOCB-operation correctly functions.

NOTE

The remaining PLCT tests perform firmware verification and—with the exception of tests 20 to 22—require an attached PLC. See the Use Information heading for details.

- Test 18 Database General Testing SUB, MOD
Exercises the database by writing valid and invalid parameters to test appropriate acceptance and rejection.
- Test 19 Command Write to #1777 SUB, MOD
Exercises the Command Register by writing valid and invalid parameters to test appropriate acceptance and rejection. Requires an attached PLC. See hardware setup for information.
- Test 20 Buffer Grant/Report IOCB and HG FIFO Management SUB
Assures that Poll and Call Up and the associated protocol work correctly. Also tests "Buffer Grant with Report."
Not currently supported when PLCG is pinned for redundancy.
- Test 21 Verify Broadcast IOCB SUB
Not currently supported.
- Test 22 Testing Without a Physical HTD SUB
Assures that the PLCG can act as a Hiway Traffic Director, including swapping between the two communication ports.
Not currently supported when PLCG is pinned for redundancy.
- Test 23 Counter Tests SUB, MOD
Tests the attached PLC using counter-type points. This is a firmware design verification test.

Test 24	Analog Tests	SUB, MOD
	Tests the attached PLC using analog-type and counter-type points. This is a firmware design verification test.	
Test 25	Digital Tests	SUB, MOD
	Tests the attached PLC using digital input/output-type points. This is a firmware design verification test.	
Test 26	Redundancy Feature Test	SUB, MOD
	Tests the attached PLC (must be on port 2) in a redundant configuration using counter-type points. This is a firmware design verification test.	

17.7 How to Interpret PLCT Alarms

The complete set of possible PLCT alarms is described below. The first line of every alarm is displayed in the standard alarm format with a generally meaningless alarm number. The first two digits of the alarm number identify the test number (or the test number, plus 50, if the alarm results from the standard housekeeping commands that precede most tests). This alarm number is advanced by two for every subtest (i.e., command or operation), and often wraps in order to avoid exceeding 99. Thus it is common for alarm numbers to be duplicated within a test. Alarm numbers do not necessarily represent the same commands or operations from one pass to another when subtests are conditionally skipped (when driven by random numbers or by pass number).

An even alarm number always indicates that the primary subtest failed (e.g., no interrupt). An odd alarm number always indicates that the follow up comparison to assure steady state failed (i.e., unexpected late interrupt).

Following the alarm number on the first alarm line is a string of terse unique codes (see the "Difference" line explanation) that summarize the errors. This should be ignored unless running at abbreviation level 1. Running at abbreviation level 1 allows tracking of one persistent, but perhaps intermittent, error while using minimum displays. This will get as many errors on one screen as possible or will reduce the stack of paper and the time required to print it. If the same terse codes are on every alarm line, it is likely that only this one type of failure is occurring.

The second and subsequent alarm lines occur only at abbreviation level 2 and higher. They generally start with three lines of "Was," "Should Be," and "Difference." The "Was" lines are shown below as the first line of each paragraph. (Note that they seldom contain the word "Was.") The remainder of each paragraph defines or describes the various fields on that line. In all cases, the "Should Be" is displayed immediately underneath the "Was" line with all fields aligned.

The "Difference" line always is displayed immediately underneath the "Should Be" line, and identifies those hex digits that are wrong. An "N" indicates that there was no change to a field that should have changed. A "C" indicates that there was a change to a field that should not have changed. A "W" indicates that the hex digit changed to a wrong value. Note that this same "N/C/W" coding is used in the terse coding at the end of the first alarm line, but is based on comparison of the entire word, not hex digit by hex digit.

Often, several discrepancies occur within one failure. If they are in the same line, that line is displayed only once (e.g., two different hardware registers); however, several triplets of "Was/Should Be/Difference lines can result if the discrepancies are in fields displayed in different lines.

When the abbreviations level is 3 or 4, a scenario describing the command, the operation, the IOCB chains, and the first eight data words is displayed after all "Was/Should Be." It is described in detail after the "Was" line descriptions.

17.7.1 Explanation of Level 1 PLCT Alarms

```
Registers were  xx  xxxx  xxxx  xxxx  xxxx  xxxx  xxxx  xxxxxxxx
Paged Stat 0- E  xxxxxxx  xxxx  xxxx  xxxxxxx  xxxxxxx  xxxxxxx  xxxxxxx  xxxxxxx
Paged Stat 10-16  xxxx  xxxxxxxxx  xxxxxxxxx  xxxxxxxxx
Pre Unsol  xx  xx  xx  xx  xx  xx  xx  xx  xx  xxx  xxx  xxx  xxx  xxx  xxx  xxx  xxx
Post Unsol xx  xx  xx  xx  xx  xx  xx  xx  xx  xxx  xxx  xxx  xxx  xxx  xxx  xxx  xxx
```

```
Registers were  aa  bbbb  cccc  dddd  00ee  ffff  gggg  hhhhhh
```

aa = "Software Register" contents:

00 = Operation completed

06 = Operation timed out

09 = No bus error exception from intentionally causing a bus parity error

(Any other values indicate software error, not PLCG error.)

bbbb = Hardware Status register

cccc = Configuration Status register before reading Operational Status register

dddd = Configuration Status register after reading Operational Status register.
Interrupt Acknowledge Semaphore (bit 2) and Module Bus Data Parity
Error (bit 15) should be cleared. Otherwise should be the same as "cccc".

00ee = Interrupt Vector register with the most significant byte cleared to "00."

ffff = Command register

gggg = Operational Status register

hhhhh = Paged Status words. During self-test, bits 0-6 show "Test in Error."

Paged Stat 0- E aaaaaaaaa bbbb ccdd eeeee fffffff gggggg hhhhhh iiiiii

aaaaaaaa = Firmware revision number (should remain constant)

bbbb = Database status

cc = Retry count

dd = Inter-block Time

00eeeeee = Address causing bus error (when no error, the value is \$FFFFFFFF)

00ffffff thru 00iiiiii = Current IOCB addresses for CCBs 0 through 3 respectively.

Paged Stat 10-16 aaaa 000bbbbbb ccccccc ddddeeee

aaaa = Database address (most significant byte = Address status, least significant byte = address)

00bbbbbb = Starting address of the FIFO Address table

ccccccc = Database Listen error

dddd = Poll counter (number of polls since last time this was read)

eeee = Poll Response counter (number of poll responses since last read)

Pre Unsol aa bb cc dd ee ff gg hh ii jjj kkk lll mmm nnn ooo ppp qq

This line profiles any unsolicited interrupts that preceded the solicited interrupt. The field descriptions are: aa = the number of unsolicited interrupts preceding the solicited interrupt; bb through ii = the first eight or fewer Interrupt Vector register values; jjj through qq = the first eight or fewer Operational Status register values.

Post Unsol aa bb cc dd ee ff gg hh ii jjj kkk lll mmm nnn ooo ppp qq

This line profiles any unsolicited interrupts that both preceded and followed the solicited interrupt (i.e., it is the continuation of "Pre Unsol" and uses the same field descriptions).

17.7.2 Explanation of Level 2 PLCT Alarms

IOCB Cx, Ix xxxx xxxx xxxxxxxx xxxx xxxx xxxxxxxx

Data Cx, Ix, Bx xxx xxx xxx xxx xxx xxx xxx xxx xxx xxx xxx xxx

SOE & FIFO Ovrfl xxxx xxxx

\$80410 & \$80412 xxxx xxxx

FIFO x Hdr & E0 xxxx xxxx xxxx xxxx xxxx xxxx xxxxxx xxxxxx xxxxxx

FIFO x E1 & E2 xxxx xxxxxx xxxxxx xxxxxx xxxx xxxxxx xxxxxx xxxxxx

FIFO x E3 & E4 xxxx xxxxxx xxxxxx xxxxxx xxxx xxxxxx xxxxxx xxxxxx

IOCB Cx, Ix aaaa bbbb cddddd eeee ffff gghhhhhh

Cx = IOCB Chain (CCB number) that this IOCB is on

Ix = IOCB's position on the chain (the first IOCB is I0)

ffff = the IOCB Status word (remainder of the IOCB should remain constant)

Data Cx, Ix, Bx aaa bbb ccc ddd eee fff ggg hhh iii jjj kkk lll

Cx = IOCB Chain (CCB number) that the IOCB using this data is on

Ix = Position on the chain for the IOCB using this data (the first IOCB is I0)

Bx = the block number of the 12 data words

aaa through lll = the data words (Note that the locations to be read into are pre-coated with \$FFF5, thus a value of \$FFF5 in the "Was" line indicates that no read took place.)

SOE & FIFO Ovrfl aaaa bbbb

aaaa = The Broadcast SOE Synchronize tally of 96 microsecond intervals (in Hex). This is only alarmed when the difference from the time standard is more than counts.

bbbb = the 16 FIFO Overflow bits

\$80410 & \$80412 aaaa bbbb

The contents of \$80410 and \$80412 after a "Test DMA Read" or a "Test DMA Write."

FIFO x Hdr & E0 000a bbb ccc ddd eee fff gghhhh 0iiiiii 0jkkkkk

x = the Error FIFO number, from 0-7

000a thru eeee = is the Error FIFO header

bbb = the Append Pointer (indicates where the next FIFO entry goes). Note that this is usually the only header field to alarm; other header fields are constant.

gg = the Hiway error code

fff thru 0jkkkkk = First position Error FIFO entry

FIFO x E1 & E2 aaa bbcccc 0ddddd 0efffff ggg hhhiii 0jjjjj 0kl111

x = the Error FIFO number, from 0-7

aaa thru 0efffff = Second position Error FIFO entry

ggg thru 0kl1111 = Third position Error FIFO entry

bb and hh = the Database Error code

FIFO x E3 & E4 aaa bbcccc 0ddddd 0efffff ggg hhhiii 0jjjjj 0kl111

x = the Error FIFO number, from 0-7

aaa thru 0efffff = Fourth position Error FIFO entry

ggg thru 0kl1111 = Last position Error FIFO entry

bb and hh = the Database Error code

Interpreting the Database Error Code:

Ignore the first three bits of the Hex value Database Error code. That is, interpret a value of 81 as an alarm code of 01; a value of F5 as 15; etc.

<u>Code</u>	<u>Description</u>
\$00	No error
\$01	No response
\$02	N/A
\$03	N/A
\$04	N/A
\$05	N/A
\$06	N/A
\$07	N/A
\$08	Database security location check failure
\$09	Hiway Grant not received from primary gateway
\$0A	N/A
\$0B	N/A
\$0C	N/A
\$0D	Database busy (Redundant Partner)
\$0E	N/A
\$0F	N/A
\$10	N/A
\$11	No response to Call-Up
\$12	N/A
\$13	N/A
\$14	Invalid PLCG security word was received
\$15	N/A

17.7.3 Explanation of Level 3 PLCT Alarms

FIFO xx Header xxxx xxxx xxxx xxxx xxxx

FIFO xx Values xxxx xxxx xxx xxx xxx xxx xxx xxx xxx xxx xxx xxx

DAC FIFO xxxx xxxx xxxx xxxx xxxx xxxxxx xxxxxx xxxxxx xxxxxx xxxxxx

	Hardwr	Config	Hardwr	Config	Hardwr	Config
Status Was	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
FIFO xx Header	000a	bbbb	cccc	dddd	eeee	

xx = the Report FIFO number (08-14)

000a thru eeee = the report FIFO header

bbbb = the Append Pointer that indicates where the next FIFO entry goes. (Note that this is usually the only header field to alarm; the other fields are constant.)

FIFO xx Values aaaa bbbb cccc ddd eeee fff ggg hhh iiii jjjj kkk lll

xx = the Report FIFO number (08-14)

aaaa thru dddd = contents of the Report FIFO entry in the first position

eeee thru hhhh = contents of the Report FIFO entry in the second position

iiii thru llll = contents of the Report FIFO entry in the last position

DAC FIFO 000a bbbb cccc dddd eeee ff00ggg hh00iii jj00kkk ll00mmm nn00ppp

000a thru eeee = the DAC Receive FIFO header

bbbb = the Append Pointer that indicates where the next FIFO entry goes. (Note that this is the only header field to alarm; other fields are constant.) ff, hh, jj, ll, and nn are the five PIU addresses. gggg, iiii, kkkk, mmmm, and pppp are the five PIU Status Word values.

Wrong "Reset Sequence" Register Values.

	After Writing a 9		After Writing a 1		One Second Later	
	Hardwr	Config	Hardwr	Config	Hardwr	Config
Status Was	aaaa	bbbb	cccc	dddd	eeee	ffff

The above alarm is displayed only during test startup. A "Reset Sequence" was initiated and the Hardware Status register and the Configuration Status register are tracked three times—after the first half of the reset sequence, after the last half, and after firmware self-test has had time to complete.

17.7.4 Interpreting Commands and Operations Which Caused Alarms

A "scenario" is included at the end of each alarm when running at abbreviation level 3 or 4. If the command issued is to the Configuration Status register address (addressable latch), the following scenario is displayed:

Flag Control Value = xxx, Registers = xxx xxx xxx xxx xxx xxx xxx xxx

If the command issued is to the Command register, this scenario is displayed:

Commanded Value = xxx, Registers = xxx xxx xxx xxx xxx xxx xxx xxx

For every CCB that has active IOCBs chained to it, and for every IOCB in each chain, the final lines of the scenario contain the CCB (IOCB chain) number, the active chain of IOCBs, and the first eight data elements for each IOCB. These scenario lines typically will be a small subset of the following:

CCB	Active Chain of IOCBs	Data Elements for this IOCB
0	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx data xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx data xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx data xxx xxxx xxx xxxx xxxx
1	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa FFF5 xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa FFF5 xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa FFF5 xxxx xxx xxxx xxx xxxx xxxx
2	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
3	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx
	opxx xxx xxxxxxx xxx xxx xxxxxxx	bapa xxxx xxxx xxx xxxx xxx xxxx xxxx

Where "op" is the 6-bit operation code, "bapa" is the 16-bit Database address, and "data" is the 16-bit write data. For most alarms resulting from some form of "write" to a box, these three fields will be most useful in identifying what is being tested. For "read" type operations, the data is **not** read into the third data word, it goes into the words which contain \$FFF5.

Note that "Active Chain of IOCBs" normally applies to chains which are set up to be commanded with a "Start IOCB" command; however, on occasion, a chain will be left "Active" while other commands are being exercised. The Scenario Alarming algorithm considers a chain to be "active" only up to the place where it points to a canned terminating "NOP" IOCB. Normally, three CCBs point to this "NOP" and thus are inactive. For exercising non "Start IOCB" commands, all four CCBs usually are pointing to the terminating "NOP" IOCB, so no chains are active.

18 Communication Network Interface Tests (CNIF)

18.1 Communication Network Interface Elements

The Communication Network Interface subsystem consists of a CNI board including a VAX RT-300 microprocessor and a CNIF paddle card all contained within a Plant Network Module (PLNM).

18.2 Use Information

NOTE

CNIF allows for an up to 2-minute response delay from the CNI to allow for the time required by a CNI to go through reset.

The major test choices depend on your system configuration. If there is only one CNI in your system, you must use loopback testing (see the `ROLE` and `NETWORK_ACTIVE` parameters) for those tests that can run in either loopback or network mode. Test 5 and tests 8-10 run in either mode. Tests 6 and 7 always run in loopback mode. Tests 1-4 and 11 are not affected by selection of loopback or network mode.

For most test situations (using the flash ROM-resident personality) the CNI boards must be pinned for normal operation (`JP3 = 0`, `JP4 = 1`, and `JP5 = 1`) and the value of parameter `FIRMWARE_TEST` must equal 0. If `FIRMWARE_TEST` is > 0 , only tests 1-4 and 11 will execute.

Test 11 requires `FIRMWARE_TEST` > 0 before any self-tests can be executed.

18.3 Setup Parameters

18.3.1 CNIF-Specific Parameters

ROLE	Defines test role of the addressed node. Allowed values are LOOPBACK—forces the test to run on one CNI board with the Ethernet messages looped back internally. MASTER—enables the test supervisor and looks for a slave node. SLAVE—waits for commands from the test supervisor in the master node.
NETWORK_ACTIVE	Allowed values are: TRUE—The presence of an Ethernet transceiver and cable allows messages to be sent to another CNI board on the same DECnet. Note that the value of <code>NETWORK_ACTIVE</code> must be TRUE if <code>ROLE</code> is not LOOPBACK.

FALSE—The lack of an Ethernet transceiver and cable forces all messages to be looped back internally on this board.

FAR_NODE Designates the other node of the master/slave pair. Allowed values are 0 thru 127 (note that the range of implemented LCN node addresses is a subset of these values).

FIRMWARE_TEST This parameter value controls whether the test expects the CNI board to be running the HVTS personality (value =0) or the self-test firmware (value >0) and, if so, what self-tests are to be run. Allowed values are \$0 thru \$F. The available self-tests are

Firmware test setup		
Parameter bits		
3 2 1 0	Value	Firmware self-test selected
x x x 1	1	Status bus read/write test
x x 1 x	2	RAM checkerboard test
x 1 x x	4	RAM address uniqueness test
1 x x x	8	EDAC integrity test

To run the self-test firmware tests under CNIF control, you must set CNI board jumpers JP3 = 0, JP4 = 0, and JP5 = 0. Be sure to set them back to JP3 = 0, JP4 = 1, and JP5 = 1 before continuing.

FIRST_RANDOM Allowed values are \$0 thru \$FFFF. A non-zero value initializes the random number generator at that value.

DATA_ERROR_LIMIT Specifies the maximum number of data errors reported for any given message. Allowed values are \$1 thru \$F000.

PRINT_LCN_COMMANDS Allowed values are

TRUE—commands are printed for operator information as they are transmitted across the LCN.

FALSE—commands are not printed.

SLOW_FOR_TESTING Allowed values are

TRUE—causes the test to run slowly so the operator can follow the sequence of events.

FALSE—causes normal-speed operation of the test.

18.3.2 Display-Only Parameters

FAR_ADDRESS Ethernet address of the test partner (format is three 16-bit Hex values making up a 48-bit address)

THIS_ADDRESS Ethernet address of local receive channel.

THIS_NODE This node's LCN address for its master/slave.

18.3.3 General Parameters

TESTS Allowed values are 1 through 11

ERROR_LIMIT Allowed values are 1 through 32767 or "-"

PASS_LIMIT Allowed values are 1 through 32767 or "-"

REPORT Allowed values are PASSNUMBER, TESTNUMBER, LOG

INHIBIT Allowed values are PASSNUMBER, TESTNUMBER, LOG

SCALE Allowed values are 0 through 999

MINUTES_LIMIT Allowed values are 0 through 32767 or "-"

ABBREVIATION Allowed values are 0 through 4

AFTER_ERROR_LIMIT Allowed values are ABORT or HALT

18.4 Preset Parameter Values for CNIF

ROLE = UNKNOWN

NETWORK_ACTIVE = TRUE

FAR_NODE = -1

FIRMWARE_TEST = 0

FIRST_RANDOM = 0

DATA_ERROR_LIMIT = 4

PRINT_LCN_COMMANDS = FALSE

SLOW_FOR_TESTING = FALSE

TESTS = 1-11

ERROR_LIMIT = -

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER

SCALE = 100

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

18.5 Mode Characteristics for CNIF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

18.6 Individual CNIF Tests

(System Modes)

Test 01 Test Reset / LEDs

SUB, MOD

Verifies that the unconditional reset, the self-test, and the reset/green LED flags all work. The test first writes to the subsystem flag control to reset the board, then reads configuration status bit 0 to verify that the board goes into reset. (Note that reset can require up to 2 minutes to complete.) Next it writes to subsystem flag control to clear the board reset and reads configuration status bit 0 to verify that it is cleared. After a delay for self-tests to run (the self-tests run only when the CNIF board is jumpered for firmware test mode and the setup parameter FIRMWARE_TEST > 0), reads hardware status bits 14 and 15 to verify that the reset is complete, the self-test has passed, and that the green LED is in the same state as before the reset.

Test 02 Load and Verify Registers

SUB, MOD

Verifies the register address/data paths as far as possible without commanding the board to start an IOCB. The test first checks the hardware status register for hardware/subsystem/sub type, then checks the interrupt response word and configuration status for initial contents. Writes to subsystem flags combinations to set and clear the flags one at a time and verifies them. Sets all flags and tries to clear all flags in the subsystem flags register. Verifies that all flags are cleared. Sends display paged status to the command register and verifies the contents of registers \$0C and \$0E for all paged status addresses.

Test 03 Basic Interrupts

SUB, MOD

Verifies that the interrupt, the interrupt control bit in the subsystem flag control register, and the board-related interrupt status bits work. First tries to generate an interrupt with interrupts disabled after the reset. Then tries to generate an interrupt with interrupts enabled in the subsystem flag control. Verifies that the interrupt bits in the configuration status, interrupt response word, and the board-related operational status work. Disables the interrupt control flag and tries again. Uses and test the NOP command. Verifies that the configuration status register's operational status semaphore bit is cleared with a read of the board-related operational status.

Test 04 Basic DMA Test

SUB

Verifies the Test DMA command, the basic DMA data path integrity, and the data swap logic. Writes and reads the paged status words to verify the data path integrity. Tests DMA write without byte swap and verifies data, then tests DMA read without byte swap and verifies the random data. Next tests DMA write with byte swap and verifies the data, then tests DMA read with byte swap and verifies the data.

Test 05 Basic CCB / IOCB Test

SUB, MOD

Verifies that the CNI board can complete a chain of one IOCB. Executes and checks the "Configure Download Initial Parameters" command. Sets up, runs and verifies a single NOP IOCB chain. Loads and checks the chip loopback mode. Executes and checks the "Configure Download Ethernet Destination Address" command to the Transmit IOCB channel. Sets up the transmit and receive IOCBs for a one IOCB long transfer of random data and loads the CCBs with the IOCB start address. Issues a start receive IOCB command to the command register. Issues a start transmit IOCB command to the command register. Verifies normal termination and valid data.

Test 06 Parity / Bus Error / Address

SUB

Verifies that the module bus error and parity error circuitry on the CNI board are functional. Begins by verifying that the initial paged status "Address Causing Bus Transaction Error" is \$FFFFFFFF. Depending on the availability of unused memory, executes a loopback IOCB test across the upper bound of system memory and verifies correct detection.

If the module processor is an HMPU, EMPU, or MCPU board, writes 0s, walking 1s, and all 1s to the Subsystem Command Flags register with bad parity to test the Status Bus Parity error. Then verifies that no bits have been set and that the configuration status Module Bus Parity Error bit is set and then cleared when the configuration status is read.

Test 07 DMA Addressing Test

SUB, MOD

Verifies that the CNI board hardware can access all areas of memory during a DMA transaction. Executes and checks the "Configure Download Initial Parameters" command. Loads and checks the Chip Loopback Mode. Executes and checks the "Configure Download Ethernet Destination Address" command to the Transmit IOCB channel. For each of the 64 k byte boundaries in the unused and available memory space beyond pool: Sets up 256 byte transmit and receive IOCBs and loads the CCBs with the IOCBs start addresses; issues a start receive IOCB command; issues a start transmit IOCB command; verifies normal termination and valid data.

Test 08 Fixed Ethernet Frames Test**SUB, MOD**

Verifies that the CNI board hardware can transmit/receive on the Ethernet LAN and that specific Ethernet frames are tested. The test executes and checks the "Configure Download Initial Parameters" command; loads and checks the Chip Loopback Mode; and executes and checks the "Configure Download Ethernet Destination Address" command to the transmit IOCB channel. Sends and verifies the following types of Ethernet frames: minimum length IOCB, maximum length IOCB, maximum length Ethernet frame, all zeros data, all ones data, and alternating 1s and 0s data.

Test 09 Random Ethernet Frames Test**SUB, MOD**

Ensures that all possible combinations of Ethernet frames and IOCBs eventually are tested. Executes and checks the "Configure Download Initial Parameters" command; loads and checks the Chip Loopback Mode; executes and checks the "Configure Download Ethernet Destination Address" command to the Transmit IOCB channel; sets up the transmit and receive IOCBs for a random length chain of random data IOCBs and loads the CCBs with the IOCB start address; issues a Start Receive IOCB command; issues a Start Transmit IOCB command; verifies normal termination and normal data.

Test 10 Random Data Exerciser Test**SUB, MOD, EXER**

Tests CNI module bus interactions with other test modules operating in the Exerciser mode. The sequence of events is similar to Test 9 with only one frame per pass.

Test 11 Test EPROM Firmware Mode**SUB**

Verifies that the EPROM firmware Command Processor and Command Self-Tests work. When pass count equals zero, a Display Paged Status command is performed and the board's Self-Test Firmware version/revision identification is printed. If the FIRMWARE_TEST parameter value is not zero and the board is jumpered for EPROM firmware, an Exercise Test command is performed and the board's register contents are verified. Firmware self-tests are enabled and run as specified by the contents of the FIRMWARE_TEST parameter.

18.7 CNIF Alarms

CNIF prints alarms in either of two formats:

- As relatively simple English language descriptions of the expected vs actual values.
- The standard HVTS alarm header line followed by additional information including:
 - Description of the operation in progress
 - Starting random number (if a random number was in use)

Description of data area in error including was and should be values, the mask used, and the differences (Exclusive OR of was and should be values ANDed with the mask).

19 Network Gateway Interface Tests (NGIF)

19.1 Network Gateway Interface Elements

The Network Gateway Interface subsystem consists of the NGI printed wiring board and NGI “paddle cards.” The NGI interfaces to the HMPU or HPK2 in a Network Gateway through the Module Bus. This includes use of Direct Memory Access, shared RAM, and interrupts.

19.2 Use Information

The NGI can be tested with or without a modem. If the modem is not present, a pin is grounded on the NGI paddle card to cause the firmware self-tests to skip the modem-related tests.

The individual NGIF tests are described in detail beginning at heading 19.6. A number of sequences that are used by several tests are described beginning at 19.7. These common sequences are

- Clear NGI (19.7.1)
- Reset NGI (19.7.2)
- Board Response Check (19.7.3)
- Execute Test IOCB (19.7.4)
- Execute a TBC Full Duplex Loopback Test IOCB (19.7.5)
- Check for Modem Present (19.7.6)

The vast majority of the alarms generated by NGIF are as a result of a Board Response Check (see heading 19.7.3 for details). Other common alarms are self-test failure, command timeout, IOCB timeout, and board not responding.

Specifics of test failures are spelled out in the text portion of the alarm messages; therefore, running at Abbreviation level 2 or higher is recommended.

19.3 Setup Parameters

19.3.1 NGIF-Specific Parameters

MODEM_AVAILABLE Allowed values are TRUE and FALSE.

EXTERNAL_LOOPBACK Allowed values are TRUE and FALSE. Set to TRUE if external loopback plug is attached, otherwise set to FALSE. EXTERNAL_LOOPBACK and MODEM_AVAILABLE affect the operation of tests 15, 16, 17, 18, 19, and 21.

ADDRESS_LIST This parameter is a list of eight decimal integers, each integer between 0 and 255 inclusive. The list is used in Test 22 to determine the PIN addresses of peer NGIs. Each element in the list must have the address of a peer NGI or the null address (0). If the PIN address of the local NGI is found in the list, it will be ignored. This allows the entry of the same address list in all NGIs. The list defaults to the null list (eight zeros).

TRANSMIT_ENABLE This parameter is used in Test 22 to coordinate peer NGI start-up. The allowed values are TRUE or FALSE. The default value is False.

19.3.2 General Parameters

TESTS Allowed values are 1 through 22

ERROR_LIMIT Allowed values are 1 through 32767 or "-"

PASS_LIMIT Allowed values are 1 through 32767 or "-"

REPORT Allowed values are PASSNUMBER, TESTNUMBER, LOG

INHIBIT Allowed values are PASSNUMBER, TESTNUMBER, LOG

SCALE Allowed values are 1 through 999

MINUTES_LIMIT Allowed values are 1 through 32767 or "-"

ABBREVIATION Allowed values are 0 through 4

AFTER_ERROR_LIMIT Allowed values are ABORT or HALT

19.4 Preset Parameter Values For NGIF

MODEM_AVAILABLE = FALSE

EXTERNAL_LOOPBACK = FALSE

TRANSMIT_ENABLE = FALSE

ADDRESS_LIST = 0, 0, 0, 0, 0, 0, 0, 0

TESTS = 1-22

ERROR_LIMIT = -

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER, TESTNUMBER

SCALE = 100

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

19.5 Mode Characteristics for NGIF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

19.6 Individual Network Gateway Interface Tests

		(System Modes)
Test 01	Reset	SUB, MOD
Puts the NGI through its Reset/Self-Test sequence to verify that all self-tests are passed. Also checks the operation of the Hardware Status and Configuration Status registers.		
Test actions are		
• Clears the NGI (see heading 19.7.1 for details). • Resets the NGI (see heading 19.7.2 for details). • If the board passes its self-test, the test delays 250 ms then does a Board Response Check (see heading 19.7.3 for details). Note that the initial Board Response Check is done in the Reset sequence.		

Test 02 Load and Verify Registers

SUB, MOD

Ensures the integrity of data paths between the memory and the NGI and checks the ability of the NGI to store and receive data from the NGI-CPU shared RAM.

Data patterns are transferred over the module bus by loading and verifying the following hardware registers:

Operational Status register

Paged Status words

Command register

As much as possible, the rest of the domain also is monitored and is alarmed on unexpected changes.

Seven basic word patterns are used: \$FFFF, \$5555, \$AAAA, \$0144, \$4401, \$DBDB, and \$0000. Two additional patterns (a random pattern and its complement) are used for the Operational Status register and the Paged Status words.

NOTE

A random pattern cannot be used to test the command register because the effect of these patterns is unpredictable and sometimes undesirable.

Test 02 actions are

- Clears the NGI (see heading 19.7.1 for details).
- Loads and verifies the Operational Status register. For each data pattern, including the random patterns
 - Writes the data pattern to the Operational Status register.
 - Reads the Operational Status register and verifies the pattern as part of a board response check (see heading 19.7.3 for details).
 - Does another Board Response Check after a 250 ms delay.
- Loads and verifies the four Paged Status words. For each data pattern, including the random patterns
 - Writes the data pattern to the Paged Status words.
 - Reads the Paged Status words and verifies the pattern as part of a board response check (see heading 19.7.3 for details).
 - Does another Board Response Check after a 250 ms delay.

- Loads and verifies the Command register. For each data pattern, except the random patterns
 - Writes the data pattern to the Command register (as a command).
 - Waits for command completion.
 - Reads the Command register and verifies the pattern as part of a board response check (see heading 19.7.3 for details). Note that the Command register's busy bit cannot be verified because it is cleared by the NGI on command completion.
 - Does another Board Response Check after a 250 ms delay.
 - Clears the NGI (see heading 19.7.1 for details) in case the Command register test did something strange to the NGI.

Test 03 Configuration Status and Subsystem Flags**SUB, MOD**

Checks the operation of the Configuration Status Register (read) and the Subsystem Flag Control Register (write) and verifies that the rest of the NGI data remains unchanged. This test includes assuring that the NGI hardware responds correctly to valid commands to the Subsystem Flag Control Register (CPU writes to the Configuration Status register). Verifies that the requested changes occur in the Hardware Status register and the Configuration Status register (read). It also verifies that the rest of the NGI domain remains unchanged.

Except for the Reset Sequence commands (checked in another test), all valid commands are written to the Subsystem Flag Control register (addressable latch). This includes the "clear all flags" command (similar to, but not the same as a Reset sequence). This test verifies that the appropriate changes occur to the first two hardware registers and that all the remaining registers and the NGI data remain unchanged. Steady state should be achieved immediately and persist for at least 250 ms. Note that this level of testing cannot verify the functionality associated with these commands, only the indications that result in the registers.

Test 03 actions are

- Clears all flags, consisting of
 - Enable (set) Reset flag
 - Disable (clear) Reset flag with the clear all flags bit set (\$41).
- Waits for self-test sequence to end.
- Does a Board Response Check (see heading 19.7.3 for details) to verify that the NGI data matches what is expected after a Reset.
- Delays for 250 ms, then does another Board Response Check.
- Aborts the NGI by writing an ABORT command, without interrupt, to the NGI's Command register.

- Does another Board Response Check to verify that the NGI data matches what is expected after an Abort.
- Delays for 250 ms, then does another Board Response Check.
- For the CPU hardware interrupt flag
 - Enables interrupts
 - Disables interrupts
 - Enables interrupts
 - Disables interrupts
- For the Passed module test flag
 - Enables (passed) module test
 - Disables (failed) module test
 - Enables (passed) module test
- For the Synchronization timer enable flag
 - Enables Synchronization timer
 - Disables Synchronization timer
 - Enables Synchronization timer
 - Disables Synchronization timer
- For the CPU/external watchdog timer enable flag
 - Enables MCPU/external watchdog timer
 - Disables MCPU/external watchdog timer
- Resets the NGI (see heading 19.7.2 for details)
- If the self-tests pass, Clears the NGI (see heading 19.7.1 for details), otherwise aborts the test.

When enabling and disabling the subsystem control flags for Test 03, two Board Response Checks are done after every operation to check for the expected changes. The first check is done immediately after the operation and the second is done after a 250 ms delay. The only exception is made when enabling the CPU/external watchdog timer that delays to allow the watchdog to expire before doing the Board Response Check.

Test 04 Paged Status

SUB, MOD

Reads and verifies contents of the Paged Status registers. All paged status words in shared RAM are precoated, then checked for proper status after commands to read Paged Status for each of the following:

- Firmware version
- Device status
- Bus error address
- Current IOCB addresses 0-3
- Synchronization timer value

As much as possible, the rest of the domain is monitored and alarmed on unexpected changes.

Seven basic word patterns are used: \$FFFF, \$5555, \$AAAA, \$0144, \$4401, \$DBDB, and \$0000. Two additional patterns (a random pattern and its complement) also are used.

Test 04 actions are

- Clears the NGI (see heading 19.7.1 for details).
- For each Paged status and each of the data patterns:
 - Precoats the Paged Status words in shared RAM.
 - Does a Board Response Check (see heading 19.7.3 for details) to make sure that the writes to the Paged Status words worked.
 - Reads the selected Paged Status word by writing a Paged Status command—without interrupt—to the NGI's Command register. The particular paged status is specified in the command word's parameter field. The NGI should write the data to the Paged Status words in shared RAM, overwriting the precoated pattern.
 - Does a Board Response Check to verify that the command worked and that the selected paged status was written to the Paged Status words in shared RAM.
 - After a 250 ms delay, does another Board Response Check to check for steadiness.

Test 05 NGI Interrupts of the CPU

SUB, MOD

Tests the NGI's board and priority related interrupt generation logic, including the logic that generates the Operational Status Word. The following interrupts are generated and checked:

- Board-related interrupt (Vector 0)
- Priority 0 related interrupt (Vector 1)
- Priority 1 related interrupt (Vector 2)
- Priority 2 related interrupt (Vector 3)
- Priority 3 related interrupt (Vector 4)

Only the board and priority-related interrupts are tested in Test 05. The Synchronization interrupt (Vector 5) is tested during the Synchronization timer test (Test 13). Although interrupt Vectors 6 and 7 are not used by the NGI, they still are checked to make sure that neither interrupt occurs during this test.

Test 05 actions are

- Clears the NGI (see heading 19.7.1 for details).
- Enables the CPU hardware interrupts by writing a word to the Subsystem Flag Control register specifying
 - CPU hardware interrupt flag
 - Enable/True
 - Do Not clear all flags
- Makes sure that the interrupts are enabled and everything else is okay by doing two Board Response Checks (see heading 19.7.3 for details); the first immediately, and the second 250 ms later.
- Generates a board-related interrupt by writing a NO OP command—requesting a command completion interrupt—to the NGI's Command register.
- Verifies that the correct interrupt occurred and all else is okay by doing two Board Response Checks; the first immediately, and the second 250 ms later.
- Puts the NGI on-line by writing an On-Line command—without a completion interrupt—to the NGI's Command register.
- For each priority (0-3) generates and verifies a priority-related interrupt by executing a NO OP IOCB (the NGI must be on-line) and specifying a completion interrupt:
 - Starts the priority by writing a PRIORITY START command—without a command completion interrupt—to the NGI's Command register. The parameter field of the command word specifies which priority is to start.
 - Causes the NGI to execute the NO OP IOCB by incrementing, by one, the number of IOCBs issued by NGIF in the priority's control structure (shared RAM).

- Waits for IOCB completion or timeout.
 - If IOCB times out, generates an alarm saying so.
 - Regardless of the completion/timeout outcome, does two board response checks; the first immediately and the second after a 250 ms delay.

- Clears the NGI (see heading 19.7.1 for details) to disable the hardware interrupts, etc.

Test 06 Module Bus Error Detection SUB

Checks the ability of the NGI to detect Module Bus transaction errors by forcing these types of errors:

- Bus error (by forcing the NGI to DMA from a nonexistent memory location)
- Parity error (generated by the CPU with different data patterns)

Test 06 actions are

- Clears the NGI (see heading 19.7.1 for details)
- Forces the NGI to DMA from a nonexistent memory location by
 - Stores a nonexistent memory address in the Priority 0 CCB.
 - Starts Priority 0 by writing a PRIORITY START command—without a command completion interrupt—to the NGI's command register. Priority 0 is specified by setting the command's parameter field to a 0.
 - Causes the NGI to execute the nonexistent IOCB by incrementing, by one, the number of IOCBs issued by NGIF in the priority 0 control structure (shared RAM). This in turn should cause a bus error when the NGI attempts to access the IOCB using DMA.
- Does two Board Response Checks to verify that the bus error occurred; the first check after a 50 ms delay, and the second 250 ms after that.
- Clears the Data Transfer Timeout indicator on the CPU that was set during the bus error transaction.
- Clears the NGI.
- Performs the parity error detection test—skipped if the CPU is a Hyper Kernel 2 (HPK2) board. This test checks the NGI's ability to detect module bus parity errors by using 16 data patterns consisting of a 1 "walking" from left to right within a word of 0s. Its actions are
 - Writes the selected data pattern to the NGI's Command register with a bad parity bit.
 - Does two Board Response Checks to verify that the bus error occurred; the first check after a 50 ms delay and the second 250 ms after that.
 - Clears the Bus error indicator on the CPU board that was set during the parity error transaction.
 - Resets the NGI (see heading 19.7.2 for details) to clear the error.

- If the NGI passes its self-tests, Test 6 clears the NGI (see heading 19.7.1 for details) to get the board back to a known state; however, if the NGI fails its self-tests, Test 6 is aborted.

Test 07 Hardware and Firmware Watchdog Timers

SUB, MOD

Verifies the proper operation and expiration of the two watchdog timers on the NGI board

- The Hardware (or External) Watchdog Timer—used by the NGI to detect CPU failure.
- The Firmware (or Internal) Watchdog Timer—used by the NGI to detect failure of the NGI's local CPU and/or firmware (test uses the Firmware Watchdog Timer Test IOCB).

The CPU/external watchdog is tested by making sure that it does not expire when kicked, expires when not kicked, and recovers after expiration. The NGI/internal firmware watchdog is tested by running the corresponding test IOCB provided by the NGI firmware.

To test that the CPU/external watchdog timer does not fail when kicked, Test 07

- Clears the NGI (see heading 19.7.1 for details).
- Enables the CPU/external watchdog timer by writing a word to the subsystem flag control register that specifies
 - CPU/external watchdog timer flag
 - ENABLE/TRUE
 - DO NOT Clear all flags
- Kicks the watchdog continuously for one second.
- Disables the CPU/external watchdog timer by writing a word to the subsystem flag control register that specifies
 - CPU/external watchdog timer flag
 - DISABLE/FALSE
 - DO NOT Clear all flags
- Delays 400 ms, then writes a NO OP command—without a command completion interrupt—to the NGI's Command register.
- Does a Board Response Check (see heading 19.7.3 for details) to make sure that nothing failed.

To test the CPU/external watchdog timer's ability to expire when not kicked, Test 07

- Enables the CPU/external watchdog timer by writing a word to the subsystem flag control register that specifies
 - CPU/external watchdog timer flag
 - ENABLE/TRUE
 - DO NOT Clear all flags
- Kicks the watchdog continuously for one second.
- Waits for 250 ms (enough time for the watchdog to expire).
- Writes a NO OP command—without a command completion interrupt—to the NGI's Command register.
- Does a Board Response Check to make sure that the watchdog expired.

To test the NGI's ability to recover after a CPU/external watchdog timer expiration, Test 7

- Disables the CPU/external watchdog timer by writing a word to the subsystem flag control register that specifies
 - CPU/external watchdog timer flag
 - DISABLE/FALSE
 - DO NOT Clear all flags
- Does a Board Response Check to make certain that the failure cleared.
- Resets the NGI (see heading 19.7.2 for details) to get the board working again.

If the NGI passes its self-tests, Test 7 clears the NGI (see heading 19.7.1 for details) to get the board back to a known state; however, if the NGI fails its self-tests, Test 07 is aborted.

To test the NGI/internal firmware watchdog timer, Test 07

- Clears the NGI (see heading 19.7.1 for details).
- Sets up the Firmware Watchdog Timer Test IOCB specifying a random number of ticks.
- Executes the test IOCB (see heading 19.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after the IOCB is complete and the second after a 250 ms delay.

Test 08 CPU and NGI Shared RAM Test Command

SUB, MOD

Tests the RAM that is shared by the CPU and NGI by using a "walking 1" and a "walking 0." This test is more extensive and takes longer (approximately 40 seconds) than the shared RAM test performed by the NGI during its self-test.

Test 08 actions are

- Clears the NGI (see heading 19.7.1 for details)
- Writes a TEST NGI SHARED RAM command—without a command completion interrupt—to the NGI's Command register. The command parameter is set to a one to specify that the NGI should run the test once.
- Waits for one second after command completion to allow the NGI time to restore the shared RAM.
- Does a Board Response Check (see heading 19.7.3 for details) to make sure that the test worked and that nothing else changed.

Test 09 CPU/NGI DMA Test Command

SUB

Tests the DMA read and write functions of the NGI board by using the DMA test commands (DMA read and DMA write) provided by the NGI. The DMA read command reads a word from a fixed location in CPU RAM and writes it to the paged status words in NGI/CPU shared RAM. The DMA write command reads a word from the paged status words in NGI/CPU shared RAM and writes it to a fixed location in CPU RAM. Both commands are run a multiple number of times with random data patterns. All of the NGI data is checked after each run.

Test 09 actions are

- Clears the NGI (see heading 19.7.1 for details)
- For each of 100 random data patterns, Test 09
 - Coats the Paged Status words in shared RAM with the complement of the data pattern.
 - Writes the pattern to the DMA read test location in CPU RAM.
 - Writes a DMA READ TEST command—without requesting a command completion interrupt—to the NGI's Command register.
 - Does two Board Response Checks; the first immediately after command completion and the second after a 250 ms delay.
- For each of 100 random data patterns, Test 09
 - Coats the Paged Status words in shared RAM with the complement of the data pattern.
 - Writes the complement of the pattern to the DMA write test location in CPU RAM.
 - Writes the pattern to the Paged status word (in shared RAM) used by the DMA write test.
 - Writes a DMA WRITE TEST command—without requesting a command completion interrupt—to the NGI's Command register.
 - Does two Board Response Checks; the first immediately after command completion and the second after a 250 ms delay.

Test 10 NGI RAM Test Command

SUB, MOD

Tests the NGI RAM using "walking 1" and "walking 0." This test is more extensive than the NGI local RAM test performed by the NGI during its self-test.

Test 10 actions are

- Clears the NGI (see heading 19.7.1 for details)
- Writes a TEST NGI LOCAL RAM command—without requesting a command completion interrupt—to the NGI's Command register. The command's parameter is set to a one to specify that the NGI should run the test once.
- Waits for 250 ms after command completion to allow the NGI time to settle.
- Does a Board Response Check (see heading 19.7.3 for details) to make sure that the test worked and that nothing else changed.

Test 11 NGI EPROM Read and Checksum Test Command

SUB, MOD

Reads the entire contents of the NGI's EPROM, verifies its checksum, and checks parity. This test is more extensive than the EPROM test performed by the NGI during its self-test.

Test 11 actions are

- Clears the NGI (see heading 19.7.1 for details)
- Writes a TEST NGI EPROM command—without requesting a command completion interrupt—to the NGI's Command register. The command's parameter is set to a one to specify that the NGI should run the test once.
- Waits for 250 ms after command completion to allow the NGI time to settle.
- Does a Board Response Check (see heading 19.7.3 for details) to make sure that the test worked and that nothing else changed.

Test 12 Token Bus Controller DMA Test IOCB

SUB, MOD

Checks the operation of the Token Bus Controller's DMA and host interface operation by using the NGI firmware test IOCB. During the test, the TBC reads data from the input part of the test buffer through DMA and writes it back to the output part of the test buffer, also through DMA. The output data is then compared against the input data. Seven fixed-byte patterns are used: \$FF, \$55, \$AA, \$14, \$41, \$DB, and \$00. Two additional patterns (a random pattern and its complement) also are used.

Test 12 actions are

- Clears the NGI (see heading 19.7.1 for details)
- For each of the nine data patterns, Test 12 sets up and executes the TBC Host Interface test IOCB as follows:
 - Sets up the test IOCB with the address of the test buffer and the proper checksum.

NOTE

NGIF uses only one test buffer at a time, so only one test buffer is allocated per running NGIF test. The test buffer is preallocated at initialization so that each test can use it without the need to allocate or deallocate it.

- Coats the input part of the test buffer with the data pattern.
- Coats the output part of the test buffer with the data pattern's complement.
- Executes the test IOCB (see heading 19.7.4 for details).
- Does two Board Response Checks; the first immediately after IOCB completion and the second after a 250 ms delay. The test buffer is checked for the expected result as part of the Board Response Check.

Test 13 Synchronization Timer Test IOCB**SUB, MOD**

Verifies proper operation of the NGI's Synchronization Timer and related interrupt logic by using the Synchronization Timer test IOCB. The timer is set to a count (number of ticks) and then allowed to expire and generate an interrupt. An NGI firmware timer stops the test and reports an error if the timer fails to generate an interrupt within six seconds.

Test 13 actions are

- Clears the NGI (see heading 19.7.1 for details)
- Sets up the Synchronization Timer Test IOCB with a random number of ticks.
- Executes the test IOCB (see heading 19.7.4 for details).
- Does two Board Response Checks to verify the result; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 14 Silence Detect Timer Test IOCB**SUB, MOD**

Verifies proper operation of the NGI's Silence Detect Timer and related interrupt logic by using the Silence Detect timer test IOCB. The timer is preset to a count (number of ticks) and then allowed to expire and generate an interrupt. A NGI firmware timer stops the test and reports an error if the timer fails to generate an interrupt within six seconds.

Test 14 actions are

- Clears the NGI (see heading 19.7.1 for details).
- Sets up the Silence Detect timer test IOCB with a random number of ticks.
- Executes the test IOCB (see heading 19.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 15 Token Bus Controller Receiver Test IOCB

SUB, MOD

Uses the LLC firmware to execute the TBC's built-in receiver test. This test can find problems in the TBC controller and support circuitry. A failure could be caused by failures in either the transmitter or receiver since this test uses both.

Seven fixed-byte patterns are used: \$FF, \$55, \$AA, \$14, \$41, \$DB, and \$00. Two additional patterns (a random pattern and its complement) also are used.

If a modem and external loopback plug are not present, each pass of this test executes the test IOCB once for each data pattern using internal loopback only.

If a modem or external loopback plug is present, each pass of this test executes the test IOCB three times for each data pattern using in turn

- Internal loopback
- Modem A loopback
- Modem B loopback

NOTE

At present, the NGI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in NGIF to provide for the eventuality of two separate modems.

Test 15 actions—for each data pattern and for each loopback type—are

- Clears the NGI (see heading 19.7.1 for details).
- Checks modem availability (see heading 19.7.6 for details).
- Sets up the TBC Receiver test IOCB specifying the loopback type and the test buffer address.
- Sets up the receiver test buffer that controls the test with
 - A random number of frames between 1 and 3
 - A random frame length between 16 and 33
 - A preamble length of 8
 - A random preamble byte pattern
 - The data pattern byte
 - Zero fills unused pieces of the test buffer
- Executes the test IOCB (see heading 19.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 16 Token Bus Controller Transmitter Test IOCB**SUB, MOD**

Executes the TBC's transmitter test IOCB to check for proper operation of the TBC's transmitter. This test finds problems in the TBC controller and support circuitry. Failures of this test can be caused by either the Transmitter or Receiver since both are used.

If a modem and external loopback plug are not present, each pass of this test executes the test IOCB once for each data pattern using internal loopback only.

If a modem or external loopback plug is present, each pass of this test executes the test IOCB three times for each data pattern using in turn

- Internal loopback
- Modem A loopback
- Modem B loopback

NOTE

At present, the NGI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in NGIF to provide for the eventuality of two separate modems.

Test 16 actions—for each loopback type—are

- Clears the NGI (see heading 19.7.1 for details)
- Checks modem availability (see heading 19.7.6 for details).
- Sets up the TBC Transmitter test IOCB specifying the loopback type and the test buffer address.
- Sets up the transmitter test buffer that controls the test with
 - A random data length between 2 and 1024
 - A random number of frames between 1 and 15
- Executes the test IOCB (see heading 19.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 17 Token Bus Controller Internal Loopback Test IOCB SUB, MOD

Checks the TBC's data paths by executing TBC Full Duplex Loopback test IOCB using an internal loopback. This IOCB checks the proper operation of the TBC's serial interface and the FCS generation/checking logic. The TBC has separate transmit and receive checksum hardware, so normal testing verifies valid checksum detection. The data is read from the input portion of the test buffer, transmitted around the loopback path and is written back to the output portion of the test buffer.

Seven fixed-byte patterns are used: \$FF, \$55, \$AA, \$14, \$41, \$DB, and \$00. Two additional patterns (a random pattern and its complement) also are used.

All of the NGI interaction in Test 17 is performed by the sequence that executes a TBC Full Duplex loopback test IOCB (see heading 19.7.5 for details).

This test will be skipped if a modem is present.

Test 18 Token Bus Controller External Loopback Test IOCB SUB, MOD

Checks the TBC's data paths by executing TBC Full Duplex Loopback test IOCB using Modem A/B (external) loopback. This IOCB checks the proper operation of the TBC's serial interface and the FCS generation/checking logic. The TBC has separate transmit and receive checksum hardware, so normal testing verifies valid checksum detection. The data is read from the input portion of the test buffer, transmitted around the loopback path and is written back to the output portion of the test buffer.

This test is not executed unless a modem or external loopback plug is present.

Seven fixed-byte patterns are used: \$FF, \$55, \$AA, \$14, \$41, \$DB, and \$00. Two additional patterns (a random pattern and its complement) also are used. Each pass of Test 18 executes the test IOCB once for Modem A and once for Modem B.

NOTE

At present, the NGI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in NGIF to provide for the eventuality of two separate modems.

All of the NGI interaction in Test 18 is performed by the sequence that executes a TBC Full Duplex loopback test IOCB (see heading 19.7.5 for details).

Test 19 Modem Jabber Test IOCB

SUB, MOD

Uses the Modem Jabber test IOCB to test the modem's ability to detect jabbering on the Plant Information Network Cables.

Skip test 19 if no modem is present.

For each modem (A&B), Test 19 actions are

- Clears the NGI (see heading 19.7.1 for details)
- Sets up the Modem Jabber test IOCB specifying the Modem (A or B) and including the proper checksum.
- Executes the test IOCB (see heading 19.7.4 for details).
- Does two Board Response Checks to verify the results; the first immediately after IOCB completion and the second after a 250 ms delay.

Test 20 Test/Normal and Node Address Switches

SUB

Checks the NGI's ability to correctly read the Test/Normal switch and the node address switches on the NGI PWB. Operator intervention is required.

Test 20 reports the state of the Test/Normal switch when the test is started and every time the switch is flipped. Also the rest of the NGI data is checked for unexpected changes.

Test 20 loops continuously until it receives a STOP or ABORT command. A warning alarm is issued on exit if the Test/Normal switch state is not Normal.

Test 20 actions are

- Clears the NGI (see heading 19.7.1 for details)
- Reads the Configuration Status register to determine the initial state of the switch.
- Reports the initial state of the switch as an information message (not an alarm).

Subsequently, Test 20 continuously reads the state of the switch and reports each change. Following each state change, Test 20 does two Board Response Checks; the first immediately after reporting the state change and the second after a 250 ms delay. It also continuously monitors the test mode looking for a STOP or ABORT command from the operator. A warning alarm is displayed if the switch is set to "Test" when Test 20 exits (NGIF tests do not run well with the switch set to "Test"). This test **must** be the only test selected.

Test 21 Token Bus Controller Internal/External Loopback

EXER

Exercises the NGI's data paths, randomly alternating between using the Internal and Modem A and Modem B loopback options of the Full duplex loopback test IOCBs. If neither a modem nor external loopback plug is available, testing is performed using internal loopback.

NOTE

At present, the NGI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in NGIF to provide for the eventuality of two separate modems.

Test 21 does not loop internally; the operator must set up for multiple passes to get the full exerciser effect. A random data pattern is used for each pass of this test.

All of the NGI interaction in Test 21 is performed by the sequence that executes a TBC Full Duplex loopback test IOCB (see heading 19.7.5 for details).

Test 22 Peer-to-Peer Communications

SUB

This utility is used to verify NGIs that are connected to a common network are able to communicate with each other. Up to eight NGIs can be connected to the network. The operator parameter "Address_List" contains the network addresses of the the NGIs that are available for communication. This test requires a modem. The operator parameter "MODEM_AVAILABLE" must be "TRUE" and "EXTERNAL_LOOPBACK" must be "FALSE."

Because candidate NGIs may be located on different LCN networks, the test program relies on the operator for coordinating the execution of the test using the following sequence.

1. Set "TRANSMIT_ENABLE" to "FALSE" for each NGI node.
2. Start NGIF Test 22 for each NGI node.

The test will perform a series of NGI startup checks and then place the NGI on-line and in-ring.

3. When all NGIs are in-ring, the operator may initiate communications and error reporting by setting the parameter “TRANSMIT_ENABLE” to “TRUE” for each node.

While “TRANSMIT_ENABLE” is “TRUE,” NGIF will transmit a message to each address in the list (except its own if found). Messages are sent one at a time to each NGI. If 5 seconds elapse without an answer, an alarm will be printed. If the NGI answers, the timeout is cancelled, and the test moves on to the next entry in the list. Each time NGIF reaches the end of the list, it increments the test pass count and returns to the beginning of the list. Note that the test does not actually complete passes. The pass count is incremented only to show test progress. None of the pass count driven control options operate (for example, halting after a given number of passes, sequential running, etc.).

4. If the operator wants to bring Test 22 to an orderly stop, “TRANSMIT_ENABLE” is set to “FALSE” for each node. Each copy of NGIF can now be stopped or aborted. When the second to the last copy is stopped, the remaining copy will announce a ring drop. This is normal since there are no remaining nodes to pass the token.

The first time NGIF communicates successfully with a given node, a brief message is printed that announces it has established communication with that node. The message contains the network address of the other node for tracking/troubleshooting purposes.

This test **must** be the only test selected.

19.7 Common Test Sequences

The following common sequences are used in several NGIF tests. Their descriptions are included here (once) to avoid repeating the information in the description of each test.

19.7.1 Clear the NGI

This sequence clears the NGI board and related data. It sets all the NGI registers and externally visible data to a known state (mostly zeros). By setting the board to a known state, an extensive Board Response Check can be performed after each test (or test phase) to check for the expected changes and to make sure that nothing else has changed.

The following is a more detailed description of how NGIF interfaces to the NGI hardware in order to perform this function:

- **ABORTs the NGI:** Writes an ABORT command, without interrupt, to the NGI's Command register.

NOTE

NGIF executes a NGI Command by:

- Writing a command word to the NGI's Command register
 - Waiting for the command to complete (Command register not busy) or for a timeout
- Timeout length depends on the command. If the NGI times out, NGIF issues an alarm that specifies which command failed.

- Sets the subsystem control flags to a default state: For each flag write a word to the Subsystem Flag Control register specifying the flag and the new state (TRUE = ENABLE; FALSE = DISABLE). The "clear all flags" function is not used. The default state of the subsystem control flags is
 - DISABLE NGI hardware interrupts
 - ENABLE NGI passed module test flag (green LED)
 - DISABLE NGI synchronization timer
 - DISABLE NGI watchdog timer
- Initializes CCBs, Priorities, and FIFOs:
 - Initialize the CCBs—Write the test IOCB address and the CCB ID to the NGI's CCBs 0-3 (the address of the NGI's CCBs is calculated based on the slot number of the NGI board; note that the CCBs are in CPU RAM).

NOTE

NGIF uses only one IOCB at a time, therefore, NGIF does not need to use the circular IOCB chains supported by the NGI. This document refers to the IOCB used by NGIF as the "test IOCB."

- Initialize the priority control structures in NGI/CPU shared RAM by—
 1. Writing the test IOCB address as the first IOCB in the priority's circular IOCB chain.
 2. Setting the number of IOCBs executed by the NGI to zero.
 3. Setting the number of IOCBs issued by NGIF to zero.
- Set up Receive buffer FIFOs for NGI priority 0—These structures in NGI/CPU shared RAM are not used during the execution of NGIF by the NGI or by NGIF, but are set up and initialized for the sake of completeness.
- Initializes the test IOCB to a NO OP: The IOCB is in CPU RAM and is initialized by the CPU writing directly to RAM and setting the IOCB fields as follows:
 - NOT a group address
 - NOT a type 3 transmission
 - Re-synchronization NOT needed
 - NO OPERATION
 - 0 Frame size
 - 0 Checksum (calculated immediately before execution)
 - Null (\$080000) Transmit buffer pointer
 - NOT Complete
 - NOT a Resynchronize error
 - Successful status
 - 0 Retries
 - No completion interrupt
 - End of Chain
 - IOCB ID
 - DO NOT Execute the next IOCB
 - Null (\$000000) next IOCB pointer
- Does a Board Response Check (see heading 19.7.3 for details).

19.7.2 Reset the NGI

This sequence puts the NGI through its Reset/Self-Test sequence and verifies that all self-tests have passed. The Configuration Status register is checked for proper operation while the NGI is in the RESET state. A Board Response Check is done after a successful RESET/Self-Test to verify that the NGI data matches what is expected after a RESET.

The following is a more detailed description of how NGIF interfaces to the NGI hardware in order to perform this function:

- Sets Passed module test flag to ENABLE (TRUE = Green LED on): This is done by writing a word to the Subsystem flag control register specifying
 - Passed module test flag
 - ENABLE/TRUE
 - DO NOT Clear all flags
- Puts NGI in RESET by setting the RESET flag to ENABLE (TRUE): This is done by writing a word to the Subsystem flag control register specifying
 - RESET flag
 - ENABLE/TRUE
 - DO NOT Clear all flags
- Checks Configuration Status register for expected contents when NGI is in RESET.
- Takes the NGI out of RESET by setting the RESET flag to DISABLE (FALSE): This is done by writing a word to the Subsystem Flag Control register specifying
 - RESET flag
 - DISABLE/FALSE
 - DO NOT Clear all flags

At this point the NGI starts its self-test sequence.

- Waits for the self-test sequence to finish or for time-out (up to 6 seconds). A successful self-test is indicated by
 - Board not failed
 - Passed module test flag = TRUE
 - Self-test that failed = 0
- If the self-test is successful, a Board Response Check is done to verify that the NGI data matches what is expected after a RESET.
- If the self-test is not successful or does not finish within the allowed time, an alarm is issued and any NGIF tests that are running are aborted.

19.7.3 Board Response Check

This sequence compares all NGI-related data to specified target (should be) data using a specified mask. The data compared includes

- In shared RAM
 - Hardware status
 - Configuration status
 - Interrupt vector
 - Command register
 - Operational status
 - Paged status words
 - Address switches
 - Network address in use
 - Group address mask
 - NGI internal error code
 - NGI internal error PC
 - Self-test that failed (valid only immediately after a self-test sequence)
- Paged status
 - Firmware revision
 - Firmware type
 - Device errors
 - Network status
 - Bus error address
 - Current IOCBs for priorities 0-3
 - Synchronization timer value
- Miscellaneous
 - Operational status for interrupt vectors 0-7
 - Test IOCB
 - Test buffer
 - DMA read test command location
 - DMA write test command location

If a mismatch occurs, an alarm is generated that—at abbreviation level 4—specifies what the test was doing, plus a description of the data item that did not match, what it was, what it should be, and the mask used for the comparison. For the test buffer only, the first 10 mismatches are listed and a summary of mismatches is displayed at the end of the alarm.

To execute a Board Response Check, NGIF first collects all the data and moves it to the CPU RAM. The comparison is then done from CPU RAM. Data is collected from the NGI as follows:

- **Shared RAM:** NGIF collects data in shared RAM by word-at-a-time copies (byte copies are not allowed).
- **Paged Status:** NGIF collects the paged status data by writing a PAGED STATUS command—without interrupt—to the NGI's Command register. The particular paged status is specified in the parameter field of the command word. Once the PAGED STATUS command is complete, NGIF reads the data from the Paged Status words in shared RAM a word at a time.
- **Miscellaneous:** This data already is in CPU RAM.

19.7.4 Execute Test IOCB

This sequence is used to execute a test IOCB through NGI priority 0 by

- Starting priority 0 by writing a PRIORITY START command—without a command completion interrupt—to the NGI's Command register with the command parameter field set to 0.
- Incrementing by one, the counter that specifies the number of IOCBs issued by NGIF. The NGI executes an IOCB when it detects differences between that counter and the counter that specifies the number of IOCBs that have been executed by the NGI. Both of these counters are in the Priority's control structure in shared RAM.
- Waiting for IOCB completion or timeout. Note that timeout is set according to IOCB type because some IOCBs take longer than others.
- If the IOCB times out, this sequence generates an alarm.

A delay of 100 ms, after IOCB completion, is included in this sequence in order to allow time for the NGI to set up the Operational status that corresponds to an End-of-Chain interrupt (generated even when interrupts are disabled).

19.7.5 Execute a TBC Full Duplex Loopback Test IOCB

This sequence is used to set up and execute a TBC Full Duplex Loopback IOCB using a specified loopback type and byte data pattern. This IOCB checks the proper operation of the TBC's serial interface and the FCS generation/checking logic. The TBC has separate transmit and receive checksum hardware, so normal testing verifies valid checksum detection. The data is read from the input portion of the test buffer, transmitted around the loopback path and written back to the output portion of the test buffer.

This test can be executed with any of three loopback types:

- Internal loopback
- Modem A loopback
- Modem B loopback

NOTE

At present, the NGI has only one physical modem so that a test using Modem A loopback does exactly the same as a test for Modem B loopback. These tests are kept separate in NGIF to provide for the eventuality of two separate modems.

For the specified loopback type and data pattern, this sequence

- Clears the NGI (see heading 19.7.1 for details)
- Sets up the TBC Full Loopback Test IOCB by specifying the loopback type, the test buffer address, and that the TBC is to generate the FCS and NOT copy it to the test buffer. Also, the proper checksum is calculated for the IOCB.
- Sets up the transmitter test buffer that controls the test with
 - The input area coated with the specified data pattern
 - The output area coated with the complement of the data pattern
 - The length output field set to the complement of the expected length
 - The error output field set to the complement of a successful status
- Executes the test IOCB (see heading 19.7.4 for details).
- After the IOCB is complete, delays 250 ms for the NGI to settle, then does a Board Response Check to verify the results.

19.7.6 Check for Modem Presence

This sequence is used to check for the presence of a modem by

- Clearing the NGI (see heading 19.7.1 for details).
- Clearing the configuration table temporary data area.
- Moving the address of the temporary data table to the paged status words.
- Executing a “Read Configuration Table” command.
- Executing a Board Response Check (see heading 19.7.3 for details) after a 250 ms delay to let the NGI settle.
- If the modem type field contains \$FFFF, a modem is not present.
- An alarm is issued if the presence/absence of a modem does not match the TRUE/FALSE value of the MODEM_AVAILABLE parameter.
- If a modem is present or EXTERNAL_LOOPBACK is TRUE, then indicate "modem present."

19.8 NGIF Alarm Categories

NGIF Alarm messages are formatted as described at heading 2.8.2 of the *Test System Executive* manual. The first digit of a 3-digit alarm code or the first two digits of a 4-digit alarm code is the number of the test reporting the failure. Specifics of the failure are contained in the test message text.

20 Process Network Interface Enhanced Tests (PNIE)

NOTE

With TPN R684, Enhanced Universal Control Network (EUCN) is introduced. In EUCN, the traditional coaxial cable system is no longer utilized and is replaced with a Fault Tolerant Ethernet (FTE) based communication network.

Throughout the document, unless specifically mentioned, the terms NIM, and UCN are interchangeably used with the terms ENIM, and EUCN respectively.

20.1 Process Network Interface Elements

The Process Network Interface consists of an EPNI/PNM board pair or an EPNI /NIM Modem in the context of a Network Interface Module.

20.2 Use Information

Only tests 1 through 13, 25, and 26 are for field use. The other tests require additional specialized equipment. In this mode, PNIE operates in a master/slave relationship with a copy of itself executing in a second node. When the master requires an incoming message, the slave will be asked to generate one. When the master wants to transmit, the slave is notified of the pending message and reports back to the master. Alarms may come from either master or slave. If a breakdown in LCN communications is detected, the master copy of PNIE will abort.

20.3 Setup Parameters

20.3.1 PNIE Specific Parameters

FAR_UCN_ADDRESS	Allowed values are 0 through 63 or “-”
FAR_LCN_ADDRESS	Allowed values are 0 through 128 or “-”
RDR_UCN_ADDRESS	Allowed values are 0 through 63 or “-”
SWITCH_REGISTER	Allowed values are 0 through 63 or “-”
MODEM_VER_REV	Allowed values are 0 to \$FFFF
FIRST_RANDOM	Allowed values are
LOOP_LIMIT	Allowed values are 1 to 100
PACKET_LIMIT	Allowed values are 0 to 1019
RDR_LIMIT	Allowed values are 0 to 1018

PERSIST_INTERVAL	Allowed values are 0 to 5
LLC_SOURCE	Allowed values are UPLOAD and EPROM
RANDOM_UCN_ADDRESS	Allowed values are ENABLED or DISABLED
DEBUG_COMMUNICATIONS	Allowed values are DISABLED
TIME_SYNC_SIMULATION	Allowed values are ENABLED or DISABLED
FAR_UCN_ADDRESS	Specifies the UCN address of the partner EPNI in decimal notation. Allowed values are 0–63 or “-”. PNIE will automatically translate this value into the appropriate address (0 = 128, 1–63 = 65–127). The default value = “-”.
FAR_LCN_ADDRESS	Sets the LCN address of the partner NIM in decimal notation. Allowed values are 0–128 or “-”. If set to the dash value, tests which require the cooperation of a second NIM will not be executed. The default value is “-”.
RDR_UCN_ADDRESS	Specifies the PM address to be used in the exerciser test (Test 25) in decimal notation. Allowed values are 0 to 63 or “-”. PNIE will automatically translate this value into the appropriate UCN address (0 = 128, 1–63 = 65–127). If “-” is entered, the exerciser test will perform modem loopback operations in place of RDR reads. The default value is “-”. Field users should set this to “-” to disable check.
SWITCH_REGISTER	Sets the expected value for the EPNI address switch register. Allowed values are 0 to 63 or “-”. If set to “-”, the contents of the EPNI address switch register will not be checked. If set to 0–63, PNIE will translate this value into the appropriate UCN address and expect the resulting value to appear in the EPNI address switch register (0 = 128, 1–63 = 65–127). The default value is 21.
MODEM_VER_REV	Set the expected value for the EPNI Modem Version Revision register in hexadecimal notation. Allowed values are 0 to \$FFFF. If set to 0, the contents of the EPNI Modem Version Revision register will not be checked. If set to \$0001–\$FFFF, PNIE will expect the specified value to appear in the EPNI Modem Version Revision register. The default value is \$0055. Field users should set this to “0”.
FIRST_RANDOM	Sets the starting random number used by the current test in hexadecimal notation. If the test is using random numbers, this number will be printed as part of the alarm message to allow faster replication of the error. All values are allowed. The default value is 0.
LOOP_LIMIT	Sets the maximum number of loops per test in decimal notation. Allowed values are 1–100. The default value is 5.

PACKET_LIMIT	Sets the maximum packet size in decimal bytes. Allowed values are 0–1019. The default value is 1019. Note: PNIE adds a two byte header to each message for tracking purposes. This header and the standard three byte UCN header added to 1019 results in a maximum transmitted frame size of 1024.
RDR_LIMIT	Sets the maximum size for an RDR reply in decimal bytes. Allowed values are 0–1019. The default value is 1018. Note: PNIE adds a 2-byte header to each message for tracking purposes. This header and the standard 4-byte UCN RDR header added to 1018, result in a maximum transmitted frame size of 1024 bytes.
PERSIST_INTERVAL	Controls the delay between initial error checks and delayed error checks in 50 millisecond intervals. Allowed values are 0–5. If 0 is entered, the delayed error check will not be performed. The default value is 1 (50 millisecond).
LLC_SOURCE	Sets the LLC source. If set to UPLOAD, the test will upload the LLC contained in the test image and use it for the duration of testing. If set to EPROM, the test will use the LLC embedded in the EPNI EPROMs. The default value is EPROM.
RANDOM_UCN_ADDRESS	Controls the generation of random UCN addresses and DSAPs will be chosen at random. If DISABLED, the EPNI will use the address encoded in the EPNI paddle card address switches and DSAPs will be based on pass number.
DEBUG_COMMUNICATIONS	Used for developer testing of the communications path between the two copies of PNIE. It should be left DISABLED.
TIME_SYNC_SIMULATION	Used only for developer testing. Should be left DISABLED.

20.3.2 General Parameters

TESTS	Allowed values are 1 through 13 and Test 26
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 1 through 32767 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

20.4 Preset Parameter Values for PNIE

RDR_NODE = 0

TESTS = 1-26

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER,SUSPENDED

SCALE = 100

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

20.5 Mode Characteristics for PNIE

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

20.6 Individual Process Network Interface Tests

(System Modes)

Test 01 EPNI Reset, Load/Verify Registers

SUB, MOD

Execute Reset/Self-Test sequence and verify that all self-tests are passed. This test also checks the integrity of the data paths between the memory and the Process Network Interface, and the ability of the Process Network Interface to store and retrieve data from the EPNI/Module processor shared RAM.

Test actions are

1. The EPNI is cleared (Clear_EPNI—Alarm 100).
2. The EPNI is run through a Reset sequence (RESET_EPNI—Alarm 110).
3. If the EPNI successfully passed self-test, the following sequence is executed:
 - a. The EPNI is checked against the model (Check_Domain using Check_Board_Response checklist—Alarm 120).
 - b. The data patterns to be used by the following operations are set up as follows:
 - #FFFF
 - \$5555
 - \$AAAA
 - \$0144
 - \$4401
 - \$DBDB
 - \$0000
 - A random 16-bit data value
 - The one complement of the random value above
 - c. The current pattern is moved to the Operational Status register, the EPNI model is updated to reflect the new operation Status contents. The EPNI is checked against the model (Check_Domain using the Check_Board_Response checklist—Alarm 130).
 - d. The following sequence is repeated for each paged status location (\$0C, \$0E, \$10, and \$12).
 - e. The current pattern is moved to the paged status register being checked and the EPNI model is updated to reflect the new register contents. The EPNI is checked against the model (Check_Domain using the Check_Board_Response checklist—Alarms 140, 141, 142, and 143 for the paged status locations #0C, #0E, #10, and \$12).
 - f. The EPNI is placed in a known state (Set_EPNI_State, Idle—Alarm 150).

Test 02 Module Bus Errors**SUB, MOD**

Verify that the EPNI can detect and report module bus parity errors during Module Processor writes. This test also verifies proper detection and reporting of DMA transaction errors using the test DMA read/write commands.

Test 02 actions are

1. If the module does not contain a full complement of memory, the following sequence is performed:
 - a. The CCB entry for priority zero is directed to a nonexistent area of memory.
 - b. The module processor is reserved to prevent interference with other tests that may be forcing bus errors.
 - c. Priority zero is started and the driver IOCB counter is set to force IOCB execution.
 - d. The module bus error indicators are cleared and the processor is released.
 - e. The model is set to reflect the final results of the operation.
 - f. The test delays for 50 ms to allow the PNIE time to settle after the bus error. The EPNI is then checked against the model for proper behavior (Check_Domain using the Check_Domain checklist—Alarm 200).
 - g. The EPNI is run through a reset sequence to clear all error indications (Reset_EPNI—Alarm 202).
 - h. The results of the reset are verified (Check_Domain using the Check_Domain checklist—Alarm 202).
 - i. The EPNI is placed into a known state and verified (Set_EPNI_State, Idle—Alarm 204).

2. If the module processor supports forcing bus parity errors, the following sequence is performed for each of the 16 bits in the command register starting at the most significant bit:
 - a. EPNI interrupts are enabled.
 - b. The module processor is reserved to prevent interference with other tests that may be forcing bus errors.
 - c. The data pattern is written to the EPNI with bad parity.
 - d. The module bus error indicators are cleared and the process is released.
 - e. The model is set to reflect the final results of the operation.
 - f. The test delays for 50 ms to allow the PNIE time to settle after the bus error. The EPNI is then checked against the model for proper behavior (Check_Domain using the Check_Domain checklist—Alarm 210).
 - g. Interrupts are disabled and the EPNI is placed into a known state and verified (Set_EPNI_State, Idle—Alarm 212).
3. If the memory board containing the test DMA cells is protected by EDAC, the following sequence is performed:
 - a. The LLC is unloaded by aborting the EPNI (Set_EPNI_State, Abort—Alarm 220).
 - b. The module processor is reserved to prevent interference with other tests which may be forcing bus errors.
 - c. A double bit EDAC error is created in the DMA read test cell (\$80410).
 - d. A test DMA read command is issued to the ENPI to force access to the test cell.
 - e. PNIE delays 50 ms to allow the EPNI to stabilize.
 - f. The EDAC error in the test cell is cleared, the module bus error indicators are cleared and the processor is released.
 - g. The model is set to reflect the final results of the operation.
 - h. The EPNI is then checked against the model for proper behavior (Check_Domain using the Check_Board_Response checklist—Alarm 222).
 - i. The EPNI is run through a reset sequence to clear all error indications (Reset_EPNI—Alarm 224).
 - j. The results of the reset are verified (Check_Domain using the Check_Domain checklist—Alarm 224).
 - k. The EPNI is placed into a known state and verified (Set_EPNI_State, Idle—Alarm 226).

Test 03 RAM/ROM TESTING**SUB,MOD**

Verify the EPROM's checksum and the operation of the EPNI's local RAM and the RAM shared by the Module Processor and the EPNI with exhaustive patterns. The EPNI's EPROM test command is used to verify the EPROM checksum. The EPNI's RAM test command is used to initiate the test of local RAM.

Test 03 actions are

1. The model is set up to reflect the conditions expected at the completion of the RAM/ROM test commands.
2. The EPNI is commanded to run one pass of the Shared RAM test.
3. The results of the test are verified (Check_Domain using the Check_Board_Response checklist—Alarm 300).
4. The EPNI is commanded to run one pass of the local RAM test.
5. The results of the test are verified (Check_Domain using the Check_Board_Response checklist—Alarm 310).
6. The EPNI is commanded to run one pass of the EPROM test.
7. The results of the test are verified (Check_Domain using the Check_Board_Response checklist—Alarm 320).
8. The EPNI is placed in a known state and verified (Set_EPNI_State,Idle — Alarm 330).

Test 04 Configuration Status/Subsystem Flag Control

SUB, MOD

Check the operation of the Configuration Status register (read) and the Subsystem Flag Control (write). This includes assuring that the EPNI hardware (not firmware) responds correctly to valid commands to the Subsystem Flag Control register (Module Processor writes to the Configuration Status register). This test verifies that the requested changes occur in the Hardware Status register and the Configuration Status register (read). It also verifies that the rest of the EPNI domain remains unchanged.

Test 04 actions are

1. The EPNI is placed in reset mode using the Subsystem Flag Control register.
2. The EPNI is released from reset mode and allowed to perform self-test by writing a “clear all flags” operation to the Subsystem Flag Control register.
3. PNIE wait for the EPNI to complete self-test.
4. The model is set to reflect the expected state of the EPNI immediately after completion of self-test.
5. The EPNI is checked against the model (Check_Domain using the Check_Board_Response checklist—alarm 400).
6. The EPNI is aborted to establish a starting point for the remainder of the test (Set_EPNI_State,Abort_State—Alarm 402).
7. EPNI hardware interrupts are enabled and the model is updated to reflect the new interrupt state.
8. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 404).
9. EPNI hardware interrupts are disabled and the model is updated to reflect the new interrupt state.
10. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 406).
11. EPNI hardware interrupts are re-enabled and the model is updated to reflect the new interrupt state.
12. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 408).
13. EPNI hardware interrupts are again disabled and the model is updated to reflect the new interrupt state.
14. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 410).

15. The “Passed Module Test” flag is reset to extinguish the green “Passed Module Test” LED and the model updated.
16. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 412).
17. The “Passed Module Test” flag is set to illuminate the green “Passed Module Test” LED and the model is updated.
18. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 414).
19. The CPU watchdog is enabled and allowed to expire.
20. The model is updated to reflect the anticipated condition of the EPNI.
21. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 416).
22. The CPU watchdog is disabled and the model is updated to reflect the anticipated condition of the EPNI.
23. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 418).
24. The EPNI is run through a reset sequence to clear all error indications (Reset_EPNI—Alarm 420).
25. If the EPNI successfully passed all self-tests, it is placed in a know state and verified (Set_EPNI_State,Idle_State—Alarm 422).
26. If the EPNI did not complete self-test, an alarm message is issued and PNIE aborts.

Test 05 Illegal Commands

SUB, MOD

The purpose of this test is to exhaustively test the EPNI's board and priority related interrupt generation logic. This includes testing the logic that generates the Operational status word.

Test 05 actions are

1. EPNI interrupts are enabled.
2. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 500).
3. Device status bits of EPNI2 are read to check the status of FTE cable A and cable B. If they are unplugged or disconnected Alarm 505 will be generated with an error.

Note: This action applies only to EPNI2 card and not to PNI, and EPNI cards.

4. The following sequence is repeated as determined by the "LOOP_LIMIT" parameter:
 - a. EPNI paged status locations \$0C, \$0E, \$10, and \$12 are set to zero.
 - b. An illegal command is created.
 - c. The model is set to reflect the expected state of the EPNI after execution of the illegal command.
 - d. The illegal command is issued to the EPNI (Command_EPNI).
 - e. The EPNI is checked against the model to verify proper operation (Check_Domain using the Check_Board_Response checklist—Alarm 510).
 - f. The model is set to reflect the expected state of the EPNI after execution of a legal command.
 - g. An NOP command is issued to the EPNI to verify it is still functional (Command_EPNI).
 - h. The EPNI is checked against the model to verify proper operation (Check_Domain using the Check_Board_Response checklist—Alarm 520).
4. The EPNI is placed in a known state and verified (Set_EPNI_State, Idle_State_Alarm 530).

Test 06 VERYPAGED STATUS**SUB, MOD**

The purpose of this test is to read and verify the contents of the Paged Status Register. This includes reading several different paged status words.

Test 06 actions are

1. The EPNI paged status values are saved.
2. The data patterns to be used by the following operations are set up as follows:
 - \$FFFF
 - \$5555
 - \$AAAA
 - \$0144
 - \$4401
 - \$DBDB
 - \$0000
 - A random 16-bit data value
 - The ones complement of the random value above
3. The following sequence is repeated for each paged status value.
 - Firmware Version/Revision
 - Device_Status
 - Bus Error Address
 - Current IOCB 0 Address
 - Current IOCB 1 Address
 - Current IOCB 2 Address
 - Current IOCB 3 Address
 - a. The current pattern is moved to all four paged status word locations and the model is updated accordingly (\$0C, \$0E, \$10, and \$12).
 - b. The EPNI is compared to the model (Check_Domain using the Check_Board_Response checklist—Alarm 601 for Firmware Version/Revision, 602 for Device_Status, 603 for Bus Error Address, 604 for Current IOCB 0 Address, 605 for Current IOCB 1 Address, 606 for Current IOCB 2 Address, 607 for Current IOCB 3 Address).
 - c. The model is restored from the paged status values saved at the start of the test.

- d. If the current paged status value is Firmware Version and Revision, the contents of the even byte ROM and odd byte ROM are swapped to make sure they match.
- e. The current paged status value is read from the EPNI.
- f. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 611 for Firmware Version/Revision, 612 for Device_Status, 613 for Bus Error Address, 614 for Current IOCB 1 Address, 615 for Current IOCB 1 Address, 616 for Current IOCB 2 Address, 617 for Current IOCB 3 Address).
- g. The EPNI is placed in a known state (Set_EPNI_State,Idle—Alarm 620).

Test 07 Local Time Sync Gate Array Operations

SUB, MOD

This test is to verify proper operation of the Time Sync Gate Array in diagnostic mode. This includes free running counter operation, UCN and AUX latching, and clearing and proper operation of the Time Sync status word.

Test 07 actions are

1. The LLC is unloaded and EPNI hardware interrupts are enabled (Set_EPNI_State, Abort_State).
2. The Time Sync Gate Array portion of the model is unmasked to make it activate.
3. The Time Sync Gate Array mode selection is tested using the following sequence:
 - a. The gate Array is set to transmit mode (Command_Time_Sync_GA).
 - b. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 700).
 - c. The Gate Array is set to receive mode (Command_Time_Sync_GA).
 - d. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 710).
4. The Time Sync Gate Array diagnostic counter operations are tested using the following sequence. The sequence is repeated as determined by the "LOOP_LIMIT" parameter:
 - a. A random 28-bit counter value is loaded into the Time Sync Gate Array (Preset_Time_Sync_Counter).
 - b. The UCN latch address is referenced to set the UCN flag and transfer the current counter value to the UCN latch (Command_Time_Sync_GA).
 - c. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 730).

- d. A random 28-bit counter value is loaded into the Time Sync Gate Array (Preset_Time_Sync_Counter).
 - e. The AUX latch address is referenced to set the AUX latch flag and transfer the current counter value to the AUX latch. (Command_Time_Sync_GA)
 - f. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 732).
 - g. A random 28-bit counter value is loaded into the Time Sync Gate Array (Preset_Time_Sync_Counter).
 - h. The UCN latch address is referenced to set the UCN error flag and transfer the current counter value to the UCN latch (Command_Time_Sync_GA).
 - i. The results are verified (Check_Domain using the Check_Board_Response Checklist—Alarm 734).
 - j. A random 28-bit counter value is loaded into the Time Sync Gate Array (Present_Time_Sync_Counter).
 - k. The AUX latch address is referenced to set the AUX error flag and transfer the current counter value to the AUX latch. (Command_Time_Sync_GA).
 - l. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 736).
 - m. The UCN clear address is referenced to clear the UCN latch and error flags.
 - n. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 738).
 - o. The AUX clear address is referenced to clear the AUX latch and error flags.
 - p. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 740)
5. The following sequence is used to test the "Clear All" function in the Time Sync Gate Array:
- a. The UCN latch address is referenced to set the UCN latch flag (Command_Time_Sync_GA).
 - b. The UCN latch address is referenced to set the UCN error flag (Command_Time_Sync_GA).
 - c. The AUX latch address is referenced to set the AUX latch flag (Command_Time_Sync_GA).
 - d. The AUX latch address is referenced to set the AUX error flag (Command_Time_Sync_GA).

- e. The diagnostic address is referenced to set the diagnostic flag (Command_Time_Sync_GA).
 - f. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 750).
 - g. The clear all address is referenced to reset the flags noted above (Command_Time_Sync_GA).
 - h. The results are verified (Check_Domain using the Check_Board_Response checklist—Alarm 752).
6. The following sequence is executed to verify proper operation of the Time Sync Gate Array message decoder:
- a. The Time Sync Gate Array is placed in the transmit mode (Command_Time_Sync_GA).
 - b. An IOCB and transmit buffer are reserved and initialized.
 - c. The following sequence is repeated for each bit in the UCN group address field:
 - (1) The EPNI IOCB is refreshed from the model.
 - (2) The current test UCN group address is transferred to the transmit buffer.
 - (3) The Time Sync Gate Array clear all address is referenced to initialize the Gate Array (Command_Time_Sync_GA).
 - (4) The Time Sync Gate Array 28-bit counter is set to a random value (Preset_Time_Sync_Counter).
 - (5) The Time Sync Gate Array message decoder is set for the current group (Command_Time_Sync_GA).
 - (6) The test IOCB is executed to set the UCN latch flag and transfer the current counter value to the UCN latch (Execute_IOCB).
 - (7) The model is set to the expected state.
 - (8) The EPNI is compared against the model (Check_Domain using the Check_All checklist—Alarm 760).
 - d. The Time Sync Gate Array 28-bit counter is set to random value (Preset_Time_Sync_Counter).
 - e. The EPNI IOCB is refreshed from the model.
 - f. The test IOCB is executed to set the UCN error flag and transfer the current counter value to the UCN latch (Execute_IOCB).

- h. The EPNI is compared against the model (Check_Domain using the Check_All checklist—Alarm 762).
 - i. The extra bit is set in the UCN group address and the results address is transferred to the transmit buffer.
 - j. The Time Sync Gate Array clear UCN latch address is referenced to clear the UCN latch and error flags (Command_Time_Sync_GA).
 - k. The test IOCB is executed; the UCN latch should remain unchanged (Execute_IOCB).
 - l. The model is set to the expected state.
 - m. The EPNI is compared against the model (Check_Domain using the Check_All checklist—Alarm 764).
 - n. The Time Sync Gate Array is set back to the receive mode (Command_Time_Sync_GA).
- 7. Place the Time Sync Gate Array in the receive mode and repeat the actions (b) through (n) listed in step (6).
 - 8. The Time Sync Gate Array clear all address is referenced to clear the Gate Array (Command_Time_Sync_GA).
 - 9. The Time Sync Gate Array portion of the model is masked out to deactivate it.
 - 10. The EPNI is placed in a known state and verified (Set_EPNI_State_Idle_State_Alarm 790).

Test 08 CCB and IOCB ERRORS, IOCB RELATED INTERRUPTS SUB, MOD

This test verifies the EPNI can detect and reject CCBs and IOCBs with format errors. Invalid CCB ID's, invalid IOCB addresses, invalid IOCB IDs, invalid IOCB checksums and random invalid IOCB op codes are used to verify proper operation of the EPNI's CCB and IOCB validity checking.

Test 08 actions are

1. The following sequences are used to verify detection of invalid CCB ID codes, invalid IOCB addresses, and odd IOCB addresses and is repeated for each EPNI priority:
 - a. A precheck NOP IOCB is built and executed.
 - b. The model is adjusted to reflect completion of the IOCB.
 - c. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 802).
 - d. An invalid CCB ID code is generated and moved to the EPNI visible CCB.
 - e. The model is adjusted to reflect the final expected state of the EPNI.
 - f. The current EPNI priority is started.
 - g. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 804).
 - h. A post-check NOP IOCB is built and executed.
 - i. The model is adjusted to reflect completion of the IOCB.
 - j. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 810).
 - k. A precheck NOP IOCB is built and executed.
 - l. The model is adjusted to reflect completion of the IOCB.
 - m. The EPNI is compared against the model (Check_Domain using the Check_Domain Checklist—Alarm 802).
 - n. An IOCB address less than \$80000 is moved to the EPNI visible CCB.
 - o. The model is adjusted to reflect the final expected state of the EPNI.
 - p. The current EPNI priority is started.
 - q. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 806).
 - r. A post-check NOP IOCB is built and executed.
 - s. The model is adjusted to reflect completion of the IOCB.
 - t. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 810).

- u. A precheck NOP IOCB is built and executed.
 - v. The model is adjusted to reflect completion of the IOCB.
 - w. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 802).
 - x. An odd IOCB address is generated and moved to the EPNI visible CCB.
 - y. The model is adjusted to reflect the final expected state of the EPNI.
 - z. The current EPNI priority is started.
 - aa. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 808).
 - ab. A post-check NOP IOCB is built and executed.
 - ac. The model is adjusted to reflect completion of the IOCB.
 - ad. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 810).
2. The following sequence is used to verify detection of invalid IOCB IDs and invalid IOCB checksums:
- a. A precheck NOP is built and executed.
 - b. The model is adjusted to reflect completion of the IOCB.
 - c. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 812).
 - d. An NOP IOCB is built with an invalid IOCB ID made visible to the EPNI.
 - e. The model is adjusted to reflect the final expected state of the EPNI.
 - f. The current EPNI priority is started.
 - g. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 814).
 - h. An NOP IOCB with the interrupt flag set is built and executed.
 - i. The model is adjusted to reflect completion of the IOCB.
 - j. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 818).

- k. A post-check NOP IOCB is built and executed.
 - l. The model is adjusted to reflect completion of the IOCB.
 - m. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 820).
 - n. A precheck NOP IOCB is built and executed.
 - o. The model is adjusted to reflect completion of the IOCB.
 - p. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 812).
 - q. An NOP IOCB is built and the checksum is invalidated by setting it to the one's complement of the correct value and made visible to the EPNI.
 - r. The model is adjusted to reflect the final expected state of the EPNI.
 - s. The current EPNI priority is started.
 - t. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 816).
 - u. An NOP IOCB with the interrupt flag set is built and executed.
 - v. The model is adjusted to reflect completion of the IOCB.
 - w. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist Alarm 818).
 - x. A post-check NOP IOCB is built and executed.
 - y. The model is adjusted to reflect completion of the IOCB.
 - z. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 820).
3. The following sequence is used to verify detection of invalid IOCB operation codes. Only one EPNI priority is tested each time this test is run. The priority tested is determined by the current PNIE pass count MOD 4. The number of illegal IOCB operation codes tested for each priority is controlled by the LOOP LIMIT parameter.
- a. A precheck NOP IOCB is built and executed.
 - b. The model is adjusted to reflect completion of the IOCB.
 - c. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 822).
 - d. An IOCB is built with an invalid IOCB operation code and made visible to the EPNI.
 - e. The model is adjusted to reflect the final expected state of the EPNI.
 - f. The current EPNI priority is started.

- g. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 824).
 - h. A post-check NOP IOCB is built and executed.
 - i. The model is adjusted to reflect completion of the IOCB.
 - j. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 826).
4. The EPNI is placed in a known state and verified (Set EPNI State, Idle —Alarm 830).

Test 09 IOCB Chaining**SUB, MOD**

This test verifies EPNI can process IOCB chains containing multiple IOCBs. The EPNI's response to both circular and straight-line IOCB chains is tested.

Test 09 actions are

1. The following sequence is repeated for each EPNI priority and for both straight-line and circular IOCB chains:
 - a. EPNI hardware interrupts are enabled and the model is adjusted as required.
 - b. The EPNI is compared to the model (Check_Domain using the Check_Board_Response checklist—Alarm 900).
 - c. Three IOCBs with the completion interrupt flag set are linked together to form the IOCB chain.
 - d. If using a straight-line chain, the end-of-chain flag is set in the last IOCB.
 - e. The model is adjusted to reflect the expected state of the EPNI.
 - f. The current EPNI priority is started (Start_Priority).
 - g. The EPNI is compared against the model (Check_Domain using the Check_Domain checklist—Alarm 910 for straight-line chains and Alarm 920 for circular chains).
2. The EPNI is placed in a known state and verified (Set_EPNI_State, Idle—Alarm 930).

Test 10 EPNI Module Memory DMA Addressing

SUB, MOD

This test verifies the EPNI's ability to access module memory via DMA.

Test 10 actions are

1. The LLC is loaded (Set EPNI State, Abort).
2. The model is set up to check the results of the Test DMA Read command.
3. The following sequence is repeated based on the "LOOP_LIMIT" parameter:
 - a. A random pattern is generated.
 - b. Paged status locations \$0C, \$0E, \$10, and \$12 are precoated with the one's complement of the current pattern.
 - c. The module processor is reserved to prevent interference with other tests that may be using the DMA test cells.
 - d. The current test pattern is written to the test DMA read cell (\$80410).
 - e. The model is adjusted to reflect the final state of the EPNI.
 - f. A Test DMA Read command is issued to the EPNI (Command EPNI).
 - g. The contents of the DMA read test cell are copied into the model to check for unexpected changes.
 - h. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 1010).
4. The model is set up to check the results of the Test DMA Write command.
5. The following sequence is repeated based on the LOOP_LIMIT parameter:
 - a. A random pattern is generated.
 - b. Paged status locations \$0C, \$0E, \$10, and \$12 are precoated with the one's complement of the current pattern.
 - c. The module processor is reserved to prevent interference with other tests which may using the DMA test cells.
 - d. The current test pattern is written to paged status location \$0E.
 - e. The model is adjusted to reflect the final state of the EPNI.
 - f. A Test DMA Write command is issued to the EPNI (Command EPNI).
 - g. The contents of the DMA write test cell are copied into the model to check for unexpected changes.
 - h. The EPNI is compared against the model (Check_Domain using the Check_Board_Response checklist—Alarm 1020).
6. The EPNI is placed in a known state and verified (Set EPNI State, Idle — Alarm 1030).

Test 11 External (MCPU) Watchdog Timer**SUB, MOD**

The purpose of this test is to verify the proper operation/expiration of the external (MCPU) watchdog timer on the EPNI board:

Test 11 actions are

1. EPNI hardware interrupts are enabled and the model is adjusted as required.
2. The EPNI is compared to the model (Check_Domain using the Check_Board_Response checklist—Alarm 1100).
3. The external watchdog is kicked to prevent a false trigger.
4. The external watch is enabled (Alarm 1110).
5. The external watchdog is kicked every 50 ms for a 1-second period to verify proper hold-off operation.
6. The external watchdog is disabled (Alarm 1120).
7. The program delays 50 ms.
8. A NOP command is issued to the EPNI to verify that it is still responsive (Command EPNI).
9. The EPNI response verified (Check_Domain using the Check_Board_Response checklist—Alarm 1130).
10. The external watch is reenabled.
11. The external watchdog is kicked every 50 ms for a 1-second period.
12. The program delays up to 500 ms waiting for the external watchdog to time out.
13. A NOP command is issued to the EPNI to verify that it is still responsive (Command EPNI).
14. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1140).
15. If the external watchdog timed out in less than 100 ms, or more than 300 ms, an alarm is generated (Alarm 1150).
16. The external watchdog is disabled.
17. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1160).
18. The EPNI is run through a reset sequence.
19. The EPNI is placed in a known state and verified (Set_EPNI_State_Idle—Alarm 1170).

Test 12 Test IOCBs

SUB, MOD

The purpose of this test is to check for proper operation of the following Test IOCBs:

- TBC Host DMA Interface Test IOCB

This portion of the test uses the LLC firmware to execute the TBC's built in DMA test. This will find problems with the TBCs DMA facilities.

- TBC Transmitter TEST

This portion of the test uses the LLC firmware to execute the TBD's built in transmitter test. This test will find problems in the TBC controller and support circuitry.

- TBC Receiver Test

This test uses the LLC firmware to execute the TBC's built in receiver test. This test will find problems in the TBC controller and support circuitry.

- TBC Full-Duplex Test

This portion of the test verifies the TBC's internal and external loop back operations. This verifies the TBC and support circuitry. It consists of two passes: one with a worst case data pattern and one with random data. A failure could be caused by failures in either the Transmitter or Receiver as both capabilities are used.

- Modem Jabber Test

This portion of the test verifies the modem's ability to detect a jabbering cable while the modem is transmitting.

- Cable Switch Static Test

The purpose of this portion of the test is to verify the proper operation of the EPNI's Silence Detect timer, cable swapping hardware and related interrupt logic.

Test 12 actions are

1. The LLC is unloaded and interrupts are enabled (Set_EPNI_State, Abort).
2. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1200).
3. The data patterns to be used by the following operations are set up as follows:
 - \$FF
 - \$55 . \$M
 - \$14
 - \$41
 - \$DB
 - \$00A random 8-bit data value
 - The one's complement of the random value above
4. The following sequence is repeated for each data pattern to verify proper operation of the TBC DMA Loopback IOCB:
 - a. An IOCB is reserved and set up with the correct operation code and data path selection.
 - b. The test buffer input area is filled with the current data pattern.
 - c. The test buffer output area is filled with the one's complement of the current data pattern.
 - d. The last word of the test buffer is set to \$0000.
 - e. The model is set to reflect the expected results.
 - f. The test IOCB is executed (Execute IOCB).
 - g. The model is set to reflect the expected operational status.
 - h. The EPNI response is verified (Check_Domain using the Check_All checklist Alarm—1210).
5. The two random data patterns are checked to verify that they do not appear to the TBC as a "Mac control request for RWR." If either pattern is invalid, two new patterns are selected.

6. The following sequence is repeated for the internal TBC receiver test, the Modem A TBC receiver test and the Modem B TBC receiver test:
 - a. An IOCB is reserved and set up with the correct operation code and data path selection.
 - b. The test buffer is set up to indicate a random number of frames between 1 and 5.
 - c. The test buffer is set up to indicate a random frame length between 16 and 33 bytes.
 - d. The test buffer is set up to indicate a random preamble pattern.
 - e. The test buffer is set up with the current data pattern.
 - f. The model is set to reflect the expected results.
 - g. The test IOCB is executed (Execute IOCB).
 - h. The model is set to reflect the expected operational status.
 - i. The EPNI response is verified (Check_Domain using the Check_All checklist—Alarm 1220 for internal data path, 1222 for Modem A path and 1224 for Modem B path).
7. The following sequence is repeated for the internal TBC transmitter test, the Modem A TBC transmitter test and the Modem B TBC transmitter test:
 - a. An IOCB is reserved and set up with the correct operation code and data path selection.
 - b. The test buffer is set up to indicate a random frame length between 2 and 1024 bytes.
 - c. The test buffer is set up to indicate a random number of frames between 1 and 15.
 - d. The model is set to reflect the expected results.
 - e. The test IOCB is executed (Execute IOCB).
 - f. The model is set to reflect the expected operational status.
 - g. The EPNI response is verified (Check_Domain using the Check_All checklist—Alarm 1230 for internal data path, 1232 for Modem A path and 1234 for Modem B path).

8. The following sequence is repeated for the internal TBC full-duplex test, the Modem A TBC full-duplex test and the Modem B TBC full duplex test:
 - a. An IOCB is reserved and set up with the correct operation code and data path selection.
 - b. The test buffer input area is filled with the current data pattern.
 - c. The test buffer output area is filled with the one's complement of the current data pattern.
 - d. The length byte of the test buffer is set to the one's complement of the expected length.
 - e. The error byte of the test buffer is set to \$FF.
 - f. The model is set to reflect the expected results.
 - g. The test IOCB is executed (Execute IOCB).
 - h. The model is set to reflect the expected operational status.
 - i. The EPNI response is verified (Check_Domain using the Check_All checklist—Alarm 1240 for internal data path, 1242 for Modem A path and 1244 for Modem B path).
9. The following sequence is repeated for the Modem A jabber timer test and the Modern B jabber timer test:
 - a. An IOCB is reserved and set up with the correct operation code and data path selection.
 - b. The model is set to reflect the expected results.
 - c. The test IOCB is executed (Execute_IOCB).
 - d. The model is set to reflect the expected operational status.
The EPNI response is verified (Check_Domain using the Check_All checklist Alarm 1250 for Modem A path and 1252 for Modem B path).
10. The following sequence is used to perform the static cable switch test:
 - a. An IOCB is reserved and set up with the correct operation code and data path selection.
 - b. The model is set to reflect the expected results.
 - c. The test IOCB is executed (Execute_IOCB).
 - d. The model is set to reflect the expected operational status.
 - e. The EPNI response is verified (Check_Domain using the Check_All checklist—Alarm 1260).
11. The EPNI is placed in a known state and verified (Set_EPNI_State, Idle—Alarm 1290).

Test 13 Local SAP Operation

SUB, MOD

This test verifies the EPNI can open and close SAPs with and without RDR responses and can recognize illegal SAP operations. Network SAP operations are tested in the various communication tests.

Test 13 actions are

1. EPNI hardware interrupts are enabled and the EPNI response is verified (Check_Domain using the Check_Board_Response checklist -Alarm 1300).
2. An attempt is made to set the UCN address to globally administered 0.
3. The model is set to reflect an illegal command status.
4. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1302).
5. An attempt is made to set the UCN address to locally administered 0.
6. The model is set to reflect an illegal command status.
7. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1304).
8. An attempt is made to set the UCN address to random, valid, locally administered address.
9. The model is set to reflect successful command completion.
10. The EPNI response is verified (Check-Domain using the Check_Board_Response checklist—Alarm 1306).
11. The current locally administered address is converted to a globally administered address and an attempt is made to set the UCN address.
12. The model is set to reflect an illegal command status.
13. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1308).
14. The current globally administered address is converted to a valid group address and an attempt is made to set the UCN address.
15. The model is set to reflect an illegal command status.
16. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1310).
17. An out of range, globally administered address is created and an attempt is made to set the UCN address.
18. The model is set to reflect an illegal command status.

19. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1312).
20. An out of range, locally administered address is created and an attempt is made to set the UCN address.
21. The model is set to reflect an illegal command status.
22. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1314).
23. A broadcast address is created and an attempt is made to set the UCN address.
24. The model is set to reflect an illegal command status.
25. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1316).
26. An attempt is made to set the UCN address to random, valid, locally administered address.
27. The model is set to reflect successful command completion.
28. The EPNI response is verified (Check Domain using the Check Board Response checklist—Alarm 1318).
29. A valid random SAP is generated.
30. Two RDR response buffers are created.
31. An odd RDR buffer address is created by adding one to a valid RDR buffer address.
32. A random RDR buffer address less than \$80000 is created.
33. An attempt is made to set up a non-RDR SAP without setting the response address to 0.
34. The model is set to reflect an illegal command status.
35. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1320).
36. An attempt is made to set up an RDR SAP without setting a valid response address.
37. The model is set to reflect an illegal command status.
38. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1322).
39. The SAP response is set to 0 and an attempt is made to open a non-RDR SAP.
40. The model is set to reflect successful command completion.

41. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1324).
42. An attempt is made to set up an RDR SAP with a response address of 0.
43. The model is set to reflect an illegal command status.
44. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1326).
45. An attempt is made to set up a non-RDR SAP with a response address of 0.
46. The model is set to reflect successful command completion.
47. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1328).
48. An attempt is made to set the SAP response while the SAP is active.
49. The model is set to reflect an illegal command status.
50. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1330).
51. An attempt is made to update a non-RDR SAP response using a valid response address.
52. The model is set to reflect an illegal command status.
53. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1332).
54. The current SAP is deactivated.
55. The model is set to reflect successful command completion.
56. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1334).
57. An attempt is made to set the SAP response to an odd value.
58. The model is set to reflect an illegal command status.
59. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1336).
60. An attempt is made to set the SAP response to a value less than \$80000.
61. The model is set to reflect an illegal command status.
62. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1338).

63. The SAP in the response buffer is corrupted and an attempt is made to set up the response.
64. The model is set to reflect an illegal command status.
65. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1340).
66. The SAP in the response buffer is corrected, the LLC control byte is corrupted and an attempt is made to set up the response.
67. The model is set to reflect an illegal command status.
68. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1342).
69. The LLC control byte in the response buffer is corrected, the LLC status byte is corrupted and an attempt is made to set up the response.
70. The model is set to reflect an illegal command status.
71. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1344).
72. A valid RDR response address is set up.
73. The model is set to reflect successful command completion.
74. The EPNI response is verified to Check_Domain using the Check_Board_Response checklist—Alarm 1346).
75. An attempt is made to activate a valid non-RDR SAP.
76. The model is set to reflect an illegal command status.
77. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1348).
78. An attempt is made to activate a valid RDR SAP.
79. The model is set to reflect successful command completion.
80. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1350).
81. An attempt is made to set the response address while the RDR SAP is active.
82. The model is set to reflect an illegal command status.
83. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1352).
84. An attempt is made to update the response address using an odd address while the RDR SAP is active.

85. The model is set to reflect an illegal command status.
86. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1354).
87. An attempt is made to update the response address using an invalid address while the RDR SAP is active.
88. The model is set to reflect an illegal command status.
89. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1356).
90. An attempt is made to update the response address using a response with a corrupt SAP while the RDR SAP is active.
91. The model is set to reflect an illegal command status.
92. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1358).
93. The SAP in the response buffer is corrected, the LLC control byte is corrupted and an attempt is made to update the response while the SAP is active.
94. The model is set to reflect an illegal command status.
95. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1360).
96. The LLC control byte in the response buffer is corrected, the LLC status byte is corrupted and an attempt is made to update the response while the SAP is active.
97. The model is set to reflect an illegal command status.
98. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1362).
99. An attempt is made to update the response using a valid RDR response while the SAP is active.
100. The model is set to reflect successful command completion.
101. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1364).
102. The RDR SAP is deactivated.
103. The model is set to reflect successful command completion.
104. The EPNI response is verified (Check_Domain using the Check_Board_Response checklist—Alarm 1366).
105. The EPNI is placed in a known state and verified (Set_EPNI_State, Idle—Alarm 1390).

Tests 14 through 24 require specific and unique hardware to be run. For information about these tests refer to the Functional Design Test, 51150365.

NOTE

Tests 14 through 24 require specific and unique hardware to be run. For information about these tests refer to the Functional Design Specification, 51150365.

Test 25 Exerciser Test

EXER

This test exercises the EPNI transmit/receive path by performing either full duplex external TBC loopback IOCBs or by performing RDR requests to the RDR test device specified by the RDR UCN Address parameter.

Test 25 actions are:

1. If the RDR UCN Address parameter is "-", the following sequence is repeated as determined by the "LOOP LIMIT" parameter:
 - a. An IOCB is reserved and set up with the correct operation code.
 - b. If the current passcount is even, the data path is set to Modem A.
 - c. If the current passcount is odd, the data path is set to Modem B.
 - d. The test buffer input area is filled with a random data pattern.
 - e. The test buffer output area is filled with the one's complement of the data pattern.
 - f. The length byte of the test buffer is set to the one's complement of the expected length.
 - g. The error byte of the test buffer is set to \$FF.
 - h. The model is set to reflect the expected results.
 - i. The test IOCB is executed (Execute IOCB).
 - j. The model is set to reflect the expected operational status.
 - k. The EPNI response is verified (Check_Domain using the Check_All checklist—Alarm 2500 for Modem A path and 2510 for Modem B path).

2. If the RDR_UCN_Address parameter is not "-" the following sequence is repeated as determined by the LOOP_LIMIT parameter:
 - a. The EPNI is placed in-ring with interrupts enabled (Set_EPNI_State, In_Ring).
 - b. The EPNI is compared to the model (Check_Domain using the Check_Domain checklist—Alarm 2520).
 - c. An RDR request message is created addressed to the RDR test device specified by the RDR_UCN_Address parameter.
 - d. The message is added the IOCB chain.
 - e. The IOCB chain is started (Execute IOCB).
 - f. The EPNI is compared to the model (Check_Domain using the Check_Domain checklist—Alarm 2530).
3. The EPNI is placed in a known state and verified (Set_EPNI_State, Idle - Alarm 2590).

Test 26 Test/Normal and Node Address Switches

SUB

This test checks the EPNI's ability to correctly read the Test/Normal switch and the node address switches on the EPNI board.

NOTE

This test is considered a utility test and must be the only test in the test list. The test requires operator intervention.

The LLC is unloaded by aborting the EPNI (SET_EPNI_STATE, ABORT).

The EPNI is checked against the model (CHECK_DOMAIN using the CHECK_BOARD_RESPONSE checklist—Alarm 2600).

The initial state of the Test/Normal switch is read and the saved value is set opposite to force an immediate report.

The following sequence is repeated until the PNIE test mode is no longer “Test.”

- a. If the current switch state does not match the saved switch state, an operator message is printed containing the current switch state.
- b. The EPNI is compared against the model to detect unexpected changes without a delayed recheck (CHECK_DOMAIN using the CHECK_BOARD_RESPONSE checklist – Alarm 2610).
- c. The current switch state is saved.

If the Test/Normal switch is not set to the Normal position, an alarm is generated (Alarm – 2620).

The EPNI is placed in a known state and verified (SET_EPNI_STATE, Idle – Alarm 2630).

20.7 Common Test Sequences

The following common sequences are used in several PNIE tests. Their descriptions are included here (once) to avoid repeating the information in the description of each test.

SET_EPNI_STATE

This sequence sets the EPNI to that designated by the caller. Two pieces of information are passed by the caller:

- Desired EPNI state
 - 0 – Reset State (runs EPNI through reset cycle)
 - 1 – Abort State (runs EPNI through abort cycle)
 - 2 – Idle State (aborts EPNI, loads LLC, off-line, not in-ring)
 - 3 – Off-Line State (LLC loaded, off-line, not in-ring)
 - 4 – On-Line State (LLC loaded, on-line, not in-ring)
 - 5 – In-Ring State (LLC Loaded, on-line, in-ring)
- Hardware Interrupt enable/disable indication is determined by the sign of the requested state.
 - Negative sign—interrupts enabled
 - Positive sign—interrupts disabled

COMMAND_EPNI

This sequence commands the EPNI and adjusts the model as required. The command is issued to COMMAND_NO_ADJUST.

COMMAND_NO_ADJUST

This sequence is used to issue a command to the EPNI and wait for completion.

1. If the EPNI command register busy bit is set and the callers command is not Abort or READ_PAGED_STATUS, an alarm message is generated and PNIE aborts.
2. Otherwise, the caller's command is issued to the EPNI.
3. A command specified timeout is calculated.
4. The program monitors the EPNI waiting for the command to complete.
5. If the command does not complete within the calculated timeout, an alarm is generated and PNIE aborts.

SET_SAP_STATUS

This sequence sets a caller-specified SAP to the state requested by the caller.

1. The indicated SAP is moved to the EPNI paged status word \$OC and the SAP status is moved to the EPNI paged status word \$OE.
2. The EPNI model is updated to reflect the new paged status word contents.
3. If requested by the caller, the command completion interrupt request is set in the SET_SAP_STATUS command.
4. A SET_SAP_STATUS command is issued to the EPNI (COMMAND_EPNI).

GRANT_PRIORITY

This sequence starts a caller-specified EPNI Priority chain.

1. A START_PRIORITY command is built for the requested priority.
2. If requested by the caller, the command completion interrupt request bit is set in the START_PRIORITY command.
3. The command is issued to the EPNI (COMMAND_EPNI).
4. If a command completion interrupt was requested, the correct EPNI Operational Status word is built and removed to the EPNI model (SET_OPERATIONAL_STATUS).

SET_OPERATIONAL STATUS

This sequence sets the operational status for the caller-specified interrupt vector to the value provided by the caller.

1. If EPNI interrupts are enabled, the caller-specified OP Status word and interrupt count are moved to the model for the indicated interrupt vector.
2. If EPNI interrupts are enabled and the caller indicated interrupt count is nonzero, then the model is set up for an indeterminate number of interrupts by masking the interrupt count field for the indicated vector.
3. If the EPNI interrupts are enabled and the caller indicated interrupt count is nonzero, then the model is set up for the specified number of interrupts by unmasking the interrupt count field for the indicated vector.
4. If the EPNI interrupts are disabled, the INTERRUPT_ACK_SEMAPHORE is set in the EPNI model.
5. The caller-specified Op Status Word is transferred to the EPNI model's shared RAM Op Status Word.
6. The caller-supplied Op Status Word Mask is transferred to the EPNI model Op Status Word Mask.

EXECUTE_IOCB

This sequence executes an IOCB that is already linked to the caller-specified IOCB chain (priority). If requested by the called, this procedure waits for the IOCB to complete before returning.

1. The driver IOCB count is advanced in the EPNI model and passed to the EPNI.
2. If the caller-supplied timeout is nonzero, the model IOCB is adjusted for completion (complete bit set).
3. If the caller-supplied timeout is zero, the model IOCB is adjusted for failure to complete (complete bit reset).
4. The indicated priority chain is started (START_PRIORITY).
5. If the caller-supplied timeout value is nonzero, the IOCB complete bit is monitored until it is set or timeout has expired.
6. A short delay is executed to allow the EPNI timeout to set up the IOCB completion status.

CREATE_EPNI_MESSAGE

This sequence builds a message using caller-supplied parameters in a caller-supplied buffer.

1. The caller-supplied buffer is coated with random numbers (GEN_RANDOM_DATA).
2. The message header is built using the caller-supplied parameters.

CREATE_RDR_REQUEST

This sequence builds an RDR request using caller-supplied parameters in a caller-supplied buffer.

1. The caller-supplied buffer is coated with random numbers (GEN_RANDOM_DATA).
2. The message header is built using the caller-supplied parameters.

SET_GROUP_MASK

This sequence sets the EPNI group mask to the caller-supplied value and updates the model.

1. The caller-supplied group mask is moved to the EPNI paged status word \$OC.
2. The EPNI model is updated to reflect the new paged status word contents.
3. ALOAD_GROUP_MASK command is issued to the EPNI (Command_EPNI).

EXPECT_MESSAGE

This sequence sets the EPNI model up for the arrival of a message.

1. The next available RX buffer is removed from the EPNI model's empty RX buffer FIFO (REMOVE_BUFFER_FROM_FIFO).
2. The expected message is built in the model buffer (CREATE_EPNI_MESSAGE).
3. The EPNI visible portion of the buffer is filled with the one's complement of the expected message.
4. The excess portion of the model buffer is coated with the one's complement of the expected message.
5. The model buffer is appended to the EPNI model's filled buffer FIFO (APPEND_BUFFER_TO_FIFO)
6. The correct EPNI Operational Status Word is built and the interrupt count is advanced.
7. The Operational Status Word and interrupt count are transferred to the model (Set_Operational_Status).

QUEUE_MESSAGE

This sequence queues a message for transmission by the EPNI using caller-supplied parameters.

1. The next IOCB in the caller-supplied EPNI priority chain is reserved (NEXT_IOCB).
2. The requested message is built in the associated TX buffer (CREAT_EPNI_MESSAGE).
3. The IOCB op code is set up.
4. If requested by the caller, the END_OF_CHAIN bit is set in the IOCB.
5. The IOCB is check summed (CHECKSUM_IOCB).
6. The IOCB is copied to the EPNI model and adjusted for completion.
7. If the END_OF_CHAIN bit in the IOCB is set or the EPNI is not in the ring, the correct EPNI Operational Status Word is built and transferred to the model (SET_OPERATIONAL_STATUS).
8. If the EPNI is not in the ring, the IOCB status code is set to the expected error code.
9. The driver IOCB count and the LLC count values in the EPNI model are advanced.
10. The updated driver IOCB count is transferred to the EPNI.

QUEUE_RDR_REQUEST

This sequence queues an RDR request for transmission by the EPNI using caller-supplied parameters.

1. The next IOCB in the caller-supplied EPNI priority chain is reserved (NEXT_IOCB).
2. The requested message is built in the associated TX buffer (CREATE_RDR_REQUEST).
3. The IOCB op code is set up.
4. If requested by the caller, the RESYNCH_REQUEST and END_OF_CHAIN bits are set in the IOCB.
5. The IOCB is check summed (CHECKSUM_IOCB).
6. The IOCB is copied to the EPNI model and adjusted for completion.
7. If the END_OF_CHAIN bit is set or the EPNI is not in the ring, the correct EPNI Operational Status Word is built and transferred to the model (SET_OPERATIONAL_STATUS).
8. If the EPNI is not in the ring, the IOCB status code is set to the expected error code.
9. The driver IOCB count and LLC IOCB count values in the EPNI model are advanced.
10. The updated driver IOCB count is transferred to the EPNI.

EXECUTE TEST IOCB

This sequence executes a test IOCB using parameters supplied by the caller.

1. The next IOCB in the indicated EPNI priority chain is reserved (NEXT_IOCB).
2. The IOCB parameters are transferred to the IOCB.
3. The IOCB is check summed (CHECKSUM_IOCB).
4. The IOCB is executed (EXECUTE_IOCB).

SET_SUCCESOR_ADDRESS

This sequence sets the successor address in the EPNI model using a parameter supplied by the caller. The caller-supplied parameter is transferred to the EPNI model.

SET_UCN_ADDRESS

This sequence sets the EPNI physical UCN address to the value provided by the caller and updates the model.

1. The caller-supplied UCN address is moved to the value provided by the caller and updates the model.
2. A Set Network Address command is issued to the EPNI (COMMAND_EPNI).
3. If the new UCN address is legal, the model is set to reflect the new UCN address in use.

CHECK_DOMAIN

This routine collects the actual EPNI parameters and compares them to the EPNI model maintained by the test. If requested by the caller, a short delay is executed and the EPNI parameters are collected and compared a second time. The items to be collected and the recheck delay are caller-supplied.

The EPNI model is built of six different block types:

1. The block that defines the Shared RAM image.
2. The block that defines the interrupt status and paged status registers.
3. The block that defines the CCB and IOCBs.
4. The block that defines the TX buffers.
5. The block that defines the RX buffers.
6. The block that defines the RDR buffers.

The caller supplies a set of two masks that specify which of the six block types are to be checked. To simplify error checking, there are four predefined masks:

1. Check_Shared_RAM_Only specifies only Shared RAM block type.
2. Check_Domain specifies the Shared RAM block, the interrupt and paged status block, and the CCB and IOCBs block.
3. Check_All specifies all block types.
4. Check_Board_Response specifies the Shared RAM block and interrupt and paged status block.

The following sequence completes the action.

1. Each indicated model block is collected from the EPNI.
2. Each block is checked for errors (Check_Block).
3. The EPNI model is updated to reflect the result of the data collection.
4. If the caller-supplied recheck delay is nonzero, the program delays the request time interval.
5. Each indicated delay check model block is recollected from the EPNI.
6. Each block is checked for errors (Check_Block).

APPEND_BUFFER_TO_FIFO

This routine performs the mechanics of appending a receive buffer to a caller-supplied FIFO.

The following steps are used to implement this routine:

1. The FIFO append and remove pointers are checked to see if the FIFO is already full. If it is, an operator alert is printed and the program aborts.
2. If there is room in the FIFO, the busy flag in the buffer is set and the buffer pointer is placed in the next FIFO position.
3. The FIFO append offset is advanced.

REMOVE_BUFFER_FROM_FIFO

This routine performs the mechanics of removing a receive buffer from a caller-supplied FIFO.

The following steps are used to implement this routine:

1. The FIFO append and remove pointers are checked to see if the FIFO is empty. If it is, and the caller has indicated this is an acceptable condition, an operator alert is printed and the program aborts.
2. If the FIFO is not empty, the next available buffer pointer is removed from the FIFO.
3. The busy flag in the FIFO entry is cleared.
4. The FIFO remove offset is advanced.

INITIALIZE_CCB_IOCBS_AND_FIFOS

This routine performs the mechanics of initializing the EPNI CCB< IOCBs and FIFOs.

The following steps are used to implement this routine.

1. The priority control and empty FIFO structures are initialized.
2. The EPNI CCB and IOCBs are initialized.
3. The EPNI transmit and receive buffers are precoated.
4. The following sequence is repeated for each receive buffer:
 - a. The buffer is added to the empty buffer FIFO assigned to the priority indicated by the caller.
 - b. If requested by the caller, the current EPNI priority is advanced.
5. The following sequence is repeated for each IOCB and transmit buffer:
 - a. The current transmit buffer address is stored in the current IOCB.
 - b. The current IOCB is added to the IOCB chain (priority) indicated by the caller (Queue_IOCB).
 - c. If requested by the caller, the current EPNI priority is advanced.

NEXT IOCB

This routine performs the mechanics of retrieving the next IOCB in the indicated chain.

The following steps are used in implementing this routine:

1. The address of the next IOCB in the chain is read from the priority control structure indicated by the caller.
2. If the value is NIL, and the caller has indicated that this condition is unacceptable, an operator alert is printed and the program aborts. If the caller has allowed for this condition, the IOCB chain status is returned to the caller and no further processing is performed by this routine.
3. If an IOCB was available, the IOCB number is extracted and saved.
4. Using the IOCB number, the current IOCB address is setup for the current EPNI priority.
5. If requested by the caller, the next IOCB address value is advanced. If this was the last IOCB in the list, the next IOCB address is set to NIL.
6. The allocated IOCB is initialized to a NOP IOCB.

QUEUE_IOCB

This routine performs the mechanics of adding an IOCB to the end of the EPNI priority chain indicated by the caller.

The following steps are used to implement this routine:

1. If this is the first IOCB in the chain, the following actions are performed:
 - a. The proper CCB entry is directed to this IOCB.
 - b. The IOCB allocation used by next IOCB is directed to this IOCB.
 - c. The IOCB link field in the current IOCB is directed back to itself, forming a circular chain.
2. If this is not the first IOCB in the chain, the following actions are performed:
 - a. The IOCB link field of the last IOCB in the chain is copied to this current IOCB link field.
 - b. The IOCB link field of the last IOCB in the chain is directed to the new IOCB, effectively appending the current IOCB to the end of the chain.
 - c. The EPNI model is updated to reflect the previous operations.

20.8 PNIE Alarm Categories

PNIE Alarm messages are formatted as described at heading 2.8.2 of the *Test System Executive* manual. The first digit of a 3-digit alarm code or the first two digits of a 4-digit alarm code is the number of the test reporting the failure. Specifics of the failure are contained in the test message text.

21 LCN Sequence Of Events Test (LSOE)

21.1 LCN Sequence Of Events Elements

The LCN Sequence Of Events subsystem consists of the EPNI printed wiring board in a Network Interface Module (NIM), and one or more Advanced Process Manager Modules (APMM) connected to a Universal Control Network (UCN) LSOE and is used to periodically send LCN time to APMMs where it is used to support Digital Input Sequence Of Events (DISOE) event time stamping. The Time Sync message recognition and Free Running Counter logic gets tested in all APMMs, regardless if DISOE test is being done or not. When there are redundant NIMs on a UCN, LSOE should be selected to run in only one of the NIMs.

21.2 Use Information

LSOE should **never** be run in conjunction with PMMTS as it will interfere with time sync testing done with the PSOE test.

LSOE operates with the APMM nodes in the APMM_USER_NODE_LIST. This is a list of the APMM nodes that have PMMTS, PMTS, PMT2, or PMT3 loaded in them. The operator can add to or delete from this list as desired.

There is only one test in LSOE and it (1) sends the specified multicast address, from the Time_Sync_Address_Bit parameter to each of the targeted APMMs and then, (2) about every six seconds, sends the current LCN time to the APMMs.

21.3 Setup Parameters

21.3.1 LSOE Specific Parameters

TIME_SYNC_ADDRESS_BIT The designated Time Sync Multicast Address bit with a range of 1–15.

NOTE

USER_NODE_LIST is an EXEC command that controls a node list used by only LCN test and exerciser programs. Whenever USER_NODE_LIST is used by a test or exerciser program, only the nodes that appear in **both** the list of LOAded nodes (as modified by use of the DEL and ADD commands), and the list controlled by the USER_NODE_LIST command are affected.

The initial value for USER_NODE_LIST is "-" (therefore, all loaded nodes). Changes to the list are invoked by the command—or command abbreviation—followed by a list of node numbers separated by commas or "-" (to signify a group of consecutive node numbers).

21.3.2 General Parameters

TESTS Allowed values is 1

ERROR_LIMIT Allowed values are 1 through 32767 or "-"

PASS_LIMIT Allowed values are 1 through 32767 or "-"

REPORT Allowed values are PASSNUMBER, SUSPENDED, TESTNUMBER, LOG

INHIBIT Allowed values are PASSNUMBER, SUSPENDED, TESTNUMBER, LOG

SCALE Allowed values are 1 through 999

MINUTES_LIMIT Allowed values are 1 through 32767 or "-"

ABBREVIATION Allowed values are 0 through 4

AFTER_ERROR_LIMIT Allowed values are ABORT or HALT

21.4 Preset Parameter Values for LSOE

TIME_SYNC_ADDRESS_BIT = 1

TESTS = 1

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER, TESTNUMBER, SUSPENDED

SCALE = 100

MINUTES_LIMIT = -

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

21.5 Mode Characteristics for LSOE

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	Only one	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

21.6 LSOE Test

(System Modes)

Test 01 Multicast Timesyncs to APMMs

SUB, MOD, EXER

Test 1 multicasts (a logical broadcast) the LCN time to the targeted APMM nodes about every six seconds. LSOE should never be run while PMMTS is running in the APMMs. When redundant NIMs are available, LSOE should be run on only one of them at a time, as multiple LSOEs on the same UCN would conflict with each other.

LSOE operates with the APMM nodes in the APMM_USER_USER_LIST. This is a list of APMM nodes that can have PMMTS, PMTS, PMT2, or PMT3 loaded in them. The operator can add or delete from this list as desired. The APMMs in this list are targeted to receive LSOE communications, which includes the LCN time in the form of a UCN Timesync message.

When started, LSOE checks for entries in the APMM_USER_NODE_LIST. If the list is empty, LSOE will wait up to five minutes for nodes to be added to the list. If the list remains empty after five minutes, the LSOE will announce that no APMMs are in the list and will turn off.

If there are entries in the APMM_USER_NODE_LIST, then the LSOE will (1), send the specified multicast address, from parameter TIME_SYNC_ADDRESS_BIT, to each of the targeted APMMs and then (2), will send the current LCN time to the APMMs about every six seconds. APMM nodes that drop out or are added to the list will be announced as they occur and will stop or begin accepting UCN timesync messages as appropriate.

In addition to multicasting the UCN time sync messages, LSOE also verifies correct operation of the Time Sync hardware in the EPNI; any discrepancies are displayed as error messages. These checks include

- (a) measuring the Free Running Counter rate against MTOS's millisecond clock,
- (b) the correct operation of the UCN and AUX Sync Latches and
- (c) correct indications in the Time Sync status word.

The Free Running Counter (FRC) interval of the EPNI is used as the standard against which the APMM FRC rate is measured. Discrepancies in any of the checks are displayed in alarm messages.

21.7 LSOE Messages

When no APMM nodes remain in the APMM_USER_NODE_LIST, LSOE will display the following message:

DD-MMM HH:MM:SS Node nn LSOE nn No APMM Nodes Remain in List—Aborting!

When LSOE detects new entries in the APMM_USER_NODE_LIST, the following message will be displayed:

DD-MMM HH:MM:SS Node nn LSOE nn PASS n

The following APMMs have been added and will begin receiving time sync messages:

a, b, c

22 Generic Gateway Serial Interface Test (GGSF)

22.1 Generic Gateway Serial Interface Elements

The Generic Gateway Serial Interface module (GGSF) consists of the GGSF printed wiring board, GGSF I/O printed wiring board, and a relay board with cables. The GGSF interfaces to the HPK2 or K2LCN in a Generic Gateway through the Module Bus. This includes use of Direct Memory Access (DMA), shared RAM, and interrupts.

22.2 Use Information

GGSF has six tests that verify the operation of the GGSF board in a GGSF module. The sixth test also verifies the serial I/O data path by executing a loopback test.

22.3 Setup Parameters

22.3.1 GGSF-Specific Parameters

FIRST_RANDOM Allowed values are 0 through 32767.

22.3.2 General Parameters

TESTS Allowed values are 1 through 6

ERROR_LIMIT Allowed values are 1 through 32767 or "-"

PASS_LIMIT Allowed values are 1 through 32767 or "-"

REPORT Allowed values are PASSNUMBER, TESTNUMBER, LOG

INHIBIT Allowed values are PASSNUMBER, TESTNUMBER,
LOG

SCALE Allowed values are 1 through 999

MINUTES_LIMIT Allowed values are 1 through 32767 or "-"

ABBREVIATION Allowed values are 0 through 4

AFTER_ERROR_LIMIT Allowed values are ABORT or HALT

22.4 Preset Parameter Values for GGSF

TESTS = 1-6

ERROR_LIMIT = 5

PASS_LIMIT = -

REPORT = LOG

INHIBIT = PASSNUMBER,TESTNUMBER

SCALE = 100

MINUTES_LIMIT = -

ABBREVIATION = 3

FIRST_RANDOM = 0

AFTER_ERROR_LIMIT = ABORT

22.5 Mode Characteristics for GGSF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

22.6 Individual Generic Gateway Serial Interface Tests

(System Modes)

Test 01 Reset Test SUB, MOD

Test 1 verifies that the GGSF hardware correctly executes a reset in response to a reset flag command to the Configuration Status Register. Test 1 then verifies that the other shared registers have the expected contents.

Test 02 Load and Verify Registers Test SUB, MOD

Test 2 assures that the GGSF hardware correctly responds to valid flag commands to the Configuration Status Register and that the other shared registers have the expected contents.

Test 03 Basic Commands and Interrupts Test SUB, MOD

Test 3 assures that the GGSF hardware correctly responds to all combinations of NOP Command Register commands with and without the use of interrupts.

Test 5 assures that the module bus data parity errors during MCPU writes to the shared Command Register are properly detected and reported. Test 5 also assures that module bus errors during DMA reads of nonexistent addresses are properly detected and reported.

Test 6 verifies that the GGSi hardware correctly responds to the loopback test command.

The PC Loopback program allows testing of the product system configuration of an ARCG module connected via serial RS-232 cables to a Personal Computer (PC) running a UNIX operating system. The PC Loopback program makes a logical connection between the two RS-232 ports on the Gateway Generic Serial Interface in the same way that the null-modem cable between the two RS-232 ports makes a physical loopback connection.

The History Archive software cannot be run concurrent with the PC Loopback program. The PC Loopback is invoked with the name LOOPBACK. The LOOPBACK program first writes data to the screen to identify the program and indicate that the escape key is used to exit the program. The LOOPBACK program sets up the PC serial ports and waits in a loop where any character received from the first serial port is sent out through the second serial port and any character received from the second serial port is sent out through the first serial port. Characters received from the first serial port are displayed as 2-digit hexadecimal values at the left margin of the screen. Characters received from the second serial port are displayed indented several spaces from the left margin of the screen.

Once the LOOPBACK program is started or a null_modem cable is installed, then the GGSF loopback test 6 can be run.

Loopback test error codes are displayed as GCSI Shared Registers Paged Status content errors and will report the following failures.

- Bit 08 set = Channel 1 transmitter won't set up.
- Bit 09 set = Channel 2 transmitter won't set up.
- Bit 10 set = Channel 1 didn't receive data.
- Bit 11 set = Channel 2 didn't receive data.
- Bit 12 set = Channel 1 receiver status bad.
- Bit 13 set = Channel 2 receiver status bad.

22.7 GGSF Alarm Messages

There are two possible alarm messages:

If the contents of the GGSF Shared Registers do not match the expected values, an alarm will be displayed as follows:

DD-MMM-YYYY HH:MM:SS NODE nn Test Slot ALARM Test#nn

WHILE EXECUTING SOME TEST ACTION - GGSF SHARED REGISTER

WAS - \$XXXX
EXPECTED - \$YYYY
MASK - \$ZZZZ

If the GGSF Shared Registers Paged Status contents do not match the expected values, an alarm will be displayed as follows:

DD-MMM-YYYY HH:MM:SS NODE nn Test Slot ALARM Test#nn

WHILE EXECUTING SOME TEST ACTION - GGSF PAGED SHARED REGISTER

WAS - \$XXXX XXXX
EXPECTED - \$YYYY YYYY
MASK - \$ZZZZ ZZZZ

If random numbers were used for a test that caused an alarm message, the following message will be appended to the alarm message.

FIRST_RANDOM USED FOR THIS TEST - \$XXXX

23 WSI Subsystem Tests (WSIF)

NOTE

The Work Station Interface (WSI) subsystem test is designed as a manufacturing stress test for the WSI and WSI2 boards. If you want to run this test on site, you must purchase either a WST or WST2 board from Honeywell's Manufacturing Test Engineering. Contact your Honeywell representative.

23.1 WSI Subsystem Elements

The common WSI and WSI2 subsystem elements tested by this program include the flag control and status registers, shared SRAM, DI02 bus, and Module bus DMA operations. Additional testing of WSI2 specific functionality includes testing of the VMEBUS interrupts, Module bus interrupts, and reading the UNIX Interface registers. This program also includes utility tests for testing the WST nonvolatile RAM, reading and displaying the contents of WST nonvolatile RAM, and resetting the WST or WST2 board.

23.2 WSIF HVTP Use Information

The WSIF HVTP will normally be used during Manufacturing stress testing of a WSI board with a WST board, or a WSI2 board with a WST2 board. WST and WST2 are test boards, developed by Manufacturing Test Engineering to support the testing of the WSI and WSI2 boards, respectively, in conjunction with the WSIF test program. WSIF can be run on any WSI/WST or WSI2/WST2 board set on the network. If a TPDG exists in the node under test, WSIF will remove itself from view (see *Test System Executive*, "Removed Command"). To restore WSIF, use the CON # command (see *Test System Executive*, "Configuration Command"); where # is the slot number of the WSI or WSI2 to be tested. Because the WSIF program will interfere with the test programs for the TPDG board, for example, test program XUSF, WSIF and TPDG test programs should not be run concurrently. Also, because WSIF will contend with XAMF for running with the WSI2, the operator will either have to specify the desired test program by name, or "Remove" the unwanted test program (see *Test System Executive*, "Removed Command"). XAMF is the test program for the Application Module^X (A^XM).

The WSIF HVTP will test the WSI or WSI2 hardware with minimum operator intervention. The operator can specify any subset (except the empty set) of the available tests to run. From the available tests in the test list, WSIF will run only the tests that are appropriate for the WSI or WSI2 being tested.

A pass will consist of completion of all tests in the test list. A sweep is one write/read/verify cycle on the bus or memory under test.

If a test board is present, for every pass, before each test in the test list is run, WSIF will verify that an operational WST or WST2 board is connected to the WSI or WSI2 board. For every WSI/WSI2 reset/restart cycle (which also causes a WST/WST2 reset/restart cycle), WSIF will verify that an operational WST/WST2 board is connected to the WSI/WSI2 board. If an operational WST/WST2 board is not connected to the WSI/WSI2 board, WSIF will report an alarm message and abort.

NOTE

The Transaction Error Test (Test 6) requires a K2LCN with less than 8 megawords of memory for proper operation.

NOTE

WSIF does not have an "exerciser" test.

23.3 Setup Parameters

23.3.1 WSIF-Specific Parameters

RANDOM_SEED

The long integer value (\$0—\$FFFFFFFF) is to be used as the seed for the generation of random number(s) during memory and bus tests. The default for this parameter will be \$00400006. This parameter can be changed only by the operator when WSIF is off.

WSI_TEST_BOARD_INSTALLED

The allowed values are YES and NO.

Indicates whether or not a test board (WST or WST2) is connected to the WSI or WSI2 board under test.

Yes—Run all specified test appropriate for the WSI or WSI2 board being tested. The WSI or WSI2 test board is installed and operational.

NO—Skip the tests and initialization procedures requiring a test board. The WSI or WSI2 test board is not installed.

The default value is YES. This parameter can be changed only by the operator when WSIF is off

23.3.2 General Parameters

TESTS	Allowed values are 1 through 12
ERROR_LIMIT	Allowed values are 1 through 32767 or "--"
PASS_LIMIT	Allowed values are 1 through 32767 or "--"
REPORT	Allowed values are PASSNUMBER, SUSPENDED, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, SUSPENDED, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 1 through 32767 or "--"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

23.3.3 Preset Parameter Values for WSIF

RANDOM_SEED = \$400006

WSI_TEST_BOARD_INSTALLED = Yes

TESTS = 1-12

ERROR_LIMIT = 5

INHIBIT_LIMIT = --

REPORT = LOG

INHIBIT = PASSNUMBER, SUSPENDED, TESTNUMBER

SCALE = 0

MINUTES_LIMIT = --

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

23.4 Mode Characteristics for WSIF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	None in WSIF	Not Applicable

23.5 Individual WSIF Tests

(System Modes)

Test 01 Control and Register Test

SUB, MOD

This test runs with either a WSI or WSI2 board. The test verifies the hardware and configuration status and the flag control registers through the Module bus, WSBUS, or VMEBUS, as well as causing the “Failed/Reset,” “Passed Module Test,” and “WSCPU Running” status indicator lamps to turn on (running status lamp on WSI board only). The WSCPU Ready flag is set through the WST/WST2 board, and the “Shutdown Request” and “WSI Reset” flags are set and cleared from the Module bus. On WSI2 boards, the “Handshake” flag (Flag #7, Configuration status bit 12) is exercised from both the WST2 and Module bus. Finally, a byte is read from the WSI/WSI2 Configuration status to verify byte access of the WSI/WSI2 registers and shared RAM. Note, this will result in a bus error if it fails.

This test will run in the subsystem and module modes and will run with or without a WST or WST2 test board. Complete testing requires a WST or WST2 board; however, partial testing will be conducted if a test board is not installed and parameter WSI_TEST_BOARD_INSTALLED is set to NO.

Test 02 Module Bus Memory SRAM Test SUB, MOD

This test runs with either a WSI or WSI2 board. WSI or WSI2 shared SRAM is tested through the Module bus by performing two sweeps where

Random words are written from Base Address + \$20 (low) to Base Address + \$FFF (high) by way of the Module bus.

The status registers are verified (that is, no errors have occurred).

The stored values are read, verified, and the inverse of the correct value is written back from low to high address.

The status registers are verified (that is, no errors have occurred).

The stored values are read and verified from low to high address.

The random numbers are generated from a long word random number generator. Both words, in the long word, are used for memory test. On the first pass, the operator defined RANDOM_SEED will be used as the random number generator seed. At the end of each test, the random number generator seed will be updated to the last random number generated.

If an error is detected during this test, the random number generator seed at the beginning of the test will be displayed in the alarm.

This method of memory test has a 100% chance of detecting any stuck bit error, any nonaddressable memory error, and any lost bit error. Each time memory is tested, using this test methodology, there is a 50% chance of catching each bit uniqueness error and a greater than 99% chance of catching any unique address error.

This test will run in the subsystem and module mode with or without a test board.

Test 03 WSBUS/VMEBUS Memory SRAM Test**SUB, MOD**

This test runs with either a WSI or WSI2 board. WSI/WSI2 shared SRAM is tested with word accesses from the WST/WST2 test board by

Random long words are written from Base Address + \$20 to Base Address + \$80E through the Module bus.

\$00 is written from Base Address + \$810 to Base Address + \$FFE through the Module bus.

The status registers are verified (that is, no errors have occurred).

The WST board transfers the random data to shared SRAM locations Base Address + \$810 to Base Address + \$FFE.

The status registers are verified (that is, no errors have occurred).

The transferred values are read and verified through the Module bus.

The status registers are verified (that is, no errors have occurred).

Random words are written from Base Address + \$810 to Base Address + \$FFE through the Module bus.

\$00 is written from Base Address + \$20 to Base Address + \$80E through the Module bus.

The status registers are verified (that is, no errors have occurred).

The WST board transfers the random data to shared SRAM locations Base Address + \$20 to Base Address + \$80E.

The status registers are verified (that is, no errors have occurred).

The transferred values are read and verified through the Module bus.

The status registers are verified (that is, no errors have occurred).

If the board under test is a WSI2, then the above steps are repeated twice more with new random data. Once with the WST2 board executing byte accesses and then with long word accesses of WSI2 shared RAM.

The random numbers are generated from a long word random number generator. Both words in the long word are used for memory test. On the first pass, the operator-defined RANDOM_SEED will be used as the random number generator seed. At the end of each test, the random number generator seed will be updated to the last random number generated.

If an error is detected during this test, the random number generator seed at the beginning of the test will be displayed in the alarm.

This method of memory test has a 100% chance of detecting any nonaddressable memory error. Each time memory is tested using this test methodology, there is a 50% chance of catching each stuck bit error, a 50% chance of catching each bit uniqueness error, and a greater than 99% chance of catching any address uniqueness error.

This test will run in the subsystem and module modes and requires a WST/WST2 board.

Test 04 DMA Address and Data Lines Test SUB, MOD

This test runs with either a WSI or WSI2 board. The DMA address and data lines are tested by

Reserving two 64-Kbyte blocks at selected random addresses above heap memory.

2 K words of random data are written to the first reserved block.

2 K words of \$0000 are written to the second reserved block.

The WST board transfers the 2 K words of random data from the first reserved block to the second reserved block.

The transferred values are read and verified by way of the Module bus.

If the board under test is a WSI2, then the above steps are repeated twice with new random data. Once with the WST2 board executing byte accesses and then with long word accesses of the module DRAM. The block size for byte accesses is 256 bytes and 1 K long words for long word accesses.

The random numbers are generated from a long word random number generator. Both words in the long word are used for memory test. On the first pass, the operator-defined RANDOM_SEED will be used as the random number generator seed. At the end of each test, the random number generator seed will be updated to the last random number generated.

If an error is detected during this test, the random number generator seed at the beginning of the test will be displayed in the alarm.

This test will run in the subsystem and module modes and requires a WST/WST2 board.

Test 05 DI02 Address and Data Lines Test SUB, MOD

This test runs with either a WSI or WSI2 board. The DI02 address and data lines are tested by:

Using 225 random data long words and random addresses.

Each random data/address pair is written to the TPDG port, through the WST board, and the data/address are verified.

Each random data/address pair is read from the TPDG port, through the WST board, and the data/address are verified.

The random numbers are generated from a long word random number generator. Both words in the long word are used for memory test. On the first pass, the operator defined RANDOM_SEED will be used as the random number generator seed. At the end of each test, the random number generator seed will be updated to the last random number generated.

If an error is detected during this test, the random number generator seed at the beginning of the test will be displayed in the alarm.

This test will run in the subsystem and module modes and requires a WST/WST2 board.

This test requires an LCN node with less than 8 megawords of memory.

Test 07	WSI2 to VMEBUS Interrupt Test	SUB, MOD
---------	-------------------------------	----------

Request the WST2 board to disable the VME interrupt by the Flag Control.

Verify the registers through the Module bus.

Request the WST2 board to copy a random data word in the range of 65-255 to the VME Interrupt Response word and prepare for the VME level 6 interrupt test.

Verify the registers through the Module bus.

Verify the registers through the VMEBUS (that is, the WST2).

Write a random word to the Command register through the Module bus.

Wait up to 1 second for the Configuration status to indicate “VME Interrupt Pending.”

Verify that no VME level 6 interrupts occurred.

Request the WST2 board to enable the VME interrupt by the Flag Control.

Wait up to 1 second for the Configuration status to indicate that the “VME Interrupt Pending” bit turns off (that is, that the VME level 6 interrupt occurred and that the WSI2 acknowledged it).

Verify the registers through the Module bus.

Verify that the WST2 board reported that only one level 6 interrupt occurred.

Request the WST2 board to disable the VME interrupt by the Flag Control.

Verify the registers through the Module bus.

March 2013

Test 08 WSI2 to Module Bus Interrupt Test SUB, MOD

This test runs only with a WSI2 board. The test verifies that the Module bus level 0 interrupt can be generated from the VMEBUS side (WST2) by a write to the LCN Interrupt Response register and controlled from the Module bus side by enabling/disabling the LCN Interrupt flag. The test will perform the following:

- Disable the LCN interrupt by the Flag Control.

- Verify the registers through the Module bus.

- Request the WST2 board to write to the LCN Interrupt Response register (offset +6) with a data word that is provided by the test program (the data word is always the WSI2 slot number multiplied by 8).

- Wait up to 1 second for the Configuration status to indicate “LCN Interrupt Pending.”

- Verify the registers through the Module bus.

- Verify the registers through the VMEBUS (that is, the WST2).

- Verify that no WSI2 board interrupts occurred.

- Enable the LCN interrupt by the Flag Control.

- Wait up to 1 second for the Configuration status to indicate that the “LCN Interrupt Pending” bit turns off (that is, that the LCN level 0 interrupt occurred and that the WSI2 board acknowledged it).

- Verify the registers through the Module bus.

- Verify that only one level 0 interrupt occurred.

- Disable the LCN interrupt by the Flag Control.

- Verify the registers through the Module bus.

This test will run in the subsystem and module modes and requires a WST2 board.

Test 09 WSI2 UNIX Interface Registers Test SUB, MOD

This test runs only with a WSI2 board. The test verifies that the UNIX Interface registers (1, 5, and \$101) can be read through the VMEBUS (WST2), and that their contents are as expected. The test will perform the following.

Request the WST2 to read and return the contents of the UNIX registers.

Verify the values returned for each of the three UNIX Interface registers (1, 5, and \$101).

This test will run in the subsystem and module modes and requires a WST2 board.

Test 10 WST Initialization Utility Test SUB

This test runs with either a WSI or WSI2 board. The utility causes the WST or WSI2 board to initiate a reset cycle.

This test is a utility test that will run in the subsystem mode if it is the only test in the test list and requires a WST or WST2 board.

Test 11 Nonvolatile RAM Test SUB

This test runs only with a WSI board. The WST's nonvolatile RAM is tested by
Clearing the first 256 bytes of nonvolatile RAM by way of the WST.

Verifying that the nonvolatile RAM was cleared by way of the WST.

Writing \$FF to the first 256 bytes of nonvolatile RAM by way of the WST.

Verifying, through the WST that \$FF was written to nonvolatile RAM.

This test is a utility test that will run in subsystem mode if it is the only test in the test list and requires a WST board.

Test 12 Read and Display Nonvolatile RAM Utility Test SUB

This test runs only with a WSI board. The utility will display the contents of the WST's nonvolatile RAM by

Reading the first 256 bytes of nonvolatile RAM through the WST and reporting the data to the operator.

This test is a utility test that will run in subsystem mode if it is the only test in the test list and requires a WST.

23.6 Alarm Messages

The following are the possible Alarm Messages and their descriptions.

23.6.1 Register

If the WSI Register contents do not match the expected values, an alarm will be displayed as follows:

DD—MMM—YYYY HH:MM:SS Node nn WSIF n Alarm nnn

While Executing <some test action>

An Error Condition was Observed — <error description>

—**WSI** <register identification>

WAS : \$XXXX

S/B : \$YYYY

MSK : \$ZZZZ

DIF : \$VVVV

23.6.2 Memory

If the contents of a memory location under test do not match the expected values, an alarm will be displayed as follows:

DD—MMM—YYYY HH:MM:SS Node nn WSIF n Alarm nnn

While Executing <some test action>

An Error Condition was Observed — <error description>

—**The Contents of Address \$WWWWWWW**

WAS : \$XXXXXXXX

S/B : \$YYYYYYYY

DIF : \$VVVVVVVV

23.6.3 Bus

If the results of a bus test (DMA or DI02) do not match the expected values, an alarm will be displayed as follows:

DD—MMM—YYYY HH:MM:SS Node nn WSIF n Alarm nnn

While Executing <some test action>

An Error Condition was Observed — <error description>

WAS : \$XXXXXXXXX

S/B : \$YYYYYYYYY

DIF : \$VVVVVVVVV

23.6.4 First Random

If random numbers were used for a test that caused an alarm message, the following message will be appended to the alarm message:

FIRST_RANDOM USED FOR THIS TEST - \$UUUUUUUU

23.6.5 Stop

If WSIF is stopped, the following operator alert will be displayed:

DD—MMM—YYYY HH:MM:SS Node nn WSIF nStopped After n Passes and n Errors

23.6.6 Abort

If WSIF is aborted, the following operator alert will be displayed:

DD—MMM—YYYY HH:MM:SS Node nn WSIF nAborted After n Passes and n Errors

23.7 Alarm Numbers

23.7.1 WSIF Pretest Device Initialization

Alarm Number	Description
1	Post clear flags status register error
2	Post set Interface Enable flag status register error
3	Post set WSI_RESET flag status register error
4	Post clear WSI_RESET flag status register error
5	Correct status register value not seen within 2 seconds after setting the WSP_RESET flag. Also, on the WSI2, after setting the WSI_RESET flag.
6	Correct status register value not seen within 1 second after clearing the WSI_RESET flag on the WSI2 (note that the WSP_RESET flag is still set).
7	Correct status register value not seen within 5 seconds after clearing the WSP_RESET flag. Also, on the WSI2 after toggling the WSI_RESET flag.
8	WST/WST2 did not complete its shared RAM initialization successfully (that is, writes “WSTRDY” or “WST2OK” to shared RAM).
9	WST/WST2 did not successfully complete the command to set the WSCPU_READY flag status register error.
10	Post set WSCPU_READY flag status register error
11	Either (1) the WST/WST2 did not successfully complete the command to read the WSI/WSI2 registers and return the values read, or (2) the correct status register value was not seen from the WST/WST2. The failure is apparent from the alarm description that accompanies the alarm.
12	Post set MODULE_TEST_PASSED flag status register error
13	Either (1) the WST/WST2 did not successfully complete the command to read the WSI/WSI2 registers and return the values read, or (2) the correct status register value was not seen from the WST/WST2. The failure is apparent from the alarm description that accompanies the alarm.
90	The contents of the Configuration status register are in error. This check is made prior to executing each test in the test list if parameter TEST_BOARD_INSTALLED is set to YES.

23.7.2 Control and Register Test (Test 1)

Alarm Number	Description
101 to 113	Same as alarms 1 through 13 for the first loop of five
114 to 126	Same as alarms 1 through 13 for the second loop of five
127 to 139	Same as alarms 1 through 13 for the third loop of five
140 to 152	Same as alarms 1 through 13 for the fourth loop of five
153 to 165	Same as alarms 1 through 13 for the fifth loop of five
180	Status register error after setting the HANDSHAKE flag (Flag 7) through the Module bus.
181	Status register error after clearing the HANDSHAKE flag (Flag 7) through the Module bus.
182	The WST2 did not successfully complete the command to set the HANDSHAKE flag.
183	Status register error, as seen through the Module bus after setting the HANDSHAKE flag (Flag 7) from the WST2 (VMEBUS).
184	Status register error, as seen from the WST2 after setting the HANDSHAKE flag (Flag 7) from the WST2 (VMEBUS).
185	The WST2 did not successfully complete the command to clear the HANDSHAKE flag.
186	Status register error, as seen through the Module bus after clearing the HANDSHAKE flag (Flag 7) from the WST2 (VMEBUS).
187	Status register error, as seen from the WST2 after clearing the HANDSHAKE flag (Flag 7) from the WST2 (VMEBUS).
Bus Error	WSI2 hardware error – byte access from the Module bus has not been implemented.

23.7.3 Modbus SRAM Test (Test 2)

Alarm Number	Description
203	Status register error after writing data to shared RAM
205	Error in value read from shared RAM
206	Status register error after writing data to shared RAM
208	Error in value read from shared RAM
209	Status register error after reading data from shared RAM

23.7.4 WSBUS/VMEBUS SRAM Test (Test 3)

Alarm Number	Description
313	Status register error after writing data to shared RAM
315	WST/WST2 did not successfully complete command to copy data from low to high addresses.
316	Status register error after copying data.
318	Error in value read from shared RAM
319	Status register error after reading data from shared RAM
321	Status register error after writing data to shared RAM
323	WST did not successfully complete command to copy data from high to low addresses.
324	Status register error after copying data
326	Error in value read from shared RAM
327	Status register error after reading data from shared RAM

23.7.5 DMA Address and Data Lines Test (Test 4)

Alarm Number	Description
403	WST/WST2 did not successfully complete command to copy data by word.
406	Error in value read from target DMA location after word copy.
413	WST/WST2 did not successfully complete the command to copy data by byte.
416	Error in value read from target DMA location after byte copy.
423	WST/WST2 did not successfully complete command to copy data by long word.
426	Error in value read from target DMA location after long word copy.

23.7.6 DI02 Address and Data Lines Test (Test 5)

Alarm Number	Description
503	WST/WST2 did not complete command to write data to the TPDG port.
504	Status register error after write
506	Error in data value read.
507	WST/WST2 did not complete command to write upper address word.
508	Status register error after writing upper address word.
509	Error in address value read.
511	WST/WST2 did not complete command to read data from the TPDG port.
512	Status register error after read.
514	Error in data value read.
515	WST/WST2 did not complete command to write upper address word to shared RAM.
516	Status register error after writing upper address word.
517	Error in address value read.

23.7.7 Transaction Error Test (Test 6)

Alarm Number	Description
603	WST/WST2 did not successfully complete command to execute the transaction error test.
604	Status register error—Expect transaction error bit to be set in the Hardware status.
611 to 623	Same as alarms 1 through 13

23.7.8 WSI2 to VMEBUS Interrupt Test (Test 7)

Alarm Number	Description
705	WST2 did not successfully complete the command to disable the VME interrupt by Flag Control.
707	Post “Disable VME Interrupt” status register error
710	WST2 did not successfully complete the command to copy a word from shared RAM to the VME Interrupt Response register.
712	Post “Write to VME Interrupt Response register” status register error
715	Either (1) the WST2 did not successfully complete the command to read the WSI2 registers and return the values read, or (2) the correct status register value was not seen from the WST2. The failure is apparent from the alarm description that accompanies the alarm.
720	Post “Write a random word to the Command register” Hardware or Configuration status register error
725	Post “Write a random word to the Command register” status register error
730	Either (1) the WST2 did not successfully complete the command to read the WSI2 registers and return the values read, or (2) the correct status register value was not seen from the WST2. The failure is apparent from the alarm description that accompanies the alarm.
735	WST2 did not successfully complete the command to enable the VME interrupt by Flag Control.
740	Post “Enable VME interrupt” status register error
745	WST2 did not successfully complete the command to return the number of times the VME level 6 interrupt occurred.
747	The number of VME level 6 interrupts is wrong.
750	Got unexpected Module bus interrupt(s) during the VME interrupt test.
755	WST2 did not successfully complete the command to return the number of times the VME level 6 interrupt occurred.
757	The number of VME level 6 interrupts is wrong.
760	WST2 did not successfully complete the command to disable the VME interrupt by Flag Control.

765 Post “Disable VME interrupt” status register error

23.7.9 WSI2 to Module Bus Interrupt Test (Test 8)

Alarm Number	Description
805	Post “Disable LCN interrupt” status register error
810	WST2 did not successfully complete the command to write to the LCN Interrupt Response register (this should trigger the LCN interrupt).
815	Post “Trigger the LCN interrupt” Hardware or Configuration status register error
820	Post “Trigger the LCN interrupt” status register error
825	Either (1) the WST2 did not successfully complete the command to read the WSI2 registers and return the values read, or (2) the correct status register value was not seen from the WST2. The failure is apparent from the alarm description that accompanies the alarm.
830	Unexpected LCN interrupt occurred while LCN interrupts are disabled.
835	Post “Enable LCN interrupt” Hardware or Configuration status register error
840	Post “Enable LCN interrupt” status register error
845	Expected one LCN level 0 interrupt to occur.
850	After delaying 200 milliseconds, expected no more LCN interrupts to occur.
855	Post “Disable LCN interrupt” status register error

23.7.10 WSI2 UNIX Interface Registers Test (Test 9)

Alarm Number	Description
905	WST2 did not successfully complete the command to read and return the three UNIX Interface registers (1, 5, and \$101).
910	UNIX Interface register—Register 1 error
915	UNIX Interface register—Register 5 error
920	UNIX Interface register—Register \$101 error

23.7.11 WSI2 Initialization Utility Test (Test 10)

Alarm Number	Description
1003	WST/WST2 did not complete the reset command.
1004	WST/WST2 did not complete its shared RAM initialization successfully (that is, writes “WSTRDY” or “WST2OK” to the shared RAM).

23.7.12 Nonvolatile RAM Test (Test 11)

Alarm Number	Description
1103	WST did not complete the command to copy from shared RAM to nonvolatile RAM.
1104	WST did not complete the command to copy from nonvolatile RAM to shared RAM.
1105	Error in the value read from nonvolatile RAM.
1106	WST did not complete the command to copy from shared RAM to nonvolatile RAM.
1107	WST did not complete the command to copy from nonvolatile RAM to shared RAM.
1108	Error in the value read from nonvolatile RAM.

23.7.13 Read Nonvolatile RAM Utility Test (Test 12)

Alarm Number	Description
1203	WST did not complete the command to copy from nonvolatile RAM to shared RAM.

24 U^XS Subsystem Tests (XUSF)

NOTE

The Universal Station^X Interface (XUSF) subsystem test is designed to run in X-Layer nodes only.

24.1 U^XS Subsystem Elements

U^XS Subsystem elements tested by this program include the TPDG, which combines the PDG and SCSI controller functions on one board, and the WSI with a WSCPU board mounted on it. Combined, these elements constitute a US with X-Windows.

24.2 XUSF HVTP Use Information

XUSF tests can be run on any U^XS on the network except the TOCS itself. XUSF tests PDG and SCSI functions of the TPDG. The keyboard(s) can be tested with KEYB. **The Work Station peripherals are not tested by any HVTS programs.**

Restriction—When testing a dual function board (for example, EPDG, EPDG2, TPDG, etc.), only one of the following test systems can run on that dual function board at a time without errors occurring in the systems. The test systems are KEYB, PDGI, SCMD, and XUSF.

Special Requirements—The nature of the device connected to a particular port must be defined to this program. Where the program requires actual devices, it bypasses those channels that are configured “Cn_HOOK_UP_TO = OPEN.” Devices on each channel are required to fully test all defined functions.

If UNIX is running, the operator must be logged out of UNIX before running this test program.

Test 12, the video output test, requires that either a CRT or 75 ohm resistors be connected to the CRT outputs on the TPDG I/O board in order to pass the test.

Special Feature—Test 15 of XUSF can be used to verify the peripheral cabling and setup of the PDG operator interface subsystem. Upon starting Test 15, a peripheral-status log is printed. If any status change occurs (that is, cables from the peripherals get disconnected and subsequently reconnected), the status is redisplayed for each state change. The status log indicates the state of all channels and the parameter setup for each channel. Note that Test 15 is a utility test and, as such, must be the only test selected in Subsystem Mode in order to run. Once started, Test 15 must be ABORTed in order to stop.

Because XUSF now tests both SCSI and PDG functionality, the error messages have the ability to report two different slot numbers, the slot number associated with the SCSI side of the board (physical slot number) and the slot number associated with the PDG side of the board (physical slot number plus one). This may cause some confusion when there is more than one TPDG board and it is necessary to communicate with only one. When typing in a command for the above mentioned situation, it is necessary to refer to the PDG side (physical slot plus one).

EXAMPLE:

TPDG boards are in the physical slots 3 and 6.

While running XUSF and attempting to set DEVice to the TPDG in physical slot 3, the command would be: DEV XUSF4

While running XUSF and attempting to set DEVice to the TPDG in physical slot 6, the command would be: DEV XUSF7

24.3 Setup Parameters

24.3.1 XUSF-Specific Parameters

In some of the information that follows, reference is made to “Cn.” This convention is used to indicate the assignment of parameter values to each of three output channels. The device assignment for each of the three channels is:

C0—Video

C2—Printer

C3—Keyboards

Use of the STATUS command displays dummy C1 and C4 values for HOOK_UP_TO and SCALE that cannot be changed.

Cn_HOOK_UP_TO Allowed values for C2 and C3 are OPEN and DEVICE.

Allowed values for C0 are OPEN and SINGLE.

The meanings are

OPEN = No device is connected

DEVICE = A physical device (Printer, Keyboard, etc.) is connected and ready

SINGLE = One video-display generator is available

Controls whether or not output operations are performed, where appropriate, on the respective channels. Only Test 6 (the Channel Configuration and Loopback test) performs loopback operations on channels 1 through 4 without regard to these parameters. The defaults are SINGLE for C0 and OPEN for C2 and C3

INITIALIZE_BOARD	Allowed values are YES and NO Controls whether or not the TPDG board is RESET following the RUN command. Normally set to YES. The default is YES.
Cn_SCALE	Allowed values for C0, C2, and C3 are 0 through 32767 Controls the frequency of channel operations for C0, C2, and C3 by Test 14, the System Exerciser test. A value of 0 will cause a new channel operation every pass, 1 will cause a new operation every other pass, etc. The defaults are 0. These parameters work in conjunction with the corresponding “Cn_HOOK_UP_TO” parameters.
EXERCISER_VIDEO_IMAGE_SELECTION	Allowed values are 0 through 3 and “—” (dash). The meanings are: 0 - Rotate through all three images on successive displays 1 - Display only image #1, the crosshatch image 2 - Display only image #2, the gray scale image 3 - Display only image #3, the blinking full white image — - Same as 0 Controls the video image(s) displayed during the System Exerciser test (Test 14). The default is “—”.
COLOR_TO_VDG	Allowed values for DARK, RED, GREEN, YELLOW, BLUE, MAGENTA, LT_BLUE, and WHITE Selects the screen color used by the Video Flood test (Test 16). The default is RED.
USE_WSCPU	Allowed values are YES and NO. Controls whether or not the WSI/WSCPU (Work Station CPU), if available, is included in the System Exerciser test (Test 14) activities. The default is YES.
RANDOM_SEED	Allowed values are 0 through 32767. This value provides the starting seed for those tests that use pseudo random values. The default is 1234.
DI02_LOOPBACK_INSTALLED	Allowed values are YES and NO. Controls whether or not the DI02 Bus Loopback test (Test 7) is run if it is in the test list. A value of YES implies that a DI02 loopback cable has been installed on the TPDG. The default is YES.

First_SCSI_Device_Address	This parameter provides either (a) the SCSI address of the first (or only) SCSI disk drive (typically a Bernoulli cartridge drive), or (b) the SCSI address of the designated "Master" controller (i.e., TPDG) used by the SCSI Controller test. The allowed values are 0-7. The default value is 0. This parameter is changeable only while XUSF is <u>off</u>.
First_SCSI_Device_Type	The parameter identifies the type of device associated with the SCSI address in First_SCSI_Device_Address. The allowed values are NOT_IN_USE—No device or not available for testing. WRITE_READ_ALLOWED—It is permissible to write to the drive. READ_ONLY_ALLOWED—It is only permissible to read from the drive. MASTER_CONTROLLER—The target device is another TPDG board. This selection is used only by the SCSI Controller test, Test 14 and implies that two or more TPDG boards are connected together via the SCSI bus. This configuration is used in the factory during board test. The default is NOT_IN_USE. This parameter is changeable only while XUSF is <u>off</u>.
Second_SCSI_Device_Address	This parameter provides either (a) the SCSI address of the second (or only) SCSI disk drive (typically a Bernoulli cartridge drive), or (b) the SCSI address of the designated "Master" controller (i.e., TPDG) used by the SCSI Controller test. The allowed values are 0-7. The default value is 0. This parameter is changeable only while XUSF is <u>off</u>.

Second_SCSI_Device_Type	This parameter identifies the type of device associated with the SCSI address in Second_SCSI_Device_Address. The allowed values are NOT_IN_USE—No device or not available for testing. WRITE_READ_ALLOWED—It is permissible to read from the drive. READ_ONLY_ALLOWED—It is only permissible to read from the drive. MASTER_CONTROLLER—The target device is another TPDG board. This selection is used only by the SCSI Controller test, Test 14, and implies that two or more TPDG boards are connected together via the SCSI bus. This configuration is used in the factory during board test. The default is NOT_IN_USE. This parameter is changeable only while XUSF is <u>off</u>.
Enable or Disable	Allows the operator to enable or disable the following functions: DISPLAY_DRIVE_ID_AND_CAPACITY—When enabled, this function causes the drive ID, from the SCSI "INQUIRY" command, and the block size and last logical block address to be displayed on starting the test. This option is then disabled by the program following the display. The default is ENABLED. KEYBOARD_ANNUNCIATORS—When enabled, allows the annunciators to "BEEP" during the exerciser test. The default is ENABLED. CONTROLLER_TO_CONTROLLER_TESTING—When enabled, allows the SCSI Controller test to run if it is in the test list. The default is DISABLED. This parameter is changeable anytime.

24.3.2 General Parameters

TESTS	Allowed values are 1 through 20
ERROR_LIMIT	Allowed values are 1 through 32767 or "-"
PASS_LIMIT	Allowed values are 1 through 32767 or "-"
REPORT	Allowed values are PASSNUMBER, SUSPENDED, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, SUSPENDED, TESTNUMBER, LOG
SCALE	not used
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

24.3.3 Preset Parameter Values for XUSF

C0_HOOK_UP_TO = SINGLE

C2_HOOK_UP_TO = OPEN

C3_HOOK_UP_TO = OPEN

INITIALIZE_BOARD = YES

C0_SCALE = 0

C2_SCALE = 0

C3_SCALE = 0

EXERCISER_VIDEO_IMAGE_SELECTION = —

COLOR_TO_VDG = RED

USE_WSCPU = YES

RANDOM_SEED = 1234

DI02_LOOPBACK_INSTALLED = YES

First_SCSI_Device_Address = 0

First_SCSI_Device_Type = NOT_IN_USE

Second_SCSI_Device_Address = 0

Second_SCSI_Device_Type = NOT_IN_USE

Enable = DISPLAY_DRIVE_ID_AND_CAPACITY, KEYBOARD_ANNUNCIATORS

Disable = CONTROLLER_TO_CONTROLLER_TESTING

24.4 Mode Characteristics for XUSF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

24.5 Individual XUSF Tests

(System Modes)

Test 01 Reset and Self-Test SUB, MOD

This test resets the TPDG by way of the PDG and SCSI Flag Control registers, thereby causing the PDG and SCSI to execute their self-tests.

This test will run in subsystem and module mode.

Test 02 Write/Read Registers Test SUB, MOD

Verifies the PDG and SCSI Status Bus logic. Testing consists of

1. On the PDG:
 - A. Writing to the Flag Control register, toggling Flags 2 through 7 and verifying them by reading and checking the Configuration Status register.
 - B. Writing and reading random words to/from the Operational Status and Paged Status registers.
2. On the SCSI:
 - A. Write to the Flag Control Register, toggling Flags 2 and 3 and verifying their effect by reading and checking the Hardware and Configuration Status registers.
 - B. Write and read random words to/from the Operational Status and Paged Status words.

This test will run in subsystem and module mode.

Test 03 Valid Commands

SUB, MOD

1. On the PDG:

- A. Verifies the correct operation of (1) the NOP command, with and without the “command complete” interrupt requested, and (2) the Display the “Firmware Revision” Paged Status (read and save this status on the first pass and, on subsequent passes, verify that there has been no change from the first pass).

2. On the SCSI:

- A. Verify the NOP command with and without the "Command Complete" interrupt requested.
- B. Execute the DOWN LINE LOAD command with random data and a valid checksum and the "Command Complete" interrupt requested.
- C. Execute the DUMP command with the "Command Complete" interrupt requested and verify the return of the data that was downloaded with the previous command.
- D. Execute the CHECKSUM command with the "Command Complete" interrupt and verify error-free termination.
- E. Execute the Display "Firmware Revision" Paged Status command. Read and save this status on the first pass, and on subsequent passes, read and verify that there has been no change.
- F. Execute the Display "Drives Present/Failed Paged Status command. Verify that the value returned in paged Status word, offset +\$C, is \$3F3F.

This test will run in subsystem and module mode.

This test executes the DUMP EXCEPTION STACK and DUMP TPDG RAM MEMORY commands. Downloaded test software loads the Data and Address registers with known values and then executes a TRAP instruction to force an exception on the LCPU. Following this, at the MCPU, the test executes a Dump Exception Stack command and evaluates the returned data for the correct values. Next, the MCPU executes a Dump TPDG RAM command and verifies the returned data against the known downloaded test software.

Test 05	Module Bus Errors Test	SUB
---------	------------------------	-----

This test verifies that the PDG can correctly detect and indicate Module Bus Errors asserted during certain failed transactions between the MCPU and the TPDG. The conditions tested are

- This test will run in subsystem mode.

Test 06 Channel Configuration and Loopback Test SUB, MOD

This test verifies the UARTs associated with channels 1 through 4. A valid configuration is selected at random and a series of fixed patterns is transferred by way of either internal or external loopback, depending on whether or not loopback plugs are installed. The patterns used are \$0000, \$FFFF, \$AAAA, and \$5555.

This test will run in subsystem and module mode.

Test 07 DI02 Bus Loopback Test SUB, MOD

This test verifies the TPDG/WSI interface using loopback. A loopback cable must be attached in order for this test to operate correctly. This test will be skipped if parameter USE_WSCPU is set to YES and there is a WSI board installed in the node, or if parameter DI02_LOOPBACK_INSTALLED is set to NO.

This test will run in subsystem and module mode.

Test 08 DRAM Test SUB, MOD

Verifies all of the TPDG DRAM memory. This test is downloaded to, and executed from, the LCPU.

This test will run in subsystem and module mode.

Test 09 GSP Translator Test SUB, MOD

This test verifies the TPDG DRAM to GSP “translator” logic. This test is downloaded to, and executed from, the GSP.

This test will run in subsystem and module mode.

Test 10 GSP VRAM Test SUB, MOD

Verifies all of the TPDG VRAM memory. This test is downloaded to, and executed from, the GSP.

This test will run in subsystem and module mode.

Test 11 GSP Video RAMDAC Signature Register Test SUB, MOD

Verifies the TPDG RAMDAC setup syndromes by way of the RAMDAC Signature register. This test is downloaded to, and executed from, the GSP.

This test will run in subsystem and module mode.

Test 12 GSP Video Output Test SUB, MOD

Verifies the TPDG RAMDAC RGB video output levels by utilizing the RAMDAC capability to compare the red and green gun outputs against the blue gun output and reference millivolts. Note that either a CRT or 75 ohm resistors must be installed on the RGB outputs for the CRT on the TPDG I/O board. This test is downloaded to, and executed from, the GSP.

This test will run in subsystem and module mode.

Test 13 SCSI Controller Test

MOD

This is a new test for XUSF 2.0.

This test is similar to SCMD Test 20 (Controller-To-Controller test)

This test first verifies that the SCSI IOCB interrupts can be generated for the Initiator and Target chains. Then the test verifies the SCSI I/O interface by communicating between two or more controllers (TPDGs) cabled together on the SCSI bus. This test has three parts (1) a controller-to-controller transfer test using random data of random block sizes of from 3—1024 words, (2) verifies that the "Buffer Overflow" IOCB status can be displayed, and (3) verifies that the "Other Than Send CDB Received When Selected" IOCB status can be displayed.

This test is skipped if parameter CONTROLLER_TO_CONTROLLER_TEST is DISABLED.

Note that this test requires (1) that two or more TPDGs be connected together via the SCSI bus connector on the respective TPDG I/O boards, (2) that the parameters

FIRST_SCSI_DEVICE_ADDRESS and FIRST_SCSI_DEVICE_TYPE (or SECOND_SCSI_DEVICE_ADDRESS and SECOND_SCSI_DEVICE_TYPE), in the copies of XUSF corresponding to the TPDGs that are connected together, be set up appropriately.

This means that the FIRST_SCSI_DEVICE_ADDRESS (or SECOND_SCSI_DEVICE_ADDRESS) be set to the (same) SCSI address of the TPDG—SCSI that is designated to be the "master" controller (note that this is an arbitrary designation and is needed only to facilitate testing), and that FIRST_SCSI_DEVICE_TYPE (or SECOND_SCSI_DEVICE_TYPE) be set to MASTER_CONTROLLER, (3) that parameter CONTROLLER_TO_CONTROLLER_TEST be ENABLED, and (5) **that all interconnected TPDG boards reside in the same node.**

This test will run in Module mode.

Test 14 System Exerciser Test

SUB, MOD, EXER

This test will optionally exercise one or more of the following: (a) the device(s) connected to the PDG, (b) one or two disks connected to the TPDG SCSI, and (c) the WSCPU Module Bus and TPDG interfaces.

For the PDG, this test outputs to the devices on channels 0 (video), 2 (printer), and 3 (keyboard) as indicated by parameters Cn_Hook_Up_To. The other parameters that affect the operation of this test are Cn_Scale, Exerciser_Video_Image_Selection, and Use_WSCPU. The activity associated with each channel, as influenced by the Cn_Hook_Up_To and Cn_Scale parameters, is

- a. Send random task codes to the keyboard electronics and verify the response.
- b. Print a line of random characters on the printer.
- c. Display any one of three images on the CRT or rotate through all three, depending on the setting of the parameter Exerciser_Video_Image_Selection.

The three images are

1. A full intensity, single element white crosshatch pattern with a line around the edge of the CRT.
2. A gray scale pattern with eight levels of shading from black to white.
3. A full screen, full intensity, blinking white screen.

If there is a WSI/WSCPU board installed, UNIX is running in the WSCPU, and parameter Use_WSCPU is set to YES, then, in addition to the above described activity, the WSCPU software is requested to load test software originating in the MCPU and execute it from the WSCPU. The test software, in turn, performs a memory test on a specified block of TPDG DRAM. All this happens interactively with the other channel activity as defined by the channel specific parameters.

If parameter FIRST_SCSI_DEVICE_TYPE and/or SECOND_SCSI_DEVICE_TYPE is/are set to either WRITE_READ_ALLOWED or READ_ONLY_ALLOWED, the disk(s) whose SCSI address(es) are entered in parameter FIRST_SCSI_DEVICE_ADDRESS and/or SECOND_SCSI_DEVICE_ADDRESS will be exercised accordingly.

This test will run in subsystem, module, and exerciser mode.

Test 15 Monitor Device Status Utility Test SUB

This test continuously monitors and logs the PDG device status, initially and then when changes are detected. This is a utility test and, therefore, must be the only test selected in Subsystem Mode in order to run. This test will run until STOPPED or ABORTED.

Test 16 Video Flood Utility Test SUB

This test fills (floods) the CRT with the color specified in the parameter Color_To_VDG. This test runs continuously and changes the screen color as the parameter is changed. This is a utility test and, therefore, must be the only test selected in Subsystem Mode in order to run. This test will run until ABORTED.

Test 17 Cursor Control Utility Test SUB

This test writes an image on the CRT consisting of a series of targets suitable for verifying cursor control by way of touchscreen, mouse, or trackball. Cursor “hits” that are off-target are reported as errors. This is a utility test and, therefore, must be the only test selected in Subsystem Mode in order to run. This test will run until STOPPED or ABORTED.

Test 18 CRT Utility Test SUB

This test writes an image on the screen consisting of 15 colored boxes on a gray background, useful for a quick check of CRT operation. This is a utility test and, therefore, must be the only test selected in Subsystem Mode in order to run.

Test 19 Video Crosshatch Pattern Utility Test SUB

This test writes a crosshatch test pattern on the screen that is useful in verifying correct operation of the video. This is a utility test and, therefore, must be the only test selected in Subsystem Mode in order to run.

Test 20 Video Grayscale Utility Test SUB

This test writes a grayscale pattern on the screen that is useful in verifying correct operation of the video. This is the same grayscale pattern described in the System Exerciser Test (Test 14). This is a utility test and, therefore, must be the only test selected in Subsystem Mode in order to run.

24.6 XUSF Alarm Messages

The following are the possible Alarm Messages and their descriptions.

24.6.1 Alarm Categories

The alarm numbers accompanying an error display are of the form “txx” or “ttxx,” where “t” or “tt” indicates the test number in which the error was discovered and “xx” indicates a specific check, within the test, which failed. If needed, this number can be used as a reference to trace back to the program listing to find the sequence of operations leading up to the error.

24.6.2 Alarm Codes

Alarm Code Number	Description
10-11	Alarms associated with attempting to shut down the UNIX XDAEMON in preparation to running the test program. These alarms indicate that there is either a problem in communicating with UNIX by way of the WSI Shared RAM space (namely, on-board WSI memory) or you have logged on to UNIX. You must be logged out of UNIX before running XUSF. These alarms can occur only if a WSI exists and the WSI Configuration Status byte indicates that UNIX is running.
20	Following a TPDG board RESET, either the PDG or SCSI Hardware Statuses were wrong or the TPDG Self-Test took too long.
30-32	Alarms associated with attempting to do a request sense for the SCSI device corresponding to Second_SCSI_Device_Address.
33-35	Alarms associated with attempting to do a request sense for the SCSI device corresponding to First_SCSI_Device_Address.
90	Just before stopping or aborting XUSF, could not enable the UNIX XDAEMON (see alarms 10-11 above).
92	TPDG Self-Tests took too long following a RESET just before stopping or aborting XUSF.
110-118	Test 01—failures associated with or immediately following a TPDG board RESET.
120	Test 01—failures associated with attempting to set the TPDG green LED (Passed Module Tests flag) following a TPDG board RESET.

Alarm Code Number	Description
220-231	Test 02—failures in the PDG and SCSI Hardware Status words following the setting or clearing of a PDG and SCSI Flag. The even-numbered alarms pertain to the setting of the Flag and odd-numbered alarms pertain to the clearing of the Flag. Alarms 220/221 are associated with Flag #2 (Interrupt Enable), 222/223 with Flag #3 (Passed Module Test), 224/225 with Flag #4 (undefined), 226/227 with Flag #5 (undefined), 228/229 with Flag #6 (undefined), and 230/231 with Flag #7 (undefined).
240-251	Test 02—failures in the PDG and SCSI Hardware Status words following the setting or clearing of a PDG or SCSI Flag. Alarms 240-251 have the same correlation with setting or clearing of the Flags as alarms 220-231, respectively.
260-264	Test 02—failures in one or more of the PDG or SCSI registers following a write of a random word to the Operational Status. Five patterns and their complements are written.
265-269	Test 02—failures following a write of the complement (inverse) of the previous pattern.
270-274	Test 02—failures following a write of a random word to PDG or SCSI Paged Status Word #1 (offset + \$E).
275-279	Test 02—failures following a write of the complement (inverse) of the previous pattern.
280-284	Test 02—failures following a write of a random word to PDG or SCSI Paged Status Word #1 (offset + \$E).
285-289	Test 02—failures following a write of the complement (inverse) of the previous pattern.
320	Test 03—failures following an attempt to initialize the execution of valid commands with and without the command complete interrupt—for the SCSI side.
330-350	Test 03—failures following an attempt to execute the Download and Dump commands—for the SCSI side.

Alarm Code Number	Description
360	Test 03—failures following an attempt to execute the Checksum commands—for the SCSI side.
370	Test 03—failures following an attempt to execute a NO OPERATION (NOP) command without the “command complete” interrupt requested for the PDG side.
380	Test 03 - failures following an attempt to execute a NO OPERATION (NOP) command with the “command complete” interrupt requested for the PDG side.
390	Test 03—failures following an attempt to read the FIRMWARE REVISION Paged Status and verify that it has not changed from previous readings for the PDG side.
410	Test 04—failures following a PDG board RESET.
420-425	Test 04—failures following the execution of the “Dump Exception Stack” command when there has been no exception.
430	Test 04—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
440	Test 04—failures following the execution of the “Dump TPDG Local Memory” command.
450	Test 04—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives, previously downloaded, are instructed to force an “exception” at the LCPU.
460	Test 04—failures following the execution of the “Dump Exception Stack” command following the previous exception.
465	Test 04—the contents of the LCPU registers, returned by the “Dump Exception Stack,” were not correct.
510-518	Test 05—failures following an attempt to force a Bus Error by having the PDG or SCSI attempt to access nonimplemented memory.
520-526	Test 05—failures following a TPDG board RESET after the previous Bus Error.
540-550	Test 05—failures following an attempt to force a Bus Error by forcing a Local Bus Parity Error.
560-572	Test 05—failures following an attempt to force a Bus Error by having the PDG or SCSI attempt to DMA read a memory location which has a double-bit EDAC error imbedded in it.

Alarm Code Number	Description
611-614	Test 06—failures following the execution of the CONFIGURE command to channel 1 through 4 as indicated by the alarm code (i.e. 611 = channel 1, 612 = channel 2, etc.).
621-624	Test 06—failures following an attempt to issue “Loopback” IOCB to channel 1 through 4 as indicated by the alarm code.
631-634	Test 06—failures following an attempt to execute the “Reset Device” command to channel 1 through 4 as indicated by the alarm code.
705	Test 07—failures following attempts to read from the “Device Status” Paged Status word.
710	Test 07—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
740	Test 07—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to execute the DI02 Loopback test from the LCPU.
750	Test 07—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
760	Test 07—failures while checking the IOCB termination status from the previous IOCB operation.
791	Test 07—failures found by the LCPU while conducting the DI02 Loopback test by writing and reading random long words to/from a specified block of the TPDG DRAM while loopback is enabled.
792	Test 07—failures found by the LCPU while conducting the DI02 Loopback test by writing and reading the complement of the random long words written before.
805	Test 08—failures following attempts to read from the “Device Status” Paged Status word.
810	Test 08—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
840	Test 08—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to execute the DI02 Loopback test from the LCPU.
850	Test 08—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
860	Test 08—failures while checking the IOCB termination status from the previous IOCB operation.

Alarm Code Number	Description
891	Test 08—failures found by the LCPU while conducting the TPDG DRAM memory test, sweep #1, by writing and reading random long words to/from the entire TPDG DRAM. The initial write operations are conducted on a byte basis and all subsequent write and read operations are done on a long word basis.
892	Test 08—failures found by the LCPU while conducting the TPDG DRAM memory test, sweep #1, by writing and reading the complement of the random data written before.
901	Test 09—there are two error situations bearing this alarm number—the content of the error message identifies the context of the error and which of the following applies: Data comparison error found by the GSP, while testing the translated DRAM address space, indicating translated data was not as expected. The GSP had an Illegal Operation interrupt while testing the translated DRAM address space.
902	Test 09—the GSP had a Bus Fault interrupt while testing the translated DRAM address space.
905	Test 09—failures following attempts to read from the “Device Status” Paged Status word.
910	Test 09—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
940	Test 09—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to “Download” the GSP-based “translator test,” start the GSP and monitor and report any errors reported by the GSP-based test.
950	Test 09—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
960	Test 09—failures while checking the IOCB termination status from the previous IOCB operation. For IOCB status of \$988x, see subsection 24.7.3

Alarm Code Number	Description
1001	Test 10—there are two error situations bearing this alarm number—the content of the error message identifies the context of the error and which of the following applies: Data comparison error found by the GSP while conducting the Video RAM (VRAM) memory test, sweep #1, while writing and reading random long words throughout the VRAM memory area. The initial write operations are conducted on a byte basis and all subsequent write and read operations are done on a long word basis. The GSP had an Illegal Operation interrupt while testing the VRAM.
1002	Test 10—there are two error situations bearing this alarm number — the content of the error message identifies the context of the error and which of the following applies: Data comparison error found by the GSP while conducting the Video RAM (VRAM) memory test, sweep #2, while writing and reading the complement of the random long words written before. The GSP had a Bus Fault interrupt while testing the VRAM.
1005	Test 10—failures following attempts to read from the “Device Status” Paged Status word.
1010	Test 10—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
1040	Test 10—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to “Download” the GSP-based VRAM test, start the GSP, and monitor and report any errors reported by the GSP-based test.
1050	Test 10—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
1060	Test 10—failures while checking the IOCB termination status from the previous IOCB operation. For IOCB status of \$988x, see subsection 24.7.3.

Alarm Code Number	Description
1102	Test 11—there are two error situations bearing this alarm number — the content of the error message identifies the context of the error and which of the following applies: The first signature was not equal to the expected value. Failure found by the GSP while testing the RAMDAC setup syndromes by way of the RAMDAC Signature register. Address “20C” refers to the red syndrome register. The GSP had a Bus Fault interrupt while testing the RAMDAC syndromes.
1103	Test 11—the second signature was not equal to the expected value. Failure found by the GSP while testing the RAMDAC setup syndromes by way of the RAMDAC Signature register. Address “20C” refers to the red syndrome register.
1104	Test 11—the first and second signature are not different. Failure found by the GSP while testing the RAMDAC setup syndromes by way of the RAMDAC Signature register. Address “20C” refers to the red syndrome register.
1105	Test 11—failures following attempts to read from the “Device Status” Paged Status word.
1110	Test 11—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
1140	Test 11—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to “Download” the GSP-based “RAMDAC Syndrome Setup test,” start the GSP, and monitor and report any errors reported by the GSP-based test.
1150	Test 11—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
1160	Test 11—failures while checking the IOCB termination status from the previous IOCB operation. For IOCB status of \$988x, see subsection 24.7.3.

Alarm Code Number	Description
1201	Test 12—there are two error situations bearing this alarm number—the content of the error message identifies the context of the error and which of the following applies: The addressed gun was never less than the reference. Failure found by the GSP while testing the video outputs. The “address” field identifies the test which failed; \$A0 is red gun compared to blue, \$90 is red gun compared to 150 mV reference, \$60 is green gun compared to blue, and \$50 is green gun compared to 150 mV reference. The GSP had a Illegal Operation interrupt while testing the Video Outputs.
1202	Test 12—there are two error situations bearing this alarm number—the content of the error message identifies the context of the error and which of the following applies: The addressed gun was below tolerance. Failure found by the GSP while testing the video outputs. The “address” field identifies the test which failed; \$A0 is red gun compared to blue, \$90 is red gun compared to 150 mV reference, \$60 is green gun compared to blue, and \$50 is green gun compared to 150 mV reference. The GSP had a Bus Fault interrupt while testing the Video Outputs.
1203	Test 12—the addressed gun was above tolerance. Failure found by the GSP while testing the video outputs. The “address” field identifies the test which failed; \$A0 is red gun compared to blue, \$90 is red gun compared to 150 mV reference, \$60 is green gun compared to blue, and \$50 is green gun compared to 150 mV reference.
1205	Test 12—failures following attempts to read from the “Device Status” Paged Status word.
1210	Test 12—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
1240	Test 12—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to “Download” the GSP-based “Video Output test,” start the GSP, and monitor and report any errors reported by the GSP-based test.
1250	Test 12—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.

Alarm Code Number	Description
1260	Test 12—failures while checking the IOCB termination status from the previous IOCB operation. For IOCB status of \$988x, see subsection 24.7.3.
1300-1319	Test 13—failures following an attempt of controller-to-controller communications. These alarm codes are for the slave(s) only.
1320-1329	Test 13—failures following an attempt to overflow the buffer. These alarm codes are for the slave(s) only.
1330-1339	Test 13—failures following an attempt to execute an “Other than send CDB.” These alarm codes are for the slave(s) only.
1350-1369	Test 13—failures following an attempt of controller-to-controller communications. These alarm codes are for the master only.
1370-1379	Test 13—failures following an attempt to overflow the buffer. These alarm codes are for the master only.
1380-1389	Test 13—failures following an attempt to execute an “Other than send CDB.” These alarm codes are for the master only.
1401	Test 14—there are two error situations bearing this alarm number—the content of the error message identifies the context of the error and which of the following applies: Data comparison error found by the WSCPU-based test while performing a memory test (that is, write/read random long words) on a 4 KB block of TPDG DRAM. The WSCPU-based test has failed to indicate that it has started within the expected time frame (for example, 15 seconds). This implies that the WSCPU-based test was not started by UNIX or that the WSCPU-based test can no longer communicate with the WSI Shared Memory space.
1402	Test 14—the WSCPU-based test did not complete its TPDG DRAM memory test within the expected time frame (for example, 15 seconds). This implies that the WSCPU-based test is hung-up, crashed, or lost communication with the WSI for some unknown reason.
1410	Test 14—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.

Alarm Code Number	Description
1420	Test 14—the WSCPU-based test failed to get loaded (from module memory) and/or started by UNIX (running in the WSCPU). Implied are problems relating to: a. An inability to write/read to/from the WSI shared memory space by either, or both, the WSCPU or MCPU, or b. The UNIX-based software running on the WSCPU has been changed so that it fails to recognize the requests from the MCPU to load, compile, and run the WSCPU-based test program.
143x	Test 14—failures detected while attempting to start an IOCB operation to the channel indicated by “x” (for example, 0 = video, 2 = printer, and 3 = keyboard electronics).
1440	Test 14—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
1445	Test 14—failures while checking the IOCB termination status from the previous IOCB operation. For IOCB status of \$988x, see subsection 24.7.3.
1454	Test 14—failures following an attempt to correct an overlap of LBAs if there is one - used for the exercising of SCSI drives. Note: This failure is a software failure.
1495-1499	Test 14—failures following the termination of the SCSI IOCBs, i.e., evaluating the data from the exercising of the SCSI drives.
1505	Test 15—failures following attempts to read from the “Device Status” Paged Status word.
1510	Test 15—failures following attempts to read from the “Device Status” Paged Status word.
1520	Test 15—failures following attempts to read from the “Device Status” Paged Status word.
1605	Test 16—failures following attempts to read from the “Device Status” Paged Status word.
1610	Test 16—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.

Alarm Code Number	Description
1640	Test 16—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to flood the screen with the color specified in parameter COLOR_TO_VDG.
1705	Test 17—failures following attempts to read from the “Device Status” Paged Status word.
1710	Test 17—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
1720	Test 17—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to display the touchscreen targets on the CRT.
1725	Test 17—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
1730	Test 17—failures associated with an attempt to send a “beep” code to the keyboard by way of a “Write Data” IOCB. This action is intended to alert the operator to a touchscreen/mouse “hit.”
1735	Test 17—a touchscreen or mouse “hit” was done with the cursor not in any of the targets.
1740	Test 17—the first code returned from the keyboard electronics following a touchscreen or mouse “hit” was expected to be an “X” coordinate code (\$50xx through \$54xx), but was not.
1750	Test 17—the second code returned from the keyboard electronics following a touchscreen or mouse “hit” was expected to be a “Y” coordinate code (\$58xx through \$5Cxx), but was not.
1805	Test 18—failures following attempts to read from the “Device Status” Paged Status word.
1810	Test 18—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
1820	Test 18—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to display the colored boxes images on the CRT.
1840	Test 18—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to write the labels for the top row of boxes previously displayed.

Alarm Code Number	Description
1850	Test 18—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to write the labels for the bottom row of boxes previously displayed.
1905	Test 19—failures following attempts to read from the “Device Status” Paged Status word.
1910	Test 19—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
1940	Test 19—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to display the crosshatch pattern on the CRT.
1950	Test 19—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
1960	Test 19—failures while checking the IOCB termination status from the previous IOCB operation.
2005	Test 20—failures following attempts to read from the “Device Status” Paged Status word.
2010	Test 20—failures following an attempt to issue the “Download Primitives” IOCB to channel 0.
2040	Test 20—failures following an attempt to issue the “Execute Primitives” IOCB to channel 0. The primitives are instructed to “Download” the GSP-based “Display the Grayscale image,” start the GSP, and monitor and report any errors reported by the GSP-based test.
2050	Test 20—failures while checking the Operational Status and associated interrupt from the completion of the current IOCB.
2060	Test 20—failures while checking the IOCB termination status from the previous IOCB operation. For IOCB status of \$988x, see subsection 24.6.3

24.6.3 Undocumented IOCB Status Values

The down-loaded test primitives indicate errors detected by the LCPU by first indicating an error condition in the IOCB termination status and then reporting any further error information, if any, to the MCPU, which will be reported on subsequent error messages. The IOCB error status codes returned by the firmware are documented in the appropriate Functional Design Specification and other support documentation. The IOCB error status codes returned by the test primitives are arbitrary, but are assigned so that they do not duplicate any of the documented codes. The IOCB error status codes returned by the down-loaded test primitives are in the \$988x series as follows:

\$9881	Illegal test primitive command (this would be a program bug)
\$9882	LCPU test failure
\$9884	GSP-based test timeout; the GSP-based test has not completed its testing in the expected time period and the LCPU has given up on waiting for it to complete. Further information may or may not follow in subsequent error messages.
\$9888	GSP-based test has found and reported one or more failures. These failures will be reported in subsequent error messages.
\$988C	GSP test lockup; the GSP-based test has failed to change the Host Data register. Further information may or may not follow in subsequent error messages.
\$988F	GSP has failed to acknowledge the Halt command directed to the Host Control Hi register.

25 LCN SUBSYSTEM TEST (LCNS)

25.1 Common LCN Subsystem Test Elements

The primary kernel elements tested by this program are the LCN portion of the K4LCN or retrofitted K2LCN processor boards. Also, the LCN cables are verified.

25.2 Use Information

Only nodes specified in the User_Node_List are tested. This list defaults to all nodes set, which allows testing of all nodes in the network; however, it may be restricted to fewer nodes through use of the command “USER_NODE_LIST,” followed by a list of nodes separated by commas.

For Test 2, LCN cable switching must be active (SWITCH YES) and both cables must be running error-free before testing for crossed cables can be done.

25.3 Setup Parameters

25.3.1 LCNS-Specific Parameters

CROSSED_NODE Allowed values and their meanings are

- 1 No node with intentionally crossed cables specified
(crossed-cable summary message printed after first and last pass, as well as any pass that finds one or more nodes with crossed cables).
- nn Node number with intentionally crossed cables
(specifies factory environment. Crossed-cable summary message suppressed if this is the only node with crossed cables).

25.3.2 General Parameters

TESTS	Allowed values are 1 through 3
ERROR_LIMIT	Allowed values are 1 through 32766 or "-"
PASS_LIMIT	Allowed values are 1 through 32766 or "-"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 0 through 32766 or "-"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

25.4 Preset Parameter Values For LCNS

TESTS = 1, 2, 3
ERROR_LIMIT = 5
PASS_LIMIT = -
REPORT = LOG
INHIBIT = PASSNUMBER, TESTNUMBER
SCALE = 0
MINUTES LIMIT = -
ABBREVIATION = 4
AFTER_ERROR_LIMIT = ABORT
CROSSED_NODE = -1

25.5 Mode Characteristics For LCNS

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

25.6 individual LCNS Tests

(System Modes)

Test 01 Check Paged Status

SUB, MOD

This test verifies that data can be written and read from each of the two LCN paged status words and that there is no interaction between these words.

Random data is written to the first paged status word (Offset C), and then both paged status words are read back and compared to expected values. If the data mismatches, alarm 101 is printed.

Next, random data is written to the second paged status word (Offset E), and both paged status words are read back and compared to expected values. If the data mismatches, alarm 102 is printed.

Test 01 alarm messages—Paged Status Register Error

101	Write: \$xxxxxxx,	Read: \$yyyyyyy,	Diff: \$zzzzzzzz
102	Write: \$xxxxxxx,	Read: \$yyyyyyy,	Diff: \$zzzzzzzz

Test 02 LCN Cable Verification

SUB, MOD, EXER

Determines if the LCN cables are correct or crossed and prints a summary message for the operator. Also tests the ability to send LCN messages on each cable separately to all other nodes in the network. This test can be run in one node or multiple nodes as long as they have either K4LCN or retrofitted K2LCN processor boards.

To verify that the send message on single cable feature works, it is necessary to have one or more nodes in the network with crossed cables. Successful operation would report these nodes with crossed cables and all others with correct cables.

For the factory environment, the setup parameter “CROSSED_NODE” can be changed to indicate the node number with crossed cables. This parameter is defaulted to -1, which indicates nonfactory environment and the summary message is to be printed after the first and last passes, as well as any pass that finds one or more nodes with crossed cables. When a node number is specified, the crossed cable results are compared to this node, and if only this node indicates crossed cables and no other, the summary message is suppressed.

Test 02 Operator Messages

- Wrong Processor Type Installed
- Wrong K2LCN Hardware Revision
- Wrong K2LCN Firmware Revision

Test 02 Alarm Messages

201 Error: Message Received From Wrong Node nn

- 202 Error: Crossed Node nn Does Not Indicate Crossed Cables
- 203 No Remote Nodes In Node List
- 204 LCN Cable Switching Is Disabled

Test 02 Summary Message

LCN cables are correct in xx nodes: n-n

LCN cables are crossed in yy nodes: n-n

LCN cables indeterminate in zz nodes: n-n (This line is not printed if zz=0)

Test 02 Run Time Estimates

Number of Nodes in Node List	Estimated Run Time in Minutes
2 - 12	2
13 - 24	4
25 - 36	6
37 - 48	8
48 - 60	10
61 - 64	12

Test 03 Diagnostic Commands Verification SUB

Tests each node in the current network list for the ability to respond to LCN diagnostic commands. The TOCS as master is not verified by this test. The Network command type prompt is recommended for starting the testing.

25.7 LCNS Alarm Messages

LCNS alarm messages describe in English text the specific problems encountered.

26 A^XM Subsystem Tests (XAMF)

26.1 XAMF Subsystem Elements

The Application Module^X (A^XM) subsystem elements required by this program include

- A WSI2 (Work Station Interface 2) board with a Work Station CPU (WSCPU)
- WSCPU hard drive(s) loaded with UNIX and supporting software
- An HMPU, K2LCN, or K4LCN board with appropriate firmware to support the initialization of the WSI2 board (and the attendant startup of UNIX).

The subsystem elements exercised by this program include the address and data line portions of the VMEBUS and module bus interfaces, access to the WSI2 board's shared RAM, and access to the LCN Module DRAM.

26.2 XAMF HVTP Use Information

XAMF has only one test, and this test exercises the VMEBUS and Module bus interfaces. To support this effort, UNIX must be running, and support software (for example, "LCN DAEMON") must exist on the WSCPU hard drive.

NOTE

The Work Station peripherals are not tested by any HVTS programs.

26.3 Setup Parameters

26.3.1 XAMF-Specific Parameters

RANDOM_SEED

The long integer value (\$0—\$FFFFFFFF) is used as the seed for the generation of random number(s) during memory and bus tests. The default for this parameter is \$00400006. This parameter can only be changed by the operator when XAMF is off.

26.3.2 General Parameters

TESTS	Allowed value is 1
ERROR_LIMIT	Allowed values are 1 through 32767 or "—"
PASS_LIMIT	Allowed values are 1 through 32767 or "—"
REPORT	Allowed values are PASSNUMBER, SUSPENDED, TESTNUMBER, LOG
INHIBIT	Allowed values are PASSNUMBER, SUSPENDED, TESTNUMBER, LOG
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 1 through 32767 or "—"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

26.3.3 Preset Parameter Values for XAMF

RANDOM_SEED = \$400006

TESTS = 1

ERROR_LIMIT = 5

INHIBIT_LIMIT = —

REPORT = LOG

INHIBIT = PASSNUMBER, SUSPENDED, TESTNUMBER

SCALE = 0

MINUTES_LIMIT = —

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

26.4 Mode Characteristics For XAMF

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

26.5 Individual XAMF Tests

(System Modes)

Test 01 A^XM Exerciser Test SUB, MOD, EXER

This test exercises the Module bus and VMEBUS interfaces on the WSI2/WSCPU board set. UNIX must be running in the Application Module^X (A^XM).

The UNIX-based support software is requested to load, compile, and execute test software that resides in the MCPU (LCN Module CPU) and execute it from the WSCPU. The test software copies a 2 K word block of random long word data from one block in the MCPU DRAM to another block. The block addresses are randomly selected every pass. The data blocks are then compared by both the WSCPU software and the LCN test software (XAMF). The run time for the test is approximately 5 seconds per pass.

26.6 XAMF Alarm Numbers

This section identifies the possible alarm messages and their descriptions that are originated by XAMF. The alarm messages contain descriptive information that should make them self-explanatory. The contents of this section, in conjunction with the displayed alarm message(s), should serve to identify the failed ORU. When a data comparison error occurs, the random number seed is also displayed.

26.6.1 Pretest Initialization

Alarm Number	Description
10	The WSI2 Configuration status register indicates that either the module CPU (HMPU, K2LCN, K4LCN) did not set the WSCPU Interface Enabled bit (bit 2), or the WSCPU software did not set the WSCPU Ready and Running bits (bits 4 and 3, respectively).

26.6.2 AXM Exerciser Test (Test 01)

Alarm Number	Description
145	The request to the UNIX software to Load, Compile, and Run the exerciser test was not accepted (the request pointer should be set to NIL after the request is accepted).
150	A comparison error between the source and destination data blocks was detected by the exerciser running in the WSCPU.
158	The exerciser test did not start running in the WSCPU. This implies that it failed in either the Load or Compile parts of the request.
159	The exerciser test that was running in the WSCPU did not complete the copy and verification parts of its execution.
160	The WSCPU failed to send an Acknowledgement (ACK) message to the LCN through the LCN Input Message Mailbox in the WSI2 board's shared RAM upon completion of the "Run script."
165	This alarm follows up on Alarm 145, in that UNIX was running, but the Load and Compile did not occur.
170	A comparison error was detected because the source data block changed sometime during the WSCPU Load, Compile, and Run activity.
180	A comparison error between the source and destination data blocks was detected by XAMF while running in the LCN mode.

27 Control And Communication Processor Tests (CCPT)

27.1 Control And Communication Processor Tests Elements

These tests provide verification of most of the CLMI and/or CCP board hardware. Although they use some firmware to perform this testing, there is no attempt to verify the firmware itself.

27.2 CCPT Use Information

The purpose of this hardware verification test is to exercise DMA transfers and internal and external address lines, including verifying data integrity and correct transfer lengths. It also performs a comprehensive memory test of the CLMI/CCP memory, and initiates the firmware test of the external ports (UARTs).

27.3 Setup Parameters

27.3.1 CCPT-Specific Parameters

none

27.3.2 General Parameters

TESTS	Allowed value is 1 through 4
ERROR_LIMIT	Allowed values are 1 through 32766 or "--"
PASS_LIMIT	Allowed values are 1 through 32766 or "--"
REPORT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
INHIBIT	Allowed values are PASSNUMBER, TESTNUMBER, LOG, SUSPENDED
SCALE	Allowed values are 0 through 999
MINUTES_LIMIT	Allowed values are 1 through 32766 or "--"
ABBREVIATION	Allowed values are 0 through 4
AFTER_ERROR_LIMIT	Allowed values are ABORT or HALT

27.4 Preset Parameter Values for CCPT

TESTS = 1-4

ERROR_LIMIT = 5

INHIBIT_LIMIT = –

REPORT = LOG

INHIBIT = PASSNUMBER, TESTNUMBER, SUSPENDED

SCALE = 100

MINUTES_LIMIT = –

ABBREVIATION = 4

AFTER_ERROR_LIMIT = ABORT

27.5 Mode Characteristics for CCPT

<u>System Mode</u>	<u>Programs Running</u>	<u>Tests Running</u>	<u>Delay Between Tests</u>
Subsystem	Only one	One or more	None
Module	One or more	One or more	Random
Exerciser	One or more	Only one	Random

27.6 Individual CCPT Tests

(System Modes)

Test 01 Fixed Transfer Test SUB, MOD

This test exercises the basic DMA and addressing functionality of the CCP using a variety of different Module Bus addresses. The test transmits a large fixed-size block of random data to a fixed CCP address using the Memory Modify command, then reads it back into the same Module Bus address using the Dump command, and then verifies the received data. This is repeated several times each pass using the same data block size and CCP address, but different Module Bus Addresses and different random data.

Test 02 CLMI/CCP Memory Test SUB, MOD

Like test 1, this test verifies the basic DMA and addressing functionality of the CCP using fixed size blocks of random data, and it uses the Memory Modify command to transmit the data and the Dump command to read it back; however, test 2 uses a fixed Module Bus address and varies the CCP address over the full range of usable CCP addresses. At the same time the CCP memory is thoroughly tested using the new T & D memory testing methodology.

Test 03 Exerciser/Random Test SUB, MOD, EXER

This test goes beyond the basic hardware verification of the other two tests, by varying several parameters at once. Although it still uses random data and transmits and receives by using the Memory Modify and Dump commands, the buffer sizes, CCP addresses, and Module Bus addresses are all randomly varied (with an emphasis on short quick transfers). Also unlike tests 1 and 2, this test receives the data back into a different module Bus address from that which was used to transmit the data. Not only does this provide better Module Bus address testing, it also allows the transmission of data to be followed immediately by its reception (because the receive buffer has been precoated in advance); however, about half the time the transmit and receive will be separated by a brief random duration pause..

Test 04 CLMI/CCP UART Loop-Back Test SUB, MOD

This test verifies the basic functionality of the four UART ports using the RS-232 and RS-485 interfaces, whenever the appropriate loop-back cables are connected. Thus, it is viewed as a factory test, although the field can use it if they connect these cables. Errors are reported as readable alarms.

27.7 CCPT ALARM NUMBERS

This section identifies the possible alarm messages and their descriptions that are originated by CCPT. The alarm messages contain descriptive information that should make them self-explanatory. The contents of this section, in conjunction with the displayed alarm message(s), should serve to identify the action or command that failed. When a data comparison error occurs, the random number seed is also displayed.

27.7.1 Start Reset Sequence

Alarm Number	Failing Command or Action
10 and 410	Addressable Latch Command.

27.7.2 Complete Reset Sequence

Alarm Number	Failing Command or Action
20 and 420	Addressable Latch Command.

27.7.3 During Reset Sequence

Alarm Number	Failing Command or Action
25 and 425	Firmware Self-Test never completed.

27.7.4 Set Passed Module Test

Alarm Number	Failing Command or Action
30 and 430	Addressable Latch Command.

27.7.5 Get First CCP Address or Get Last CCP Address or Test All Four UART Ports

Alarm Number	Failing Command or Action
40 and 440	Paged Status Command.

27.7.6 Memory Dump or Memory Modify

Alarm Number	Failing Command or Action
--------------	---------------------------

150/250/350	Command Register Command.
-------------	---------------------------

27.7.7 Data Comparison Error Writing and Reading Back Random Data

Alarm Number	Failing Command or Action
--------------	---------------------------

160 and 360	A data comparison error was detected between writing and reading back random data.
-------------	--

27.7.8 Memory Test Data Comparison Error Using Random Data

Alarm Number	Failing Command
--------------	-----------------

270	A memory data comparison error was detected between writing and reading back random data.
-----	---

27.7.9 Memory Test Data Comparison Error Using Inverted Random Data

Alarm Number	Failing Command
--------------	-----------------

280	A memory data comparison error was detected between writing and reading back inverted random data.
-----	--

27.7.10 Firmware Detected and Reported a Loopback UART Failure

Alarm Number	Failing Command
--------------	-----------------

490	Firmware detected and reported a loopback UART failure.
-----	---

Appendix A

Terms And Abbreviations

\$	Hexadecimal value
ALARM	Displays error information and increments the error count
BASE ADDRESS	Hexadecimal value that is the first accessible address of the DUT (that is, \$040000 + [\$Module Slot * \$1000])
BYTE	Eight bits
DAC	Digital to Analog Converter
DI02 BUS	Bus connecting the TPDG to the WSCPU (namely, the WSBUS
DMA	Direct Memory Access
DRAM	Dynamic RAM
DUT	Device Under Test
DVTP	Device Verification Test Program
EPROM	Erasable Programmable Read Only nonvolatile Memory
FIRMWARE	Software whose ultimate source is nonvolatile memory
GSP	Texas Instruments 34020 Graphics System Processor
HVTP	Hardware Verification Test Program (the individual test programs within HVTS)
HVTS	Hardware Verification Test System (resides at the LCN Module level and tests the LCN Module application boards)
IOCB	Input Output Control Block
LCN	Local Control Network
LCPU	TPDG board Local MC68020 processor
LONG WORD	Thirty-two bits, four bytes, or two words
MCPU	The LCN Module Central Processing Unit (K2LCN/K4LCN)
MTOS	Multi-Tasking Operating System
OPERATOR ALERT	Displays test program information and does not increment the errorcount
PRIMITIVES	Video graphics drawing procedures

PDG	Peripheral Display Generator; or those functions associated with the video, cursor control, keyboard, and serial printer subsystems. On multifunction boards, like the TPDG and EPDG, "PDG" is used to differentiate between the PDG functions and the SCSI functions.
RAM	Random Access Memory
RAMDAC	Brooktree Bt459 integrated circuit
ROM	Read Only Memory
SOFTWARE OVERLAY	Down-loaded code that overwrites, replaces, and is run instead of firmware
SCSI	Small Computer System Interface
SOFTWARE PROCEDURES	Down-loaded code that coexists with, and is called by, firmware
SRAM	Static RAM
TBD	To Be Determined
TOCS	Test Operator's Control Station
TPDG	Turbo Peripheral Display Generator board. A dual function board that supports both SCSI and PDG functions on a UXS
UART	Universal Asynchronous Receiver/Transmitter
UXS	Universal Station with X-Windows that consists of a WSI, WSI I/O, TPDG, TPDG I/O, K2LCN/K4LCN, LCN I/O, and high resolution 1280 x 1024 monitor
VIDEO ATTRIBUTES	The color, intensity, or blink characteristics of the video screen or some object on the video screen
VRAM	Video RAM
WORD	Sixteen bits or two bytes
WSBUS	Bus connecting the TPDG to the WSCPU (namely, the DI02 bus)
WSCPU	Workstation Coprocessor board (Hewlett Packard)
WSI	Workstation Interface Subsystem
WSIF	WSI Hardware Verification Test Program
WST	WSI Test sequencer board

Appendix B

UCLM: CCP and UCIM Redesigned

CCP (51402426) and UCIM (51402436) boards have been redesigned and are now known as UCLM-51306409. UCLM is a combination of CCP and UCIM.

Ordering Information

To order the boards use the Honeywell part numbers as provided in the following table:

Part Number	Board Name
51306409-100	CCP
51306409-200	UCIM

Switch Settings of UCLM board

Figure C-1 illustrates the redesigned CCP and UCIM settings.

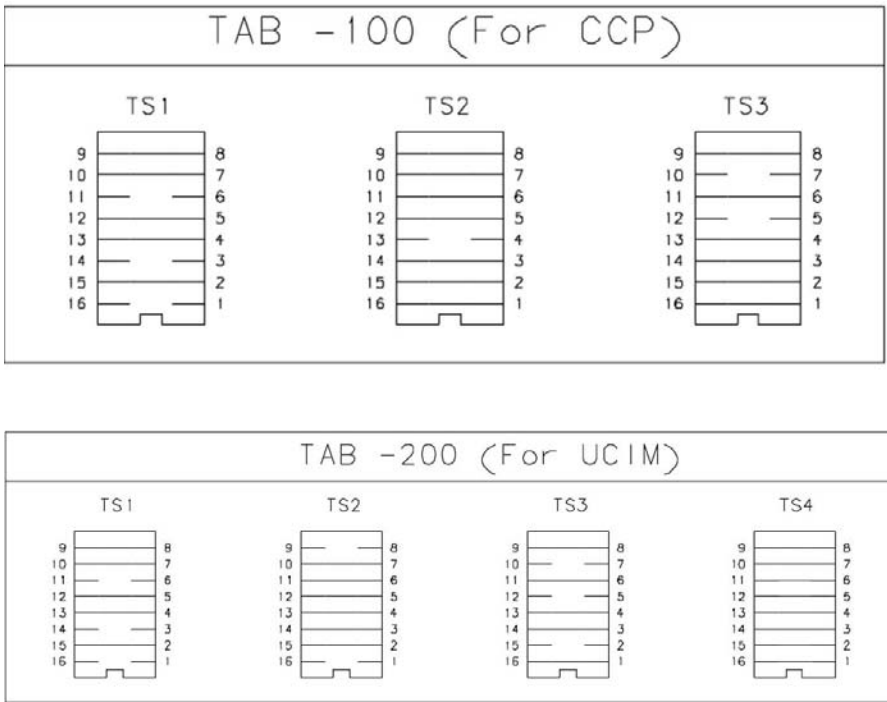


Figure 1 – Switch settings of UCLM board

There is no change in the procedure to load AM personality on UCLM. Procedure remains the same as it was in CCP and UCIM.

Honeywell

Honeywell Process Solutions
1860 W. Rose Garden Lane
Phoenix, AZ 85027 USA